



A Graph Neural Network and Blockchain-Based Framework for Detecting Temporal-Echo Topology Poisoning Attacks in Software-Defined Vehicular Networks

An undergraduate project progress report submitted to the
Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfillment of the requirements for the
**Degree of the Bachelor of the Science of Engineering
Honours**

by

Kumara K.N.Y.	- EG/2021/4624
Bandara A.W.M.L.M.	- EG/2021/4426
Wijegurusinghe O.W.D.K.M	- EG/2021/4890
Hewawasam A.D.K.	- EG/2021/4556

.....
Dr. Nilmantha Wijesekara
(Supervisor)

.....
Dr. Prabath Weerasingha
(Co-Supervisor)

Abstract

Software-Defined Vehicular Networks (SDVNs) depend on temporally consistent control-plane beacon and heartbeat messages for safety-critical services including collision avoidance and cooperative driving, where controller topology corruption can directly impact network performance. Existing topology poisoning defences share six fundamental limitations: they target only structurally invalid or volumetrically anomalous traffic with no formal characterisation of timestamp forgery, heartbeat identity spoofing, or false witness attestation; apply fixed detection thresholds unadapted to vehicular mobility parameters; omit cryptographic placement analysis per attacker position; fail to bind topology observation reports to the reporter’s physical position and signal strength; and ground blockchain verification in controller-reported data, rendering trust architectures invalid under a malicious controller assumption. This paper formally characterises three novel attack variants: (1) Topology Time-Warp (TTW), which forges beacon timestamps to artificially prolong stale topology states beyond physical link lifetime bounds; (2) Beacon State Heartbeat Hijack (BSHH), which spoofs heartbeat sender identity to make the controller falsely believe a vehicle is active at a location or time inconsistent with physical reality; and (3) Multipath Echo (ME), which fabricates phantom topology paths either through false witness attestations from physically inconsistent reporters or through direct controller topology database injection, inflating path diversity beyond physically plausible bounds. To counter these attacks, we propose TETA-Guard, a dual-mode framework with five contributions. First, nine formally derived attack signatures grounded in a mobility-induced vulnerability model that links link lifetime, RSU handover windows, and dynamic vehicle density to the temporal ambiguity each attack exploits. Second, a Lightweight (LW) version deploying these signatures with HMAC-timestamp-nonce pre-detection filtering on resource-constrained trusted nodes within the 100 ms beacon interval. Third, a Full-Stack (FS) version applying a Temporal Graph Neural Network modelling beacon-based topology graph evolution under vehicular mobility, with distributed evidence collection across trusted node Hyperledger Fabric peers under PBFT consensus, eliminating the controller as a single point of trust. Fourth, a position-aware cryptographic placement analysis formally mapping mitigation primitives — ML-DSA-87 threshold aggregate signatures over ML-KEM-1024+HQC-5 hybrid post-quantum key agreement, location-binding signatures for ME false witness prevention, and LKH-based key revocation — to attacker position across all three variants. Fifth, blockchain smart contracts deployed in a dual active role as forensic detection enabler through immutable temporal event correlation and automated mitigation executor through flow rule enforcement, replacing the passive audit function of all prior SDVN blockchain deployments. Evaluation across five structured benchmark experiments on NS-3/SUMO vehicular simula-

tions, assessed via twelve performance evaluation metrics against three topology poisoning baselines and fourteen ablation configurations, demonstrates detection accuracy, latency compliance within the 100 ms safety bound, and routing performance recovery across all twelve attack scenarios spanning four attacker positions per variant.

Keyword : Software-Defined Vehicular Networks, Temporal-Echo Topology Poisoning, Graph Neural Networks, Post-Quantum Cryptography, Blockchain, Cybersecurity.

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Dr. Nilmantha Wijesekara, for his invaluable guidance, continuous support, and encouragement throughout the development of this undergraduate project titled “*A Transfer-Learned Graph Neural Network and Blockchain-Based Framework for Temporal Fraud Detection: From Countering Temporal-Echo Topology Poisoning Attacks in SDVNs to Tax Evasion Analysis.*” His expertise, insightful feedback, and dedication have been instrumental in shaping the direction and success of this work. We also extend our heartfelt thanks to our co-supervisor, Dr. Prabath Weerasingha, for his constructive suggestions, guidance, and support in refining this proposal. His contributions have greatly enhanced the quality of our research. Furthermore, we would like to acknowledge all the academic staff and coordinators involved in the undergraduate project module for their guidance and support throughout this journey. Finally, we express our sincere appreciation to everyone who has contributed directly or indirectly to the successful completion of this project. Their encouragement and support have been truly valuable.

Contents

Abstract	ii
Acronyms	xii
1 Introduction	1
1.1 Background	1
1.1.1 Control Plane Protocols and Timing Dependence	1
1.2 Problem Statement	2
1.3 Limitations of Existing Approaches	2
1.4 Research Objectives	3
1.4.1 Research Questions	4
1.5 Research Scope	4
1.6 Motivations	4
1.7 Methodological Contributions	5
1.8 Novelty Statement	6
1.9 Implications of the Study	6
2 Background and Related Work	7
2.1 Background: Temporal Attacks in SDN and SDVN	7
2.1.1 Phantom Link and Time-Delay Attacks	7
2.1.2 Timing Side-Channels and Reconnaissance	7
2.1.3 Detection Challenges	8
2.1.4 Critical Implications for SDVN	8
2.2 Related Work	11
2.2.1 Cluster A: Topology Poisoning and Control-Plane Attacks in SDN/SDVN	11
2.2.2 Cluster B: AI/ML-Based Detection in VANET and SDN Security	12
2.2.3 Cluster C: Cryptographic Mechanisms for V2X/VANET Se- curity	13
2.2.4 Cluster D: Blockchain for Detection and Mitigation in SDN/VANET	14
2.3 Research Gaps	14
3 Methodology	16
3.1 System Architecture	16
3.1.1 Notation Definitions	18
3.1.2 Data Plane Layer	20
3.1.3 Control and Intelligence Layers	20
3.1.4 Verification and Application Layers	21
3.2 High-Level Functional Workflow	21

3.3	Formulation of Temporal-Echo Attacks in SDVNs	22
3.3.1	Topology Time Warp (TTW) Attack	22
3.3.2	Beacon State Heartbeat Hijack Attack (BSHH) Attack . . .	31
3.3.3	Multipath Echo (ME) Attack	39
3.4	Detection and Mitigation Framework	46
3.4.1	Temporal-Echo Attack Formalization	46
3.4.2	Lightweight Detection	51
3.4.3	Full-Stack Temporal Graph Network Detection	53
3.4.4	TGN Training and Testing Pipeline	59
3.4.5	Threshold Aggregate Signature Authentication	64
3.4.6	ME-Specific Mitigation: Location-Binding Signatures and Quorum Attestation	65
3.4.7	Attack Signature Characterization	66
3.4.8	Mobility-Induced Attack Amplification in SDVNs	67
3.4.9	Detection Logic	72
3.4.10	Cryptographic Placement Analysis	80
3.4.11	Mitigation Algorithms	82
3.4.12	Dynamic Peer Selection and the RSU Demotion Pipeline . .	94
4	Results	98
4.1	Attack Injection	98
4.2	Baseline Selection and Benchmarking Rationale	98
4.2.1	External Baselines	99
4.2.2	Internal Ablation Study	99
4.3	Performance Evaluation Metrics	108
4.3.1	Detection Quality Metric (M1)	109
4.3.2	Attack-Specific Metrics (M2–M4)	109
4.3.3	System Pipeline Latency Metric (M5)	110
4.3.4	Network Performance Metrics (M6)	111
4.3.5	Communication Overhead Metric (M7)	111
4.3.6	Scalability Metric (M8)	111
4.3.7	Component-Level Metrics for Ablation Study (M9–M12) . .	112
4.3.8	Research Questions	116
4.3.9	Benchmark Experiment Definitions	116
4.4	Simulation Settings	119
4.5	Detection Performance Results	142
4.5.1	MCC vs. Attack Injection Rate	143
4.5.2	AUROC vs. Attack Injection Rate	145
4.5.3	Detection Latency vs. Attack Injection Rate	146
5	Timeline and Resource Required	148
5.1	Timeline	148
5.2	Resource Required	148
5.2.1	Hardware Resources	148
5.2.2	Software Resources (All Open-Source/Free)	149
5.2.3	Budget Summary	149
6	Conclusion	150

List of Figures

3.1	Proposed Layered System Architecture for Secure Temporal Anomaly Detection in SDVN	17
3.2	Proposed Attack Detection and Mitigation Functional Workflow . .	22
3.3	TTW Attack via Malicious Vehicle (Without RSU): V1 manipulates timestamps of HELLO messages sent to the controller.	23
3.4	TTW Attack via Malicious RSU: RSU manipulates timestamps in forwarded vehicle HELLO messages.	25
3.5	TTW Attack via Malicious Controller (Without RSU): The controller internally manipulates timestamps of received HELLO messages.	27
3.6	TTW Attack via Malicious Controller (With RSU): The controller internally manipulates timestamps of received HELLO messages. . .	29
3.7	BSHH Attack via Malicious Vehicle (Without RSU): V2 hijacks heartbeat messages from V3 to V1, inducing false adjacency.	31
3.8	BSHH Attack via Malicious RSU: The RSU manipulates the sender identity of heartbeat messages to induce false adjacency.	33
3.9	BSHH Attack via Malicious Controller (Without RSU): The controller internally hijacks heartbeat identities to induce false adjacency. .	35
3.10	BSHH Attack via Malicious Controller (With RSU): The controller internally hijacks heartbeat identities to induce false adjacency. . .	37
3.11	Vehicle-based Multipath Echo Attack (Without RSU): V3 and V4 replay V1–V2 topology to the controller, creating false paths.	39
3.12	The RSU injects duplicated topology information related to V1–V2, creating false paths. (With RSU)	41
3.13	Controller-based Multipath Echo Attack (Without RSU): Controller internally duplicates V1–V2 link across multiple inferred paths. . .	43
3.14	Controller-based Multipath Echo Attack (With RSU): The controller internally duplicates the V1–V2 link across multiple inferred paths.	45
3.15	Offline TGN training pipeline (<code>tgn/tgn_train.py</code>).	61
3.16	Online TGN testing / inference pipeline (<code>TGN_ProcessEventInline</code>). .	64
3.17	Sequence diagram of the lightweight (LW) Temporal-Echo attack detection and mitigation workflow.	74
3.18	Sequence diagram of the full-stack (FS) Temporal-Echo attack detection and mitigation workflow.	78
4.1	Real-world OpenStreetMap-based Colombo urban map used for SUMO GUI mobility simulation.	120
4.2	NetAnim visualization of the SDN-enabled VANET simulation topology consisting of 200 vehicles, 64 RSUs arranged in an 8×8 grid . .	121

4.3	MCC vs. attack percentage for TTW-BSHH-ME attack scenarios (detection-enabled vs. detection-disabled).	145
4.4	AUROC vs. attack percentage for TTW-BSHH-ME attack scenarios (detection-enabled vs. detection-disabled).	146
4.5	Detection latency (T_{det}) vs. attack percentage for TTW-BSHH-ME attack scenarios (detection-enabled vs. detection-disabled).	147
5.1	Project Timeline (Gantt Chart)	148

List of Tables

2.1	Comparative Analysis of Related Work: Coverage of Temporal-Echo Topology Poisoning Attacks in SDVNs (Part 1 of 2). Y = Yes; P = Partial; N = No; NP = None Proposed; - = Not Applicable. Columns G1–G4 indicate coverage of the four identified research gaps (G1: All three attack variants addressed; G2: Dual-mode LW + FS versions provided; G3: Formal attack signatures derived; G4: Cryptographic role analyzed per attacker position).	9
2.2	Comparative Analysis of Related Work: Coverage of Temporal-Echo Topology Poisoning Attacks in SDVNs (Part 2 of 2). G1 – All three variants addressed; G2 – Dual-mode LW + FS provided; G3 – Formal signatures derived; G4 – Cryptographic role analysed per attacker position.	10
3.1	A summary of the notations.	18
3.2	TGN training hyperparameters (<code>tgn/tgn_train.py</code> defaults). . . .	62
3.3	Summary of nine formally characterized attack signatures mapped to observable control-plane features and effective attacker positions.	71
3.4	Cryptographic placement analysis per attack variant and attacker position.	82
3.5	Trust and peer-selection parameters	97
4.1	External Baseline Systems	99
4.2	Ablation study configurations. Each row targets one design component in isolation. X : independent variable; PEMs : applicable metrics from Table 4.4 (M_n = metric number; † denotes ablation-only metrics not reportable for B1–B3 baselines).	100
4.3	Ablation coverage: design component $\times$ ablation ID and ablation-only PEM $\times$ ablation ID.	108
4.4	Performance evaluation metrics: design component coverage, equation reference, and baseline comparability. Ablation-only metrics are applicable to ablation configurations A1–A6 but not to external baselines B1–B3.	115
4.5	Research questions and the experiment or metric set that answers each.	116
4.6	Experiment 5 result placeholder: per-attack-variant comparison of TETA-Guard against B1–B3 under default settings. ↑ : higher is better. ↓ : lower is better. [TBD: populate with simulation results.]	119
4.7	DSRC channel settings and estimated communication range — urban scenario	122
4.8	Simulation settings.	122

4.9	Parameters requiring empirical calibration	124
5.1	Hardware Resource Requirements	149
5.2	Software Resource Requirements	149
5.3	Overall Project Budget	149

Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
ARP	Address Resolution Protocol
AUROC	Area Under the Receiver Operating Characteristic
BSHH	Beacon State Heartbeat Hijack
DDoS	Distributed Denial of Service
DoS	Denial of Service
FN	False Negative
FP	False Positive
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
IDS	Intrusion Detection System
LLDP	Link Layer Discovery Protocol
ME	Multipath Echo
ML	Machine Learning
NS-3	Network Simulator Version 3
OBU	On-Board Unit
PBFT	Practical Byzantine Fault Tolerance
PKI	Public Key Infrastructure
QoS	Quality of Service
RSU	Roadside Unit
SDN	Software-Defined Networking

SDVN Software-Defined Vehicular Network

SUMO Simulation of Urban Mobility

TGN Temporal Graph Network

TN True Negative

TP True Positive

TTW Topology Time-Warp

VANET Vehicular Ad-hoc Network

Chapter 1

Introduction

1.1 Background

The rapid development of intelligent transportation systems and autonomous driving technologies depends heavily on reliable, low-latency, and adaptive communication networks. Vehicular Ad-hoc Networks (VANETs) enable direct communication among vehicles and roadside units, but their highly dynamic topology makes traditional distributed routing and management inefficient [25, 26]. Software-Defined Vehicular Networks (SDVNs) integrate the principles of Software-Defined Networking (SDN) with vehicular networking, enabling centralized control, programmability, and global visibility of the network state [25, 27].

In SDVN architecture, a centralized controller maintains a global view of vehicles, roadside units, and communication links, and dynamically manages routing, traffic engineering, and network policies [18, 27]. The separation of control and data planes allows rapid adaptation to mobility, traffic density changes, and application requirements. These capabilities make SDVN a promising foundation for safety-critical services such as collision avoidance, cooperative driving, and emergency response systems [26, 27].

However, this architectural shift also changes the security model. Traditional vehicular networks relied on distributed decision-making, whereas SDVN concentrates control intelligence in a central controller. The correctness of the entire network depends on the accuracy and timeliness of topology and status information reported to the controller. Any manipulation of this control-plane information can lead to incorrect routing, service disruption, and potentially dangerous decisions affecting vehicle safety [18, 24, 27].

1.1.1 Control Plane Protocols and Timing Dependence

The SDVN control plane relies on a set of standardized communication protocols to support centralized monitoring and network control [25]. The southbound interface between the controller and RSUs is implemented using the OpenFlow protocol (version 1.5) over TCP port 6653 [25]. Wireless communication between vehicles and RSUs is supported through IEEE 802.11p and Wireless Short Message Protocol (WSMP), with periodic beaconing used to convey mobility and link-state

information.

Topology discovery in the considered system model is performed through the aggregation of beacon messages periodically broadcast by vehicles, rather than through LLDP-based link-layer discovery. Lightweight beacon messages are transmitted at fixed intervals to report node presence, neighborhood information, and link conditions to RSUs, which forward aggregated topology state updates to the controller [20, 26]. In addition, heartbeat-style beacon mechanisms are employed to maintain network liveness, with small control messages transmitted at regular intervals.

To secure control-plane communications, Transport Layer Security (TLS 1.3) is assumed to provide cryptographic protection against eavesdropping and message tampering [7]. Despite these protections, the control plane remains highly dependent on correct timing, sequencing, and message freshness. As a result, SDVN control mechanisms are inherently vulnerable to attacks that exploit temporal inconsistencies, delayed or replayed messages, and out-of-context reuse of valid control information [20, 23]

1.2 Problem Statement

In safety-critical Software-Defined Vehicular Networks, where incorrect routing decisions caused by corrupted topology information directly endanger collision avoidance and cooperative driving under a 100 ms latency constraint, existing security mechanisms designed for structural and volumetric anomalies cannot detect three novel temporal-echo attack variants - Topology Time-Warp, Beacon State Heartbeat Hijack, and Multipath Echo - that reuse structurally valid control messages through timestamp forgery, identity spoofing, and echo duplication respectively, leaving SDVN controllers with no lightweight or cryptographically secured detection and mitigation solution

1.3 Limitations of Existing Approaches

Most existing SDN and SDVN security mechanisms are designed to detect structural anomalies such as fake links, unknown nodes, or abnormal traffic volumes. Techniques such as rule-based validation (e.g., TopoGuard), active probing, and shallow machine learning models are effective against overt topology poisoning and denial-of-service attacks [18, 24, 28]. However, they suffer from several limitations:

- **G1 – No integrated detection across all three attack variants and attacker positions.** Existing topology poisoning defences address structural attack classes only — fake links, fake nodes, and LLDP relay [27, 31, 32, 42] — and no single framework simultaneously addresses TTW, BSHH, and ME across vehicle-origin, RSU-origin, and controller-origin vectors.
- **G2 – No dual-mode lightweight and full-stack architecture.** Rule-based defences [32, 42] are deployable but detect only overt structural violations. Deep detection models [19, 37] provide superior accuracy but impose computational overhead incompatible with resource-constrained OBUs and

RSUs. No framework provides both versions with a formally justified boundary between them.

- **G3 – No formal attack signatures for temporal-echo variants.** No existing work derives mathematically expressible, observable conditions from the definitions of TTW, BSHH, or ME that enable rule-based detection without model training. All current ML-based detectors [37, 39] require labeled training data and cannot detect attack variants not represented in training distributions.
- **G4 – No cryptographic placement analysis per attacker position.** Prior cryptographic schemes apply authentication uniformly at the message layer [29] without determining whether enforcement should precede or follow detection as a function of attacker position. No work identifies that insider vectors (controller-origin) render pre-detection cryptographic filtering architecturally ineffective.
- **G5 – No mobility-aware detection calibration.** All structural topology defences use fixed detection thresholds designed for static or wired SDN infrastructure. No existing work calibrates signature thresholds to vehicular mobility parameters including link lifetime, RSU handover windows, or dynamic vehicle density [27].
- **G6 – No distributed trust architecture resilient to malicious controller.** Existing blockchain deployments in SDVN use the controller as the primary data source for ledger entries [7, 31], producing a passive audit role that is entirely ineffective when the controller itself is the attacker. No prior work grounds blockchain evidence collection in RSU-level independent beacon observations verified against controller topology claims under Byzantine fault-tolerant consensus.

As a result, attacks that reuse valid data but manipulate timing, ordering, or repetition can bypass existing defenses [19–21].

1.4 Research Objectives

1. To analyze why existing SDVN security mechanisms fail to detect these attacks.
2. To formally define three temporal-echo attack variants (TTW, BSHH, ME) and their manipulation mechanisms.
3. To evaluate TETA-Guard across all twelve attack scenarios using the twelve performance evaluation metrics (Table 4.4), benchmarked against three external baselines and fourteen ablation configurations under NS-3/SUMO simulation.

1.4.1 Research Questions

Six research questions structure the evaluation of TETA-Guard. RQ1–RQ5 are each answered by one of five benchmark experiments (defined in Section 4.3.9); RQ6 is answered by the overhead metrics reported across all experiments.

RQ1: How effectively does TETA-Guard detect and mitigate each attack variant against each origin vector compared to the B1–B3 baselines under default deployment settings?

RQ2: How does TETA-Guard performance degrade as joint attack penetration and injection intensity increase from a no-attack baseline to full-network compromise?

RQ3: How robust is TETA-Guard detection and mitigation performance across the vehicular mobility range supported by DSRC-equipped SDVNs?

RQ4: Does TETA-Guard detection and mitigation performance scale sub-linearly with increasing network size, consistent with the $O(|\mathcal{W}|)$ analytical complexity bound (Eq. 3.14)?

RQ5: How does TETA-Guard performance degrade as the total false evidence volume generated per attacker per attack episode (attack aggressiveness index Ψ , Eq. 4.28) increases?

RQ6: What is the communication and pipeline latency overhead imposed by the dual-layer PQC authentication and blockchain enforcement architecture?

1.5 Research Scope

This study covers the SDVN control plane only; data-plane forwarding performance, physical-layer behaviour, and hardware-level validation are outside scope. Within the control plane, the scope is restricted to topology manipulation through timestamp forgery, identity spoofing, and echo duplication; jamming, physical tampering, and cryptographic-primitive compromise are not addressed.

The paper encompasses: the formal characterisation of three attack variants (TTW, BSHH, ME) and derivation of nine attack signatures; design and NS-3/SUMO simulation of the dual-mode (LW/FS) detection and mitigation architecture; offline TGN training and C++ inference deployment; specification of the Hyperledger Fabric `TemporalEchoMitigator` smart contract; and a five-experiment benchmark evaluation with twelve performance evaluation metrics against three external baselines and fourteen ablation configurations. Full TGN and Fabric runtime results and systematic B1–B3 comparative evaluation are within scope and in progress.

1.6 Motivations

We pursue this research driven by two key factors.

Security of Future Transportation Systems: With autonomous and connected vehicles proliferating, network control failures pose direct risks to human lives. Subtle attacks on control timing stand out as particularly insidious—they evade detection and can linger undetected for extended durations.[19, 20].

Inadequacy of Current Defenses: Today’s SDVN protections target blatant topology changes or traffic floods, leaving stealthy temporal deceptions unaddressed . [18, 24].

1.7 Methodological Contributions

Contribution 1: Formal Attack Signature Characterization. We derive nine formally specified, mathematically expressible attack signatures across TTW, BSHH, and ME attack variants, constituting the first rigorous behavioral fingerprint framework for temporal-echo topology attacks in SDVNs. These signatures enable systematic rule-based detection independent of machine learning training data.

Contribution 2: Dual-Mode Detection Framework. We propose a lightweight rule-based detection system and a full-stack Temporal GNN detection system as independent, evaluable alternatives targeting the same three attack variants. The lightweight version provides practical deployability on resource-constrained OBUs and RSUs; the full-stack version achieves maximum detection performance. Both are fully specified, implemented, and compared.

Contribution 3: Cryptographic Placement Analysis. We formally analyze, for each combination of attack variant and attacker position, whether cryptographic enforcement (HMAC+timestamp, threshold/aggregate signatures, ML-KEM-1024 + HQC-5 hybrid post-quantum key agreement) eliminates or merely reduces the attack surface, and derive the architectural placement rule: pre-detection for external vectors, post-detection for insider vectors. This analysis has not been performed in prior work.

Contribution 4: PQC-Secured Hybrid KEM Threshold Signature Mitigation. We integrate a hybrid post-quantum KEM — combining ML-KEM-1024 (Kyber-1024, NIST FIPS 203, Level 5, lattice-based) with HQC-5 (NIST-selected for standardization March 2025, NIST IR 8545, Level 5, code-based) — with a lattice-based threshold aggregate signature scheme [30] to provide post-quantum authenticated topology report verification for RSU-aggregated beacon data, specifically designed to prevent external-vector BSHH and TTW attacks. The two KEM primitives share no mathematical overlap: a break in ML-KEM-1024 (Module-LWE) provides no advantage against HQC-5 (quasi-cyclic syndrome decoding), making the hybrid strictly stronger than either primitive alone under an algorithmic diversity argument [66, 67].

Contribution 5: Blockchain Smart Contract Automated Mitigation. We design and specify the `TemporalEchoMitigator` smart contract on Hyperledger Fabric under PBFT consensus, which provides both (a) tamper-evident temporal event correlation for forensic detection of low-rate attacks, and (b) automated flow rule enforcement triggered by confirmed detection events, demonstrating blockchain’s dual role as detection enabler and mitigation executor.

1.8 Novelty Statement

This work introduces temporal-echo topology manipulation as a formally characterisable and deployably mitigable attack primitive in SDVNs. Four directions advance the state of the art beyond prior work. First, attack signatures are derived directly from the mathematical attack definitions, enabling training-free rule-based detection without labelled data. Second, a Temporal Graph Network is applied to beacon-based SDVN topology evolution under vehicular mobility — a regime where node arrival and departure are governed by beacon intervals rather than switch failures, which no existing TGN application in wired SDN or financial networks addresses. Third, cryptographic enforcement is mapped to attacker position rather than attack type: location-binding signatures prevent false-witness echo attestation through physically verifiable position and signal-strength binding, a threat that content-only authentication cannot reach. Fourth, blockchain smart contracts are activated in a dual role — forensic temporal event correlation and automated flow-rule enforcement — replacing the passive audit function of all prior SDVN blockchain deployments, with the LW/FS detection boundary formally derived from the cryptographic placement analysis rather than chosen heuristically.

1.9 Implications of the Study

This study yields key implications across theoretical, practical, policy, and future research domains. **Theoretical Implications.** It advances network security theory by positioning temporal deception as a core attack primitive, on par with structural and volume threats. New taxonomies must now elevate time to a primary security attribute . [18, 20]. Formalizing temporal-echo attacks as (Gt, T, E, I) equips researchers with a solid lens for dissecting timing manipulations in networked control systems.

Practical Implications. The results spotlight vulnerabilities in SDVN control planes that underpin vital functions like collision avoidance and platooning. Temporal attacks maintain structural validity yet trigger flawed routing and forwarding - stealthy risks to smart transportation that elude current detectors . [19, 27]. Operators must rethink topology validation and monitoring in live SDVN setups.

Policy Implications. The identified gaps in existing SDVN security mechanisms call for unified standards on temporal integrity in safety-critical networked systems. Regulators and standards groups need to mandate temporal checks in safety-critical protocols, mirroring today’s structural and cryptographic requirements.

Future Research Implications. The work lays groundwork for adaptable temporal anomaly detection in SDVNs. Evasion features such as inter-packet delays and sequence gaps guide GNN detector design, while the formal attack characterisations provide a foundation for experimental validation and implementation in subsequent research phases. Above all, it shifts validation focus: trust demands verifying not just what arrives, but when and in what order—reshaping security for tomorrow’s networks.

Chapter 2

Background and Related Work

2.1 Background: Temporal Attacks in SDN and SDVN

While no research specifically addresses “Temporal-Echo Attacks,” related temporal manipulation techniques have been identified in Software-Defined Networks (SDN) and Software-Defined Vehicular Networks (SDVN) that fundamentally differ from traditional structural attacks [1, 2, 26, 28]. Unlike topology poisoning that creates fake links or nodes, these temporal attacks manipulate the timing, sequencing, and ordering of legitimate control-plane messages. This corrupts the controller’s temporal perception of network events rather than its structural understanding [18, 19].

The attack vector lies in *when* events are perceived, rather than in the content of packets, creating a uniquely challenging detection problem [1, 4, 26].

2.1.1 Phantom Link and Time-Delay Attacks

Phantom Link Attacks: Weintraub et al. (2024) demonstrated that attackers can exploit transient synchronization windows during flow rule installation to create unauthorized connectivity within brief 50+ millisecond windows, leaving no trace once the network converges [20].

Time-Delay Attacks: Ramani et al. (2022) showed that intercepting and delaying Link Layer Discovery Protocol (LLDP) packets by 10–50 milliseconds causes topology convergence failures [21]. In SDVNs, where high vehicle mobility creates dynamic topologies, even minor delays can cause incorrect routing and compromise safety-critical communications [27].

2.1.2 Timing Side-Channels and Reconnaissance

Shoaib et al. (2021) showed that measuring flow rule response times allows adversaries to infer recent communications, cache states, and potentially vehicle proximity [22].

Liu et al. (2017) demonstrated that timing analysis combined with active probing enables reconstruction of network topology and communication patterns [23]. These studies reveal that timing itself is an information channel, exploitable for both passive reconnaissance and active temporal manipulation [24, 28].

2.1.3 Detection Challenges

Temporal attacks evade conventional defenses due to several inherent characteristics:

Temporal-Echo Topology Attacks are particularly challenging to detect due to a combination of properties that distinguish them from conventional topology poisoning. Individual messages within these attacks are factually correct — deception arises not from false content but from the temporal arrangement in which valid messages are delivered [18]. This stealth through legitimacy means that standard content-validation mechanisms cannot identify an attack from any single message in isolation.

Compounding this, the attacks succeed with a minimal operational footprint, requiring only low traffic volume to achieve their effect and thereby bypassing volumetric anomaly detection systems that rely on identifying abnormal traffic spikes [19, 26]. They further exploit transient states of inconsistency that arise naturally during topology updates, operating within legitimate timing windows that existing defences treat as normal behaviour [1, 4].

From a detection standpoint, these attacks present no distinctive structural signatures amenable to conventional pattern-matching approaches [8, 9, 20]. Identifying them instead demands temporal reasoning — specifically, an understanding of event causality and sequence consistency across successive control messages [5, 6, 18]. This requirement places Temporal-Echo attacks beyond the reach of existing rule-based and shallow machine learning defences, motivating the graph-based temporal detection approach proposed in this work.

2.1.4 Critical Implications for SDVN

These attacks are particularly concerning in vehicular contexts. Safety applications such as collision avoidance demand latencies under 100 milliseconds [1, 2]. Temporal attacks introducing 50+ millisecond delays or misordered events can directly compromise these guarantees [4, 27]. Frequent topology changes due to vehicle mobility expand the attack surface of exploitable temporal windows, making this a pressing concern for intelligent transportation security [2, 3, 26, 28].

Table 2.1: Comparative Analysis of Related Work: Coverage of Temporal-Echo Topology Poisoning Attacks in SDVNs (Part 1 of 2). **Y** = Yes; **P** = Partial; **N** = No; **NP** = None Proposed; – = Not Applicable. Columns G1–G4 indicate coverage of the four identified research gaps (G1: All three attack variants addressed; G2: Dual-mode LW + FS versions provided; G3: Formal attack signatures derived; G4: Cryptographic role analyzed per attacker position).

#	Reference	Year	Attack get	Tar-	Detection	Mitigation	Temp. Model	SDVN Spec.	Beacon Based	RSU Vector	G1	G2	G3	G4	Key Limitation
1	Hong et al. [32] (TopoGuard)	2015	Host hijacking; LLDLP fabrication	Rule-based port validation	Flow rule validation	N	N	N	N	N	N	N	N	N	Structural validity only; timestamp replay undetected
2	Bui et al. [18]	2015	Fake nodes; fake links (taxonomy)	Analytical only	NP	N	N	N	N	N	N	N	N	N	No temporal primitive; structural model only
3	Alharbi [33]	2015	LLDP link spoofing	HMAC authentication	Packet drop	N	N	N	N	N	N	N	N	N	HMAC without nonce cannot prevent valid-message replay (TTW)
4	Weintraub et al. [20]	2024	Temporal flow-rule ordering	IBN Timing analysis	Detection only	P	N	N	N	N	N	N	N	N	IBN-specific; no beacon forgery or heartbeat hijacking
5	Ramani et al. [21]	2022	LLDP time-delay injection	RTT measurement	Flow modification	P	N	N	N	N	N	N	N	N	External delays only; no internal timestamp forgery
6	Jain et al. [19] (Sec-Topo)	2024	LLDP topology poisoning	Autoencoder in P4	Diff-network blocking	N	N	N	N	N	N	N	N	N	LLDP-specific; inapplicable to beacon-based SDVN
7	Adjou et al. [31] (TopoTrust)	2022	HTS; OFDP poisoning	Blockchain verification	Smart contract isolation	N	N	N	N	N	N	N	N	N	Valid replayed beacons pass blockchain undetected
8	Marionette [36]	2024	Control-plane flow forwarding	NP	NP	N	N	N	N	N	N	N	N	N	No detection; no temporal manipulation addressed
9	TopoSleuth [42]	2025	Link fabrication; replay; flooding	Decoy + behavior profiling	Correlation response	P	N	N	N	N	N	N	N	N	LLDP-based; cannot detect TTW, BSHH, or ME in SDVN
10	Deng et al. [34]	2024	Five structural VANET threats only	Empirical study	Discussion only	N	Y	P	P	P	N	N	N	N	Valid-message reuse in wrong context outside threat model

Table 2.2: Comparative Analysis of Related Work: Coverage of Temporal-Echo Topology Poisoning Attacks in SDVNs (Part 2 of 2).
G1 – All three variants addressed; G2 – Dual-mode LW + FS provided; G3 – Formal signatures derived; G4 – Cryptographic role analysed per attacker position.

#	Reference	Year	Attack get	Tar- geted	Detection	Mitigation	Temp. Model	SDVN Spec.	Beacon Based	RSU Vector	G1	G2	G3	G4	Key	Limitation
11	Jayakrishna & Prasanth [13]	2025	DDoS	in VANETs	CNN + LSTM	Traffic filtering	P	P	N	N	N	N	N	N	N	Volumetric; inapplicable to low-volume temporal attacks
12	Mekonen et al. [39]	2025	BSM falsification	position	Ensemble ML	RSU alert	N	N	P	Y	N	N	N	N	N	BSM payload features unrelated to control-plane timing
13	Tariq [35]	2024	GNSS spoofing in VANET	spoofing	Autoencoder + LSTM	Attribute encryption	P	N	N	N	N	N	N	N	N	GNSS features unrelated to beacon timestamps or heartbeat
14	Yang et al. [1] (GNN-NAD)	2025	DDoS; web attacks in SDN	web attacks	GraphSAGE + RNP		P	N	N	N	N	N	N	N	N	Static graph; no beacon-interval evolution or mobility
15	DMSTG-AD [37]	2026	DDoS; topology spoofing	topology	GRU + BiGRU + NP		Y	N	N	N	N	N	N	N	N	Wired SDN; no mobility, beacons, or mitigation
16	PQC ML-KEM-1024+HQC-5 [64, 66]	2025	– (key agreement)	agreement	NP	Hybrid-KEM + HMAC	–	P	N	N	N	N	N	N	N	No insider or echo attack coverage; no anomaly detection
17	Yadav et al. [30]	2024	– (VANET authentication)	authentication	NP	Threshold signature	–	N	P	Y	N	N	N	N	N	No detection; cryptographic role not analysed per position
–	TETA-Guard (This work)	2025	TTW ; all variants; all positions	BSHH ; all variants; all positions	LW : 9 formal signatures; TGN; beacon-graph modeling	FS : HMAC + nonce; PQC + blockch smart tract	Y	Y	Y	Y	Y	Y	Y	Y	Y	First framework addressing all temporal-echo variants with formal signatures, dual-mode design, PQC mitigation, and per-position cryptographic analysis

Abbreviations: LW = Lightweight; FS = Full-Stack; TGN = Temporal Graph Network; PQC = Post-Quantum Cryptography; HTS = Host Tracking Service; LLDP = Link Layer Discovery Protocol; BSM = Basic Safety Message; GNSS = Global Navigation Satellite System; NP = None Proposed.

2.2 Related Work

The related work is organized into four thematic clusters, followed by a gap analysis table that maps existing limitations to the research questions addressed in this paper.

2.2.1 Cluster A: Topology Poisoning and Control-Plane Attacks in SDN/SDVN

The foundational taxonomy of SDN topology attacks — covering fake links, fake nodes, and LLDP relay attacks — is established by Bui et al. [18], which classifies all attack primitives as structurally based. No temporal manipulation is represented as a distinct category, confirming that temporal-echo attacks constitute a genuinely new primitive absent from the established threat model.

Rule-based defenses address structural validity exclusively. TopoGuard [32] detects link fabrication by validating LLDP packet source ports, but checks structural validity only — it does not verify timestamp freshness, message sequencing, or detect replayed valid packets. All three temporal-echo attacks preserve structural validity and therefore evade TopoGuard completely. SecTopo [19] uses a hybrid data-plane inspection and ML classification model for LLDP-based topology discovery in wired SDN; however, the SDVN system model uses beacon-based topology discovery over IEEE 802.11p rather than LLDP, making SecTopo’s feature set inapplicable. Furthermore, SecTopo does not model temporal message ordering, heartbeat liveness, or echo duplication patterns. Link-Guard [33] and similar HMAC-based countermeasures authenticate the source of topology messages but do not bind authentication to timestamp freshness or sequence numbers: an authorised node that replays its own previously authenticated beacon (TTW attack) passes HMAC verification because the key is valid, demonstrating that message authentication alone without freshness verification cannot prevent temporal replay attacks.

Blockchain-based defenses address structural authenticity, not temporal consistency. TopoTrust [31] verifies whether a packet originates from a legitimate node and achieves high detection rates for spoofing and packet-tampering attacks. However, a compromised node that replays its own previously valid topology update (TTW attack) or heartbeat (BSHH attack) passes TopoTrust’s blockchain verification because the message is structurally authentic. TopoTrust cannot detect attacks that reuse legitimate messages in the wrong temporal context. The most recent defense, TopoSleuth [42] (2025), employs a decoy-based tripwire mechanism that targets unauthorised discovery probes rather than temporal manipulation of legitimate update messages.

The most advanced recent attack, Marionette [36] (CCS 2024), demonstrates that the topology attack surface is still expanding — forwarding manipulation enables global topology distortion without injecting invalid packets. Even so, Marionette, like all prior structural attacks, introduces topologically invalid or

redirected packets and does not address the class of attacks that reuse valid packets in incorrect temporal contexts.

In SDVNs specifically, Deng et al. [34] represent the current state of the art, addressing fake node injection and link fabrication under vehicular mobility. Their attack model is restricted to structurally invalid topology data; the temporal-echo attacks proposed in this work reuse structurally valid topology data in incorrect temporal contexts, which falls outside the threat model and detection capability of Deng et al.

2.2.2 Cluster B: AI/ML-Based Detection in VANET and SDN Security

Ensemble and deep learning methods achieve near-perfect detection of position falsification attacks in VANETs [39], where the anomaly resides in BSM payload content (claimed position, speed, direction). However, position falsification features — sender-claimed location, RSSI-based distance estimation, plausibility checks — are fundamentally different from the control-plane temporal features (beacon timestamp, sequence number, inter-beacon interval, reporter multiplicity) that characterise temporal-echo attacks. Models trained on VeReMi-style features provide no detection signal for TTW, BSHH, or ME attacks.

The most methodologically comparable work is Tariq [35], which combines Autoencoder+LSTM anomaly detection with attribute-based encryption mitigation for BeiDou spoofing. PSAU-Defender [38] explicitly proposes a lightweight dual-component framework combining XGBoost/RF detection with geospatial message authentication, paralleling the dual-mode architecture of this paper. However, both operate at the GNSS signal and positioning layer, not the SDN control-plane layer, and their temporal features (satellite signal consistency, carrier phase, Doppler residuals) differ entirely from control-plane beacon timestamps and heartbeat sequence numbers.

GNN-NAD [1] represents the state of the art in applying GNNs to SDN anomaly detection, fusing a static vulnerability-aware attack graph with dynamic traffic features. However, it does not model the continuous temporal evolution of the topology graph as a sequence of snapshots. In the SDVN context, where the topology graph changes with vehicle mobility at beacon intervals, a purely static attack graph is insufficient. DMSTG-AD [37] is the most advanced temporal GNN applied to SDN intrusion detection, modelling both node-level short-term behavioural variations and graph-level long-term temporal trends simultaneously. However, DMSTG-AD is designed for wired SDN with static infrastructure; it does not model the high-mobility vehicular topology graph where nodes appear and disappear at beacon intervals, nor does it address RSU-aggregated reporting or heartbeat identity spoofing. The proposed TGN must demonstrate superior performance over DMSTG-AD on SDVN-specific temporal-echo scenarios.

Hybrid deep learning for DDoS detection in VANETs [13] demonstrates that deep learning is applicable to the VANET security domain, but DDoS detection

relies on volume-based and rate-based features orthogonal to the temporal ordering and replay features required for temporal-echo attack detection. The stacked ensemble IDS for SDN-based VANETs [13] is architecturally closest to this paper’s system model yet detects position falsification using BSM-layer features and the VeReMi dataset. No temporal control-plane attack dataset exists for SDVNs — one of the experimental challenges addressed in this paper by generating an NS-3/SUMO simulation dataset for TTW, BSHH, and ME scenarios.

2.2.3 Cluster C: Cryptographic Mechanisms for V2X/VANET Security

The ML-KEM-1024 + HQC-5 hybrid post-quantum key encapsulation mechanism [64, 66] provides the PQC-based key agreement foundation for the full-stack mitigation. ML-KEM-1024 (NIST FIPS 203) provides IND-CCA2-secure lattice-based key encapsulation resistant to quantum adversaries. HQC-5 (NIST-selected March 2025, NIST IR 8545) provides code-based key encapsulation under the quasi-cyclic syndrome decoding assumption, which is mathematically independent of ML-KEM-1024’s Module-LWE assumption; a break in one provides no advantage against the other. The HMAC+timestamp+nonce layer provides message freshness verification that directly prevents TTW replay attacks when applied as a pre-detection filter for external attack vectors. Quantitative latency evaluation of the hybrid KEM configuration is presented in the experimental section.

Prajapat et al. [30] establish that NTRU lattice-based aggregate signatures are feasible for VANETs, providing quantum resistance alongside signature compression for RSU-aggregated reports. This directly supports the threshold/aggregate signature component of the full-stack mitigation, where RSU-aggregated topology updates require at least t -of- n vehicle signatures. Yadav et al. [30] provide the key latency benchmark: 1.6 ms per signature generation and verification at the 2024 security parameter level. Given the 100 ms SDVN beacon interval, a threshold signature overhead of 1.6 ms per beacon is operationally feasible, providing the concrete latency justification for the full-stack mitigation.

For the lightweight cryptographic baseline, Jayashree and Kumar propose an ECC-based certificateless aggregate signature achieving low overhead on OBU hardware but providing no post-quantum security. The full-stack version of this paper upgrades this to lattice-based PQC; the lightweight version retains the HMAC-only approach from [29] without full aggregate signature verification, trading security for deployability.

Alharbi [33] establishes the earliest HMAC application to SDN topology message authentication and demonstrates that HMAC prevents unsigned packet injection. However, HMAC alone without timestamp binding cannot prevent a legitimate node from replaying its own previously signed valid topology update — precisely the TTW attack mechanism. This limitation of bare HMAC authentication motivates the combined HMAC+timestamp+nonce construction in the lightweight mitigation. Combining threshold signatures with blockchain verifi-

cation in VANETs is established by [10], with the threshold serving as a Sybil-resistance mechanism and the blockchain providing tamper-evident event logging, analogous to the RSU-aggregated threshold signature feeding into the Hyperledger Fabric layer in this paper.

Consortium blockchain for SDVN [7] provides tamper-evident logging and trust management but operates passively — recording events after they occur without triggering automated detection or mitigation responses. This motivates the addition of smart contract-driven automated mitigation in the proposed framework, converting the blockchain from a passive audit tool into an active response executor.

2.2.4 Cluster D: Blockchain for Detection and Mitigation in SDN/VANET

Blockchain deployments in SDN and VANET security uniformly adopt a passive audit role. Zhao et al. [7] and TopoTrust [31] use blockchain for tamper-evident logging and identity verification respectively but do not trigger automated countermeasures upon detecting anomalies. The anonymous authentication scheme in incorporates a blockchain incentive layer for Sybil-resistance but again treats the blockchain as a verification and logging backend rather than an active mitigation executor.

No existing SDVN or SDN blockchain deployment uses smart contracts to enforce flow rule changes, quarantine suspicious nodes, or correlate low-rate temporal attack patterns across multiple beacon intervals automatically. The proposed **TemporalEchoMitigator** smart contract on Hyperledger Fabric activates blockchain in a dual role: forensic detection enabler through immutable temporal event correlation, and automated mitigation executor through smart contract-triggered flow rule enforcement — a function not present in any prior SDVN blockchain deployment.

2.3 Research Gaps

Gap 1 — No Integrated Detection for All Three Temporal-Echo Variants in SDVN. Rows 1–9 show that all existing topology poisoning defenses — rule-based (rows 1, 2, 3), ML-based (rows 6, 9), or blockchain-based (row 7) — address structural attack types only. Row 9 (Deng et al.) is the only work in SDVN specifically, yet targets structural poisoning only. The temporal-echo attack class remains entirely unaddressed.

Gap 2 — No Framework Provides Both Lightweight and Full-Stack Versions for Temporal Attacks. Rows 10–14 show that existing AI/ML detection works do not provide a lightweight deployment alternative alongside the full AI-based solution. No existing work proposes both a computationally constrained

rule-based detection version and a full-stack AI+cryptography version targeting the same attack class.

Gap 3 — No Formal Attack Signatures Defined for Temporal-Echo Attacks. Rows 1, 2, and 6 show that existing defenses use implicit or informal anomaly indicators derived from structural features. No equivalent formal signature characterisation exists for TTW, BSHH, or ME attacks, leaving the rule-based detection baseline undefined.

Gap 4 — Cryptographic Role Not Analysed Per Attack Variant and Attacker Position. Rows 3, 13, 14, and 17 show that cryptographic mechanisms are applied without formal analysis of whether they prevent or merely reduce specific attack variants. No prior work determines the optimal placement of cryptography — pre-detection versus post-detection — for each combination of attack variant and attacker position (vehicle, RSU, controller).

Chapter 3

Methodology

3.1 System Architecture

This research proposes a comprehensive, hierarchical security framework designed to secure Software-Defined Vehicular Networks (SDVNs) against sophisticated temporal-echo attacks. The architecture necessitates a defense mechanism that transcends traditional rule-based filtering by enforcing temporal consistency and immutable accountability within the control plane. As illustrated in Figure 3.1, the system is structured into six logical layers that bridge the gap between experimental simulation and cross-domain fraud analytics

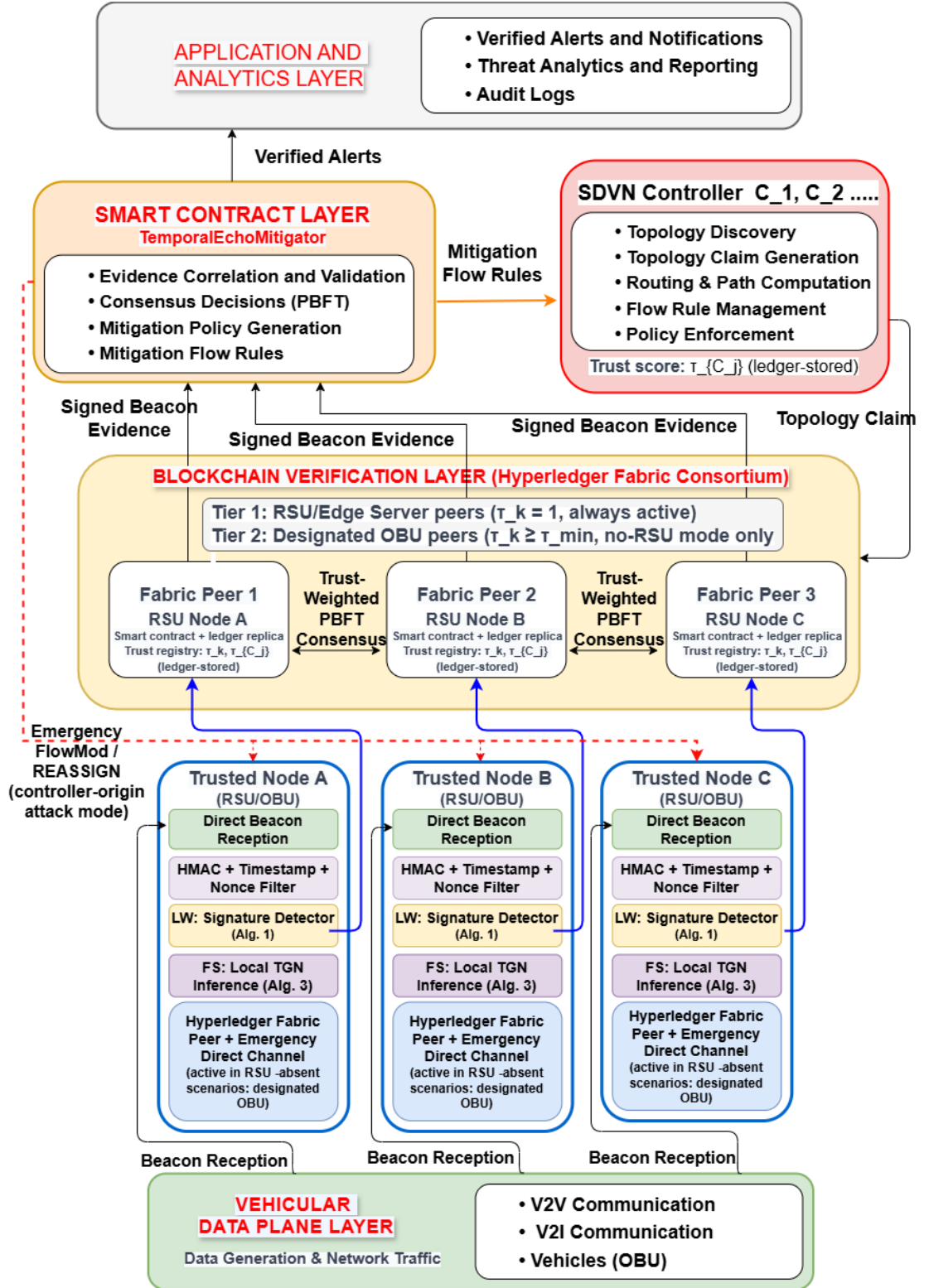


Figure 3.1: Proposed Layered System Architecture for Secure Temporal Anomaly Detection in SDVN

As illustrated in Fig. 3.1, the framework is organized as a six-layer stack in which the LW and FS detection modes share the Data Plane, SDN Control, and Blockchain layers, but diverge at the Intelligence Layer. The Data Plane Layer comprises OBU-equipped vehicles and RSUs operating as OpenFlow-enabled forwarding devices over IEEE 802.11p, generating the beacon and heartbeat traffic that constitutes the attack surface for TTW, BSHH, and ME attacks. The Pre-Detection Cryptographic Filter applies HMAC+timestamp+nonce verification to every beacon before it reaches the detection engine, eliminating unsigned injections and authenticated replays from external attack vectors. At the Intelligence Layer, the LW path executes rule-based signature detection on OBU/RSU-class hardware while the FS path runs Temporal GNN inference at each trusted RSU node independently, with both paths feeding confirmed detections to the Blockchain Layer where the **TemporalEchoMitigator** smart contract issues automated FlowMod isolation commands to the SDN controller.

3.1.1 Notation Definitions

Table 3.1: A summary of the notations.

Notation	Description
$A = (G_t, T, E, \mathbf{I})$	Four-tuple attack model. G_t : ground-truth topology at time t ; $T \in \mathbb{R}^{ V \times V }$: timestamp manipulation matrix; $E \in \{0, 1\}^{ V \times V }$: echo duplication matrix; $\mathbf{I} \in \{0, 1\}^{ V \times V }$: identity spoofing matrix.
$\mathbf{I} \in \{0, 1\}^{ V \times V }$	Identity spoofing matrix; $I_{jk} = 1$ if V_j claims identity V_k in a transmitted beacon or heartbeat.
$\delta(G_t^C, G_t^R) = E_t^C \Delta E_t^R $	Topology divergence: symmetric difference between controller-perceived edge set E_t^C and real edge set E_t^R ; non-zero value indicates a successful attack.
$G_t = (V_t, E_t, X_t, A_t)$	Temporal graph snapshot at beacon interval t . V_t : active vehicles/RSUs; E_t : reported topology links; $X_t \in \mathbb{R}^{ V_t \times d}$: node feature matrix; $A_t \in \mathbb{R}^{ V_t \times V_t }$: weighted adjacency matrix.
$\mathbf{x}_v = [\tau_{\text{dev}}^{(v)}, c_v^W, \Delta s_v, \rho_v, \iota_v] \in \mathbb{R}^5$	Node feature vector (5 elements): timestamp age deviation (BSHH signal), beacon count in window W (BSHH-S3 signal), sequence-number gap (TTW-S2 signal), reporter count (ME-S1 signal), identity mismatch indicator (BSHH-S1 signal). Node-ID embedding excluded to prevent identity-based overfitting.
$\tau_{\text{dev}}^{(v)} = \text{clip}((\tau_r^{(v)} - \tau_s^{(v)})/T_b, -50, 50)$	Timestamp age deviation: normalised clipped difference between reception time and claimed sender timestamp. Near zero for legitimate beacons and TTW-forged beacons (forged timestamp mimics current time); large positive for BSHH non-forged replays. Replaces raw $\tau_s^{(v)}$, which encodes simulation time-position and causes distribution shift.
$\iota_v \in \{0, 1\}$	Identity mismatch indicator (Eq. 3.21): 1 when the MAC-layer physical transmitter $\text{mac}_{\text{phys}}(e)$ does not match the application-layer claimed identity $\text{id}_{\text{claim}}(e)$, with suppression conditioned on the RSU's signed beacon evidence record $\mathcal{B}_{n_k}(t)$ (Eq. 3.46): (i) 0 for direct vehicle transmission; (ii) 0 if forwarding RSU is a consortium peer and $\text{id}_{\text{claim}}(e) \in \mathcal{B}_{\text{macphys}}(e)(t)$ — claimed vehicle in RSU's direct beacon observation record, confirming physical presence; (iii) 1 if forwarding RSU is a consortium peer and $\text{id}_{\text{claim}}(e) \notin \mathcal{B}_{\text{macphys}}(e)(t)$ — vehicle absent from RSU's beacon evidence, indicating impersonation; (iv) 0 for controller sentinel (physical_sender_id = 9999).
$A_{uv}(t) = \exp\left(-\frac{\tau_r^{(t)} - \tau_s^{(uv)}}{\gamma \cdot T_b}\right)$	Freshness-decayed edge weight: down-weights stale TTW-replayed links; $\gamma > 0$ is the decay hyperparameter.

Continued on next page

Table 3.1 – continued from previous page

Notation	Description
$C = \{(C_j, \tau_{C_j}, Z_j)\}$	Controller registry on ledger: controller C_j , its trust score $\tau_{C_j} \in [0, 1]$, and assigned zone Z_j .
T_b	Beacon interval (100 ms).
$L_{\text{link}} = 2r_{\text{comm}}/v_{\text{rel}}$	Mobility-derived link lifetime bound; r_{comm} : communication range; v_{rel} : relative vehicle speed.
$N_{\text{beacon}} = \lfloor L_{\text{link}}/T_b \rfloor$	Number of beacon intervals within one link lifetime.
$\tau_{\text{conv}} \leq T_b + \tau_{\text{prop}}$	Maximum topology convergence delay; τ_{prop} : propagation delay.
$W_{\text{ho}} \approx r_{\text{overlap}}/v_i$	RSU handover window; r_{overlap} : RSU overlap distance; v_i : vehicle speed.
$\tau_s^C(e_{ij}, V_i), \tau_r^{\text{last}}(V_i)$	Sender timestamp of topology report for link e_{ij} at the controller; reception time of the most recent legitimate beacon from V_i (TTW-S2).
$R(e_{ij}, t)$	Reporter set: vehicles claiming to have witnessed link e_{ij} at time t .
$\rho_{\text{max}}(\hat{\lambda}, r_{\text{comm}}) = \lfloor (1 + \mu) \cdot 2r_{\text{comm}} \cdot \hat{\lambda}(t) \rfloor$	1D road-segment observer count bound for ME-S1; $\hat{\lambda}(t)$: instantaneous vehicle density; $\mu = 0.2$ default inter-lane margin.
$\delta_{\text{max}} = \max(1, \lceil \lambda \cdot v_{\text{rel}} \cdot T_b \rceil)$	Mobility-consistent path-growth bound for ME-S2. Default: $\delta_{\text{max}} = 1$ (urban and highway).
$E[\lceil R^*(e_{ij}, t) \rceil] = 2r_{\text{comm}} \cdot \lambda(t)$	Expected observer count for link e_{ij} at vehicle density $\lambda(t)$ (1D road model).
$s(e) = \sum_{k=1}^9 w_k \cdot \mathbf{1}[\text{Sig}_k \text{ triggered}]$	LW weighted anomaly score; $w_k \geq 0$, $\sum w_k = 1$; alert raised when $s(e) > \theta_{\text{LW}}$.
$T_{\text{LW}}(W , S) = O(W \cdot S)$	LW per-event time complexity; reduces to $O(W)$ since $ S = 9$ is constant.
$m_v(t) = \text{GRU}(h_v(t^-), x_v(t), \varphi(\Delta t_v)), \varphi(\Delta t_v) = \log(1 + \Delta t_v/T_b)$	TGN temporal memory update; $h_v(t^-)$: prior embedding; $\varphi(\cdot)$: time-elapsd encoding.
$h_v^{(l+1)} = \sigma(W^{(l)} \cdot \text{MEAN}\{h_u^{(l)} \odot A_{uv}(t) : u \in \mathcal{N}(v, t)\} + b^{(l)})$	L -round neighbourhood aggregation; $\odot$: element-wise multiply; σ : ReLU; $\mathcal{N}(v, t)$: neighbours of v .
$\hat{y}_v(t) = \sigma(w^\top h_v^{(L)}(t))$	TGN anomaly score (σ = sigmoid here); alert when $\hat{y}_v(t) > \theta_{\text{FS}}$.
$\hat{\alpha}_v(t) = \text{softmax}(W_{\text{cls}} \cdot h_v^{(L)}(t))$	Predicted variant label from FS path; $W_{\text{cls}} \in \mathbb{R}^{3 \times d}$: classification head.
$h_{V_{\text{new}}}(t^-) = \mathbf{0} \in \mathbb{R}^d$	Zero initialization for a newly appearing vehicle node.
$\text{HMAC-SHA256}(K_{V_i, n_k}, m') = \text{MAC}_i^{\text{received}}$	HMAC-SHA256 integrity check; K_{V_i, n_k} : session key established via ML-KEM-1024 + HQC-5 [64, 66] KEM.
$ \tau_r - \tau_s \leq T_b + \epsilon$	Timestamp freshness bound; ϵ : max propagation tolerance.
$\text{nonce}_i \notin N_{\text{seen}}$	Nonce novelty check; N_{seen} : cache of previously observed nonces.
$\text{Verify}(\sigma_{\text{agg}}, PK_{\text{agg}}) = 1 \Leftrightarrow \{i : \text{Verify}(\sigma_i, m_i, PK_{V_i}) = 1\} \geq t$	Threshold aggregate signature validity; $t \geq \lfloor n/2 \rfloor + 1$ (strict majority).
$m'_{V_k} = e_{ij} \parallel \text{pos}_{V_k} \parallel \text{RSSI}_{V_k \leftarrow V_i} \parallel \tau_s \parallel \text{nonce}$	Location-binding message signed by reporter V_k (GPS position + RSSI bound to the observed link).
$\sigma_{V_k} = \text{Sign}(SK_{V_k}, m'_{V_k})$	ML-DSA-87 signature over location-binding message; SK_{V_k} : long-term ML-DSA-87 signing key of V_k .
ML-DSA-87 (NIST FIPS 204, Level 5)	PQC signing algorithm (CRYSTALS-Dilithium5, NIST Category 5); produces threshold aggregate signing key SK_{V_k} [43, 65].
ML-KEM-1024 + HQC-5 (hybrid KEM)	ML-KEM-1024 (NIST FIPS 203, Level 5): lattice-based (Module-LWE) KEM that establishes session key K_{V_i, n_k} ; not used for signing [64]. HQC-5 (NIST-selected for standardization March 2025, NIST IR 8545, Level 5): code-based KEM (quasi-cyclic syndrome decoding); provides algorithmic diversity because its hardness assumption is <i>mathematically independent</i> of Module-LWE, so a break in one cannot assist in breaking the other. Draft FIPS expected 2026; final standard ≈ 2027 . Available in liboqs as <code>OQS_KEM_alg_hqc.5</code> ("HQC-5") [66, 67].
$C_{\text{revoke}} = O(\log n)$	LKH session-key revocation cost for n vehicles; $d = \lceil \log_2 n \rceil$: tree depth.

Continued on next page

Table 3.1 – continued from previous page

Notation	Description
$\tau_k^{(t+1)}$	Node trust score update (Eq. 3.38): incremented by Δ^+ on correct participation, decremented by Δ^- on failure, zeroed on detection flag.
$\tau_{C_j}^{(t+1)} = \max(0, \tau_{C_j}^{(t)} - \Delta_-^C)$	Controller trust (penalty-only): decremented Δ_-^C per confirmed divergence; no reward path.
$E_t^{\text{trusted}} = \bigcup_{\substack{n_k \in P_{\text{active}} \\ \tau_k \geq \tau_{\min}^{\text{gt}}}} B_{n_k}(t)$	Trust-filtered RSU beacon evidence set; $B_{n_k}(t)$: signed beacon record from RSU n_k .
$\delta = E_t^C \Delta E_t^{\text{trusted}}$	Controller-vs-evidence divergence for controller-origin attack detection (Eq. 3.45).
$\delta_{\text{thresh}} = \left\lceil \left(1 + \frac{\tau_{\text{prop}}}{T_b}\right) \cdot \lambda \cdot 2r_{\text{comm}} \right\rceil + 1$	Controller-origin divergence detection threshold; urban/highway default: $\delta_{\text{thresh}} = 14$ (see Table 4.1).
$\text{Accept}(e_{ij}) \Leftrightarrow \{V_k : \text{Accept}_{V_k}(e_{ij}) = 1\} \geq t$	Quorum acceptance of link e_{ij} : requires t physically consistent witnesses to pass Eq. 3.29.
$P_{\text{active}} = \arg \max_{ P =n_p} \sum_{k \in P} \tau_k$ ($V_k \in P \Rightarrow \text{Eligible}(V_k)$)	Active peer set (size $n_p = 3f + 1$) selected by highest aggregate trust; f : max Byzantine peers.
$\text{Accept} \Leftrightarrow \frac{\sum_{k \in P_{\text{approve}}} \tau_k}{\sum_{k \in P_{\text{active}}} \tau_k} > \frac{2}{3}$	Trust-weighted PBFT acceptance condition (Eq. 3.47).
$R_{\min} = \left\lceil (\tau_{\min}^{\text{gt}} - \tau_0) / \Delta^+ \right\rceil$	Bootstrap phase duration in consensus rounds; default $R_{\min} = 8$ rounds.
$O = (V_{\text{id}}, \alpha, \hat{y}_v, S_{\text{trig}}, t_{\text{alert}})$	Alert object: offending node ID, attack variant $\alpha \in \{\text{TTW}, \text{BSHH}, \text{ME}\}$, anomaly score, triggered signature set, detection timestamp.

3.1.2 Data Plane Layer

The **Data Plane Layer** serves as the foundational tier of the proposed framework, representing the physical forwarding infrastructure of the SDVN. This layer comprises vehicles equipped with On-Board Units (OBUs) and Roadside Units (RSUs) that function as OpenFlow-enabled switches. These devices are responsible for forwarding network traffic based exclusively on flow tables provided by the control plane, acting as the physical entry point for both legitimate traffic and malicious topology discovery packets injected by compromised nodes.

While the proposed architecture is designed for real-world deployment, for the purpose of this research, this layer is experimentally modeled using **NS-3** and **SUMO**. These tools generate realistic vehicular mobility patterns and network traffic, allowing for the systematic injection of specific temporal threats—Topology Time-Warp (TTW), Beacon State Heartbeat Hijack (BSHH), and Multipath Echo (ME)—to train and validate the detection models.

3.1.3 Control and Intelligence Layers

The SDVN deployment operates a multi-controller architecture in which the network is partitioned into geographic coverage zones per Eq. 3.39, each assigned a primary and at least one backup controller registered in the Fabric consortium.

Each controller’s trust score τ_{C_j} is maintained on the ledger per Eq. 3.41 and is the sole basis for subnetwork reassignment decisions. The SDN Control Layer is the *recipient* of topology enforcement commands — FlowMod rules issued by the Fabric smart contract — and the *submitter* of topology claims to the Fabric network for cross-checking; it does not feed the detection pipeline. In controller-origin attack scenarios, the smart contract issues FlowMod rules directly to RSU OpenFlow agents via the emergency channel, bypassing the malicious controller entirely, and triggers subnetwork reassignment per Eqs. 3.44–3.45.

Detection logic is embedded within each trusted node n_k (RSU or designated OBU), not in a centralised intelligence layer fed by the controller. Each trusted node runs the LW signature detector (Algorithm 1) locally on beacons it directly receives, and the FS Temporal GNN (Algorithm 2) on its local graph snapshot $\mathcal{G}_t^{n_k}$. Detection outputs are submitted directly to the node’s local Fabric peer via the emergency channel — never routed through the controller. The controller submits only its topology claim G_t^C to the Fabric for cross-checking against $\mathcal{E}_t^{\text{trusted}}$ per Eq. 3.47.

3.1.4 Verification and Application Layers

The Blockchain Verification Layer uses Hyperledger Fabric for audit trails and fault tolerance. Smart Contracts verify Intelligence Layer alerts, log attack meta-data to an immutable ledger, and trigger automatic isolation of malicious nodes.

3.2 High-Level Functional Workflow

While the system architecture outlines structural layers, Figure 3.2 depicts the operational data pipeline as a functional block diagram. This workflow converts live network traffic into security actions across four processing stages.

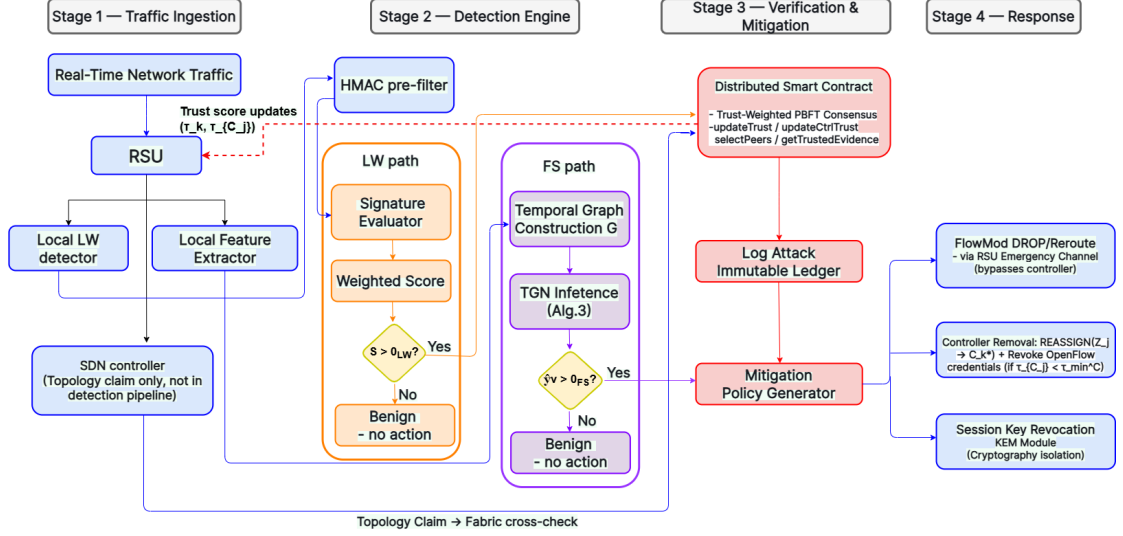


Figure 3.2: Proposed Attack Detection and Mitigation Functional Workflow

As shown in Fig. 3.2, beacon traffic is parsed into structured topology events in Stage 1 through RSU-based traffic ingestion, local lightweight (LW) detection, and feature extraction before forwarding topology claims to the SDN controller. In Stage 2, the proposed dual-mode detection engine performs both lightweight signature-based analysis and full-stack Temporal Graph Network (TGN) inference. The LW path applies HMAC pre-filtering, signature evaluation, and weighted scoring to rapidly identify temporal inconsistencies, while the FS path constructs temporal graph representations and performs TGN-based anomaly inference for advanced attack detection. In Stage 3, suspicious topology claims are verified using trust-weighted PBFT consensus per Eq. 3.49 among RSU Fabric peers, with controller topology claims cross-checked against $\mathcal{E}_t^{\text{trusted}}$ per Eq. 3.47. Detected attacks are immutably logged and mitigation policies are generated. For node-origin attacks, FlowMod isolation is issued via the RSU emergency channel directly to OpenFlow agents. For controller-origin attacks where $\tau_{C_j} < \tau_{\min}^C$, subnetwork re-assignment per Eq. 3.44 is triggered, transferring zone management to C_k^* per Eq. 3.45 without any involvement of the malicious controller. Finally, in Stage 4, automated response mechanisms enforce FlowMod-based traffic isolation and session key revocation to mitigate Temporal-Echo topology poisoning attacks in the SDVN environment.

3.3 Formulation of Temporal-Echo Attacks in SD-VNs

3.3.1 Topology Time Warp (TTW) Attack

Unlike link fabrication attacks, the topology time warp attack preserves spatial correctness while corrupting temporal ordering, making it particularly difficult

to detect using conventional topology validation mechanisms. The TTW attack targets the temporal consistency of topology information in SDVNs rather than introducing false links.

TTW Attack via Malicious Vehicle

Figure 3.3 illustrates the TTW attack executed by a malicious vehicle operating without RSU involvement, where the compromised vehicle replays forged timestamp topology updates directly to the controller after physical link failure.

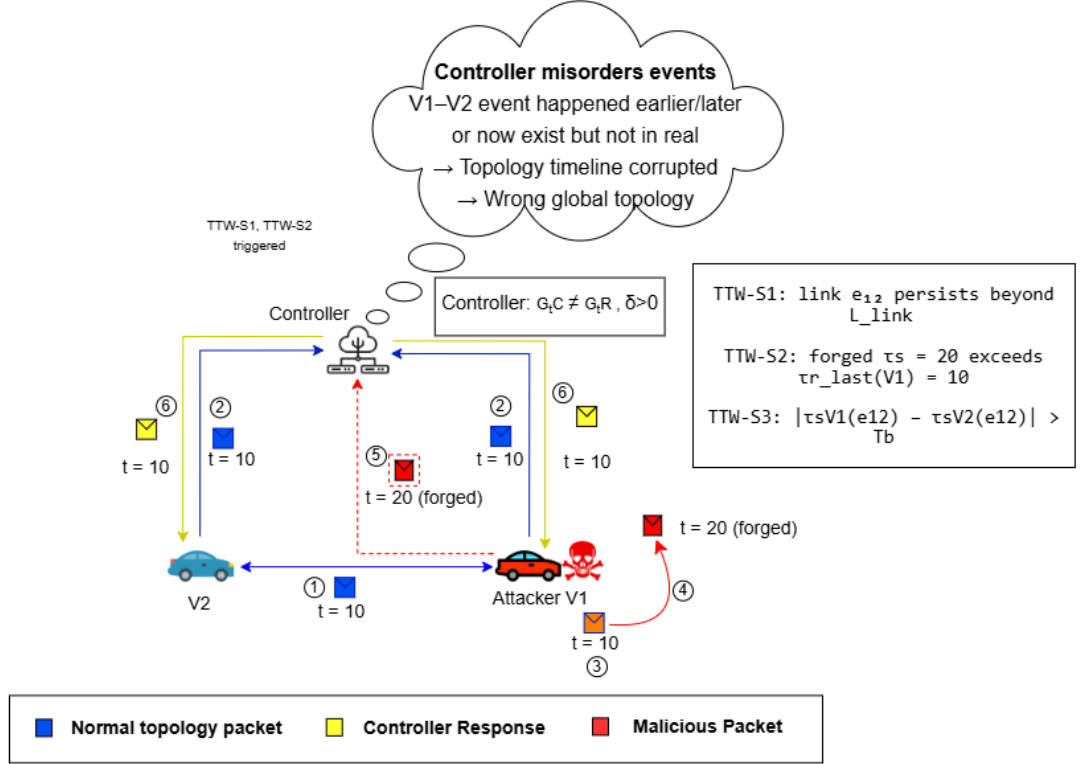


Figure 3.3: TTW Attack via Malicious Vehicle (Without RSU): V1 manipulates timestamps of HELLO messages sent to the controller.

Topology Time Warp Attack (Malicious Vehicle) The Topology Time-Warp (TTW) Attack via malicious vehicle (without RSU) occurs when a compromised vehicle delays and replays legitimate topology updates with forged timestamps, causing the controller to perceive stale links as currently active (Fig. 3.3). This temporal manipulation creates inconsistencies between the real network state and the controller's view without altering physical connectivity.

The attack proceeds as follows:

1. **V2V Neighbor Discovery:** At time $t = 10$, V2 broadcasts a HELLO

beacon that is received by V1, allowing V1 to detect V2 as a neighboring vehicle. V2 → V1: HELLO V1 → V2: HELLO

2. **Topology Update:** Based on the V2V HELLO exchange, both V1 and V2 independently generate legitimate topology updates. V1 sends $\langle V1 \text{ sees } V2, t = 10 \rangle$ and V2 sends $\langle V2 \text{ sees } V1, t = 10 \rangle$ to the controller, which records the bidirectional V1–V2 link as active. V1 → Controller: $\langle V1 \text{ sees } V2, t = 10 \rangle$ V2 → Controller: $\langle V2 \text{ sees } V1, t = 10 \rangle$
3. **Malicious Vehicle Stores Legitimate Update:** The malicious vehicle intercepts and stores a copy of the legitimate topology update for later replay. Old packet: $\langle V1 \text{ sees } V2, t = 10 \rangle$
4. **Link Failure and Attack Execution:** At time $t = 15$, vehicles V1 and V2 move out of communication range and the physical link breaks. At $t = 20$, the malicious vehicle replays the old legitimate topology update with a forged current timestamp, causing the controller to incorrectly believe the V1–V2 link is still active. Forged packet: $\langle V1 \text{ sees } V2, t = 20 \rangle$
5. **Faulty Routing Decisions:** The controller installs routing rules based on the forged "fresh" topology update, directing traffic through the non-existent V1–V2 link, resulting in packet loss and routing instability.

Topology Time-Warp Attack (Malicious Vehicle, No RSU): An attack where a compromised vehicle replays legitimate topology updates with forged timestamps after physical link failure, creating false perceptions of network connectivity and poisoning controller routing decisions.

Figure 3.4 illustrates the TTW attack executed via a malicious RSU, where the compromised RSU intercepts legitimate vehicle topology updates, modifies their timestamps, and replays them to the controller to maintain a false perception of active links.

TTW Attack via Malicious RSU

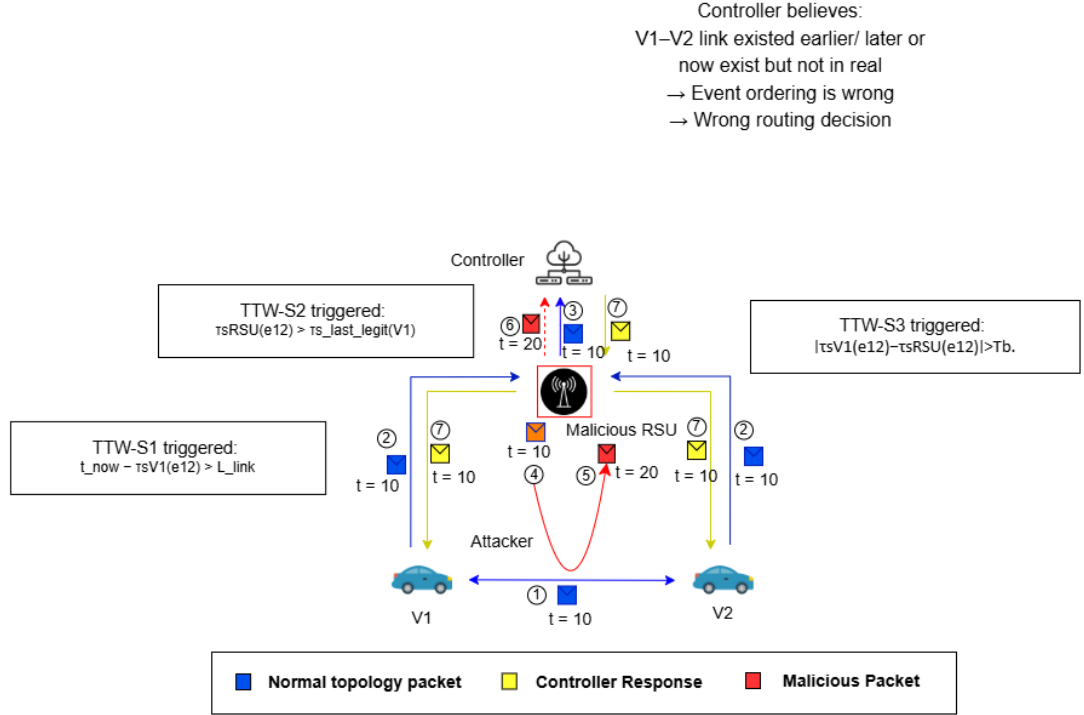


Figure 3.4: TTW Attack via Malicious RSU: RSU manipulates timestamps in forwarded vehicle HELLO messages.

Topology Time Warp Attack (Malicious RSU) The Topology Time-Warp (TTW) Attack via malicious RSU occurs when a compromised RSU intercepts legitimate topology updates, stores them, and later replays them with forged timestamps after physical link failure, causing the controller to perceive stale links as currently active (Fig. 3.4). This temporal manipulation creates inconsistencies between the real network state and the controller's view without altering physical connectivity.

The attack proceeds as follows:

1. **V2V Neighbor Discovery:** At time $t = 10$, V2 broadcasts a HELLO beacon that is received by V1, allowing V1 to detect V2 as a neighboring vehicle. V2 → V1: HELLO V1 → V2: HELLO
2. **Topology Update to RSU:** Based on the V2V HELLO exchange, both V1 and V2 independently generate legitimate topology updates and send them to the RSU. V1 → RSU: $\langle V1 \text{ sees } V2, t = 10 \rangle$ V2 → RSU: $\langle V2 \text{ sees } V1, t = 10 \rangle$
3. **RSU Aggregation and Forwarding:** The RSU aggregates the received topology updates $\langle V1 \text{ sees } V2, t = 10 \rangle$ and $\langle V2 \text{ sees } V1, t = 10 \rangle$ and

forwards them to the controller, establishing the bidirectional V1–V2 link as active.

4. **Malicious RSU Stores Legitimate Update:** The malicious RSU stores a copy of the legitimate topology update for later replay. Old packet: $\langle V1 \text{ sees } V2, t = 10 \rangle$
5. **Link Failure and Attack Preparation:** At time $t = 15$, vehicles V1 and V2 move out of communication range and the physical link breaks. At $t = 20$, the malicious RSU forges the stored legitimate topology update by modifying its timestamp from $\langle V1 \text{ sees } V2, t = 10 \rangle$ to $\langle V1 \text{ sees } V2, t = 20 \rangle$.
6. **Replay of Forged Update:** The malicious RSU replays the timestamp-forged topology update to the controller, causing stale connectivity information to appear fresh. RSU $\rightarrow$ Controller: $\langle V1 \text{ sees } V2, t = 20 \rangle$
7. **Faulty Routing Decisions:** The controller installs routing rules based on the forged "fresh" topology update, directing traffic through the non-existent V1–V2 link, resulting in packet loss and routing instability.

Topology Time-Warp Attack (Malicious RSU): An attack where a compromised RSU replays stored legitimate topology updates with forged timestamps after physical link failure, creating false perceptions of active connectivity and poisoning controller routing decisions.

TTW Attack via Malicious Controller

Figure 3.5 illustrates the TTW attack executed by a malicious controller operating without RSU involvement, where the controller internally replays stored topology updates with forged timestamps after the physical link has broken.

Topology Time Warp Attack (Malicious Controller – Timestamp Injection, No RSU) The Topology Time-Warp (TTW) Attack via malicious controller occurs when a compromised controller stores legitimate topology updates and later replays them with forged timestamps after physical link failure, causing itself to perceive stale links as currently active (Fig. 3.5). This internal temporal manipulation creates inconsistencies between the real network state and the controller's topology database without altering physical connectivity.

The attack proceeds as follows:

1. **V2V Neighbor Discovery:** At time $t = 10$, V2 broadcasts a HELLO beacon that is received by V1, allowing V1 to detect V2 as a neighboring vehicle. V2 $\rightarrow$ V1: HELLO V1 $\rightarrow$ V2: HELLO

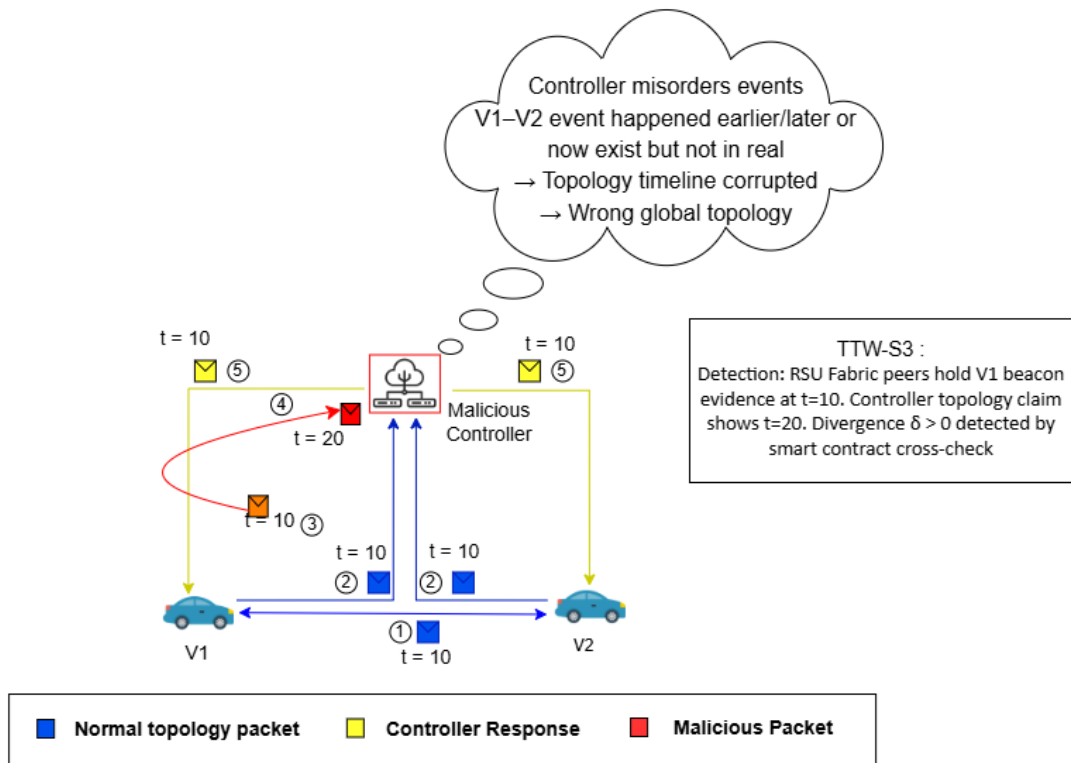


Figure 3.5: TTW Attack via Malicious Controller (Without RSU): The controller internally manipulates timestamps of received HELLO messages.

2. **Topology Update:** Based on the V2V HELLO exchange, both V1 and V2 independently generate legitimate topology updates and send them directly to the controller, which records the bidirectional V1–V2 link as active. V1 $\rightarrow$ Controller: $\langle V1 \text{ sees } V2, t = 10 \rangle$ V2 $\rightarrow$ Controller: $\langle V2 \text{ sees } V1, t = 10 \rangle$
3. **Malicious Controller Stores Legitimate Update:** The malicious controller stores a copy of the received legitimate topology update for later replay. Old packet: $\langle V1 \text{ sees } V2, t = 10 \rangle$
4. **Link Failure and Attack Preparation:** At time $t = 15$, vehicles V1 and V2 move out of communication range and the physical link breaks. At $t = 20$, the malicious controller forges the stored legitimate topology update by modifying its timestamp to appear current. Forged packet: $\langle V1 \text{ sees } V2, t = 20 \rangle$
5. **Replay into Topology Database:** The controller reinserts the forged topology update into its internal topology database as a "fresh" observation, causing itself to incorrectly believe the V1–V2 link remains active.
6. **Faulty Routing Decisions:** The controller installs routing rules based on the forged "fresh" topology update, directing traffic through the non-existent V1–V2 link, resulting in packet loss and routing instability.

Topology Time-Warp Attack (Malicious Controller): An attack where a compromised controller internally replays stored legitimate topology updates with forged timestamps after physical link failure, creating false perceptions of active connectivity and poisoning its own routing decisions.

TTW Attack via Malicious Controller

Figure 3.6 illustrates the TTW attack executed by a malicious controller in an RSU-assisted deployment, where the controller stores RSU-aggregated topology updates and later reinjects them with forged timestamps into its own topology database.

Topology Time Warp Attack (Malicious Controller – Reorder/Replay), with RSU The Topology Time-Warp (TTW) Attack via malicious controller (with RSU) occurs when a compromised controller stores legitimate topology updates received through RSU aggregation and later replays them with forged timestamps after physical link failure, causing itself to perceive stale links as currently active (Fig. 3.6). This internal temporal manipulation creates inconsistencies between the real network state and the controller's topology database without altering physical connectivity.

The attack proceeds as follows:

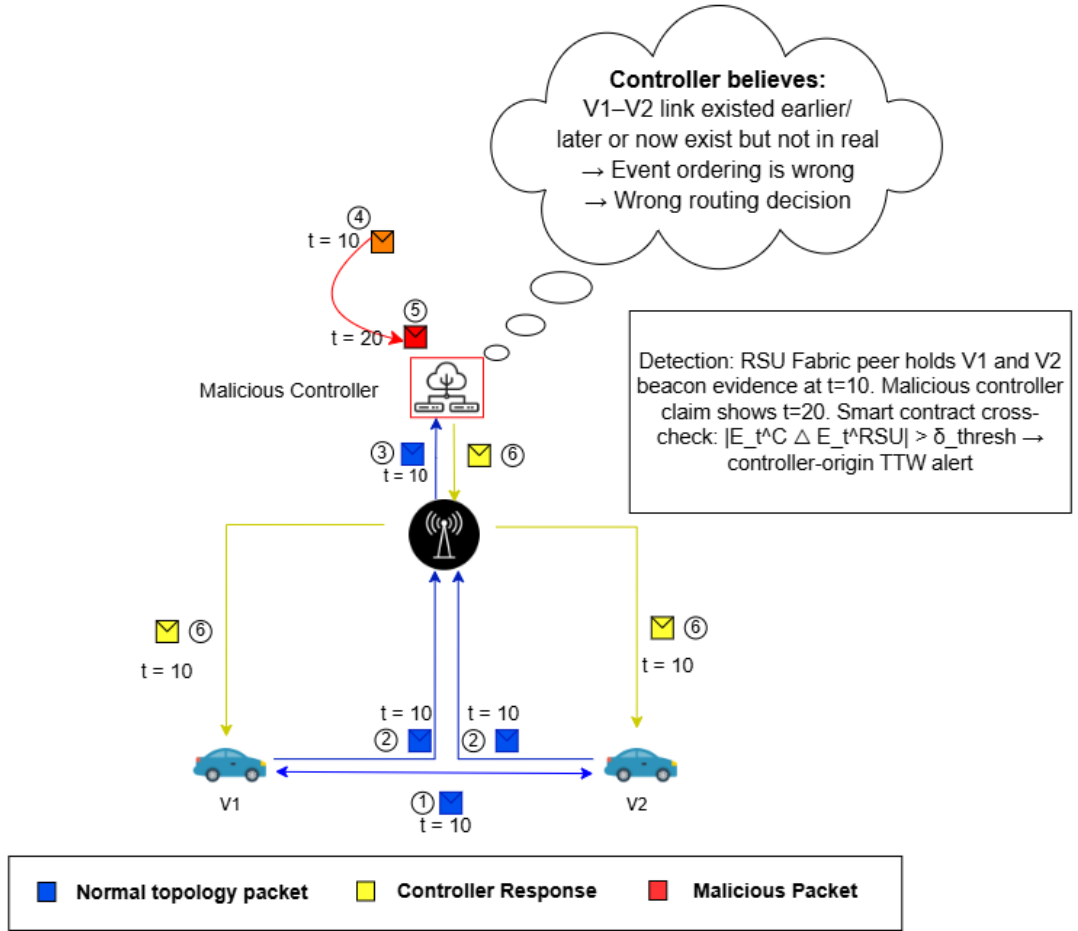


Figure 3.6: TTW Attack via Malicious Controller (With RSU): The controller internally manipulates timestamps of received HELLO messages.

1. **V2V Neighbor Discovery:** At time $t = 10$, V2 broadcasts a HELLO beacon that is received by V1, allowing V1 to detect V2 as a neighboring vehicle. V2 $\rightarrow$ V1: HELLO V1 $\rightarrow$ V2: HELLO
2. **Topology Update to RSU:** Based on the V2V HELLO exchange, both V1 and V2 independently generate legitimate topology updates and send them to the RSU. V1 $\rightarrow$ RSU: $\langle V1 \text{ sees } V2, t = 10 \rangle$ V2 $\rightarrow$ RSU: $\langle V2 \text{ sees } V1, t = 10 \rangle$
3. **RSU Aggregation and Forwarding:** The RSU aggregates the received topology updates $\langle V1 \text{ sees } V2, t = 10 \rangle$ and $\langle V2 \text{ sees } V1, t = 10 \rangle$ and forwards them to the controller, establishing the bidirectional V1–V2 link as active.
4. **Malicious Controller Stores Legitimate Update:** The malicious controller stores a copy of the received legitimate topology update (via RSU) for later replay. Old packet: $\langle V1 \text{ sees } V2, t = 10 \rangle$
5. **Link Failure and Attack Preparation:** At time $t = 15$, vehicles V1 and V2 move out of communication range and the physical link breaks. At $t = 20$, the malicious controller forges the stored legitimate topology update by modifying its timestamp to appear current. Forged packet: $\langle V1 \text{ sees } V2, t = 20 \rangle$
6. **Replay into Topology Database:** The controller reinserts the forged topology update into its internal topology database as a "fresh" observation, causing itself to incorrectly believe the V1–V2 link remains active.
7. **Faulty Routing Decisions:** The controller installs routing rules based on the forged "fresh" topology update, directing traffic through the non-existent V1–V2 link, resulting in packet loss and routing instability.

Topology Time-Warp Attack (Malicious Controller with RSU): An attack where a compromised controller replays stored legitimate topology updates (received via RSU aggregation) with forged timestamps after physical link failure, creating false perceptions of active connectivity and poisoning its own routing decisions.

3.3.2 Beacon State Heartbeat Hijack Attack (BSHH) Attack

The Beacon State Heartbeat Hijack (BSHH) attack undermines SDVN topology inference by tampering with heartbeat identity fields, while leaving payloads untouched and avoiding new message injections. It capitalizes on the inherent trust in heartbeat signals, which vehicles rely on to discover neighbors and sustain consistent global topology views.

BSHH Attack via Malicious Vehicle (Without RSU)

Figure 3.7 illustrates the BSHH attack executed by a malicious vehicle without RSU involvement, where the compromised vehicle replays stored heartbeat messages from a legitimate peer to both the peer and the controller, creating false liveness perceptions.

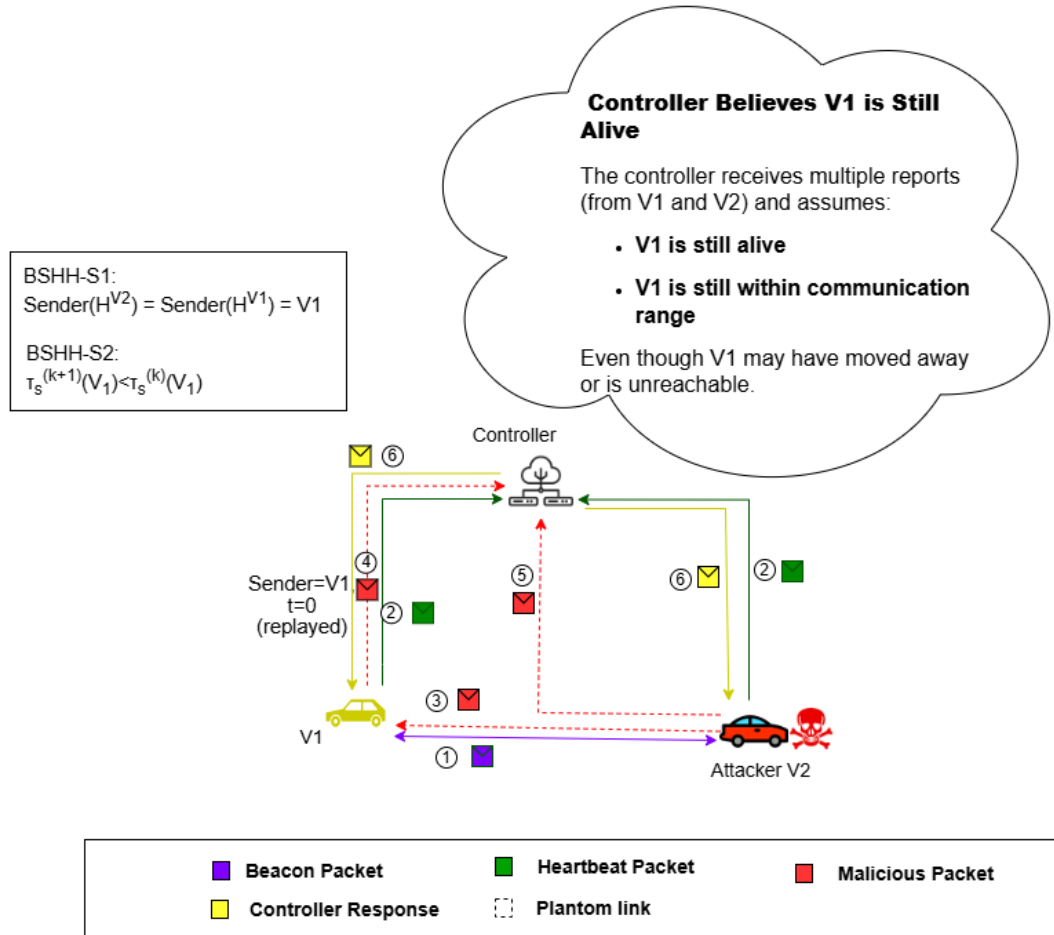


Figure 3.7: BSHH Attack via Malicious Vehicle (Without RSU): V2 hijacks heartbeat messages from V3 to V1, inducing false adjacency.

Beacon State Heartbeat Hijack Attack (Malicious Vehicle) The Beacon State Heartbeat Hijack (BSHH) Attack via malicious vehicle (without RSU) occurs when a compromised vehicle (V2) intercepts legitimate heartbeat exchanges, stores old heartbeats, and replays them to both peer vehicles and the controller—creating false liveness perceptions and hijacking the network state (Fig. 3.7). This zero-payload temporal attack maintains stale topology perceptions without fabricating new links.

The attack proceeds as follows:

1. **Normal Beacon Exchange ($t = 5$):** Vehicles V1 and V2 exchange beacons normally. V1 $\rightarrow$ V2: "I am alive" ($t = 5$) V2 $\rightarrow$ V1: "I am alive" ($t = 5$)
2. **Both Forward Legitimate Heartbeats to Controller ($t = 5$):** Both vehicles send honest heartbeat reports to the controller. V1 $\rightarrow$ Controller: Heartbeat (Sender = V1, Timestamp = 5) V2 $\rightarrow$ Controller: Heartbeat (Sender = V2, Timestamp = 5)
3. **V2 Replays Old Beacon to V1 ($t = 10$):** V2 (malicious) stores an old heartbeat from V1 (from $t = 5$) and replays old beacon of V1 (from $t = 5$) to V2 at $t = 10$. V2 $\rightarrow$ V1: Heartbeat (Sender = V1, Timestamp = 5) (old legitimate packet replayed)
4. **V1 Forwards Old Heartbeat to Controller:** V1 believes the received heartbeat is valid and forwards it to the controller. V1 $\rightarrow$ Controller: Heartbeat (Sender = V1, Timestamp = 5)
5. **V2 Also Sends Old Heartbeat to Controller (Hijack):** V2 sends the stored old heartbeat of the V1 directly to the controller, impersonating V1. V2 $\rightarrow$ Controller: Heartbeat (Sender = V1, Timestamp = 5) (fake packet — impersonating V1)
6. **Faulty Routing Decisions:** Using the conflicting heartbeat timestamps and incorrect liveness information, the controller installs wrong routes: Packets routed through non-existent/stale links; Packet loss and delays occur; Network performance degrades significantly.

Beacon State Heartbeat Hijack (Malicious Vehicle, No RSU): An attack where a compromised vehicle replays stored legitimate heartbeats to both peers and controller—impersonating liveness through zero-payload temporal hijacking, poisoning network state without fabricating structural elements.

BSHH Attack via Malicious RSU

Figure 3.8 illustrates the BSHH attack executed via a malicious RSU, where the compromised RSU intercepts legitimate vehicle heartbeat exchanges and replays stale heartbeats to the controller through aggregated topology reports.

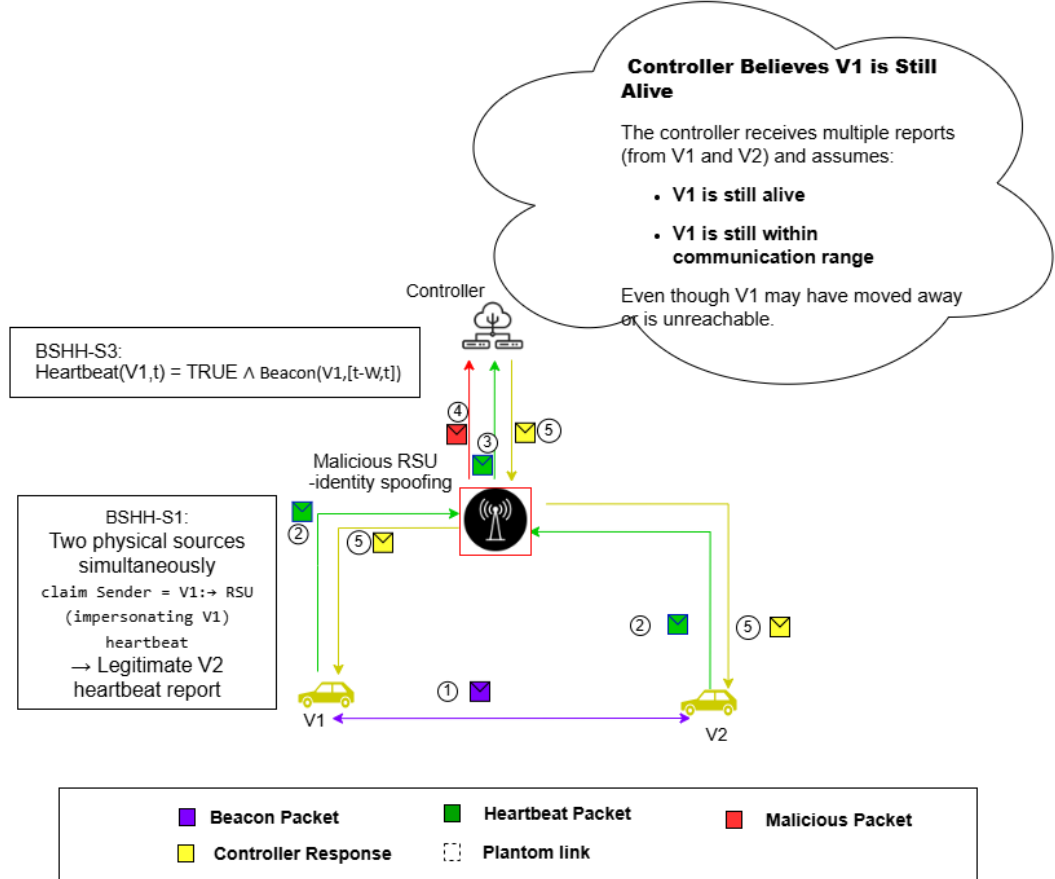


Figure 3.8: BSHH Attack via Malicious RSU: The RSU manipulates the sender identity of heartbeat messages to induce false adjacency.

Beacon State Heartbeat Hijack Attack (Malicious RSU) The Beacon State Heartbeat Hijack (BSHH) Attack via malicious RSU occurs when a compromised RSU intercepts legitimate heartbeat exchanges from vehicles, stores old heartbeats, and replays them to the controller—creating false liveness perceptions and hijacking the network state through aggregated topology poisoning (Fig. 3.8). This zero-payload temporal attack maintains stale topology perceptions without fabricating new links.

The attack proceeds as follows:

1. **Normal Beacons Exchange ($t = 5$):** Vehicles V1 and V2 exchange beacons normally. V1 → V2: "I am alive" ($t = 5$) V2 → V1: "I am alive" ($t =$

5)

2. **Vehicles Forward Legitimate Heartbeats to RSU ($t = 5$):** Both vehicles send honest heartbeat reports to the RSU. $V1 \rightarrow \text{RSU}$: Heartbeat (Sender = V1, Timestamp = 5) $V2 \rightarrow \text{RSU}$: Heartbeat (Sender = V2, Timestamp = 5)
3. **RSU Forwards Aggregated Heartbeat to Controller ($t = 5$):** The RSU aggregates and forwards legitimate heartbeat summary to the controller. $\text{RSU} \rightarrow \text{Controller}$: [Heartbeat (Sender = V1, Timestamp = 5), Heartbeat (Sender = V2, Timestamp = 5)]
4. **RSU Replays Old Heartbeat ($t = 10$):** The malicious RSU stores the old heartbeat from V1 (from $t = 5$) and replays it to the controller at $t = 10$, while suppressing the genuine current heartbeat updates from V1, impersonating V1's identity. $\text{RSU} \rightarrow \text{Controller}$: Heartbeat (Sender = V1, Timestamp = 5) (old legitimate packet replayed — impersonating V1 via RSU)

RSU Injects Additional Conflicting Old Heartbeat (Hijack) ($t = 10$): The malicious RSU injects a second stale heartbeat with an even older timestamp, creating false liveness signals and timestamp inconsistencies at the controller. $\text{RSU} \rightarrow \text{Controller}$: Heartbeat (Sender = V1, Timestamp = 5) (duplicate attack packet — identity hijacking via RSU aggregation)

5. **Faulty Routing Decisions:** Using the conflicting heartbeat timestamps and incorrect liveness information from RSU aggregation, the controller installs wrong routes: Packets routed through non-existent/stale links; Packet loss and delays occur; Network performance degrades significantly.

Beacon State Heartbeat Hijack (Malicious RSU): An attack where a compromised RSU replays stored legitimate heartbeats through aggregated topology reports—impersonating vehicle liveness via zero-payload temporal hijacking, poisoning controller network state perception.

BSHH Attack via Malicious Controller, without RSU

Figure 3.9 illustrates the BSHH attack executed by a malicious controller without RSU involvement, where the controller internally reprocesses stored stale heartbeats as current topology updates, maintaining false vehicle liveness in its own state database

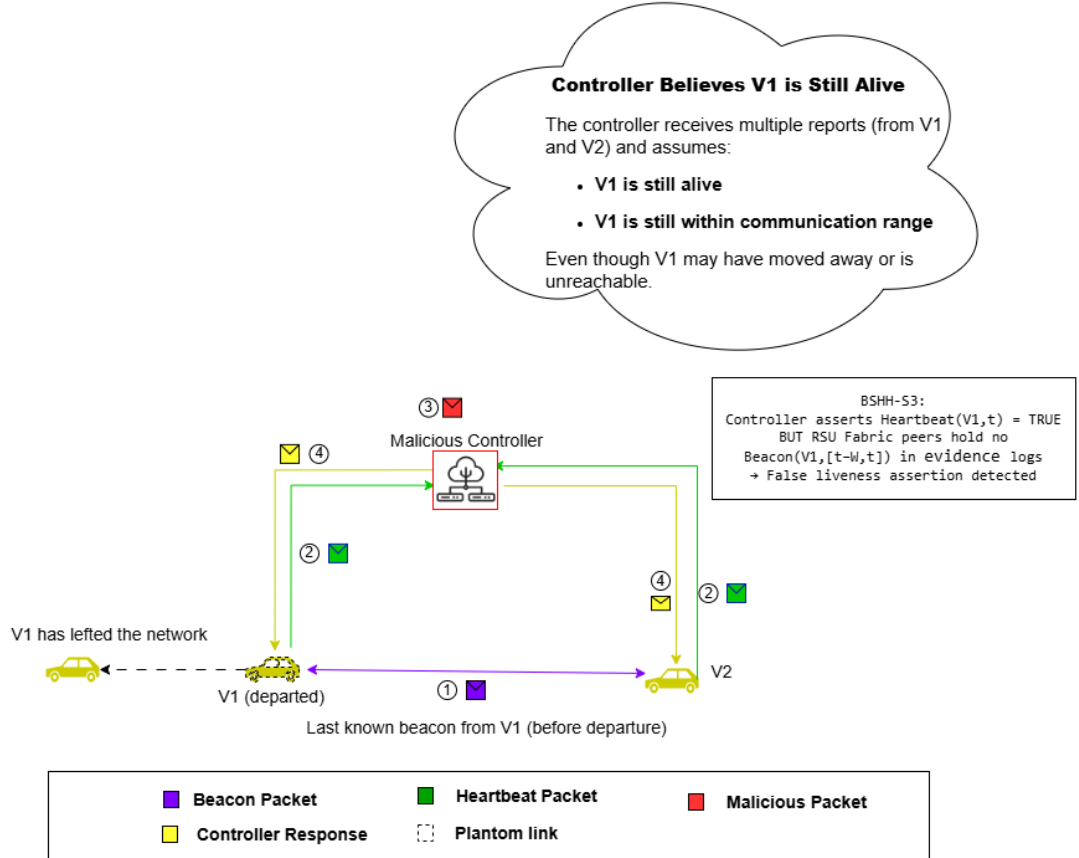


Figure 3.9: BSHH Attack via Malicious Controller (Without RSU): The controller internally hijacks heartbeat identities to induce false adjacency.

The Beacon State Heartbeat Hijack (BSHH) Attack via malicious controller (without RSU) occurs when a compromised controller stores legitimate heartbeat reports from vehicles and internally treats old heartbeats as current—creating false liveness perceptions and hijacking its own network state view (Fig. 3.9). This internal zero-payload temporal attack maintains stale topology perceptions without external communication or fabricating new links.

The attack proceeds as follows:

1. **Normal Beacon Exchange ($t = 5$):** Vehicles V1 and V2 exchange beacons normally. V1 → V2: "I am alive" ($t = 5$) V2 → V1: "I am alive" ($t = 5$)

2. **Both Forward Legitimate Heartbeats to Controller ($t = 5$):** Both vehicles send honest heartbeat reports directly to the controller. $V1 \rightarrow \text{Controller: Heartbeat (Sender = V1, Timestamp = 5)}$ $V2 \rightarrow \text{Controller: Heartbeat (Sender = V2, Timestamp = 5)}$
3. **Controller Silently Stores Old Heartbeats ($t = 10$):** The malicious controller logs and retains old heartbeats from a previous interval alongside the current legitimate ones. Stored internally: Heartbeat (Sender = V1, Timestamp = 5) Stored internally: Heartbeat (Sender = V2, Timestamp = 5)

Controller Internally Reinstates Old Heartbeats as Current ($t = 10$): At $t = 10$, the malicious controller ignores the actual liveness state and internally reactivates the stale stored entries, overwriting the topology view with outdated data. Internal override: Controller topology DB $\leftarrow$ Heartbeat (Sender = V1, Timestamp = 5) [treated as active] Internal override: Controller topology DB $\leftarrow$ Heartbeat (Sender = V2, Timestamp = 5) [treated as active]

Internal State Hijacking — False Liveness Maintained: The controller now falsely believes V1 and V2 remain alive and connected based on stale timestamps, even if the vehicles have physically moved or disconnected. No external communication is made; the entire attack is internal. Controller state: V1 — ALIVE (stale, $t = 5$ treated as current) Controller state: V2 — ALIVE (stale, $t = 5$ treated as current)

4. **Faulty Routing Decisions:** Using the manipulated heartbeat-based topology, the controller installs wrong routes: Packets routed through non-existent/stale links; Packet loss and delays occur; Network performance degrades significantly.

Beacon State Heartbeat Hijack (Malicious Controller, No RSU): An attack where a compromised controller internally reprocesses stored old legitimate heartbeats as current topology updates—creating false liveness through zero-payload temporal state hijacking without external communication.

BSHH Attack via Malicious Controller (With RSU)

Figure 3.10 illustrates the BSHH attack executed by a malicious controller in an RSU-assisted deployment, where the controller treats previously received RSU-aggregated heartbeats as fresh, sustaining a false liveness state without external communication.

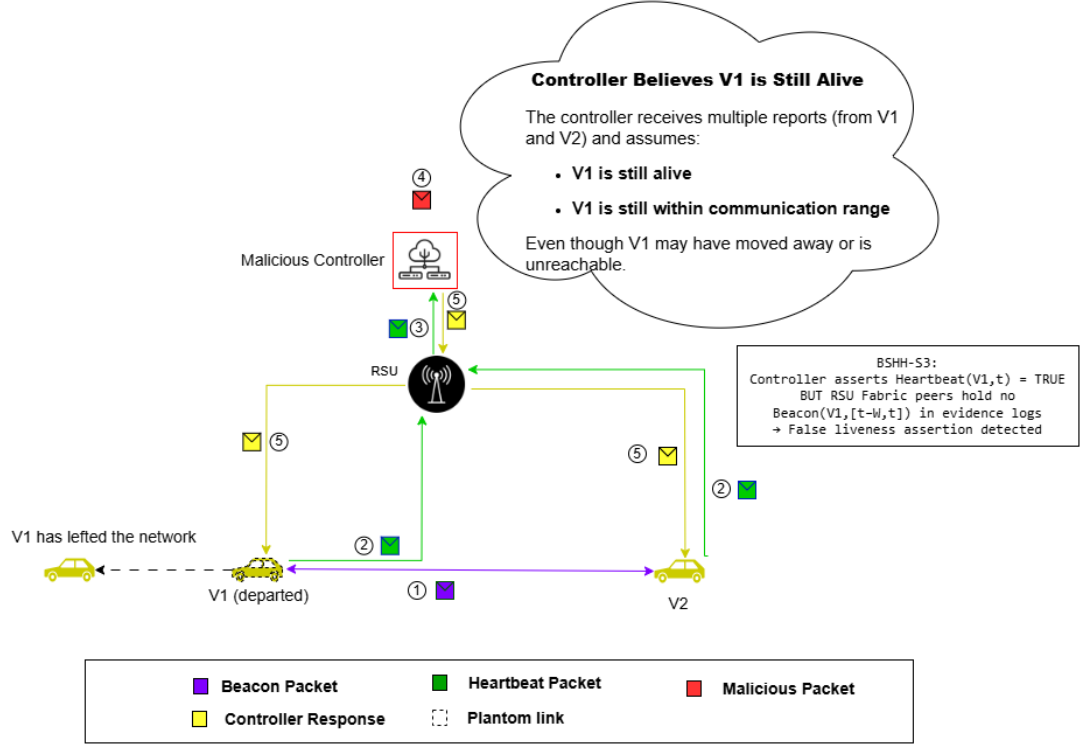


Figure 3.10: BSHH Attack via Malicious Controller (With RSU): The controller internally hijacks heartbeat identities to induce false adjacency.

Beacon State Heartbeat Hijack Attack (Malicious Controller – Direct Injection) The Beacon State Heartbeat Hijack (BSHH) Attack via malicious controller (direct injection with RSU) occurs when a compromised controller receives legitimate heartbeat reports through RSU aggregation, stores old heartbeats, and internally treats them as current—creating false liveness perceptions and hijacking its network state view (Fig. 3.10). This internal zero-payload temporal attack maintains stale topology perceptions without external communication or fabricating new links.

The attack proceeds as follows:

1. **Normal Beacons Exchange ($t = 5$):** Vehicles V1 and V2 exchange beacons normally. V1 → V2: "I am alive" ($t = 5$) V2 → V1: "I am alive" ($t = 5$)

2. **Both Forward Legitimate Heartbeats to RSU ($t = 5$):** Both vehicles send honest heartbeat reports to the RSU. $V1 \rightarrow \text{RSU}$: Heartbeat (Sender = V1, Timestamp = 5) $V2 \rightarrow \text{RSU}$: Heartbeat (Sender = V2, Timestamp = 5)
3. **RSU Aggregates and Forwards Legitimate Heartbeats to Controller ($t = 5$):** The honest RSU aggregates the vehicle heartbeats and forwards the legitimate summary to the controller. $\text{RSU} \rightarrow \text{Controller}$: [Heartbeat (Sender = V1, Timestamp = 5), Heartbeat (Sender = V2, Timestamp = 5)]
4. **Controller Silently Stores Previously Received Stale Heartbeats ($t = 10$):** The malicious controller retains old heartbeats received from prior RSU aggregation cycles alongside the current legitimate updates. Stored internally: Heartbeat (Sender = V1, Timestamp = 5) (from prior RSU forwarding) Stored internally: Heartbeat (Sender = V2, Timestamp = 5) (from prior RSU forwarding)

Controller Internally Reinstates Old RSU-Forwarded Heartbeats as Current ($t = 10$): At $t = 10$, the malicious controller discards the newly received legitimate RSU-forwarded update and internally reactivates the stale RSU-aggregated heartbeats, overriding the topology database with outdated liveness data. Internal override: Controller topology DB $\leftarrow$ Heartbeat (Sender = V1, Timestamp = 5) [treated as active, sourced from old RSU report] Internal override: Controller topology DB $\leftarrow$ Heartbeat (Sender = V2, Timestamp = 5) [treated as active, sourced from old RSU report]

5. **Faulty Routing Decisions:** Using the manipulated heartbeat-based topology derived from stale RSU-aggregated data, the controller installs wrong routes: Packets routed through non-existent/stale links; Packet loss and delays occur; Network performance degrades significantly.

Beacon State Heartbeat Hijack (Malicious Controller with RSU): An attack where a compromised controller internally reprocesses stored old legitimate heartbeats (received via RSU aggregation) as current topology updates—creating false liveness through zero-payload temporal state hijacking.

3.3.3 Multipath Echo (ME) Attack

The Multipath Echo (ME) attack exploits duplicated topology observations to mislead the SDN controller into inferring multiple logical paths that do not exist in the physical network. By replaying or reinterpreting legitimate topology messages, an attacker can induce false path diversity and disrupt routing and forwarding decisions.

ME Attack via Malicious Vehicles

Figure 3.11 illustrates the ME attack executed by malicious vehicles without RSU involvement, where compromised vehicles V3 and V4 intercept and echo legitimate V1–V2 HELLO messages to the controller, inducing false multi-hop path inference.

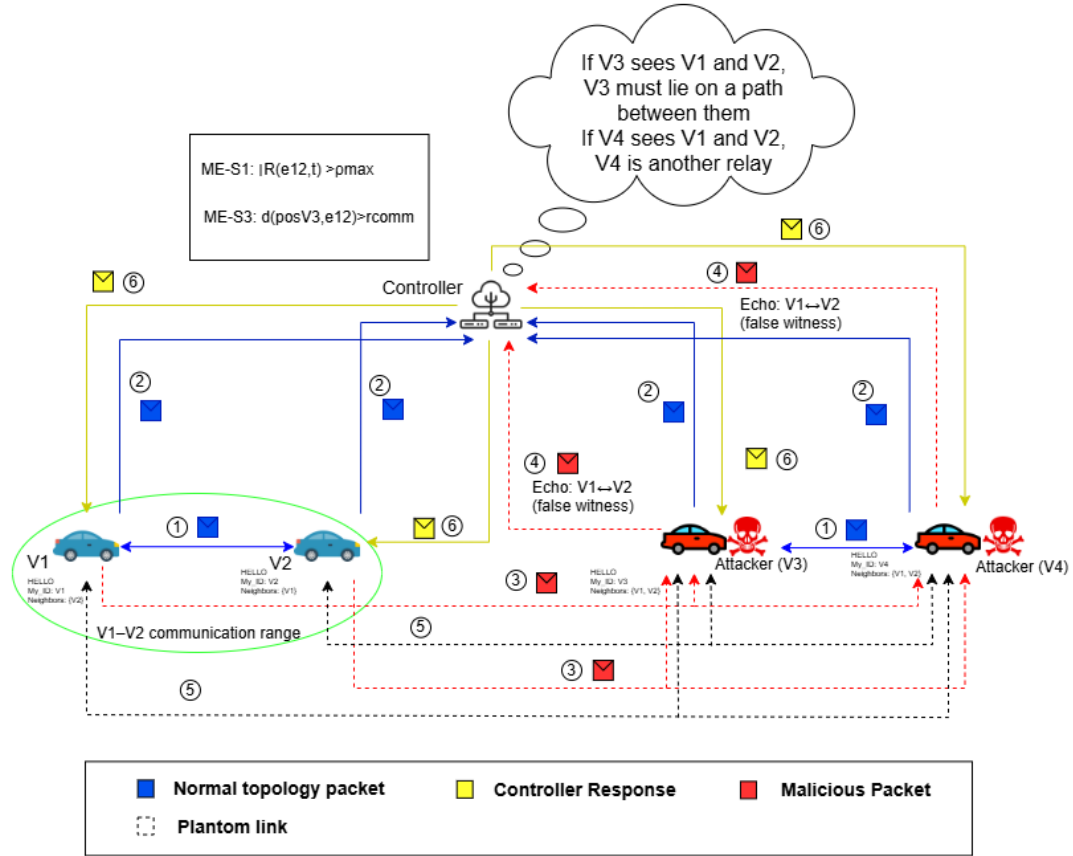


Figure 3.11: Vehicle-based Multipath Echo Attack (Without RSU): V3 and V4 replay V1–V2 topology to the controller, creating false paths.

ME Attack Steps (Vehicle Overhearing Variant) The Multipath Echo (ME) attack via compromised vehicles (without RSU) occurs when malicious vehicles V3 and V4 intercept and echo legitimate V1–V2 HELLO messages, causing the controller to perceive false multi-hop paths. Although physical links remain

unchanged, the echoed reports create non-existent logical paths. This deception leads to incorrect routing decisions without fabricating physical links (Fig. 3.11).

The attack proceeds as follows:

1. **Normal V2V Topology Discovery:** Vehicles periodically exchange topology HELLO messages over V2V wireless links to announce neighbor connectivity.
2. **Legitimate Link Establishment:** A legitimate wireless link exists between vehicles V1 and V2, while vehicles V3 and V4 are within overhearing range but are not part of the V1–V2 link. No direct links exist between V1–V3, V1–V4, V2–V3, or V2–V4.
3. **HELLO Message Interception (Echo Preparation):** HELLO messages exchanged between V1 and V2 are overheard and copied by vehicles V3 and V4, which act as echoing adversaries.
4. **Echoed Topology Observation Reports:** Vehicles V3 and V4 replay the observed HELLO information toward the controller, reporting the same legitimate link ($V1 \leftrightarrow V2$) as if it were observed through different intermediate nodes: Observed link: $V1 \leftrightarrow V2$, reported by V3; Observed link: $V1 \leftrightarrow V2$, reported by V4. These reports do not fabricate new links but duplicate an existing link across multiple observation paths.
5. **Multipath Inference at the Controller (Attack Effect):** The controller aggregates the echoed observations and incorrectly infers multiple distinct paths involving intermediate nodes V3 and V4. As a result, the controller assumes the existence of non-existent paths: Path 1: $V1 \rightarrow V2$ (real); Path 2: $V1 \rightarrow V3 \rightarrow V2$ (false); Path 3: $V1 \rightarrow V4 \rightarrow V2$ (false); Path 4: $V1 \rightarrow V3 \rightarrow V4 \rightarrow V2$ (false).
6. **Faulty Routing Decisions:** Based on the falsely inferred multipath topology, the controller installs incorrect routing rules that rely on non-existent paths, leading to packet loss, increased delay, and routing instability.

Multipath Echo Attack (Compromised Vehicles): An attack where malicious vehicles intercept and echo legitimate topology messages to the controller, inducing multiple non-existent logical paths and poisoned path computation without fabricating physical links.

ME Attack via Compromised RSU

Figure 3.12 illustrates the ME attack executed via a malicious RSU, where the compromised RSU injects echoed topology observations into its aggregated reports, causing the controller to infer multiple non-existent paths through vehicles V3 and V4.

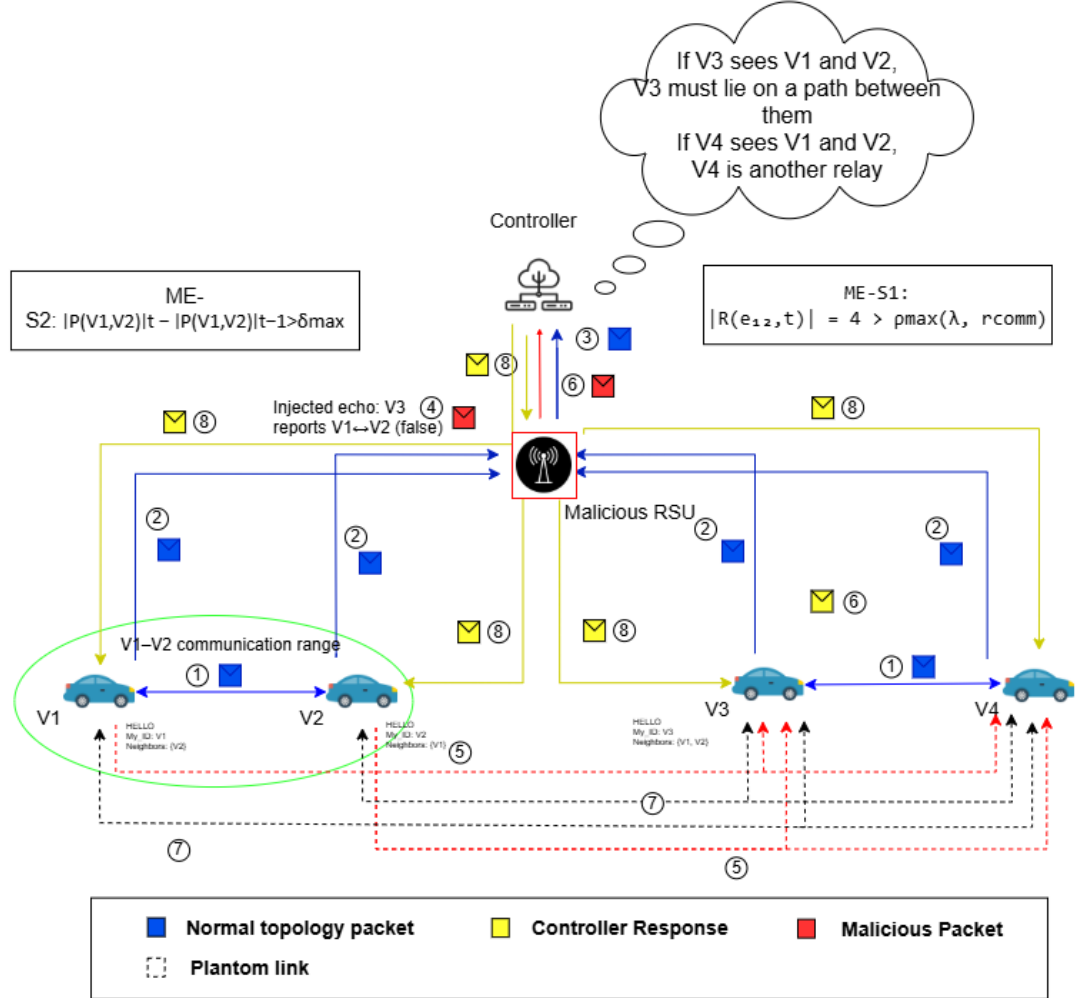


Figure 3.12: The RSU injects duplicated topology information related to V1–V2, creating false paths. (With RSU)

ME Attack Steps (RSU Aggregation Variant) The Multipath Echo (ME) attack via compromised RSU occurs when a malicious RSU injects echoed observations into aggregated topology reports from vehicles, causing the controller to perceive false multi-hop paths (Fig. 3.12). Although the physical links remain unchanged, the manipulated aggregation creates logical paths that do not exist in reality. This deception leads to incorrect routing decisions without fabricating physical links.

The attack proceeds as follows:

1. **Normal V2V Topology Discovery:** Vehicles exchange topology HELLO messages over V2V wireless links to advertise neighbor connectivity.
2. **Legitimate Link Establishment:** Legitimate wireless links exist between vehicles V1–V2 and V3–V4. Vehicles V3 and V4 are within overhearing range of the V1–V2 communication but do not have direct links with either V1 or V2.
3. **Normal Topology Reporting to the RSU:** Vehicles forward their legitimate topology observations to the RSU for aggregation.
4. **Echo Injection by the Malicious RSU:** The RSU, acting maliciously, injects forged HELLO observations that falsely indicate the V1–V2 HELLO messages were overheard by vehicles V3 and V4. These injected observations, illustrated as red dotted links in the figure, do not fabricate new links but instead duplicate an existing legitimate link across multiple reporting nodes.
5. **Observed HELLO Reports (Forged):** The forged topology observations indicate: Observed link: V1 $\leftrightarrow$ V2, reported by V3; Observed link: V1 $\leftrightarrow$ V2, reported by V4.
6. **Aggregated Topology Forwarding to the Controller:** The malicious RSU forwards an aggregated topology report to the controller containing both legitimate and injected echo observations: V3 reports observing the V1 $\leftrightarrow$ V2 link; V4 reports observing the V1 $\leftrightarrow$ V2 link.
7. **False Multipath Inference at the Controller:** Upon aggregating the echoed observations, the controller incorrectly infers the existence of multiple distinct paths involving vehicles V3 and V4: Path 1: V1 $\rightarrow$ V2 (real); Path 2: V1 $\rightarrow$ V3 $\rightarrow$ V2 (false); Path 3: V1 $\rightarrow$ V4 $\rightarrow$ V2 (false); Path 4: V1 $\rightarrow$ V3 $\rightarrow$ V4 $\rightarrow$ V2 (false).
8. **Faulty Routing Decisions:** Based on the falsely inferred multipath topology, the controller installs incorrect routing rules that rely on non-existent paths, leading to packet loss and routing instability.

Multipath Echo Attack (Compromised RSU): An attack where a malicious RSU injects echoed topology observations into aggregated vehicle reports, inducing multiple non-existent logical paths at the controller and poisoned path computation without fabricating physical links.

ME Attack via Compromised Controller

Figure 3.13 illustrates the ME attack executed by a malicious controller without RSU involvement, where the controller directly injects echoed HELLO observations into its internal topology database, fabricating false multi-hop paths from V2V communications.

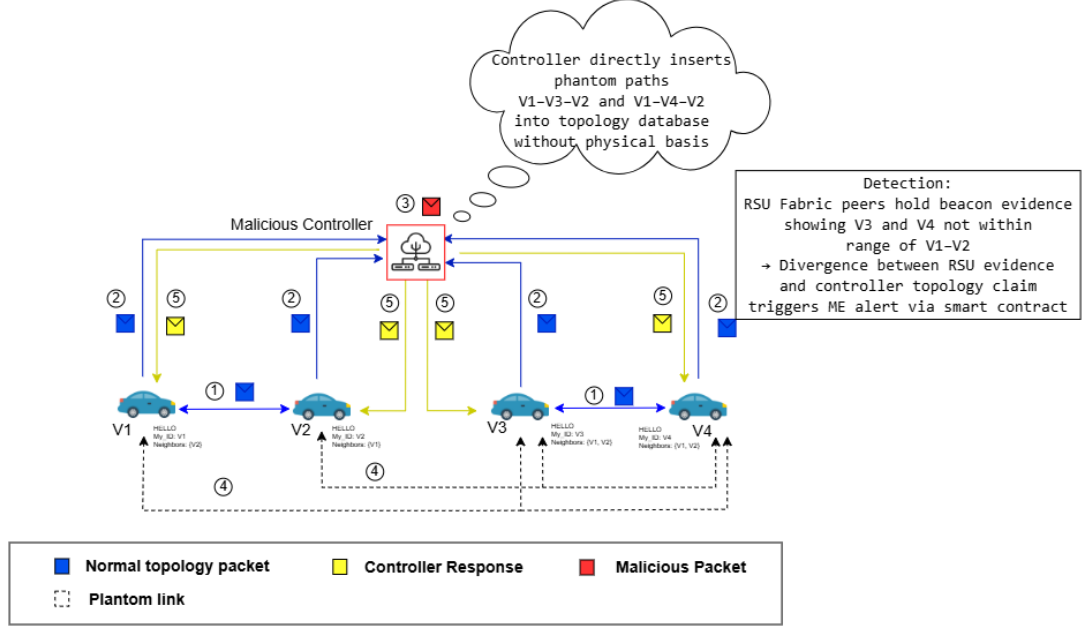


Figure 3.13: Controller-based Multipath Echo Attack (Without RSU): Controller internally duplicates V1–V2 link across multiple inferred paths.

ME Attack Steps (Vehicle-based: Direct Injection) The Multipath Echo (ME) attack via compromised controller (without RSU) occurs when a malicious controller directly injects echoed observations into its topology database from V2V communications, causing it to perceive false multi-hop paths (Fig. 3.13). Although the physical links remain unchanged, the manipulated internal topology creates logical paths that do not exist in reality. This deception leads to incorrect routing decisions without fabricating physical links.

The attack proceeds as follows:

1. **Normal V2V Topology Discovery:** Vehicles exchange topology HELLO messages over V2V wireless links to advertise neighbor connectivity.
2. **Legitimate Link Establishment:** Legitimate wireless links exist between vehicles V1–V2 and V3–V4. Vehicles V3 and V4 are within overhearing range of the V1–V2 communication but do not have direct links with either V1 or V2. Each vehicle independently reports its topology observations directly to the controller.

3. **Echo Injection by the Malicious Controller:** The controller, acting maliciously, injects forged HELLO observations into its internal topology database, making it appear as if the legitimate V1–V2 HELLO messages were overheard and relayed through vehicles V3 and V4. These injected observations duplicate an existing legitimate link across multiple reporting paths. Echoed topology observations: Observed link: $V1 \leftrightarrow V2$, relayed via V3; Observed link: $V1 \leftrightarrow V2$, relayed via V4.
4. **False Multipath Inference:** Based on the duplicated observations, the controller incorrectly infers the existence of multiple paths involving vehicles V3 and V4: Path 1: $V1 \rightarrow V2$ (real); Path 2: $V1 \rightarrow V3 \rightarrow V2$ (false); Path 3: $V1 \rightarrow V4 \rightarrow V2$ (false); Path 4: $V1 \rightarrow V3 \rightarrow V4 \rightarrow V2$ (false).
5. **Faulty Routing Decisions:** The controller installs routing policies based on the falsely inferred multipath topology, resulting in incorrect packet forwarding and routing instability.

Multipath Echo Attack (No RSU): An attack where a compromised controller injects echoed topology observations directly into its database from V2V reports, inducing multiple non-existent logical paths and poisoned path computation without fabricating physical links.

ME Attack Steps via Compromised Controller (with RSU)

Figure 3.14 illustrates the ME attack executed by a malicious controller in an RSU-assisted deployment, where the controller injects echoed observations into RSU-aggregated topology reports, inflating path diversity beyond physically plausible bounds.

The Multipath Echo (ME) attack occurs when a malicious controller, receiving topology information aggregated by RSUs, injects echoed observations into otherwise legitimate topology reports, causing itself to perceive false multi-hop paths (Fig. 3.14). Although the physical links remain unchanged, the manipulated reports create logical paths that do not exist in reality. This deception leads to incorrect routing decisions without fabricating physical links.

The attack proceeds as follows:

1. **Normal V2V Topology Discovery:** Vehicles exchange topology HELLO messages over V2V wireless links to advertise neighbor connectivity.
2. **Legitimate Link Establishment:** Legitimate wireless links exist between vehicles V1–V2 and V3–V4. Vehicles V3 and V4 are within overhearing range of the V1–V2 communication but do not have direct links with either V1 or V2. Each vehicle forwards both its own and observed topology information to the RSU.
3. **Topology Reporting via RSU:** Vehicles forward their normal topology observations to the RSU. The RSU aggregates the received topology reports and forwards the aggregated topology information to the controller.

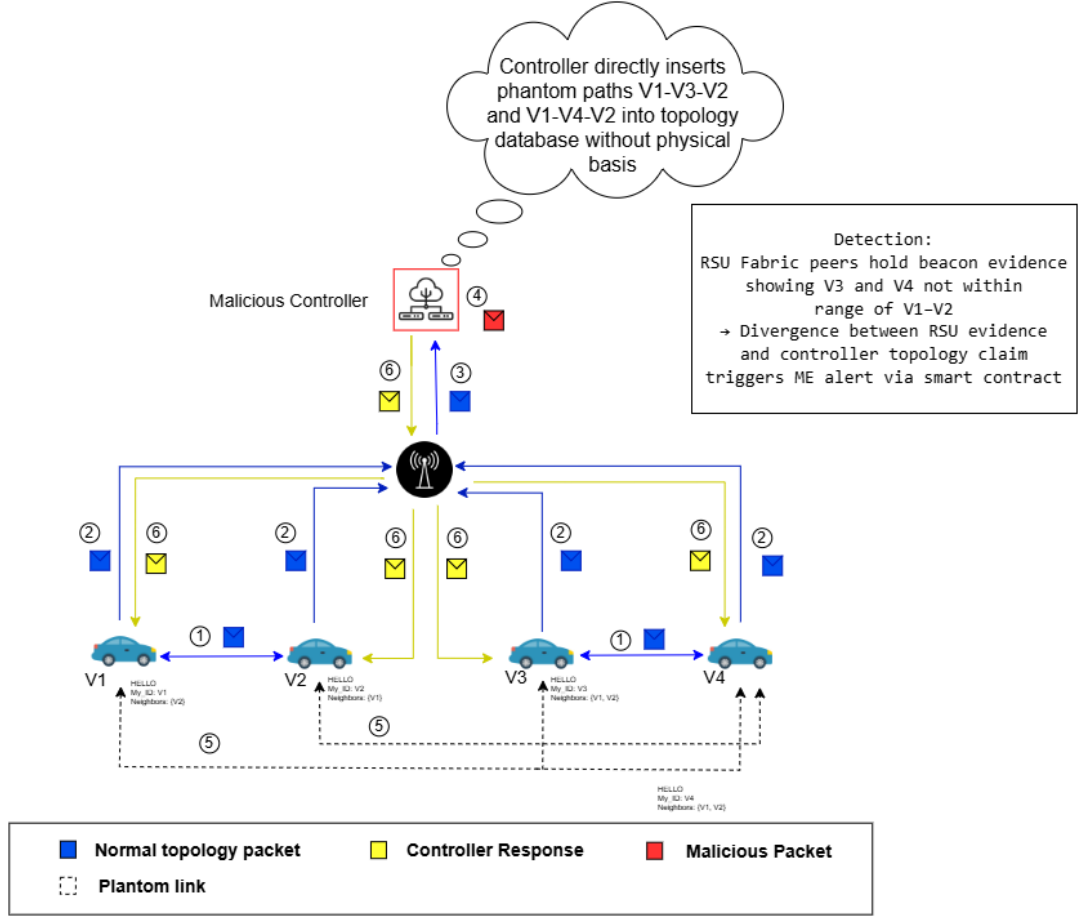


Figure 3.14: Controller-based Multipath Echo Attack (With RSU): The controller internally duplicates the V1–V2 link across multiple inferred paths.

4. **Echo Injection by the Malicious Controller:** The controller injects forged HELLO observations into the aggregated topology information, making it appear as if the legitimate V1–V2 HELLO messages were overheard and relayed through vehicles V3 and V4. These injected observations duplicate an existing legitimate link across multiple reporting paths.

Echoed topology observations: Observed link: $V1 \leftrightarrow V2$, relayed via V3;
Observed link: $V1 \leftrightarrow V2$, relayed via V4.

5. **False Multipath Inference:** Based on the duplicated observations, the controller incorrectly infers the existence of multiple paths involving vehicles V3 and V4: Path 1: $V1 \rightarrow V2$ (real); Path 2: $V1 \rightarrow V3 \rightarrow V2$ (false); Path 3: $V1 \rightarrow V4 \rightarrow V2$ (false); Path 4: $V1 \rightarrow V3 \rightarrow V4 \rightarrow V2$ (false).
6. **Faulty Routing Decisions:** The controller installs routing policies based on the falsely inferred multipath topology, resulting in incorrect packet forwarding and routing instability.

Multipath Echo Attack: An attack in which legitimate topology messages are replayed or misinterpreted to induce multiple non-existent logical paths, resulting in poisoned controller path computation without fabricating physical links.

3.4 Detection and Mitigation Framework

This work proposes a dual-mode framework. The Lightweight Version (LW) employs rule-based attack signature detection and HMAC + timestamp + nonce verification, targeting resource-constrained OBU and RSU deployment. The Full-Stack Version (FS) employs Temporal Graph Network-based detection combined with ML-DSA-87 threshold aggregate signature-based authentication using ML-KEM-1024 + HQC-5 hybrid key agreement [64, 66] and Hyperledger Fabric smart contract mitigation. Both versions are evaluated independently and comparatively. This design explicitly addresses deployment feasibility concerns while providing maximum detection performance when computational resources permit

3.4.1 Temporal-Echo Attack Formalization

The class of temporal-echo topology attacks is formally modelled as a four-tuple, defined in Equation 3.1 below, where the topology manipulation operates through timestamp and echo matrices rather than structural link fabrication.

$$\mathcal{A} = (\mathcal{G}_t, \mathbf{T}, \mathbf{E}, \mathbf{I}), \mathbf{T} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}, \mathbf{E}, \mathbf{I} \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{V}|}, \delta(\mathcal{G}_t^C, \mathcal{G}_t^R) = |\mathcal{E}_t^C \Delta \mathcal{E}_t^R| > 0 \quad (3.1)$$

In Eq. 3.1, $\mathbf{T}$ is the timestamp manipulation matrix ($\mathbf{T}_{ij}$ is the forged timestamp on link e_{ij} , capturing TTW); $\mathbf{E}$ is the echo duplication matrix ($\mathbf{E}_{ij} = 1$ if link e_{ij} has been echoed by an unauthorised reporter, capturing ME); and $\mathbf{I}$ is the identity spoofing matrix ($\mathbf{I}_{jk} = 1$ if physical sender V_j claims identity V_k in a transmitted heartbeat or beacon), which captures BSHH — a mechanism absent from both $\mathbf{T}$ and $\mathbf{E}$. Each attack variant realises a distinct instantiation: TTW has $\mathbf{T} \neq \mathbf{0}$, $\mathbf{E} = \mathbf{I} = \mathbf{0}$; BSHH has $\mathbf{I} \neq \mathbf{0}$, $\mathbf{T} = \mathbf{E} = \mathbf{0}$; ME has $\mathbf{E} \neq \mathbf{0}$, $\mathbf{T} = \mathbf{I} = \mathbf{0}$.

The rightmost term, $\delta(\mathcal{G}_t^C, \mathcal{G}_t^R) = |\mathcal{E}_t^C \Delta \mathcal{E}_t^R|$, is the *topology divergence*: the symmetric difference $\mathcal{E}_t^C \Delta \mathcal{E}_t^R$ produces the set of links on which controller perception and physical reality disagree—links that the controller wrongly records as active, or real links it has been deceived into ignoring. Its cardinality $|\cdot|$ counts how many such mismatches exist. A non-zero divergence indicates a successful temporal-echo attack: the controller perceives a topology that differs from reality *without any structurally invalid packet having been injected*. This divergence is what existing structural defenses—which check only whether packets are well-formed—cannot detect. The three attack variants TTW, BSHH, and ME each realise a different instantiation of the triple $\mathcal{A}$, with their respective signatures derived in Equations 3.2–3.11.

Credential-integrity assumption. The attack model assumes all registered vehicles maintain the confidentiality of their long-term ML-DSA-87 signing key SK_{V_i} and session key K_{V_i, n_k} . Voluntary disclosure of both credentials by a colluding vehicle to a malicious RSU constitutes a PKI-level infrastructure failure outside the protocol-level threat model; no authentication protocol can distinguish a legitimate principal from one that has willingly transferred all its credentials. Two residual cases arise within this boundary and are handled by existing mechanisms. First, if the colluding vehicle goes silent while the malicious RSU impersonates it, the vehicle is absent from $\mathcal{B}_{n_k}(t)$ (Eq. 3.46); $\iota_v = 1$ (Eq. 3.21, case 3) fires as a direct TGN detection signal. Second, if the colluding vehicle maintains legitimate physical transmission while the RSU simultaneously impersonates it, two distinct physical MAC senders claim the same identity concurrently; BSHH-S1 (Eq. 3.5) detects this case independently.

Topology Time-Warp (TTW) Attack Signatures

The TTW attacker corrupts the controller’s temporal perception by replaying or reordering legitimate beacon messages with forged timestamps. Three signatures characterise its observable effects.

TTW-S1 (Topology State Persistence Anomaly): The TTW attacker forges fresh timestamps on stale topology content, causing link e_{ij} to persist in the controller topology beyond its physically plausible lifetime under the prevailing mobility model. Let $t_{\text{first}}^{e_{ij}}$ denote the time link e_{ij} was first recorded by the controller in the current session, and L_{link} the mobility-derived link lifetime bound (Eq. 3.33). A link remaining active beyond this bound indicates artificial timestamp-based prolongation:

$$(t - t_{\text{first}}^{e_{ij}}) > L_{\text{link}} \quad (3.2)$$

In Eq. 3.2, the left-hand side $(t - t_{\text{first}}^{e_{ij}})$ is the *observed age* of link e_{ij} —how many seconds have elapsed since this link first appeared in the controller’s records for the current session. $L_{\text{link}} = 2r_{\text{comm}}/v_{\text{rel}}$ is the physical upper bound on that age: two vehicles moving at relative speed v_{rel} with communication range r_{comm} can maintain a link for at most L_{link} seconds before they exit each other’s radio range. Under legitimate operation, every link expires naturally within L_{link} as vehicles move out of communication range. A link that persists *beyond* this bound while continuing to receive beacon support indicates that forged-timestamp injections are artificially maintaining a topology state that no longer reflects physical reality.

TTW-S2 (Timestamp-Topology Age Inconsistency): A forged topology timestamp attributed to reporter V_i cannot legitimately exceed the reception time of the most recent authentic beacon the controller received from V_i through any path. Let $\tau_s^C(e_{ij}, V_i)$ denote the sender timestamp of the topology report for link e_{ij} recorded at the controller, and $\tau_r^{\text{last}}(V_i)$ the reception time of the most recent legitimate beacon from V_i . Violation of this bound exposes timestamp manipulation is expressed in the 3.3:

$$\tau_s^C(e_{ij}, V_i) > \tau_r^{\text{last}}(V_i) \quad (3.3)$$

In Eq. 3.3, $\tau_s^C(e_{ij}, V_i)$ is the timestamp *written inside* the topology report: the time at which the attacker (posing as reporter V_i) claims to have observed link e_{ij} . $\tau_r^{\text{last}}(V_i)$ is the wall-clock time at which the controller last genuinely heard from V_i via any delivery path. Under normal operation, a vehicle cannot produce a beacon whose sender timestamp is *later* than the moment the controller last heard from it—such a timestamp is only possible if it was forged. TTW-S2 is evaluated per reporter per link over sliding window $\mathcal{W}$, and is particularly effective against the RSU-origin TTW vector, where forged topology timestamps are not corroborated by independent vehicle beacons arriving at the controller.

TTW-S3 (Cross-Reporter Inconsistency): In a multi-RSU environment, two distinct reporters of the same link should carry timestamps within one beacon interval of each other. The cross-reporter consistency check is given in Equation 3.4.

$$\exists V_a \neq V_b : |\tau_s^{V_a \rightarrow C}(e_{ij}) - \tau_s^{V_b \rightarrow C}(e_{ij})| > T_b \quad (3.4)$$

In Eq. 3.4, $V_a \neq V_b$ are two distinct vehicles that both reported link e_{ij} to the controller. $\tau_s^{V_a \rightarrow C}(e_{ij})$ and $\tau_s^{V_b \rightarrow C}(e_{ij})$ are the sender timestamps embedded in their respective reports, as received by controller C . Because all legitimate reporters of the same link observe it within the same beacon cycle, the absolute difference between any two legitimate reporter timestamps cannot exceed $T_b = 100$ ms. A gap exceeding T_b proves that at least one report is a replay of a stale observation, not a fresh witness account. The existential quantifier $\exists$ means the signature fires as soon as *any single pair* of reporters for the same link exhibits this inconsistency.

Beacon State Heartbeat Hijack (BSHH) Attack Signatures

The BSHH attack spoofs the sender identity field in heartbeat messages, maintaining a phantom vehicle presence in the topology after the legitimate vehicle has departed. Because the attack does not alter message payloads or inject new packets, content-authentication alone is insufficient for detection. Three signatures detect this identity misuse. The first signature, formalized in Eq. 3.5, captures duplicate liveness assertions from distinct physical senders claiming the same identity.

$$\exists V_a \neq V_b : \text{Sender}(H^{V_a}) = \text{Sender}(H^{V_b}) = V_k, \quad \tau_r^{V_a} \approx \tau_r^{V_b} \quad (3.5)$$

BSHH-S1 (Duplicate Liveness Assertion), Eq. 3.5: H^{V_a} and H^{V_b} are heartbeat messages physically transmitted by distinct vehicles V_a and V_b ($V_a \neq V_b$). $\text{Sender}(\cdot)$ extracts the claimed sender identity from the heartbeat header. The condition asserts that two physically distinct senders *both claim to be* the same vehicle V_k —an impossibility in an uncompromised network, since a vehicle cannot be in two locations simultaneously. $\tau_r^{V_a} \approx \tau_r^{V_b}$ requires that both heartbeats

arrived at the controller within the same observation window W , confirming simultaneous (not sequential) duplicate identity claims. The existential quantifier $\exists$ means the signature triggers as soon as any such pair is found. The second signature, formalized in Eq. 3.6, detects timestamp regression caused by replayed stale heartbeats.

$$\tau_s^{(k+1)}(V_i) < \tau_s^{(k)}(V_i) \quad (3.6)$$

BSHH-S2 (Timestamp Regression), Eq. 3.6: $\tau_s^{(k)}(V_i)$ is the sender timestamp embedded in the k -th heartbeat received from vehicle V_i . The superscripts (k) and $(k+1)$ index consecutive controller-received messages from the same source. Under legitimate operation, a vehicle's heartbeat timestamps must be strictly increasing: $\tau_s^{(k+1)} > \tau_s^{(k)}$. A reversal—the $(k+1)$ -th heartbeat carrying an *earlier* timestamp than the k -th—is only possible if an older heartbeat has been replayed out of sequence. This signature directly detects the stale-heartbeat replay that characterises the BSHH attack. The third signature, formalized in Eq. 3.7.

$$\text{Heartbeat}(V_i, t) = \text{TRUE} \wedge \nexists \text{Beacon}(V_i, [t - W, t]) \quad (3.7)$$

BSHH-S3 (Liveness-Beacon Inconsistency), Eq. 3.7: $\text{Heartbeat}(V_i, t) = \text{TRUE}$ means the controller currently holds an active liveness assertion for vehicle V_i at time t . $\nexists \text{Beacon}(V_i, [t - W, t])$ is the logical assertion that no beacon from V_i was received within the most recent observation window $[t - W, t]$. The conjunction $\wedge$ requires *both* conditions to hold simultaneously: a vehicle is being declared alive *while* no beacon corroborates its presence. Under legitimate operation, heartbeat liveness and beacon activity are correlated—a vehicle transmitting heartbeats will also transmit beacons within the same window. A liveness assertion unsupported by any beacon indicates that the heartbeat originates from an impersonator rather than from V_i itself. BSHH-S3 is the only BSHH signature effective against the controller-origin attack vector, and its window W is calibrated to $2W_{ho}$ where W_{ho} is the expected RSU handover duration, preventing false positives during legitimate coverage transitions.

Multipath Echo (ME) Attack Signatures

The ME attack is qualitatively different from TTW and BSHH: the attacker does not forge content or spoof identities—it falsely *witnesses* a legitimate link from a physically impossible position, inflating the reporter count and path diversity metrics. Content-authenticated schemes provide no pre-detection value because the attacker produces correctly signed reports. Three signatures detect ME attacks and the reporter count excess is defined by 3.8.

$$|R(e_{ij}, t)| > \rho_{\max}(\hat{\lambda}(t), r_{\text{comm}}), \quad \rho_{\max} = \left\lfloor (1 + \mu) \cdot 2r_{\text{comm}} \cdot \hat{\lambda}(t) \right\rfloor \quad (3.8)$$

ME-S1 (Reporter Count Excess), Eq. 3.8: $2r_{\text{comm}} \cdot \hat{\lambda}(t)$ is the expected number of vehicles within a *1D road segment* of length $2r_{\text{comm}}$ centred on link e_{ij} , derived from the linear road topology of vehicular networks. The prior formula $\lfloor \lambda \cdot \pi r_{\text{comm}}^2 \rfloor$ assumed a 2D disc, overstating the bound by a factor $\pi r_{\text{comm}}/2 \approx 471$ at $r_{\text{comm}} = 300$ m, which would prevent ME-S1 from ever firing. The factor $(1 + \mu)$ is a conservative margin accounting for inter-lane density variation, where $\mu = 0.2$ is the recommended default. $\hat{\lambda}(t)$ is the instantaneous vehicle density estimated from RSU-observed beacon rates, making the threshold mobility-adaptive. The density estimate $\hat{\lambda}$ is updated dynamically from RSU-observed beacon rates at each time step, making the threshold adaptive to both dense urban and sparse highway deployment conditions. And also the path diversity inflation is defined by 3.9.

$$|P(V_i, V_j)|^t - |P(V_i, V_j)|^{t-1} > \delta_{\text{max}} \quad (3.9)$$

ME-S2 (Path Diversity Inflation), Eq. 3.9: $|P(V_i, V_j)|^t$ is the number of distinct paths inferred by the controller between vehicles V_i and V_j at time step t ; $|P(V_i, V_j)|^{t-1}$ is the same count at the immediately preceding beacon interval. Their difference is the *path growth rate* over one beacon interval. δ_{max} is the mobility-consistent upper bound on new legitimate paths per beacon interval, formally defined in Eq. 3.10:

$$\delta_{\text{max}} = \max(1, \lceil \lambda \cdot v_{\text{rel}} \cdot T_b \rceil) \quad (3.10)$$

In Eq. 3.10, $\lambda \cdot v_{\text{rel}} \cdot T_b$ is the expected number of new relay vehicles entering the communication range of both V_i and V_j within one beacon interval — the only physical mechanism by which a legitimate new multi-hop path can appear. At urban speeds ($v_{\text{rel}} \approx 14$ m/s, $\lambda = 0.02$ veh/m): $\delta_{\text{max}} = 1$. At highway speeds ($v_{\text{rel}} \approx 67$ m/s): $\delta_{\text{max}} = 1$. In practice $\delta_{\text{max}} = 1$ across all realistic SDVN conditions; any path-count increase exceeding one per beacon interval indicates ME injection and triggers ME-S2. The third signature of ME attack is defined by 3.11

$$V_k \in R(e_{ij}, t) \wedge (d(\text{pos}_{V_k}, e_{ij}) > r_{\text{comm}} \vee \text{RSSI}_{V_k \leftarrow V_i} < \text{RSSI}_{\text{min}}) \quad (3.11)$$

ME-S3 (Reporter-Range and Signal Inconsistency), Eq. 3.11: $V_k \in R(e_{ij}, t)$ asserts that vehicle V_k claims to have witnessed link e_{ij} , i.e., it is a member of the reporter set. $d(\text{pos}_{V_k}, e_{ij})$ is the Euclidean distance from V_k 's GPS-attested position pos_{V_k} to the nearest endpoint of link e_{ij} : if this exceeds r_{comm} , V_k is outside the physical communication range and cannot legitimately have observed the link. $\text{RSSI}_{V_k \leftarrow V_i}$ is the received signal strength measured at V_k from endpoint V_i : if this falls below $\text{RSSI}_{\text{min}}(r_{\text{comm}})$, the signal is too weak to be consistent with the proximity required to observe the link. The logical OR ($\vee$) means the signature triggers if *either* physical consistency check fails. Unlike ME-S1 and ME-S2, which are statistical signatures, ME-S3 is a per-reporter physical consistency check that directly identifies the individual false witness.

3.4.2 Lightweight Detection

The LW detector evaluates all nine signatures over sliding window $\mathcal{W}$ and computes a weighted match score $s(e)$ for each incoming topology event e . The scoring function is defined in Equation 3.12; the complete detection procedure is given in Algorithm 1.

$$s(e) = \sum_{k=1}^9 w_k \cdot \mathbf{1}[\text{Sig}_k \text{ triggered}], \quad \sum_{k=1}^9 w_k = 1, \quad w_k \geq 0 \quad (3.12)$$

In Eq. 3.12, the sum runs over all nine signatures ($k = 1, \dots, 9$, corresponding to TTW-S1 through ME-S3). $\mathbf{1}[\text{Sig}_k \text{ triggered}]$ is the indicator function: it equals 1 if signature Sig_k fired on event e under window $\mathcal{W}$, and 0 otherwise. $w_k \geq 0$ is the weight assigned to signature k ; the constraint $\sum w_k = 1$ normalises the score to $[0, 1]$. For the current NS-3/SUMO evaluation, uniform weights $w_k = 1/9$ are used as a calibration-neutral baseline, consistent with the zero false-positive rate observed across all nine signatures under controlled simulation conditions (MCC = 1.0, AUROC = 1.0). Proportional calibration — assigning higher weights to signatures with lower false-positive rates — will be applied in future real-network evaluation where non-zero FP rates are expected under environmental noise and adversarial mimicry. A detection alert is raised when $s(e) > \theta_{\text{LW}}$, where θ_{LW} is determined by maximising the F1 score on the validation split of the simulation dataset.

The per-event time complexity of the lightweight detector over a window of $|\mathcal{W}|$ events and $|\mathcal{S}| = 9$ fixed signatures is defined as Eq3.13:

$$T_{\text{LW}}(|\mathcal{W}|, |\mathcal{S}|) = O(|\mathcal{W}| \cdot |\mathcal{S}|) \quad (3.13)$$

In Eq. 3.13, $|\mathcal{W}|$ is the number of events retained in the sliding window and $|\mathcal{S}|$ is the number of signatures evaluated per event. $O(\cdot)$ denotes standard big-O asymptotic notation. Since $|\mathcal{S}| = 9$ is a compile-time constant determined by the attack model and does not scale with network size, this reduces to Eq3.14:

$$T_{\text{LW}}(|\mathcal{W}|) = O(|\mathcal{W}|) \quad (3.14)$$

As shown in Eq. 3.14, detection overhead scales *linearly* with the window size $|\mathcal{W}|$ only, and is entirely independent of network scale (the number of vehicles or RSUs). This confirms deployability on resource-constrained OBU and RSU hardware within the 100 ms SDVN latency budget.

HMAC-Timestamp-Nonce Authentication

Each beacon is pre-authenticated before reaching the signature detector. The verifier accepts a message from vehicle V_i only when three conditions hold simul-

taneously: HMAC integrity (Eq. 3.15), timestamp freshness (Eq. 3.16), and nonce novelty (Eq. 3.17). The complete procedure is given in Algorithm 3.

$$\text{HMAC-SHA256}(K_{V_i, n_k}, m') = \text{MAC}_i^{\text{received}} \quad (3.15)$$

In Eq. 3.15, $\text{HMAC-SHA256}(K_{V_i, n_k}, m')$ recomputes the authentication tag locally at the verifier, using session key K_{V_i, n_k} (shared between vehicle V_i and trusted node n_k via ML-KEM-1024 + HQC-5 hybrid key encapsulation mechanism [64, 66]) over the authenticated message string m' . $\text{MAC}_i^{\text{received}}$ is the tag field extracted from the received packet. Equality of the two tags proves that the message was generated by an entity holding K_{V_i, n_k} and that the content was not altered in transit. This check alone, however, does not prevent a legitimate but compromised node from replaying its own past messages—which motivates Equations 3.16 and 3.17.

$$|\tau_r - \tau_s| \leq T_b + \epsilon \quad (3.16)$$

In Eq. 3.16, τ_r is the verifier’s local clock at the moment of reception and τ_s is the sender timestamp embedded in the message. $|\tau_r - \tau_s|$ is the measured one-way delay between creation and reception. $T_b + \epsilon$ is the maximum acceptable delay: $T_b = 100$ ms is one beacon interval, and ϵ is the maximum tolerable network propagation delay. A message arriving with a delay exceeding this bound was created in an earlier beacon cycle and is therefore a replay. This check directly blocks TTW-class attacks in which a stored beacon is replayed long after its creation time.

$$\text{nonce}_i \notin \mathcal{N}_{\text{seen}} \quad (3.17)$$

In Eq. 3.17, nonce_i is a cryptographic nonce (a number used exactly once) appended by vehicle V_i to the message before signing. $\mathcal{N}_{\text{seen}}$ is the set of all nonces previously observed by the verifier from any sender. The non-membership condition $\notin$ confirms that this nonce has never been seen before, guaranteeing that even a message within the freshness window of Eq. 3.16—which might result from a very rapid replay—has not been replayed from any prior session. Together, Equations 3.15–3.17 close the replay gap left by bare HMAC: Eq. 3.16 blocks inter-session TTW replay; Eq. 3.17 blocks rapid intra-session BSHH replay.

Logical Key Hierarchy (LKH) for Session Key Management

Key Bootstrapping and Initial Trust Establishment. All vehicles are assumed to be pre-registered with a V2X Public Key Infrastructure (PKI) and to possess a valid long-term certificate binding their identity to a ML-DSA-87 public signing key. Prior to operational deployment, the consortium enrollment authority provisions each vehicle with its certified credential set and the consortium trust anchor.

When a vehicle V_i first contacts a trusted node n_k (RSU or consortium gateway), the trusted node transmits its certified ML-KEM-1024 (Kyber-1024) public key. Vehicle V_i verifies the certificate chain using the pre-installed trust anchor and performs a ML-KEM-1024 + HQC-5 [64, 66] hybrid-KEM key establishment procedure. Both parties derive a shared session key K_{V_i, n_k} , which subsequently serves as the root key for HMAC authentication Eq. 3.15 and enrollment into the Logical Key Hierarchy (LKH).

This bootstrapping process ensures that all session keys originate from PKI-verified identities rather than self-asserted credentials. Certificate lifecycle management, credential renewal, and revocation handling are outside the scope of the present work and are considered future extensions.

Trusted-Node-Managed Logical Key Hierarchy. The session keys K_{V_i, n_k} used in Eq. 3.15 are managed within a Logical Key Hierarchy. The ML-DSA-87 signing keys SK_{V_i} used in Eq. 3.29 are long-term identity keys managed separately under the PKI. The LKH organises the n vehicles currently registered with n_k into a binary tree $\mathcal{T}$ of depth $d = \lceil \log_2 n \rceil$. Each leaf node ℓ_i stores the individual session key K_{V_i, n_k} of vehicle V_i , and each internal node j stores a Key Encryption Key (KEK) K_j^{kek} used to encrypt the keys of its children. The root stores the group key $K_{\mathcal{G}}$ shared by n_k and all registered vehicles. When the smart contract in Algorithm 4 executes $\text{REVOKE-SESSION-KEY}(V_k)$, only the d KEKs on the path from leaf ℓ_k to the root require updating, giving a revocation cost of as in Eq. 3.18:

$$C_{\text{revoke}} = O(\log n) \quad (3.18)$$

In Eq. 3.18, C_{revoke} is the number of key update operations required to revoke a single vehicle from a group of n vehicles. $O(\log n)$ means this cost grows logarithmically with group size: for 1,000 registered vehicles, only approximately 10 KEK updates are needed, versus 1,000 updates under naïve full re-keying ($O(n)$). The updated KEKs are encrypted under the individual keys of the remaining $n - 1$ legitimate vehicles and distributed within the next beacon interval T_b , ensuring continuity of HMAC authentication and threshold signing for all non-revoked vehicles without interrupting beacon processing.

3.4.3 Full-Stack Temporal Graph Network Detection

The full-stack detector models the SDVN topology as a sequence of graph snapshots, one per beacon interval. The snapshot definition—the foundation of all subsequent TGN computations—is given in Eq. 3.19. The temporal memory update rule is defined in Eq. 3.23, the neighbourhood aggregation pass in Eq. 3.24, and the anomaly scoring function in Eq. 3.25.

$$\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \mathbf{X}_t, \mathbf{A}_t) \quad (3.19)$$

In Eq. 3.19, $\mathcal{V}_t$ is the set of active vehicles and RSUs at beacon interval t ; $\mathcal{E}_t$ is the set of reported topology links; $\mathbf{X}_t \in \mathbb{R}^{|\mathcal{V}_t| \times d}$ is the node feature matrix, one row per node; $\mathbf{A}_t \in \mathbb{R}^{|\mathcal{V}_t| \times |\mathcal{V}_t|}$ is the weighted adjacency matrix, whose entry $\mathbf{A}_{t,uv}$ encodes the normalised timestamp freshness of the most recent link report between u and v via exponential decay (Eq. 3.22). Stale-timestamped TTW-replayed edges receive near-zero weights in $\mathbf{A}_t$, directly downweighting their contribution during neighbourhood aggregation without requiring explicit threshold tuning.

$$\mathbf{x}_v = \left[\tau_{\text{dev}}^{(v)}, c_v^W, \Delta s_v, \rho_v, \iota_v \right] \in \mathbb{R}^5, \quad \tau_{\text{dev}}^{(v)} = \text{clip} \left(\frac{\tau_r^{(v)} - \tau_s^{(v)}}{T_b}, -50, 50 \right) \quad (3.20)$$

In Eq. 3.20, $\mathbf{x}_v \in \mathbb{R}^5$ is the feature vector of node v . The node-identifier embedding id_v is deliberately excluded: including it allows the model to memorise “attacker node $V_k \rightarrow \text{attack}$ ” rather than learning temporal behaviour patterns, preventing generalisation to attacker nodes not seen during training.

$\tau_{\text{dev}}^{(v)}$ is the *timestamp age deviation*, defined as the normalised clipped difference between the event reception time $\tau_r^{(v)}$ (RSU wall-clock at receipt) and the claimed sender timestamp $\tau_s^{(v)}$ embedded in the packet. Under legitimate operation $\tau_r^{(v)} - \tau_s^{(v)}$ approximates the propagation delay, producing a value near zero. Under BSHH attacks, the replayed old heartbeat carries an unforged stale timestamp, so $\tau_r^{(v)} \gg \tau_s^{(v)}$ and τ_{dev} is large and positive, providing a direct BSHH detection signal. Under TTW attacks the attacker forges $\tau_s^{(v)}$ to match the current time, so $\tau_{\text{dev}} \approx 0$; the TTW-specific temporal signal is carried instead by Δs_v (sequence-number gap, TTW-S2) which the attacker cannot forge without disclosing the replay. The raw absolute timestamp $\tau_s^{(v)}$ is not used: it encodes simulation time-position (events at $t = 100$ s versus $t = 250$ s look systematically different) rather than any attack-relevant deviation, causing distribution shift that prevents generalisation beyond the training window. The clip bound of ± 50 intervals (± 5 s) suppresses outliers from long-delay or malformed events without discarding attack signal, since even a 5-interval staleness gap already exceeds the legitimate propagation delay by two orders of magnitude.

c_v^W is the beacon count from v within sliding window W : a low count signals liveness anomalies (BSHH-S3 signal). Δs_v is the sequence-number gap between consecutive beacons from v : non-zero gaps encode TTW replay sequence inversions (TTW-S2 signal). ρ_v is the reporter count for links adjacent to v : inflated values encode ME echo injection (ME-S1 signal). $\iota_v \in \{0, 1\}$ is the *identity mismatch indicator*, formally defined in Eq. 3.21. It is 1 when the MAC-layer physical transmitter does not match the application-layer claimed identity under conditions that cannot be explained by legitimate RSU forwarding. Suppression to 0 for RSU-forwarded events is conditioned on membership of the claimed vehicle in the forwarding RSU’s signed beacon evidence record $\mathcal{B}_{n_k}(t)$ (Eq. 3.46): if the vehicle appears in $\mathcal{B}_{n_k}(t)$, the RSU directly observed it in the current beacon interval and forwarding is legitimate (case 2); if the vehicle is absent, the RSU is forwarding a beacon it never physically received, signalling impersonation (case 3).

The controller-sentinel suppression (`physical_sender_id = 9999`) is unchanged (case 4).

The identity mismatch indicator is formally defined as:

$$\iota_v(e) = \begin{cases} 0 & \text{mac}_{\text{phys}}(e) = \text{id}_{\text{claim}}(e) & [\text{direct transmission}] \\ 0 & \text{mac}_{\text{phys}}(e) \in \mathcal{P}_{\text{consortium}} \wedge \text{id}_{\text{claim}}(e) \in \mathcal{B}_{\text{macphys}}(e)(t) & [\text{legitimate RSU forwarding}] \\ 1 & \text{mac}_{\text{phys}}(e) \in \mathcal{P}_{\text{consortium}} \wedge \text{id}_{\text{claim}}(e) \notin \mathcal{B}_{\text{macphys}}(e)(t) & [\text{impersonation signal}] \\ 0 & \text{mac}_{\text{phys}}(e) = 9999 & [\text{controller sentinel}] \end{cases} \quad (3.21)$$

$\mathcal{B}_{n_k}(t)$ is the RSU’s signed beacon evidence record (Eq. 3.46), containing identities of vehicles from which n_k directly received a legitimate 802.11p beacon in interval t . Case 2 suppresses ι_v for legitimate forwarding: the RSU is a registered consortium peer and the claimed vehicle appears in its own direct observation record for the current interval. This requires no additional tracking variable — $\mathcal{B}_{n_k}(t)$ is already generated and cryptographically signed by each RSU as part of the existing beacon aggregation pipeline. Case 3 fires the impersonation signal: the RSU claims to forward from V_i but V_i is absent from $\mathcal{B}_{n_k}(t)$, meaning the RSU never directly received V_i ’s beacon in this interval. A malicious RSU using V_i ’s stolen credentials cannot add V_i to $\mathcal{B}_{n_k}(t)$ without V_i physically transmitting within radio range; if V_i does transmit, its physical presence is independently verifiable through other means.

In the implementation, the time-elapsd encoding $\phi(\Delta t_v) = \log(1 + \Delta t_v/T_b)$ defined in Eq. 3.23 is concatenated with $\mathbf{x}_v$ before the GRU cell, giving a total GRU input dimension of $32 + 5 + 1 = 38$ (hidden state + feature vector + time encoding). The features Δs_v and ρ_v directly embed the TTW and ME anomaly signals at the node level, while $\tau_{\text{dev}}^{(v)}$ and ι_v embed the BSHH signals, allowing the TGN to learn attack-specific graph evolution patterns from labelled simulation data.

The edge weight entry used in $\mathbf{A}_t$ is computed via as in Eq3.22:

$$\mathbf{A}_{uv}(t) = \exp\left(-\frac{\tau_r^{(t)} - \tau_s^{(uv)}}{\gamma \cdot T_b}\right) \quad (3.22)$$

In Eq. 3.22, $\tau_r^{(t)}$ is the controller’s clock at the current time step and $\tau_s^{(uv)}$ is the most recent sender timestamp on the link report for edge (u, v) . Their difference $(\tau_r^{(t)} - \tau_s^{(uv)})$ is the *report age*: how old the most recent link observation is. The exponential function $\exp(-\cdot)$ maps this age to a weight in $(0, 1]$: perfectly fresh reports receive weight close to 1, while stale reports receive weight close to 0. $\gamma > 0$ is a decay rate hyperparameter tuned on validation data; it is chosen so that $\mathbf{A}_{uv}(t) \approx 0.5$ when the report age equals $L_{\text{link}}/2$ —the halfway point of the expected link lifetime—ensuring that edges approaching natural expiry are already

downweighted, while TTW-replayed edges significantly past their expected lifetime receive near-zero weight. T_b in the denominator scales the decay relative to the beacon interval, maintaining consistent behaviour across deployment speeds. The temporal memory of node v can be defined as in Eq. 3.23.

$$\mathbf{m}_v(t) = \text{GRU}(\mathbf{h}_v(t^-), \mathbf{x}_v(t), \phi(\Delta t_v)), \quad \phi(\Delta t_v) = \log(1 + \Delta t_v/T_b) \quad (3.23)$$

In Eq. 3.23, $\mathbf{m}_v(t)$ is the updated temporal memory of node v : a persistent vector maintained by the TGN that captures v 's behavioral history across all past beacon intervals. $\text{GRU}(\cdot)$ is the Gated Recurrent Unit, a recurrent neural network cell that selectively retains or forgets past information via learned update and reset gates. $\mathbf{h}_v(t^-)$ is the node embedding from the *immediately preceding* event, feeding the memory of past behavior into the current update. $\mathbf{x}_v(t)$ is the current feature vector from Eq. 3.20, carrying the attack-relevant signals at the current time step. $\phi(\Delta t_v) = \log(1 + \Delta t_v/T_b)$ is the *time-elapsd encoding*: Δt_v is the number of seconds since the last event at v , divided by T_b so that one normal beacon gap encodes as $\log 2 \approx 0.69$. Nodes silent for an unusually long interval—characteristic of the liveness spoofing in BSHH-S3—produce large ϕ values that the GRU learns to associate with anomalous temporal patterns.

Per-Node Window-Based Truncated Backpropagation Through Time.

The GRU memory update in Eq. 3.23 requires that gradients propagate through each node's hidden-state trajectory across multiple events for the model to learn temporal patterns. One-step gradient truncation — detaching $\mathbf{h}_v(t^-)$ between every individual event — prevents the loss from propagating through the GRU's temporal trajectory, reducing the GRU to a stateless per-event MLP with a stale uninformative context vector. TTW-S1 persistence anomalies span up to $\lceil L_{\text{link}}/T_b \rceil = 430$ beacon intervals in the urban scenario; BSHH-S3 liveness gaps span the RSU handover window W_{ho} . Neither is learnable with 1-step truncation.

A global-window formulation — detaching all node states every W_{BPTT} events in the global stream — is also insufficient. With $|\mathcal{V}| = 200$ vehicles sharing a global window of 100, each node contributes on average $100/200 = 0.5$ gradient steps per detach boundary, which is functionally equivalent to 1-step truncation at the node level.

Training therefore uses *per-node TBPTT*: each node v maintains an independent event counter n_v . The global event stream is processed in strict reception-time order (preserving the temporal correctness of edge weights $\mathbf{A}_{uv}(t)$ in the forward pass). For node v , the hidden state $\mathbf{h}_v$ is detached and carried forward as a constant only after v 's counter reaches W_{BPTT} , at which point n_v resets to zero. This gives every node a full W_{BPTT} -step gradient horizon along its own temporal trajectory regardless of how many other nodes' events interleave in the global stream.

During message passing on the three-node subgraph $\{\text{reporting_node}, \text{link_src}, \text{link_dst}\}$, the hidden states $\mathbf{h}_{\text{link_src}}$ and $\mathbf{h}_{\text{link_dst}}$ must be treated as *detached context* — frozen constants contributing to the forward-pass aggregation

but excluded from the backward graph for this event. Allowing gradient flow through both $\mathbf{h}_v$ and the context node states in the same backward pass entangles the per-node gradient horizons and breaks the independence property.

The recommended default is $W_{\text{BPTT}} = 100$ events per node ($= 10$ s of observation context per node at $T_b = 100$ ms); validation sweeps $W_{\text{BPTT}} \in \{50, 100, 200\}$.

The embedding of node v is defined as in Eq. 3.24.

$$\mathbf{h}_v^{(l+1)} = \sigma(\mathbf{W}^{(l)} \cdot \text{MEAN}(\{\mathbf{h}_u^{(l)} \odot \mathbf{A}_{uv}(t) : u \in \mathcal{N}(v, t)\}) + \mathbf{b}^{(l)}) \quad (3.24)$$

In Eq. 3.24, $\mathbf{h}_v^{(l+1)}$ is the embedding of node v after $l + 1$ rounds of neighbourhood aggregation. $\sigma(\cdot)$ is the ReLU activation function ($\sigma(x) = \max(0, x)$), introducing non-linearity. $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the trainable weight matrix and bias at layer l , learned during training on NS-3/SUMO simulation data. $\text{MEAN}\{\cdot\}$ averages the embeddings of all neighbours $u \in \mathcal{N}(v, t)$. $\odot$ is element-wise multiplication: each neighbour embedding $\mathbf{h}_u^{(l)}$ is scaled component-wise by the freshness weight $\mathbf{A}_{uv}(t)$, so stale-linked neighbours (TTW-replayed) contribute proportionally less to the aggregated representation. $\mathcal{N}(v, t)$ is the set of topological neighbours of v in graph snapshot $\mathcal{G}_t$. This aggregation is repeated L times, allowing the model to incorporate information from L -hop neighbourhoods.

Neighbourhood Implementation. Eq. 3.24 describes aggregation over $\mathcal{N}(v, t)$, the full set of topological neighbours of node v in snapshot $\mathcal{G}_t$. In the implementation, the neighbourhood for each event is realised as a fixed three-node induced subgraph $\{\text{reporting_node}, \text{link_src}, \text{link_dst}\}$, weighted by the freshness weight $\mathbf{A}_{uv}(t)$ (Eq. 3.22). `link_src` maintains temporal memory state for graph consistency but is excluded from the score readout; only `reporting_node`'s final-layer embedding $\mathbf{h}_v^{(L)}$ is passed to the anomaly scorer (Eq. 3.25). This per-event local subgraph differs from full snapshot aggregation in that it does not pool embeddings across all simultaneous reporters of the same link e_{ij} within a single beacon interval. The reporter count feature ρ_v (Eq. 3.20) partially compensates by carrying the aggregate reporter count as a direct node-level signal, ensuring the ME-S1 evidence remains accessible to the GRU across event windows regardless of neighbourhood pooling depth. **Implementation requirement:** the training pipeline must construct the identical three-node induced subgraph $\{\text{reporting_node}, \text{link_src}, \text{link_dst}\}$ that the deployed inference engine uses. A trainer that uses only a two-node pair $\{\text{reporting_node}, \text{link_dst}\}$ produces weights optimised for a different computational graph and introduces a training–inference distribution shift that will degrade detection accuracy on RSU-relayed and echo-reporter events (BSHH and ME attack families) in an unpredictable and unquantifiable manner. Any checkpoint trained on a two-node subgraph must be considered a first-order approximation only and must be retrained on the three-node subgraph before final evaluation.

The anomaly score for the node v is defined as in Eq. 3.25.

$$\hat{y}_v(t) = \sigma(\mathbf{w}^\top \mathbf{h}_v^{(L)}(t)) \quad (3.25)$$

In Eq. 3.25, $\hat{y}_v(t) \in (0, 1)$ is the anomaly score for node v at time t . $\sigma(\cdot)$ here denotes the sigmoid function ($\sigma(x) = 1/(1 + e^{-x})$), mapping the linear logit to the probability interval $(0, 1)$. $\mathbf{w} \in \mathbb{R}^d$ is a trainable scoring vector; its inner product $\mathbf{w}^\top \mathbf{h}_v^{(L)}(t)$ with the final-layer embedding $\mathbf{h}_v^{(L)}(t)$ produces a scalar logit measuring how anomalous v 's embedding is relative to the learned distribution of benign behaviour. A full-stack detection alert is raised when $\hat{y}_v(t) > \theta_{\text{FS}}$, where θ_{FS} is selected by maximising the Matthews Correlation Coefficient (MCC) on held-out validation data, ensuring robust detection performance under the natural class imbalance between benign and attack events.

Variant Classification Head. The binary score $\hat{y}_v(t)$ determines whether node v is anomalous; Algorithm 4 additionally requires the variant label $\alpha \in \{\text{TTW}, \text{BSHH}, \text{ME}\}$ to select the appropriate mitigation branch. The variant is predicted by a multi-class head defined in Eq. 3.26:

$$\hat{\alpha}_v(t) = \text{softmax}(\mathbf{W}_{\text{cls}} \cdot \mathbf{h}_v^{(L)}(t)) \in \mathbb{R}^3 \quad (3.26)$$

In Eq. 3.26, $\mathbf{W}_{\text{cls}} \in \mathbb{R}^{3 \times d}$ is a trainable weight matrix producing a probability vector over the three attack families; the predicted variant is $\alpha = \arg \max \hat{\alpha}_v(t)$. The head is trained jointly with the binary scorer using multi-class cross-entropy loss on the NS-3/SUMO simulation labels. A full-stack alert is raised only when $\hat{y}_v(t) > \theta_{\text{FS}}$, at which point α is passed to Algorithm 4 for variant-specific mitigation.

Training Objective. The binary scorer (Eq. 3.25) and variant classification head (Eq. 3.26) are trained jointly by minimising a three-term loss over each mini-batch:

$$\mathcal{L} = \underbrace{w_c \cdot \mathcal{L}_{\text{BCE}}(\hat{y}_v, y_v)}_{\text{(i) weighted binary detection}} + \underbrace{0.3 \cdot \mathcal{L}_{\text{CE}}(\hat{\alpha}_v, \alpha_v) \cdot \mathbf{1}[y_v = 1]}_{\text{(ii) variant classification (attack events only)}} + \underbrace{0.5 \cdot \max(0, m - (\bar{\hat{y}}_{\text{atk}} - \bar{\hat{y}}_{\text{ben}}))}_{\text{(iii) score-separation margin}} \quad (3.27)$$

In Eq. 3.27: $\mathcal{L}_{\text{BCE}}$ is binary cross-entropy between predicted score $\hat{y}_v$ and ground-truth binary label $y_v \in \{0, 1\}$; $w_c = N_{\text{total}}/(2N_{\text{attack}})$ is the per-batch class-imbalance weight applied to attack events; $\mathcal{L}_{\text{CE}}$ is multi-class cross-entropy for variant prediction, masked by $\mathbf{1}[y_v = 1]$ so benign events do not contribute a variant-classification gradient; $\bar{\hat{y}}_{\text{atk}}$ and $\bar{\hat{y}}_{\text{ben}}$ are the mini-batch mean anomaly scores for attack and benign events respectively; $m = 0.35$ is the target score-separation margin. Term (iii) is a hinge penalty that fires whenever the mean attack score is less than $m = 0.35$ above the mean benign score, preventing the score-collapse failure mode in which a GRU trained under extreme class imbalance learns to assign uniformly low scores to all events and achieves low BCE loss while producing $\text{MCC} = 0$. Term (iii) activates only when the separation is insufficient and contributes zero loss once the separation exceeds m .

3.4.4 TGN Training and Testing Pipeline

Sections 3.4.3 and Algorithm 2 define the *mathematical* model of the Temporal Graph Network (TGN) detector: the feature vector x_v (Eq. 3.20), the edge-freshness weight $A_{uv}(t)$ (Eq. 3.22), the temporal memory update (Eq. 3.23), the neighbourhood-aggregation embedding (Eq. 3.24), and the anomaly/variant scoring heads (Eq. 3.25–Eq. 3.26). This subsection describes the concrete, two-stage *engineering* pipeline that produces the trained parameters consumed by those equations and that subsequently operationalises them at run time. The pipeline spans two implementation artefacts that are compiled/executed together with the routing logic: an offline PyTorch training script (`tnn/tnn_train.py`) and a C++ inference engine (`.tnn_src/tnn_core.cc`) that is included directly into `scratch/routing.cc` via `#include ".tnn_src/tnn_core.cc"`, so that the same weighted computation graph executes during both the data-generation runs and the evaluation runs. The pipeline is presented in two phases: an *offline training phase* (Fig. 3.15) and an *online testing/inference phase* (Fig. 3.16).

Offline Training Pipeline

Figure 3.15 illustrates the training pipeline. Labelled event traces are first produced by replaying all twelve attack scenarios (three variants $\times$ four attacker positions) through the NS-3/SUMO simulation with `tnn_core.cc` compiled in but no weight file supplied, so that every reported PEM event is logged to `tnn_events.csv` together with its Eq. 3.20 feature values and its ground-truth attack label. The per-event feature vector is re-assembled in Python exactly as `TGN_ExtractFeatures` computes it in C++, augmented with the time-elapsed encoding $\phi(\Delta t_v) = \log(1 + \Delta t_v/T_b)$, giving the six-element GRU input $[\tau_{dev}, c_v^W, \Delta s_v, \rho_v, \iota_v, \phi]$. Because a naive chronological 70/15/15 split would leave several of the twelve scenarios with zero benign or zero attack events in the held-out folds, the split is performed *per scenario and per class*: for every (scenario, benign/attack) group the first 70% of events by reception time form the training fold, the next 15% the validation fold, and the remaining 15% the test fold; the three folds are then independently re-sorted by reception time before being fed to the model, so that the GRU still observes each fold in correct temporal order.

The model (`TGNModel`) mirrors the C++ detector parameter-for-parameter: three GRU gate matrices (W_z, U_z, b_z) , (W_r, U_r, b_r) , (W_n, U_n, b_n) implementing Eq. 3.23 $L = 2$ message-passing layers $(W^{(l)}, b^{(l)})$ implementing Eq. 3.24 as a mutual pairwise interaction between the reporting node and `link_dst` (see the parity note below for how this differs from the deployed model’s full three-node aggregation); a scoring vector w and bias b_{score} implementing Eq. 3.25; and a $3 \times d$ classification head W_{cls}, b_{cls} implementing Eq. 3.26. Training uses per-node windowed truncated back-propagation through time (TBPTT) with $W_{BPTT} = 100$ events per node, matching the thesis’s per-node TBPTT formulation in §3.4.3: each node’s hidden state is detached from the autograd graph only after that specific node has contributed 100 events, so nodes with sparse activity retain a full 100-event gradient horizon regardless of how many other nodes’ events interleave

in the global stream.

The training objective combines three terms: a class-weighted binary cross-entropy loss on the anomaly logit (Eq. 3.25), a multi-class cross-entropy loss on the variant logits (Eq. 3.26, restricted to attack events, weighted at 0.30 relative to the BCE term), and a margin term, weighted at 0.50 relative to the BCE term, that penalises the model whenever the mean attack score does not exceed the mean benign score by a margin of 0.35, which prevents the optimiser from collapsing both classes to similar scores under heavy class imbalance: (Eq. 3.27) The model is optimised with Adam (weight decay 10^{-4}) under a cosine-annealed learning-rate schedule; at fixed intervals the validation-set MCC is computed after sweeping the decision threshold, and the checkpoint with the highest validation MCC (ties broken by AUROC) is retained. If, after completing all epochs, the best validation MCC has not reached the target of 0.975, the model is re-initialised and training restarts, up to five attempts, with the single best checkpoint across all attempts kept as the final model. Using the current defaults ($d = 32$, $L = 2$ – see Table 4.9 for pending sensitivity analysis across $d \in \{32, 64, 128\}$ and $L \in \{1, 2, 3\}$), the retained checkpoint’s weight tensors are then serialised to `tnn_weights.bin` in the fixed binary layout. expected by `TGNDetector::LoadWeights()` (row-major `float64`, dimension and layer count as a header), and its performance is finally reported on the untouched test fold.

Implementation note — message-passing parity. The offline trainer’s message-passing step aggregates only the reporting node and `link_dst` as a mutually-interacting pair. The online detector’s message-passing step (Eq. 3.24, `MessagePassingRound` in `tnn_core.cc`) instead constructs the full three-edge induced subgraph over $\{\text{reporting node}, \text{link_src}, \text{link_dst}\}$ and mean-aggregates over however many of those neighbours are active, so the two implementations do not currently compute an identical function whenever the reporting node differs from `link_src` (RSU-relayed and echo-reporter events). Any checkpoint trained via the current `tnn_train.py` should therefore be treated as a first-order approximation to the deployed model’s computation graph rather than an exact replica.

Testing / Online Inference Pipeline

The compiled detector (`TGNDetector` in `tnn_core.cc`) is reused directly inside `routing.cc` at evaluation and deployment time, rather than being replayed through a separate offline scorer. `ProcessEvent()` supports two scoring modes, selected at start-up by whether a trained checkpoint is supplied via `--tnn_weights`. In both modes, every PEM event recorded by `PemRecordObservation()` inside the NS-3 event loop is passed immediately to `TGNProcessEventInline()` for online, per-event scoring (not batched at beacon-interval boundaries), and the GRU memory update (Eq. 3.23) and freshness-weighted message-passing rounds (Eq. 3.22, 3.24) are always executed, so that each node’s temporal state stays current irrespective of which scorer is active. Pure BEACON events update only the per-node sliding window used for c_v^W and do not trigger a scoring pass.

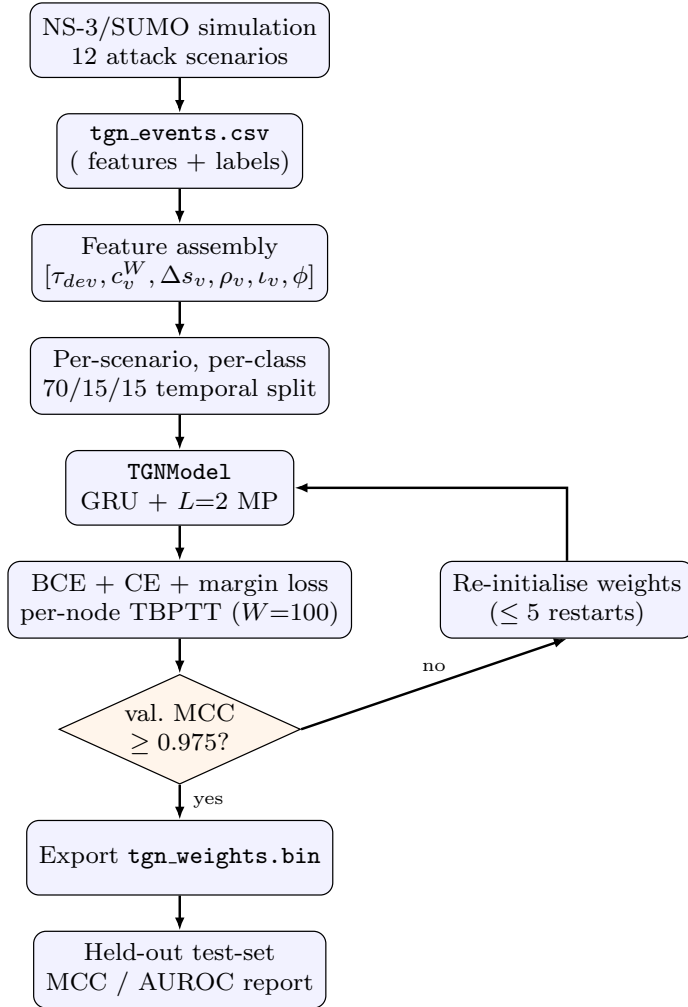


Figure 3.15: Offline TGN training pipeline (`tgn/tgn_train.py`).

Table 3.2: TGN training hyperparameters (`tgn/tgn_train.py` defaults).

Symbol / flag	Default value	Description
d (<code>--dim</code>)	32	GRU/embedding dimension (must match <code>tgn_dim</code> at inference)
L (<code>--layers</code>)	2	Message-passing rounds (Eq. 3.24)
epochs (<code>--epochs</code>)	50	Training epochs per restart attempt
η (<code>--lr</code>)	10^{-3}	Adam learning rate
weight decay	10^{-4}	Adam L2 regularisation
L_{link} (<code>--l_link</code>)	43 s (urban) / 9 s (highway)	Expected link lifetime, calibrates γ and W_{max}
γ (<code>--gamma</code>)	auto: $L_{link}/(2T_b \ln 2)$	Edge-freshness decay rate (Eq. 3.22)
W_{BPTT}	100	Per-node truncated-BPTT window
W_{max}	$\lceil L_{link}/T_b \rceil$	Sliding-window event retention bound
CE weight (<code>--ce_weight</code>)	0.30	Weight of the variant-classification cross-entropy term relative to BCE
margin λ	0.50	Weight of the class-separation margin loss term
margin target	0.35	Required gap between mean attack and benign scores
target validation MCC	0.975	Early-restart criterion (max. 5 restarts)
θ_{FS} (<code>--theta</code>)	auto: $\arg \max_{\theta} \text{MCC}$	Detection threshold, selected on validation data

Heuristic-score mode. When no checkpoint is loaded, `ProcessEvent()` scores each event with `HeuristicScore()`: a fixed five-term additive function combining four raw Eq. 3.20 feature terms — TTW staleness ($\text{clip}(\tau_{dev} - 1, 0, 2)$), TTW-S2 timestamp regression ($\min(1, \Delta s_v/5)$), the BSHH identity-mismatch indicator ($\iota_v \times 1.5$), and ME reporter-density excess ($\min(2, (\rho_v - 1) \times 0.8)$ when $\rho_v > 1.5$), plus an edge-staleness term ($2(1 - A_{uv}(t))$), passed through a sigmoid centred at 2.0. This mode does not consult the trained GRU embedding or the learned score/variant weights; Eq. 3.26 variant classification is disabled, and the alert’s reported variant is instead read from the ground-truth `attack_scenario` label (`alpha_source = scenario_id_fallback` in `tn.alerts.json`), which must not be interpreted as a classifier output.

Trained-model mode. When `--tn.weights` supplies a checkpoint produced by the training pipeline above, `ProcessEvent()` instead scores each event from the trained GRU/message-passing embedding (Eq. 3.25) and, only when that score exceeds θ_{FS} , computes the trained variant classification (Eq. 3.26), subject to the message-passing parity caveat noted above.

Each event increments a per-reporting-node round-count tally; once the run completes, this tally is used to retrospectively gate which events’ evidence qualifies for the trusted-evidence set (`TGN_BuildTrustedEvidence()`): Tier-1 RSU reporters qualify unconditionally ($\tau_k = 1$), while Tier-2 OBU reporters must have accumulated $R_{min} = 8$ bootstrap rounds ($\tau_k \geq \tau_{min}^{gt} = 0.50$) by run’s end. This bootstrap/trust computation is local to `tn_core.cc` and is independent of the separate node-trust bookkeeping (`g_trust_table`) that `routing.cc` itself maintains for the controller-divergence gate elsewhere in the pipeline — the two are conceptually aligned (same R_{min}, τ_{min}^{gt} derivation) but are separate, non-communicating implementations of Eq. 3.46.

Concretely, for each non-beacon event the pipeline: (i) extracts the Eq. 3.20 feature vector and the Eq. 3.22 edge-freshness weight; (ii) always updates the per-node GRU memory and runs the two-round message-passing aggregation (Eqs. 3.23–3.24); (iii) branches on whether weights are loaded, using the trained score head (Eq. 3.25) if so and `HeuristicScore()` otherwise; (iv) accumulates TP/FP/TN/FN confusion counts against the ground-truth label, split into controller-origin and behavioural-origin sub-totals; and (v) if the score exceeds θ_{FS} , computes the variant classification (Eq. 3.26) when weights are loaded, or falls back to the ground-truth `attack_scenario` label otherwise, adds the flagged vehicle to a run-time exclusion set so it is no longer accepted as a trusted verifier, issues an LKH key revocation, and — once the reporting node’s running round tally has reached $R_{min} = 8$ — dispatches the tier-appropriate mitigation primitive (`FlowMod` for Tier-1, `BlacklistBeacon` for Tier-2). Every event’s feature/score row is logged to `tn.events.csv` regardless of outcome.

At the end of the run, the trusted-evidence set $E_t^{trusted}$ (Eq. 3.46) is assembled, controller-divergence checking reclassifies controller-origin false negatives as true positives where the ledger diverges, the MCC-optimal threshold is found by sweeping every distinct observed score, and the AUROC is computed via the Mann–Whitney U statistic. The confusion matrix, threshold, AUROC, per-event

log, and alert set are written to `tgn_events.csv`, `tgn_alerts.json`, and a run summary, after which `tgn_alerts.json` is handed off to `submit_alerts.py` for on-chain revocation (Algorithm 4), which lies outside the TGN detector itself.

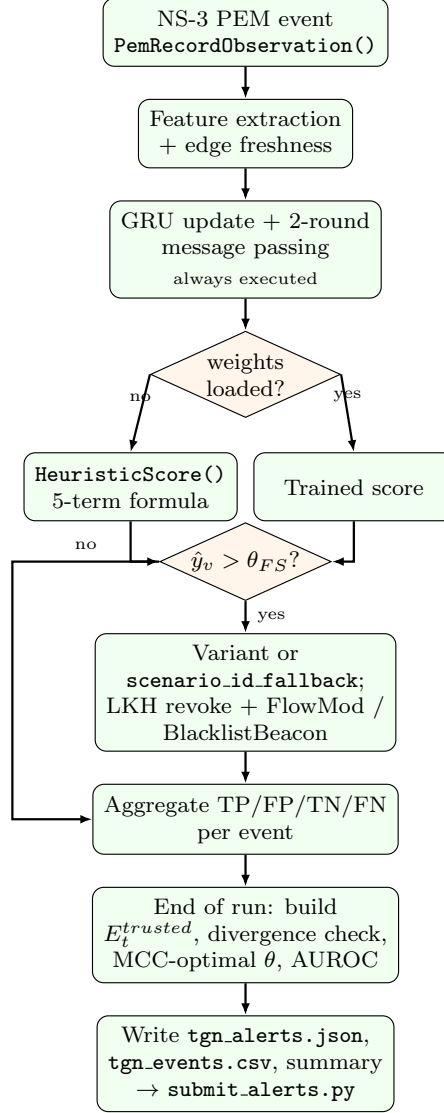


Figure 3.16: Online TGN testing / inference pipeline (`TGN_ProcessEventInline`).

3.4.5 Threshold Aggregate Signature Authentication

For RSU-aggregated beacon reports, a threshold aggregate signature ensures that no single compromised vehicle can forge a valid report. The validity condition, derived from the lattice-based scheme, is defined in Equation 3.28.

$$\text{Verify}(\sigma_{\text{agg}}, \text{PK}_{\text{agg}}) = 1 \iff |\{i : \text{Verify}(\sigma_i, \text{msg}_i, \text{PK}_{V_i}) = 1\}| \geq t, \quad t \geq \lfloor n/2 \rfloor + 1 \quad (3.28)$$

In Eq. 3.28, the left-hand side $\text{Verify}(\sigma_{\text{agg}}, \text{PK}_{\text{agg}}) = 1$ checks whether the *aggregate* signature is valid under the corresponding aggregate public key. The biconditional

$\Longleftrightarrow$ states that this passes if and only if the set on the right-hand side has cardinality at least t . The inner $\text{Verify}(\sigma_i, \text{msg}_i, \text{PK}_{V_i}) = 1$ checks the *individual* signature from vehicle V_i : σ_i is the signature, msg_i is the signed message (the topology report), and PK_{V_i} is V_i 's public key. The cardinality $|\{\cdot\}|$ counts how many individual signatures are valid. The threshold $t \geq \lfloor n/2 \rfloor + 1$ enforces a strict majority: at least one more than half of all n contributing vehicles must sign legitimately before the aggregate is accepted, preventing any colluding minority smaller than t from forging a valid report.

3.4.6 ME-Specific Mitigation: Location-Binding Signatures and Quorum Attestation

To prevent false witness attestation in the ME attack, each topology observation report is constructed by binding the observed link to the reporter's GPS-attested position and RSSI measurement before signing and that has defined in Eq. 3.29

$$m'_{V_k} = e_{ij} \parallel \text{pos}_{V_k} \parallel \text{RSSI}_{V_k \leftarrow V_i} \parallel \tau_s \parallel \text{nonce} \quad (3.29)$$

In Eq. 3.29, m'_{V_k} is the location-binding message constructed by reporter V_k before it is signed. The concatenation operator $\parallel$ joins five fields end-to-end: the observed link identifier e_{ij} ; the GPS-attested position pos_{V_k} of reporter V_k at the time of observation; the received signal strength $\text{RSSI}_{V_k \leftarrow V_i}$ measured at V_k from link end-point V_i ; the sender timestamp τ_s ; and a cryptographic nonce. By including pos_{V_k} and $\text{RSSI}_{V_k \leftarrow V_i}$ inside the signed message, any attempt to relay a legitimate report from a different position becomes detectable: the signature binds the claimed observation to a specific physical location and signal environment, which an out-of-range attacker cannot reproduce without being physically present. The digital signature produced by reporter V_k over the location-binding message is defined in 3.30

$$\sigma_{V_k} = \text{Sign}(SK_{V_k}, m'_{V_k}) \quad (3.30)$$

In Eq. 3.30, σ_{V_k} is the digital signature produced by reporter V_k over the location-binding message m'_{V_k} . $\text{Sign}(SK_{V_k}, m'_{V_k})$ applies ML-DSA-87 [43], a lattice-based post-quantum signature scheme standardised as NIST FIPS 204, using V_k 's long-term ML-DSA-87 signing key SK_{V_k} . ML-KEM-1024 + HQC-5 [64, 66] is used exclusively for session key encapsulation (K_{V_i, n_k} in Eq. 3.15); it is a Key Encapsulation Mechanism and does not produce signing keys. The two primitives serve distinct roles: ML-KEM-1024 + HQC-5 [64, 66] establishes shared session secrets for HMAC authentication, while ML-DSA-87 provides unforgeable identity-bound signatures for topology observation reports and threshold aggregation. The resulting signature σ_{V_k} proves both the identity of the reporter and the integrity of the location-bound message, enabling the controller to verify physical plausibility before accepting any topology report.

The controller accepts a single topology observation report from V_k for link e_{ij} if and only if all three conditions hold simultaneously and it is defined as Eq3.31:

$$\text{Accept}_{V_k}(e_{ij}) \iff \begin{cases} \text{Verify}(\sigma_{V_k}, PK_{V_k}) = 1 \\ d(\text{pos}_{V_k}, e_{ij}) \leq r_{\text{comm}} \\ \text{RSSI}_{V_k \leftarrow V_i} \geq \text{RSSI}_{\min}(r_{\text{comm}}) \end{cases} \quad (3.31)$$

In Eq. 3.31, $\text{Accept}_{V_k}(e_{ij})$ is a Boolean predicate: TRUE if and only if all three conditions in the cases block are satisfied simultaneously. The biconditional $\iff$ makes this an exact characterisation. The three conditions enforce three independent physical consistency requirements: (i) $\text{Verify}(\sigma_{V_k}, PK_{V_k}) = 1$ confirms cryptographic authenticity—the report was signed by a vehicle holding the private key corresponding to PK_{V_k} ; (ii) $d(\text{pos}_{V_k}, e_{ij}) \leq r_{\text{comm}}$ confirms spatial plausibility—the reporter’s GPS-attested position is within communication range of the link endpoints; (iii) $\text{RSSI}_{V_k \leftarrow V_i} \geq \text{RSSI}_{\min}(r_{\text{comm}})$ confirms signal plausibility—the measured received signal strength is consistent with the claimed proximity. An attacker operating outside r_{comm} can satisfy (i) (if it possesses valid keys) but will fail either (ii), (iii), or both, causing its report to be rejected.

To resist collusion among multiple distant attackers, link e_{ij} is accepted into the controller topology only when a quorum of at least t physically consistent witnesses independently confirm it and it is defined as in Eq3.32:

$$\text{Accept}(e_{ij}) \iff |\{V_k : \text{Accept}_{V_k}(e_{ij}) = 1\}| \geq t \quad (3.32)$$

In Eq. 3.32, $\text{Accept}(e_{ij})$ is the final Boolean decision on whether link e_{ij} is admitted into the controller’s topology. The set $\{V_k : \text{Accept}_{V_k}(e_{ij}) = 1\}$ collects every reporter that individually passed all three checks of Eq. 3.31. $|\cdot|$ counts the size of this set, and the condition $\geq t$ requires at least $t = \lfloor n/2 \rfloor + 1$ such individually accepted reports before the link is admitted—the same majority threshold as in Eq. 3.28. This quorum requirement ensures that no single distant attacker, and no colluding group smaller than t , can fabricate a link observation that passes both cryptographic verification and physical consistency checking: every colluding attacker would need to be physically co-located within r_{comm} of the link endpoints, which defeats the purpose of a false-witness echo attack.

3.4.7 Attack Signature Characterization

Effective detection of temporal-echo attacks requires that observable, measurable behavioral conditions be formally derived from the mathematical definition of each attack variant. An attack signature is defined as a necessary and sufficient logical condition on control-plane beacon or heartbeat traffic that is triggered if and only if the corresponding attack is in progress. As shown in Eq. 3.1, a non-zero divergence is the consequence; the nine signatures below are its observable antecedents.

Nine signatures are expressed as a condition on the variables defined in Section 3.1: the reception time $r(k)$, sender timestamp $s(k)$, beacon interval T_b , propagation tolerance τ_{prop} , reporter set $R(e_{ij}, t)$, path set $P(V_i, V_j)$, communication range r_{comm} , and heartbeat window W .

3.4.8 Mobility-Induced Attack Amplification in SDVNs

A defining characteristic of Software Defined Vehicular Networks (SDVNs) is the impact of vehicle mobility on network topology dynamics. Unlike wired SDN environments, where topology changes are infrequent and often suspicious, SDVNs experience continuous link formation and breakdown due to vehicle movement. Therefore, the controller must tolerate temporary uncertainty in its topology view. Temporal-echo attacks exploit this unavoidable tolerance window.

Link Lifetime and Mobility The lifetime of a communication link between two vehicles depends on their relative speed. Consider two vehicles V_i and V_j moving with relative velocity v_{rel} . The minimum duration for which the link e_{ij} remains active is bounded by Eq. 3.33:

$$L_{link} = \frac{2r_{comm}}{v_{rel}} \quad (3.33)$$

where r_{comm} represents the communication range, as shown in Eq. 3.33. At urban speeds ($v_{rel} \approx 14$ m/s), the link lifetime is approximately 43 seconds, while at highway speeds ($v_{rel} \approx 67$ m/s), it decreases to around 9 seconds. The number of beacons exchanged during this period is given by Eq. 3.34:

$$N_{beacon} = \left\lfloor \frac{L_{link}}{T_b} \right\rfloor \quad (3.34)$$

where T_b denotes the beacon interval, as expressed in Eq. 3.34. Since links naturally appear and disappear rapidly in vehicular environments, the controller continuously accepts topology changes as legitimate behavior. This creates the structural condition exploited by temporal-echo attacks.

Topology Convergence Delay After a physical link e_{ij} breaks at time t_b , the controller does not immediately learn about the failure. Instead, the update is received only after the next beacon cycle completes. The maximum divergence duration between the real topology and the controller's topology view is bounded by Eq. 3.35:

$$\tau_{conv} \leq T_b + \tau_{prop} \quad (3.35)$$

where τ_{prop} is the propagation delay between vehicles, RSUs, and the controller, as captured in Eq. 3.35. During this convergence window, it is legitimate for the controller to temporarily maintain stale topology information. The Temporal Topology Window (TTW) attack exploits this condition by replaying old beacons before the controller updates its topology state.

RSU Handover Window Vehicles frequently transition between adjacent RSU coverage zones. During this handover process, there exists a temporary interval in which neither RSU has fresh information about the vehicle. This handover window is approximated by Eq. 3.36:

$$W_{ho} \approx \frac{r_{overlap}}{v_i} \quad (3.36)$$

where $r_{overlap}$ is the overlap distance between neighboring RSUs and v_i is the vehicle speed, as defined in Eq. 3.36. Since short liveness interruptions are expected during handovers, the controller tolerates temporary communication gaps. The Beacon Suppression with Heartbeat Hijacking (BSHH) attack exploits this behavior by replaying stale heartbeat messages during the legitimate handover window.

Dynamic Observer Population The set of vehicles capable of observing a link changes continuously in SDVNs because vehicles constantly move in and out of communication range. The expected number of observers for a link e_{ij} at time t under vehicle density $\lambda(t)$ is formalized in Eq. 3.37:

$$E [|R^*(e_{ij}, t)|] = 2r_{comm} \cdot \lambda(t) \quad (3.37)$$

In dense urban environments, as quantified by Eq. 3.37, many vehicles may legitimately observe the same link, making it difficult to distinguish genuine reports from replayed echo reports. This amplifies the effectiveness of the Multi-Echo (ME) attack. Conversely, in sparse highway scenarios, even a small number of replayed reports becomes suspicious and easier to detect. Because vehicle density changes dynamically, a static detection threshold would either produce excessive false positives in urban environments or fail to detect attacks in sparse traffic conditions. Therefore, the ME-S1 signature must use mobility-aware adaptive thresholds.

Summary These mobility-induced properties explain why temporal-echo attacks are unique to SDVNs. Unlike wired SDN environments, links in SDVNs naturally appear and disappear due to physical movement, temporary liveness gaps occur during RSU handovers, and observer populations fluctuate with traffic density. Attackers therefore exploit normal mobility behavior to hide malicious replay activity within legitimate topology uncertainty.

As above explanations, signatures TTW-S1 through TTW-S2 and BSHH-S1 through BSHH-S2 are effective against vehicle-origin and RSU-origin vectors but not controller-origin vectors. Signatures BSHH-S3 and ME-S1 through ME-S3 provide the broadest coverage. The nine signatures collectively form the input to the weighted scoring function introduced in Eq. 3.12 (defined in Section 3.4.5 below), which serves as the decision boundary for the lightweight detector.

Mobility Amplification of TTW The TTW attack is directly amplified by the topology convergence delay. After a physical link breaks, the controller may

still temporarily maintain stale topology information because delayed updates are expected under normal SDVN mobility conditions. The attacker exploits this tolerance window by replaying stale beacons before topology convergence completes.

At high vehicle speeds, the link lifetime defined in Eq. 3.33 becomes shorter, causing links to appear and disappear more frequently. As a result, replayed beacons become more believable because the controller has only recently observed the link and cannot immediately distinguish between a delayed legitimate update and a replayed beacon.

TTW-S1 detects timestamp staleness using the freshness-offset condition defined in Eq. 3.2. Since replayed beacons experience larger timing offsets, the probability of detection increases with increasing relative vehicle speed v_{rel} .

TTW-S2 detects replayed beacons arriving out of temporal order using the sequence inversion condition in Eq. 3.3. This signature identifies cases where an older beacon is replayed after a newer legitimate beacon from the same source.

TTW-S3 uses the cross-reporter inconsistency condition defined in Eq. 3.4. Since legitimate observers of the same link operate within the same beacon cycle, timestamp differences larger than one beacon interval T_b indicate stale replay activity. Therefore, TTW-S3 remains mobility-aware through the beacon interval while remaining independent of absolute vehicle speed.

Mobility Amplification of BSHH The BSHH attack is amplified by the RSU handover window defined in Eq. 3.36. During transitions between adjacent RSU coverage zones, the controller legitimately expects temporary liveness gaps because fresh updates from the vehicle may not yet be available. The attacker exploits this expected interruption by replaying stale heartbeat messages during the handover period, making them appear as legitimate post-handover liveness updates.

This effect becomes more significant at high vehicle speeds because handover events occur more frequently. Although the handover duration becomes shorter, the repeated occurrence of handovers creates multiple opportunities for replay attacks.

BSHH-S1 detects structurally impossible duplicate liveness assertions using the condition defined in Eq. 3.5. Since a legitimate vehicle cannot simultaneously transmit from multiple physical senders, this signature remains mobility-independent and provides a strong detection signal.

BSHH-S2 detects replayed stale heartbeats using the timestamp regression condition in Eq. 3.6. A decreasing heartbeat timestamp indicates that an older liveness message has been replayed to maintain a false active state for the node.

BSHH-S3 detects inconsistency between heartbeat assertions and beacon activity using the condition defined in Eq. 3.7. The observation window W must be configured relative to the expected handover duration W_{ho} to avoid false positives during legitimate RSU transitions. In practice, W is selected to exceed W_{ho} , making BSHH-S3 a mobility-calibrated signature whose threshold adapts to

infrastructure geometry and vehicle speed.

Mobility Amplification of ME The ME attack is amplified by two mobility-induced effects: dynamic observer density and delayed topology reporting. According to the observer population model defined in Eq. 3.37, dense urban environments naturally contain many vehicles capable of observing the same link. This makes it more difficult to distinguish legitimate observations from malicious echo reports.

ME-S1 addresses this challenge through a dynamic reporter threshold defined in Eq. 3.8. The threshold adapts to prevailing traffic conditions: in dense urban environments it increases to reduce false positives, while in sparse highway environments it decreases to improve sensitivity to even small echo injections.

ME-S2 detects abnormal increases in controller-inferred path diversity using the condition defined in Eq. 3.9. Echo-generated topology reports artificially inflate the controller’s network graph, causing sudden increases in inferred communication paths that exceed mobility-consistent bounds.

The ME attacker also exploits delayed topology reporting caused by vehicle mobility. A vehicle that was previously within communication range may submit a delayed observation after moving away from the link endpoints. ME-S3 addresses this using the reporter-range and signal consistency condition defined in Eq. 3.11. Unlike statistical signatures, ME-S3 performs a direct physical consistency check using the reporter’s current GPS-attested position and received signal strength. Even if the vehicle was historically within range, a replayed report becomes invalid once the reporter moves beyond the communication radius, thereby closing the mobility-induced delay window exploited by the ME attacker.

Table 3.3: Summary of nine formally characterized attack signatures mapped to observable control-plane features and effective attacker positions.

Signature ID	Observable Condition	Attacker Positions Detected		
		Vehicle	RSU	Controller
TTW-S1	Link e_{ij} persists in controller topology beyond mobility-derived lifetime L_{link} : $(t - t_{\text{first_eij}}) > L_{\text{link}}$. Forged fresh timestamps artificially prolong a stale topology state beyond physical plausibility.	✓	✓	~
TTW-S2	Forged topology timestamp $\tau_s^C(e_{ij}, V_i)$ attributed to reporter V_i exceeds reception time of the most recent authentic beacon $\tau_r^{\text{last}}(V_i)$: $\tau_s^C(e_{ij}, V_i) > \tau_r^{\text{last}}(V_i)$. Particularly effective against the RSU-origin TTW vector.	✓	✓	~
TTW-S3	Two distinct reporters of the same link e_{ij} carry timestamps differing by more than one beacon interval T_b .	✓	✓	~
BSHH-S1	Two distinct physical senders submit heartbeats claiming to originate from the same node V_k in the same observation window.	✓	✓	~
BSHH-S2	Heartbeat timestamp of a given node decreases between consecutive controller-received messages, revealing stale replay.	✓	✓	~
BSHH-S3	Heartbeat asserts liveness of V_i within window W in which no beacon was received from V_i .	✓	✓	~
ME-S1	Reporter count $ R(e_{ij}, t) $ exceeds mobility-aware bound $\rho_{\text{max}}(\hat{\lambda}(t), r_{\text{comm}}) = \lfloor (1 + \mu) \cdot 2r_{\text{comm}} \cdot \hat{\lambda}(t) \rfloor$ derived from 1D road segment density.	✓	✓	~
ME-S2	Controller-inferred path count between V_i and V_j increases beyond mobility-consistent bound δ_{max} within one beacon interval.	~	~	—
ME-S3	Reporter V_k 's GPS-attested position exceeds r_{comm} from link e_{ij} , or reported RSSI is below minimum consistent with direct-range reception.	✓	✓	—

As shown in Table 3.3, nine formally characterized attack signatures spanning TTW (S1–S3), BSHH (S1–S3), and ME (S1–S3) are mapped to observable control-plane features across three attacker positions, where $\checkmark$ denotes full detection, $\sim$ partial, and $-$ not applicable. TTW and BSHH signatures are fully detectable from Vehicle and RSU positions, while ME-S2 yields only partial detection and ME-S2/ME-S3 remain inapplicable at the Controller level.

3.4.9 Detection Logic

Detection logic is provided in two versions. The lightweight version (**LW**) applies rule-based evaluation of the nine signatures and is suitable for deployment on resource-constrained OBUs and RSUs. The full-stack version (**FS**) applies a Temporal Graph Neural Network that models the sequential evolution of the beacon-based SDVN topology graph under vehicular mobility, providing superior detection performance against adaptive adversaries that may evade individual signature conditions.

Detection logic is provided in two independent versions. The lightweight version (LW) applies rule-based evaluation of the nine signatures defined in Eqs. 3.2–3.11 and computes the weighted anomaly score defined in Eq. 3.12. The full-stack version (FS) applies a Temporal Graph Neural Network that models the sequential evolution of the beacon-based SDVN topology graph, operating on the snapshot structure defined in Eq. 3.19.

Lightweight Rule-Based Detection

The LW detector maintains a sliding window W of the most recent $W_{\max}$ topology events. For each incoming event e , it evaluates all nine signatures using the conditions from Eqs. 3.2–3.11 and computes the following weighted score.

As shown in Eq. 3.12, the score accumulates evidence from all nine signatures, with weights reflecting each signature’s reliability on benign traffic. A detection alert of variant α is raised when $s(e) > \theta_{\text{LW}}$. The complete detection procedure is formalised in Algorithm 1, and the end-to-end detection and mitigation workflow is illustrated in Fig. 3.17.

The per-event time complexity of Algorithm 1 is bounded by the expression in Eq. 3.14, confirming OBU and RSU deployability within the 100 ms SDVN latency budget.

As shown in Eq. 3.14, since $|S| = 9$ is a constant determined by the attack model, the detection overhead scales linearly with the size of the window only and is independent of the network scale, which is essential for the deployability of OBU and RSU constrained by resources.

Algorithm 1 operationalises the lightweight detection logic in three coordinated routines. The main procedure **LW-DETECT** governs window maintenance and

Algorithm 1 Lightweight Rule-Based Signature Detection (LW-DETECT).

```

1: procedure LW-DETECT( $e, \mathcal{W}, w, \theta_{\text{LW}}$ )
2:   APPEND( $\mathcal{W}, e$ )
3:   if  $|\mathcal{W}| > W_{\text{max}}$  then
4:     REMOVE_OLDEST( $\mathcal{W}$ )
5:   end if
6:    $s \leftarrow 0$ 
7:   for  $k \leftarrow 1$  to 9 do
8:      $f_k \leftarrow \text{EVAL\_SIG}(k, e, \mathcal{W})$ 
9:      $s \leftarrow s + w_k \cdot f_k$ 
10:  end for
11:  if  $s > \theta_{\text{LW}}$  then
12:     $\alpha \leftarrow \text{CLASSIFY\_ATTACK}(s, e)$ 
13:    return ALERT( $\alpha, e, s$ )
14:  end if
15:  return BENIGN
16: end procedure

17: function EVAL_SIG( $k, e, \mathcal{W}$ )
18:  if  $k = 1$  then
19:    return 1  $[(t - t_{\text{first}}^{e_{ij}}) > L_{\text{link}}]$  ▷ TTW-S1: Eq. (3.2)
20:  else if  $k = 2$  then
21:    return 1  $[\tau_s^C(e_{ij}, V_i) > \tau_r^{\text{last}}(V_i)]$  ▷ TTW-S2: Eq. (3.3)
22:  else if  $k = 3$  then
23:    return 1  $[\exists j \in \mathcal{W} : |\tau_s^{(j)}(e_{ij}) - \tau_s^{(e)}(e_{ij})| > T_b]$  ▷ TTW-S3: Eq. (3.4)
24:  else if  $k = 4$  then
25:    return 1  $[\exists V_a \neq V_b : \text{Sender}(H^{V_a}) = \text{Sender}(H^{V_b})]$  ▷ BSHH-S1: Eq. (3.5)
26:  else if  $k = 5$  then
27:    return 1  $[\tau_s^{(e)}(V_i) < \tau_s^{\text{prev}}(V_i)]$  ▷ BSHH-S2: Eq. (3.6)
28:  else if  $k = 6$  then
29:    return 1  $[\text{HB}(V_i, t) \wedge \nexists \text{Beacon}(V_i, [t - W, t])]$  ▷ BSHH-S3: Eq. (3.7)
30:  else if  $k = 7$  then
31:    return 1  $[|\mathcal{R}(e_{ij}, t)| > \rho_{\text{max}}(\lambda, r_{\text{comm}})]$  ▷ ME-S1: Eq. (3.8)
32:  else if  $k = 8$  then
33:    return 1  $[|\mathcal{P}(V_i, V_j)|_t - |\mathcal{P}(V_i, V_j)|_{t-1} > \delta_{\text{max}}]$  ▷ ME-S2: Eq. (3.9)
34:  else if  $k = 9$  then
35:    return 1  $[d(\text{pos}_{V_k}, e_{ij}) > r_{\text{comm}} \vee \text{RSSI}_{V_k} < \text{RSSI}_{\text{min}}]$  ▷ ME-S3: Eq. (3.11)
36:  end if
37: end function

38: function CLASSIFY_ATTACK( $s, e$ )
39:   $\mathcal{T} \leftarrow \{k : f_k = 1, k \in \{1, 2, 3\}\}$ 
40:   $\mathcal{B} \leftarrow \{k : f_k = 1, k \in \{4, 5, 6\}\}$ 
41:   $\mathcal{M} \leftarrow \{k : f_k = 1, k \in \{7, 8, 9\}\}$ 
42:  return  $\arg \max(|\mathcal{T}|, |\mathcal{B}|, |\mathcal{M}|)$ 
43: end function

```

score accumulation: it appends each incoming topology event e to the sliding window $\mathcal{W}$, evicts the oldest entry when $|\mathcal{W}|$ exceeds $W_{\max}$, and iterates over all nine signatures to aggregate the weighted anomaly score s defined in Eq. 3.12. The helper function `EVAL_SIG` encapsulates the per-signature binary evaluation, mapping each index $k \in \{1, \dots, 9\}$ to the corresponding indicator condition from Eqs. 3.2–3.11, covering the three attack families: Topology Time-Warp (TTW, $k = 1-3$), Beacon-State Heartbeat Hijack (BSHH, $k = 4-6$), and Multipath Echo (ME, $k = 7-9$). Finally, `CLASSIFY_ATTACK` partitions the triggered signatures into their respective family sets $\mathcal{T}$, $\mathcal{B}$, and $\mathcal{M}$, returning the variant with the greatest evidence count as the attack label α .

Fig. 3.17 illustrates the lightweight (LW) detection and mitigation workflow, where cryptographic pre-filtering and signature-based anomaly scoring combine to produce a low-latency, per-event response pipeline bounded by $\mathcal{O}(|W|)$ complexity.

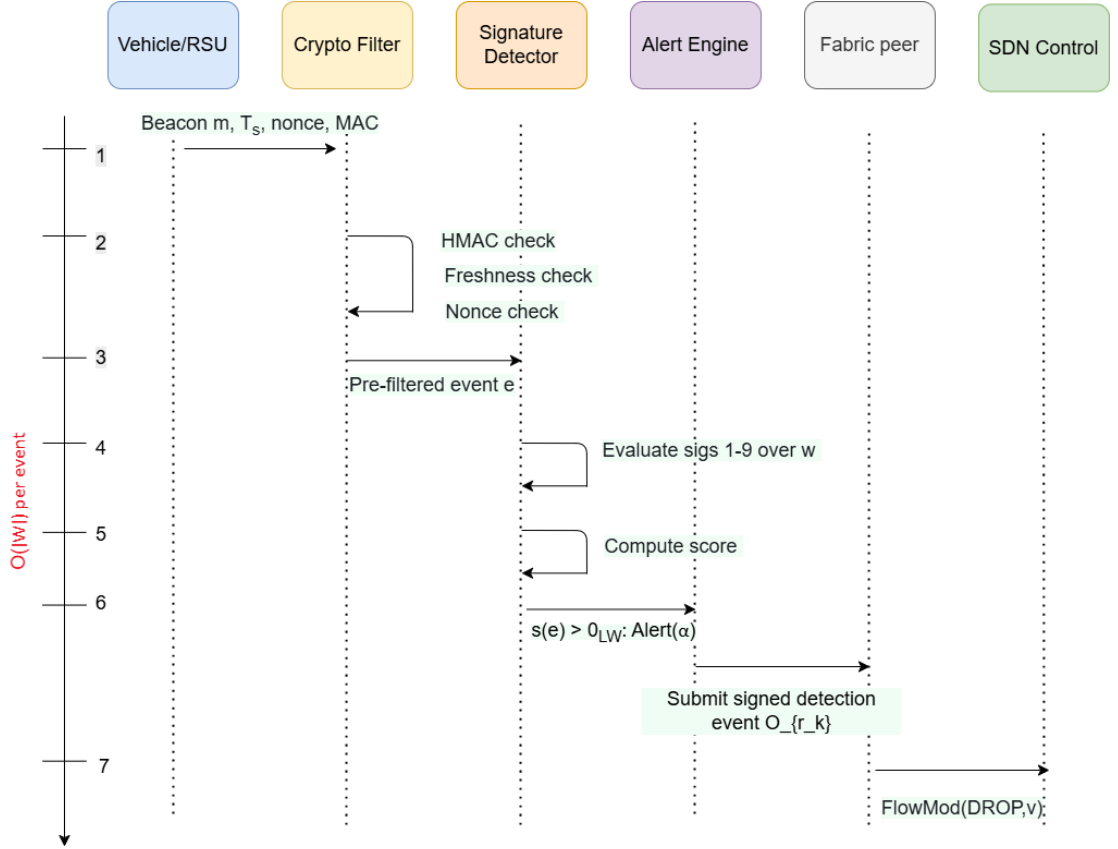


Figure 3.17: Sequence diagram of the lightweight (LW) Temporal-Echo attack detection and mitigation workflow.

As shown in Fig. 3.17, the lightweight detection and mitigation workflow proceeds in seven sequential steps: the Vehicle/RSU transmits a beacon comprising message m , timestamp T_s , nonce, and MAC to the Crypto Filter (step 1), which performs HMAC verification 3.15, freshness checking 3.16, and nonce validation 3.17 before forwarding the pre-filtered event e to the Signature Detector (steps 2–3); the Signature Detector then evaluates all nine signatures over obser-

vation window w via Algorithm 1 (step 4) and computes the anomaly score $s(e)$ using 3.12 (step 5), after which a threshold-exceeding score $s(e) > \theta_{LW}$ triggers an Alert α to the Alert Engine (step 6), which submits a signed detection event $\mathcal{O}_{r_k}$ to the Fabric peer, culminating in a `FlowMod(DROP, v)` command issued by SDN Control (step 7), with the entire per-event complexity bounded by $\mathcal{O}(|W|)$ as annotated on the vertical axis.

Full-Stack Temporal GNN Detection

The FS detector models the SDVN topology as a sequence of graph snapshots, one per beacon interval. At time t , the snapshot is defined in Eq. 3.19. The node feature vector is given in Eq. 3.20, while the freshness-aware edge weighting function is defined in Eq. 3.22.

As defined in Eqs. 3.22, the adjacency matrix entry $A_u v(t)$ down-weights stale edges using exponential decay—this directly penalizes TTW-injected edges during neighbourhood aggregation without requiring an explicit threshold. The features Δs_v and ρ_v in Eq. 3.20 encode the TTW and ME anomaly signals respectively at the node level, allowing the TGN to learn attack-specific graph evolution patterns from labeled simulation data.

The TGN processes the evolving graph sequence through three main computational stages. The temporal memory update, which amplifies the signal from nodes that have been silent for anomalously long intervals (BSHH-S3 indicator), is defined in Eq. 3.23. The neighbourhood aggregation pass is given in Eq. 3.24. The final anomaly score is computed by Eq. 3.25.

As shown in Eqs. 3.23–3.25, the TGN models the full temporal evolution of the beacon-based topology graph, unlike DMSTG-AD [37] which operates on wired SDN topologies, the proposed TGN explicitly models high-mobility SDVN dynamics, including rapid node appearance/disappearance and beacon-driven temporal topology evolution.

Edge Weight Computation. The adjacency matrix entry $\mathbf{A}_{uv}(t)$ encodes the freshness of the most recent topology report for link (u, v) using the exponential decay function in Eq. 3.22, where $\tau_s^{(uv)}$ is the most recent sender timestamp on the link report between u and v , and $\gamma > 0$ is a decay rate hyperparameter tuned on validation data. This formulation assigns unit weight to perfectly fresh reports and approaches zero for stale reports, directly penalizing TTW-injected stale-timestamped edges during neighbourhood aggregation without requiring explicit threshold tuning.

Online Operation Mode. The TGN operates in online mode, updating node memories and computing anomaly scores at each incoming topology event rather than at the end of each beacon interval. This is essential because a single anomalous beacon injected mid-interval would be masked by averaging in batch operation. Each topology event e received by the SDN controller immediately triggers a memory update and anomaly scoring pass, with the persistent memory

state $\mathbf{m}_v(t)$ maintained incrementally across events, as captured by the edge weight decay in Eq. 3.22.

New Node Initialization. When a vehicle V_{new} appears for the first time — a frequent occurrence in the high-mobility SDVN context — its initial embedding is set as defined in Eq. 3.38:

$$\mathbf{h}_{V_{\text{new}}}(t^-) = \mathbf{0} \in \mathbb{R}^d \quad (3.38)$$

As shown in Eq. 3.38, this zero initialization ensures that a newly appearing node contributes no spurious prior embedding signal, while still allowing immediate anomaly detection from its first event if it triggers any of the nine attack signatures or produces an anomalous topology observation pattern. The edge freshness weight from Eq. 3.22 further ensures that any stale reports submitted by a new node are appropriately down-weighted during aggregation.

Mobility-Aware TGN Design. Three TGN design decisions are specifically motivated by the SDVN mobility model.

First, the time-elapsed encoding $\phi(\Delta t_v) = \log(1 + \Delta t_v/T_b)$ captures the expected inter-event gap under normal vehicular mobility. A gap of exactly T_b (one beacon interval) encodes as $\phi = \log 2 \approx 0.69$, representing normal operation. A gap of L_{link} (3.33) without a subsequent beacon encodes as $\phi = \log(1 + L_{\text{link}}/T_b)$, signaling a mobility-consistent link termination. A gap that significantly exceeds L_{link} encodes as a large ϕ value that the GRU memory update learns to associate with anomalous temporal patterns, since no legitimate mobile link should produce such gaps under the prevailing speed regime.

Second, the freshness-decayed edge weight $\mathbf{A}_{uv}(t)$ (3.19) is calibrated by decay rate γ , which is set based on the expected link lifetime at the prevailing vehicle speed. Specifically, γ is chosen such that $\mathbf{A}_{uv}(t) \approx 0.5$ when $\tau_r^{(t)} - \tau_s^{(uv)} = L_{\text{link}}/2$ — the halfway point of expected link lifetime — ensuring that edges approaching natural expiry are already downweighted before they are expected to break, while edges significantly past their expected lifetime (TTW-replayed stale edges) receive near-zero weight.

Third, the sliding window size W_{max} of the lightweight detector is set to retain events from the most recent $\lceil L_{\text{link}}/T_b \rceil$ beacon intervals, ensuring that the window spans at least one full link lifetime under the prevailing speed. This prevents the window from being too short (missing cross-interval TTW sequence inversions detectable by TTW-S2) or too long (retaining events from vehicles that have long since departed, producing false positive TTW-S3 detections from legitimately stale cross-reporter timestamps). The window size is therefore a mobility-adaptive parameter that should be updated when the estimated vehicle speed changes significantly, as detected from the rate of RSU handover events.

Temporal Memory Update. Unlike static GNN methods, the Temporal Graph Network maintains a persistent memory state for each node that captures its behavioral history across beacon intervals. At each event, the memory of node v is updated as defined in Eq. 3.23, where $\mathbf{h}_v(t^-)$ is the node embedding from the immediately preceding beacon event, $\mathbf{x}_v(t)$ is the current feature vector from

Eq. 3.20, and $\phi(\Delta t_v) = \log(1 + \Delta t_v/T_b)$ is a time-elapsd encoding that scales the inter-event gap by the beacon interval T_b to account for vehicular mobility dynamics. This encoding ensures that nodes that have been silent for an unusually long interval receive a proportionally amplified temporal signal, which is directly relevant to the liveness spoofing detection in BSHH-S3.

Message Passing. Node embeddings are refined through L rounds of neighbourhood aggregation over the topology graph as defined in Eq. 3.24, where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are trainable weight matrix and bias at layer l , $\sigma(\cdot)$ denotes the ReLU activation function, $\odot$ is element-wise multiplication, and $\mathcal{N}(v, t)$ is the set of topological neighbours of v in $\mathcal{G}_t$. The edge weight $\mathbf{A}_{uv}(t)$ from Eq. 3.22 modulates the contribution of each neighbour by the freshness of the link report, causing stale-timestamped edges — characteristic of TTW attacks — to receive lower aggregation weights and propagate weaker embedding signals to the central node.

Anomaly Scoring. The anomaly score for node v at time t is computed from the final-layer embedding as defined in Eq. 3.25, where $\mathbf{w} \in \mathbb{R}^d$ is a trainable scoring vector and $\sigma(\cdot)$ here denotes the sigmoid function mapping the score to the interval $(0, 1)$. A full-stack detection alert for node v is raised when $\hat{y}_v(t) > \theta_{\text{FS}}$, where θ_{FS} is selected by maximizing the Matthews Correlation Coefficient (MCC) on held-out validation data to ensure robust performance under class imbalance between benign and attack events. The complete inference procedure is given in Algorithm 2.

The TGN is trained on labeled topology event traces generated by the NS-3/SUMO simulation framework under all twelve attack scenarios (three variants $\times$ four attacker positions). A standard 70/15/15 train/validation/test split is used. The model is trained using binary cross-entropy loss with class-weight balancing to account for the natural imbalance between benign and attack events in realistic vehicular traffic traces.

Fig. 3.18 illustrates the full-stack (FS) detection and mitigation workflow, where TGN-based temporal graph inference is integrated with PBFT consensus and smart contract execution to deliver end-to-end attack response within a 100 ms latency budget.

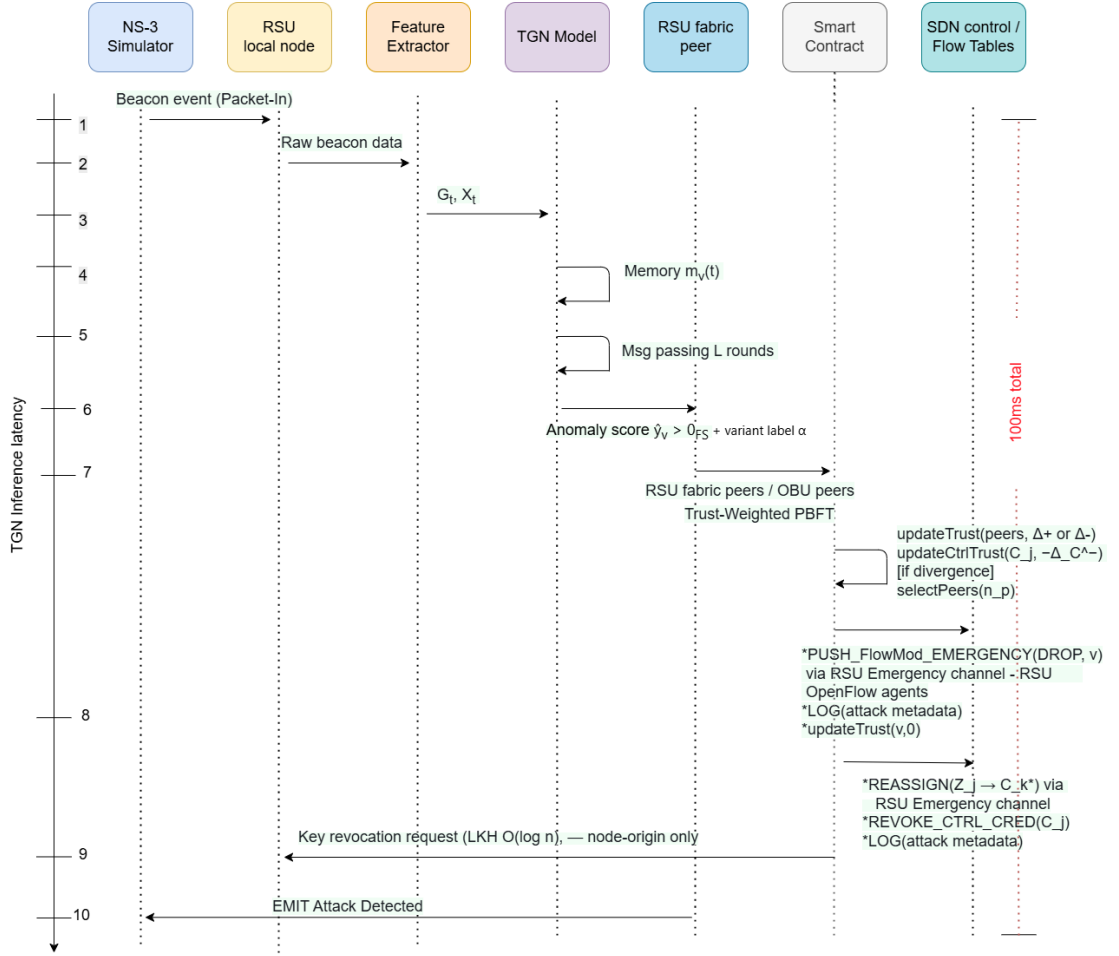


Figure 3.18: Sequence diagram of the full-stack (FS) Temporal-Echo attack detection and mitigation workflow.

As shown in Fig. 3.18, the full-stack Temporal-Echo attack detection and mitigation workflow spans ten sequential steps within a 100 ms total TGN inference latency budget: the NS-3 simulator emits a beacon Packet-In event to the RSU local node (step 1), which forwards raw beacon data to the Feature Extractor (step 2); the extractor constructs the temporal graph snapshot G_t and feature matrix X_t per Eq. 3.22 and passes them to the TGN Model (step 3), which internally updates node memory $m_v(t)$ via Eq. 3.23 (step 4) and executes L -round message passing per Eq. 3.24 (step 5) to produce an anomaly score $\hat{y}_v > \theta_{FS}$ per Eq. 3.25 (step 6); a positive detection triggers PBFT consensus across RSU fabric peers (step 7), after which the Smart Contract logs attack metadata and issues a **FlowMod(DROP, v)** command to SDN Control/Flow Tables (step 8), dispatches a key revocation request to the RSU local node (step 9), and finally broadcasts an **EMIT Attack Detected** notification (step 10).

Algorithm 2 Trusted-Node-Level Full-Stack Temporal GNN Detection (FS-DETECT).

```

1: procedure FS-DETECT( $\mathcal{G}_t^{n_k}$ ,  $\Theta$ ,  $\theta_{\text{FS}}$ ,  $n_k$ )  $\triangleright n_k$ : trusted node — RSU or designated OBU
2:    $\mathbf{X}_t \leftarrow \text{EXTRACT\_FEATURES}(\mathcal{G}_t^{n_k})$   $\triangleright$  RSU-local beacon observations only; no controller data  $\triangleright$  Eq. (3.20)
3:   for each  $v \in \mathcal{V}_t$  do
4:      $\Delta t_v \leftarrow t - t_v^{\text{last}}$ 
5:      $\phi_v \leftarrow \log(1 + \Delta t_v / T_b)$ 
6:      $\mathbf{m}_v(t) \leftarrow \text{GRU}(\mathbf{h}_v(t^-), \mathbf{x}_v(t), \phi_v)$   $\triangleright$  Eq. (3.23)
7:      $\mathbf{h}_v^{(0)} \leftarrow \mathbf{m}_v(t)$ 
8:   end for
9:   for  $l \leftarrow 0$  to  $L - 1$  do
10:    for each  $v \in \mathcal{V}_t$  do
11:       $\mathbf{h}_v^{(l+1)} \leftarrow \sigma(\mathbf{W}^{(l)} \cdot \text{MEAN}(\{\mathbf{h}_u^{(l)} \odot \mathbf{A}_{uv}(t) : u \in \mathcal{N}(v, t)\}) + \mathbf{b}^{(l)})$   $\triangleright$  Eq. (3.24)
12:    end for
13:  end for
14:   $\mathcal{A} \leftarrow \emptyset$ 
15:  for each  $v \in \mathcal{V}_t$  do
16:     $\hat{y}_v \leftarrow \sigma(\mathbf{w}^\top \mathbf{h}_v^{(L)})$   $\triangleright$  Eq. (3.25)
17:     $\hat{\alpha}_v \leftarrow \text{softmax}(\mathbf{W}_{\text{cls}} \cdot \mathbf{h}_v^{(L)})$   $\triangleright$  Eq. (3.26): variant label
18:    if  $\hat{y}_v > \theta_{\text{FS}}$  then
19:       $\alpha \leftarrow \arg \max \hat{\alpha}_v$ 
20:       $\mathcal{A} \leftarrow \mathcal{A} \cup \{(v, \hat{y}_v, \alpha)\}$ 
21:    end if
22:  end for
23:  SUBMIT_TO_FABRIC( $n_k$ ,  $\mathcal{A}$ ,  $\mathcal{B}_{n_k}(t)$ )  $\triangleright$  Trusted node submits directly to Fabric peer
24:  return  $\mathcal{A}$ 
25: end procedure

26: function EXTRACT_FEATURES( $\mathcal{G}_t$ )
27:   for each  $v \in \mathcal{V}_t$  do
28:      $\mathbf{x}_v \leftarrow [\tau_{\text{dev}}^{(v)}, c_v^W, \Delta s_v, \rho_v, \iota_v]$   $\triangleright$  Eq. (3.20):  $\tau_{\text{dev}}^{(v)} = \text{clip}((\tau_r^{(v)} - \tau_s^{(v)})/T_b, -50, 50)$ ;
       id_v excluded (identity overfitting)
29:      $t_v^{\text{last}} \leftarrow \tau_r^{(v)}$ 
30:   end for
31:   return  $\mathbf{X}_t$ 
32: end function

```

Algorithm 2 formalises the full-stack inference pipeline executed at a trusted node n_k (an RSU or a designated OBU) and comprises two coordinated routines. The main procedure **FS-DETECT** operates over three successive stages. First, node features are extracted exclusively from RSU-local beacon observations via **EXTRACT_FEATURES**, producing the feature matrix $\mathbf{X}_t$ as defined in Eq. 3.20. Second, a temporal memory update is performed for each node $v \in \mathcal{V}_t$: the time-elapsing encoding $\phi_v = \log(1 + \Delta t_v / T_b)$ amplifies the inter-event gap relative to the beacon interval T_b , and the GRU cell fuses this signal with the prior embedding $\mathbf{h}_v(t^-)$ and the current feature vector $\mathbf{x}_v(t)$ to produce the updated memory $\mathbf{m}_v(t)$, which initialises the first-layer embedding $\mathbf{h}_v^{(0)}$. Third, L rounds of neighbourhood aggregation refine each node’s embedding by accumulating freshness-weighted neighbour signals via Eq. 3.22, where the edge weight $\mathbf{A}_{uv}(t)$ suppresses the contribution of stale-timestamped links characteristic of TTW attacks. An anomaly score $\hat{y}_v \in (0, 1)$ is then derived from the final-layer embedding (Eq. 3.25),

and any node exceeding threshold θ_{FS} is appended to the alert set $\mathcal{A}$, which is submitted directly to the Hyperledger Fabric peer via `SUBMIT_TO_FABRIC`. The helper function `EXTRACT_FEATURES` constructs the five-dimensional feature vector $\mathbf{x}_v = [\tau_{\text{dev}}^{(v)}, c_v^W, \Delta s_v, \rho_v, \iota_v]$, where $\tau_{\text{dev}}^{(v)} = \text{clip}((\tau_r^{(v)} - \tau_s^{(v)})/T_b, -50, 50)$ is the timestamp age deviation and the node-ID embedding id_v is excluded to prevent the model learning identity-based rather than behaviour-based attack signatures (Eq. 3.20). The sequence gap Δs_v and reporter count ρ_v directly encode the TTW and ME anomaly signals; the mismatch indicator ι_v encodes the BSHH signal; together they allow the TGN to learn attack-specific graph evolution patterns from labelled simulation data.

3.4.10 Cryptographic Placement Analysis

This section formally determines, for each combination of attack variant and attacker position, whether cryptographic enforcement eliminates or merely reduces the attack surface. The central placement principle is that a cryptographic primitive provides *pre-detection* value if and only if the attack cannot be executed by a node that successfully passes the cryptographic verification condition. If an attacker possesses valid credentials and can still perform the attack while producing valid authenticated messages, then the cryptographic primitive provides no pre-detection value and must instead be applied post-detection together with TGN-based behavioral analysis.

Three cryptographic mechanisms are considered. HMAC-timestamp-nonce verification, defined in Eqs. 3.15–3.17, provides message authentication, freshness validation, and replay protection. Threshold aggregate signature verification, defined in Eq. 3.28, provides quorum-based identity authentication for cooperative beacon reporting. In addition, location-binding verification using the physical consistency conditions in Eq. 3.11 binds topology observations to GPS-attested location and RSSI evidence.

For TTW attacks, the vehicle-origin and RSU-origin vectors are effectively prevented at the pre-detection stage using the HMAC-timestamp-nonce triple defined in Eqs. 3.15–3.17. A replayed beacon fails because the nonce has already been consumed and the freshness condition in Eq. 3.16 is violated. Therefore, these vectors are eliminated before reaching the TGN detector. However, for controller-origin TTW attacks, the malicious controller already possesses valid keys and can generate correctly authenticated messages internally. In this case, cryptographic verification provides no pre-detection protection, and TGN-based temporal anomaly detection becomes the primary defense mechanism.

For BSHH attacks, the vehicle-origin and RSU-origin vectors are mitigated at the pre-detection stage through the threshold aggregate signature condition defined in Eq. 3.28. An attacker cannot impersonate another node without controlling at least the required threshold number of participants. Thus, external identity spoofing is cryptographically prevented before topology events enter the detection pipeline. However, in the controller-origin BSHH vector, the controller manipulates the topology state internally while already holding valid credentials.

Consequently, no message-layer authentication check can prevent the attack, and post-detection behavioral analysis using the TGN and the liveness inconsistency condition in Eq. 3.7 becomes necessary.

The ME attack differs fundamentally from TTW and BSHH because the attacker does not forge identities or modify message contents. Instead, the attacker submits correctly authenticated but physically inconsistent topology observations. Consequently, HMAC and threshold signatures provide limited pre-detection value because the attacker still passes standard authentication checks. The relevant protection mechanism is the location-binding and signal-consistency verification defined in Eq. 3.11. For vehicle-origin ME attacks, physically inconsistent reporters fail the range or RSSI validation conditions and can therefore be filtered before full-stack analysis. For RSU-origin ME attacks, colluding infrastructure nodes may still pass individual verification checks, requiring both cryptographic verification and TGN-based cross-observer behavioral analysis. In the controller-origin ME vector, the controller directly injects false topology observations into the graph state, making external cryptographic validation impossible. Therefore, detection relies primarily on TGN-based topology anomaly analysis together with the dynamic reporter consistency checks defined in Eq. 3.8.

Overall, the placement analysis shows that cryptographic enforcement is highly effective for preventing external replay and impersonation attacks, but provides limited protection against insider and controller-origin attacks where the attacker already possesses valid credentials. Consequently, the proposed TETA-Guard framework adopts a hybrid architecture in which cryptographic verification acts as a lightweight pre-detection filter, while TGN-based behavioral analysis provides post-detection protection against insider and topology-level attacks.

Table 3.4: Cryptographic placement analysis per attack variant and attacker position.

Variants	Vehicle	RSU	Controller
TTW	Pre-detection – Eliminates	Pre-detection – Eliminates	Post-detection only
	HMAC-timestamp-nonce	HMAC-timestamp-nonce	<i>No pre-detection value</i>
	Freshness + nonce cache blocks replay.	Consumed-nonce + staleness check prevents stale topology forwarding.	Valid key bypasses crypto filter.
	ML-KEM-1024 + HQC-5 [64, 66] establishes session key K_{V_i, n_k} .	ML-KEM-1024 + HQC-5 [64, 66].	TGN temporal anomaly is primary. FlowMod DROP + key revocation post-alert.
BSHH	Pre-detection – Eliminates	Pre-detection – Eliminates	Post-detection only
	Threshold aggregate signature	Threshold aggregate signature	<i>No message-layer auth applies</i>
	Identity forgery fails threshold check ($t \geq \lfloor n/2 \rfloor + 1$).	Sender spoofing breaks threshold signing.	The controller writes directly to the topology state, bypassing signatures.
	ML-DSA-87: threshold aggregate signing key $SK_{\{V_k\}}$. ML-KEM-1024 + HQC-5 [64, 66]: session key $K_{\{V_i, n_k\}}$ (KEM only)	RSU key hierarchy enforces integrity. ML-DSA-87: threshold aggregate signing key $SK_{\{V_k\}}$. ML-KEM-1024 + HQC-5 [64, 66]: session key $K_{\{V_i, n_k\}}$ (KEM only)	TGN is primary. Smart contract isolation post-alert.
ME	Hybrid – Partial prevention	Hybrid – Partial prevention	Post-detection only
	Location-binding signature	Location-binding signature	<i>No external measurement applies</i>
	Eliminates s1/s3 where position is verifiable via GPS+RSSI. TGN required for S2.	RSU position evidence filters false witness reports (s1/s3). Standard HMAC is ineffective.	Controller injects false observations – no GPS/RSSI verifiable.
	ML-DSA-87: threshold aggregate signing key $SK_{\{V_k\}}$. ML-KEM-1024 + HQC-5 [64, 66]: session key $K_{\{V_i, n_k\}}$ (KEM only)	TGN supplements S2. ML-DSA-87: threshold aggregate signing key $SK_{\{V_k\}}$. ML-KEM-1024 + HQC-5 [64, 66]: session key $K_{\{V_i, n_k\}}$ (KEM only)	TGN scoring (S1/S2/S3) sole detection path. FlowMod reroute via smart contract.

3.4.11 Mitigation Algorithms

The full-stack detection and mitigation architecture is designed under the explicit assumption that the SDN controller may itself be a malicious attacker. This assumption invalidates any architecture in which the blockchain is driven exclusively by controller-produced data — a malicious controller can fabricate anomaly scores,

suppress genuine detections, or selectively log events to produce a false audit trail. The correct architectural response is to ground the blockchain in *RSU-level evidence* collected independently of and verifiable against the controller's topology claims.

Let $\mathcal{N} = \{n_1, n_2, \dots, n_m\}$ denote the set of trusted infrastructure nodes in the Hyperledger Fabric consortium, organised into two tiers. *Tier 1 peers* are fixed infrastructure nodes — RSUs in standard deployments, or permanent edge servers at highway interchanges in RSU-absent deployments — running full Fabric peer software with complete ledger replication. *Tier 2 peers* are designated OBUs active exclusively in no-RSU scenarios, running Fabric in native Go binary mode without containerisation and maintaining a checkpoint-based ledger of the $K = \lceil T_{\text{dwell}}/T_b \rceil$ most recent blocks, where T_{dwell} is the OBU's current dwell time. To preserve PBFT agreement across peers with heterogeneous windows, every Tier-1 peer periodically produces an *anchor checkpoint* — a PBFT-signed ledger digest committing the global state root at a fixed block height — and publishes it to all consortium peers. Before a Tier-2 OBU peer participates in any consensus round, it must synchronise from the most recent anchor checkpoint, ensuring all active peers validate transactions from an identical committed base state regardless of their individual window sizes. The anchor checkpoint interval is set to $\lfloor T_{\text{min}}/T_b \rfloor$ blocks, guaranteeing synchronisation before any newly promoted peer joins consensus.

All remaining OBUs operate as Fabric *clients* only, submitting signed beacon data and detection events without joining consensus or storing the ledger. Tier 2 peers are promoted from client OBUs only when no Tier 1 peers are reachable and at least $n_p = 3f + 1$ eligible OBUs are available.

In deployments where Tier 1 infrastructure is entirely absent - as opposed to transiently unreachable - every slot in $\mathcal{P}_{\text{active}}$ is filled by appointed Tier 2 OBUs rather than by RSUs or edge servers. Concretely, all n_p active peer (endorser) slots are populated from the pool of registered, eligible OBUs; any eligible OBUs in excess of n_p remain Fabric clients, since $n_p = 3f + 1$ acts as a hard admission cap rather than a minimum-availability gate. For example, with 15 registered OBUs and $n_p = 8$, the eight highest-trust OBUs are appointed as active peers and the remaining seven are excluded by the cap alone, independent of their individual eligibility.

The SDVN deployment operates under a multi-controller architecture in which the network is partitioned into geographic coverage zones, each assigned a primary controller and at least one backup controller. The Fabric consortium maintains a controller registry on the ledger, defined in Eq. 3.39:

$$\mathcal{C} = \{(C_j, \tau_{C_j}, \mathcal{Z}_j) : j = 1, \dots, p\} \quad (3.39)$$

In Eq. 3.39, C_j is controller j , $\tau_{C_j} \in [0, 1]$ is its ledger-stored trust score, and $\mathcal{Z}_j$ is its currently assigned subnetwork zone. Each RSU in the deployment maintains a *direct emergency channel* - a persistent TLS connection to its local Fabric peer - through which the smart contract can issue FlowMod rules directly to RSU OpenFlow agents, bypassing the controller entirely. This channel is used for all iso-

lation and reassignment commands so that a malicious controller cannot intercept or suppress them.

Each node n_k and controller C_j carries a trust score (τ_k and τ_{C_j} respectively) stored on the Fabric ledger and updated by the **TemporalEchoMitigator** smart contract. Tier 1 nodes and correctly-behaving controllers are initialised with trust 1; newly admitted Tier 2 OBUs receive $\tau_0 = 0.1$. In pure no-RSU deployments where all peers start at $\tau_k = \tau_0 = 0.1 < \tau_{\min}^{\text{gt}}$, no node immediately satisfies the trusted evidence threshold and $\mathcal{E}_t^{\text{trusted}} = \emptyset$, making controller-origin divergence detection blind at startup. A *bootstrap phase* of $R_{\min}$ consensus rounds is therefore enforced before normal trust thresholds apply. During the bootstrap phase, all eligible peers with $\tau_k \geq \tau_{\min}$ contribute beacon evidence regardless of $\tau_{\min}^{\text{gt}}$, and all detections are flagged as *preliminary* pending full trust accumulation. The bootstrap ends when at least $n_p = 3f + 1$ peers reach $\tau_k \geq \tau_{\min}^{\text{gt}}$, at which point normal threshold enforcement resumes. The minimum bootstrap duration is $R_{\min} = \lceil (\tau_{\min}^{\text{gt}} - \tau_0) / \Delta_+ \rceil$ rounds, giving $R_{\min} = \lceil (0.5 - 0.1) / 0.05 \rceil = 8$ rounds for recommended parameters — a startup latency of $8 \times T_{\text{PBFT}} \ll T_{\min}$. The node trust update rule is defined in Eq. 3.40:

$$\tau_k^{(t+1)} = \begin{cases} \min(1, \tau_k^{(t)} + \Delta_+), & \text{correct participation} \\ \max(0, \tau_k^{(t)} - \Delta_-), & \text{failure or inconsistent evidence} \\ 0, & \text{flagged by LW or FS detector (attacker)} \end{cases} \quad (3.40)$$

In Eq. 3.40, $\Delta_+ < \Delta_-$ (recommended $\Delta_+ = 0.05$, $\Delta_- = 0.10$) makes trust harder to gain than to lose; a detection flag immediately zeros τ_k , permanently excluding the attacker from peer eligibility. Controller trust τ_{C_j} follows a *penalty-only* update, defined in Eq. 3.41:

$$\tau_{C_j}^{(t+1)} = \max(0, \tau_{C_j}^{(t)} - \Delta_-^C) \quad \text{on each confirmed divergence detection} \quad (3.41)$$

In Eq. 3.41, there is no reward mechanism for controllers — trust only decreases — reflecting the higher-stakes consequence of controller compromise. Both node and controller trust scores are tamper-evident and consensus-driven; no entity can inflate its own score.

A Tier 2 OBU V_k is eligible for peer promotion if and only if all five conditions hold, as formalised in Eq. 3.42:

$$\text{Eligible}(V_k) \iff \left\{ \begin{array}{ll} \text{Cert}(V_k) \in \text{CA}_{\text{consortium}} & \text{valid consortium} \\ C_{V_k} \geq C_{\min} & \text{certificate} \\ & \text{hardware} \\ & \text{capacity} \\ T_{\text{dwell}}(V_k) \geq T_{\min} & \text{minimum} \\ & \text{dwell time} \\ \tau_k \geq \tau_{\min} & \text{trust threshold} \\ & \text{(primary criterion)} \\ V_k \notin \mathcal{F}_{\text{flagged}} & \text{no detection flag} \\ & \text{this session} \end{array} \right. \quad (3.42)$$

In Eq. 3.42, $C_{\min}$ is the hardware floor (ARM Cortex-A53 class, ≥ 2 GB RAM, ≥ 8 GB storage); $T_{\min} = \max(3 \cdot T_{\text{PBFT}}, L_{\text{link}}/2)$ guarantees at least three complete consensus rounds before possible departure; $\tau_{\min}$ is the admission trust threshold; and $\mathcal{F}_{\text{flagged}}$ is the set of nodes flagged as attackers this session. Trust is the primary criterion because it encodes a cumulative behavioral history rather than an instantaneous presence or capability.

In pure no-RSU deployments, conditions 2 and 3 (hardware floor $C_{\min}$ and minimum dwell time $T_{\min}$) are bypassed for appointed OBUs, since no Tier 1 checkpoint infrastructure exists against which dwell-adequacy or hardware parity would otherwise be benchmarked. Certificate validity (condition 1) and the trust threshold (condition 4) remain mandatory and become the *sole* effective admission filters in this mode, consistent with the bootstrap-phase treatment of uniformly low initial trust τ_0 described above.

Nodes in the QUARANTINED_CLIENT or REMOVED states carry $\tau_k = 0$ from the Stage 1 demotion action of Section 3.4.12, so they already fail condition 4 ($\tau_k \geq \tau_{\min}$) above; the explicit state_k filter in the eligibility set of Section 3.4.12 is therefore a confirmatory defensive check, not an independent exclusion mechanism.

The active peer set of size $n_p = 3f + 1$ is formed by trust-ranked nomination, as defined in Eq. 3.43:

$$\mathcal{P}_{\text{active}} = \arg \max_{\substack{|\mathcal{P}|=n_p, \\ V_k \in \mathcal{P} \Rightarrow \text{Eligible}(V_k)}} \sum_{k \in \mathcal{P}} \tau_k \quad (3.43)$$

In Eq. 3.43, selecting the highest aggregate trust directly excludes compromised OBUs, which carry low or zeroed trust scores and rank below established peers. When active peers drop below $3f + 1$ due to mobility, the highest-ranked eligible client OBU is promoted after checkpoint synchronization, bounded by $T_{\text{recruit}} \leq T_{\min}$.

In the pure no-RSU case, Eq. 3.43 is evaluated over the full registered OBUs pool from system initialization — not incrementally as peers drop out — so $\mathcal{P}_{\text{active}}$ is populated entirely by appointed vehicles from the outset, with the n_p cap, rather than eligibility failure, determining exclusion for any remaining registered OBUs.

When controller trust falls below the controller-specific threshold $\tau_{\min}^C$, the smart contract triggers subnetwork reassignment. The reassignment condition is defined in Eq. 3.44:

$$\tau_{C_j} < \tau_{\min}^C \implies \text{REASSIGN}(\mathcal{Z}_j \rightarrow C_k^*) \quad (3.44)$$

The backup controller is selected as the highest-trust available controller other than C_j , as defined in Eq. 3.45:

$$C_k^* = \arg \max_{\substack{k \neq j, \\ \tau_{C_k} > \tau_{\min}^C}} \tau_{C_k} \quad (3.45)$$

In Eqs. 3.44–3.45, the REASSIGN command is issued by the smart contract directly to the RSU Fabric peers in zone $\mathcal{Z}_j$ via the emergency channel — not through

the malicious controller. Upon receiving this command, RSU OpenFlow agents switch their southbound connection from C_j to C_k^* ; C_j 's OpenFlow credentials are revoked via the consortium CA; and C_k^* receives a full topology snapshot of $\mathcal{Z}_j$ from the RSU beacon evidence on the ledger, assuming management of the zone immediately.

In Tier 2 (no-RSU) mode, where there are no RSU OpenFlow agents to redirect, the controller removal reduces to three actions executed by OBU Fabric peers via the Fabric ledger: (i) C_j 's CA certificate is revoked, preventing it from re-establishing V2X sessions with any vehicle; (ii) a signed CONTROLLERREVOKED-BEACON is distributed via OBU peers to all vehicles in $\mathcal{Z}_j$, causing them to cease submitting topology updates to C_j ; (iii) C_k^* is notified via the Fabric ledger and begins soliciting topology observations directly from vehicles in $\mathcal{Z}_j$ through its own V2X interface. Routing in $\mathcal{Z}_j$ temporarily falls back to distributed vehicle-level decisions until C_k^* accumulates sufficient topology state, bounded by $\lceil L_{\text{link}}/T_b \rceil$ beacon intervals per Eq. 3.33.

Three independent data flows feed the Fabric network, none of which passes through the controller:

1. **RSU Beacon Evidence Log.** At each beacon interval, each RSU r_k submits a signed beacon evidence record $\mathcal{B}_{r_k}(t)$ directly to its local Fabric peer: $\mathcal{B}_{r_k}(t) = \{(v_{\text{id}}, \tau_s, \text{pos}_v, \text{RSSI}_v, \sigma_{n_k}) : v \in \mathcal{V}_{n_k}(t)\}$ where $\mathcal{V}_{r_k}(t)$ is the set of vehicles from which r_k received beacons in interval t and σ_{r_k} is the RSU's own signature over the record. This establishes a tamper-evident ground truth of what each RSU directly observed, independent of the controller. In Tier 1 deployments, RSU-signed beacon records (all $\tau_k = 1$) provide full-strength ground truth. In Tier 2 (no-RSU) deployments, ground truth is derived from OBU peers exceeding a stricter threshold $\tau_{\min}^{\text{gt}} > \tau_{\min}$, as defined in Eq. 3.46:

$$\mathcal{E}_t^{\text{trusted}} = \bigcup_{\substack{n_k \in \mathcal{P}_{\text{active}} \\ \tau_k \geq \tau_{\min}^{\text{gt}}}} \mathcal{B}_{n_k}(t) \quad (3.46)$$

In Eq. 3.46, only peers with demonstrated multi-round reliability contribute; coordinating false evidence across multiple high-trust nodes would simultaneously degrade all their scores via Eq. 3.40 and trigger detection before consensus could pass. The controller topology divergence check is redefined in Eq. 3.47:

$$\delta = |\mathcal{E}_t^C \triangle \mathcal{E}_t^{\text{trusted}}| \quad (3.47)$$

In Eq. 3.47, $\mathcal{E}_t^C$ is the controller-claimed edge set and $\mathcal{E}_t^{\text{trusted}}$ follows from Eq. 3.46. In Tier 1 mode all $\tau_k = 1$, so Eq. 3.46 reduces to full RSU evidence; in Tier 2 mode it uses the high-trust OBU subset. A divergence $\delta > \delta_{\text{thresh}}$ flags a controller-origin attack and triggers Eq. 3.41. The threshold separates legitimate propagation-delay-induced divergence from malicious manipulation and is defined in Eq. 3.48:

$$\delta_{\text{thresh}} = \left\lceil \left(1 + \frac{\tau_{\text{prop}}}{T_b}\right) \cdot \lambda \cdot 2r_{\text{comm}} \right\rceil + 1 \quad (3.48)$$

In Eq. 3.48, $(1 + \tau_{\text{prop}}/T_b) \cdot \lambda \cdot 2r_{\text{comm}}$ bounds the number of links legitimately in transit between RSU evidence submission and controller topology update during propagation delay τ_{prop} . The +1 margin absorbs single-link transient divergence. With $\lambda = 0.02 \text{ veh/m}$, $r_{\text{comm}} = 300 \text{ m}$, and $\tau_{\text{prop}} \approx T_b/10$: $\delta_{\text{thresh}} = \lceil 1.1 \cdot 12 \rceil + 1 = 14$ for both urban and highway conditions.

2. **RSU-Level LW Detection Events.** Each RSU runs Algorithm 1 locally on beacons received directly from vehicles. When signatures trigger, the RSU submits a signed detection event $\mathcal{O}_{r_k} = (v_{\text{id}}, \alpha, s(e), \mathcal{S}_{\text{trig}}, t_{\text{alert}})$ from 3.52 directly to its Fabric peer, without routing through the controller.
3. **Controller Topology Claim.** The controller submits its current topology view G_t^C to the Fabric network. The smart contract compares G_t^C against the RSU beacon evidence $\{\mathcal{B}_{r_k}(t)\}$ aggregated across all RSU peers. Divergence exceeding the attack divergence bound 3.1 triggers a controller-origin attack alert without relying on the controller's own anomaly reporting.

RSU-Level TGN Inference. The Temporal GNN defined above runs at each RSU independently, constructing its local graph snapshot $\mathcal{G}_t^{r_k}$ from beacons directly received. RSU-level TGN anomaly scores are submitted to the Fabric peer alongside the LW detection events. The controller does not participate in TGN inference and cannot influence its outputs.

PBFT Consensus and Smart Contract Execution. Detection events from $\mathcal{P}_{\text{active}}$ are confirmed under trust-weighted PBFT consensus, where each peer's vote is weighted by τ_k rather than counted equally. The acceptance condition is defined in Eq. 3.49:

$$\text{Accept} \iff \frac{\sum_{k \in \mathcal{P}_{\text{approve}}} \tau_k}{\sum_{k \in \mathcal{P}_{\text{active}}} \tau_k} > \frac{2}{3} \quad (3.49)$$

In Eq. 3.49, $\mathcal{P}_{\text{approve}} \subseteq \mathcal{P}_{\text{active}}$ is the set of approving peers; the threshold $\frac{2}{3}$ preserves the standard PBFT safety margin. Trust scores are read from the ledger at the start of each round, so zero-scored peers contribute no voting weight immediately upon detection. A confirmed consensus triggers the **TemporalEchoMitigator** smart contract, which executes variant-specific mitigation defined in Algorithm 4.

Safety of Trust-Weighted PBFT. Classical PBFT safety rests on node counts: with $n_p = 3f + 1$ nodes and at most f Byzantine, the $2f + 1$ honest nodes always form a majority. Under trust weighting, the equivalent condition is that Byzantine aggregate trust must remain below $\frac{1}{3}$ of total trust. With f Byzantine peers each holding trust τ_B and $2f + 1$ honest peers each holding trust τ_H , safety holds when:

$$\frac{f \cdot \tau_B}{f \cdot \tau_B + (2f + 1) \cdot \tau_H} < \frac{1}{3} \iff \frac{\tau_B}{\tau_H} < \frac{2f + 1}{2f} \quad (3.50)$$

In Eq. 3.50, the right-hand side exceeds 1 for all $f \geq 1$, so the condition reduces to $\tau_B < \tau_H$. Since Byzantine peers are admitted at most at $\tau_{\text{min}} \leq 1$ and Tier 1

honest peers hold $\tau_H = 1$, the safety inequality is satisfied at initialisation by construction. Over time, Byzantine peers can accumulate trust at rate Δ_+ per round, but any detectable misbehaviour immediately zeros τ_B via Eq. 3.40, resetting their aggregate weight to zero. The safety condition therefore holds throughout the session provided $\tau_{\min} < 1$, which is enforced by the eligibility predicate in Eq. 3.42.

Lightweight Mitigation: HMAC-Timestamp-Nonce Verification

Each beacon or topology update message m from node V_i at sender timestamp τ_s is constructed as Eq. 3.51:

$$m' = m \parallel \tau_s \parallel \text{nonce}_i \quad (3.51)$$

As shown in Eq. 3.51, the message is authenticated using a session key $K_{V_i, \text{RSU}}$ established via the ML-KEM-1024 + HQC-5 hybrid key encapsulation mechanism [64, 66].

The verifier (RSU or controller) accepts the message if and only if all three conditions hold simultaneously. First, the MAC must be valid as defined in Eq. 3.15. Second, the message must be temporally fresh as bounded by Eq. 3.16. Third, the nonce must not have been previously observed as checked by Eq. 3.17, where $\mathcal{N}_{\text{seen}}$ is a per-sender nonce cache maintained at the verifier.

Messages failing any of the conditions in Eqs. 3.15–3.17 are silently dropped before reaching the signature detector or TGN inference engine, eliminating the TTW and BSHH external attack vectors at the filter layer. The per-message overhead of HMAC-SHA256 is approximately 1–2 μs on ARM Cortex-M class OBU hardware [29], which is less than 2% of the 100 ms beacon interval and confirms lightweight deployability. Session key establishment uses the ML-KEM-1024 + HQC-5 [64, 66] hybrid key encapsulation mechanism, which achieves millisecond-level handshake latency compliant with the 10 ms V2X timing constraint. The complete lightweight mitigation procedure is given in Algorithm 3.

Algorithm 3 Lightweight HMAC-Timestamp-Nonce Mitigation (LW-MITIGATE).

```

1: procedure LW-MITIGATE( $m, \tau_s, \tau_r, \text{nonce}, K_{V_i}, \mathcal{N}_{\text{seen}}$ )
2:    $m' \leftarrow m \parallel \tau_s \parallel \text{nonce}$  ▷ Eq. (3.51)
3:    $\text{MAC}_{\text{exp}} \leftarrow \text{HMAC-SHA256}(K_{V_i}, m')$  ▷ Eq. (3.15)
4:   if  $\text{MAC}_{\text{exp}} \neq \text{MAC}^{\text{received}}$  then ▷ Eq. (3.15)
5:     return DROP(invalid MAC)
6:   end if
7:   if  $|\tau_r - \tau_s| > T_b + \epsilon$  then ▷ Eq. (3.16)
8:     return DROP(stale message)
9:   end if
10:  if  $\text{nonce} \in \mathcal{N}_{\text{seen}}$  then ▷ Eq. (3.17)
11:    return DROP(replayed nonce)
12:  end if
13:  ADD( $\mathcal{N}_{\text{seen}}, \text{nonce}$ )
14:  return ACCEPT( $m$ )
15: end procedure

```

Algorithm 3 formalises the lightweight pre-detection filter that intercepts every incoming beacon or topology update message m from node V_i before it can reach either the signature detector or the TGN inference engine. The procedure enforces three independent conditions in strict sequential order, dropping the message at the first failure and incurring no further overhead. First, the verifier reconstructs the authenticated payload $m' = m \parallel \tau_s \parallel \text{nonce}$ (Eq. 3.51) and recomputes the expected MAC using the ML-KEM-1024 + HQC-5 [64, 66]-established session key K_{V_i} . A MAC mismatch immediately signals forgery or tampering and the message is silently dropped (Eq. 3.15). Second, a temporal freshness check $|\tau_r - \tau_s| \leq T_b + \epsilon$ rejects messages whose reception-to-sender timestamp offset exceeds one beacon interval plus propagation tolerance, directly neutralising the TTW external injection vector. Third, a nonce lookup against the per-sender cache $\mathcal{N}_{\text{seen}}$ (Eq. 3.17) detects and discards replayed messages, closing the BSHH replay pathway. Only messages passing all three checks are admitted to $\mathcal{N}_{\text{seen}}$ and forwarded for further processing. The per-message overhead of HMAC-SHA256 is approximately 1–2 μs on ARM Cortex-M class OBU hardware, well under 2% of the 100 ms beacon interval and confirming deployability within the SDVN latency budget.

Full-Stack Mitigation: Threshold Aggregate Signatures, Location-Binding Signatures, and Smart Contract Enforcement

Threshold Aggregate Signature Verification. For RSU-aggregated topology reports, each vehicle V_i signs its individual beacon observation under its private key. The RSU collects n such signatures and constructs a threshold aggregate, defined in Eq. 3.28, valid only if at least $t \geq \lfloor n/2 \rfloor + 1$ individual signatures are verified. The threshold aggregate signature is constructed using ML-DSA-87 [43], which provides post-quantum security under the Module-LWE hardness assumption. Per-verification latency is sub-millisecond on modern hardware, well within the 100 ms SDVN latency budget. The signing key SK_{V_k} is provisioned via the consortium CA at vehicle registration and is distinct from the session key K_{V_i, n_k} established by ML-KEM-1024 + HQC-5 [64, 66] hybrid key encapsulation mechanism. Per-verification latency is approximately 1.6 ms [30], which is within the SDVN latency budget. As shown in Eq. 3.28, this construction eliminates vehicle-origin and RSU-origin BSHH vectors by preventing any single node from forging a signature under another node’s identity without breaking the underlying lattice assumption.

Location-Binding Signatures for ME Mitigation. The ME attack is a false witness attestation problem that cannot be addressed by content-authentication alone. Each topology observation report is therefore constructed by binding the observed link to the reporter’s GPS-attested position and RSSI measurement before signing, as defined in Eqs. 3.29 and 3.30, where SK_{V_k} is the private signing key of V_k established via the ML-KEM-1024 + HQC-5 [64, 66] hybrid key encapsulation mechanism, pos_{V_k} is the GPS-attested position of V_k at time τ_s , and $\text{RSSI}_{V_k \leftarrow V_i}$ is the received signal strength from V_i measured at V_k .

The controller accepts a single observation report from V_k for link e_{ij} if and only if the signature is valid and both physical consistency conditions are jointly

satisfied, as defined in Eq. 3.31. To resist collusion among multiple distant attackers, link e_{ij} is inserted into the controller topology database only when a quorum of at least t physically-consistent independent witnesses confirm it, as defined in Eq. 3.32, where $\text{Accept}_{V_k}(e_{ij})$ follows from Eq. 3.31 and t is the same threshold as in Eq. 3.28. This condition ensures that no single distant attacker, and no colluding group smaller than t , can fabricate a link observation that passes both cryptographic verification and physical-consistency checking, eliminating vehicle-origin and RSU-origin ME vectors at the pre-detection filter layer.

Smart Contract Automated Mitigation. Upon trust-weighted PBFT consensus on a confirmed detection event, the **TemporalEchoMitigator** smart contract executes the following actions. Three trust management functions are called first at each round: (i) **updateTrust**(nodeId, Δ) applies Eq. 3.40 to each participating peer; (ii) **updateCtrlTrust**(C_j) applies Eq. 3.41 to the reporting controller on each confirmed divergence; (iii) **selectPeers**(n_p) returns $\mathcal{P}_{\text{active}}$ per Eq. 3.43, demoting peers below $\tau_{\min}$ and promoting the next-ranked eligible client; (iv) **getTrustedEvidence**($\tau_{\min}^{\text{gt}}$) returns $\mathcal{E}_t^{\text{trusted}}$ per Eq. 3.46. The contract then executes five mitigation actions: (1) attack metadata is logged to the immutable ledger using trusted-node-signed beacon evidence as ground truth; (2) for TTW and BSHH, a FlowMod DROP rule is pushed via the RSU emergency channel directly to RSU OpenFlow agents and the session key is revoked through LKH ($O(\log n)$ updates); (3) for ME, false paths are invalidated and a reroute FlowMod is pushed via the emergency channel; (4) for controller-origin attacks where $\tau_{C_j} < \tau_{\min}^C$, subnetwork reassignment is executed per Eqs. 3.44 and 3.45 and C_j 's OpenFlow credentials are revoked via the consortium CA; (5) the offending node or controller is flagged for mandatory re-authentication and an AttackDetected event is emitted to the application layer. The complete procedure is given in Algorithm 4. First, the attack metadata is logged to the immutable distributed ledger using RSU-signed beacon evidence as the ground truth record, not controller-reported data. Second, variant-specific mitigation is executed: for TTW and BSHH, a FlowMod DROP rule is pushed to the SDN controller and the associated session key is revoked through the KEM module; for ME, falsely inferred paths are invalidated and a reroute FlowMod is pushed. Third, the offending node is flagged for mandatory re-authentication. Fourth, an AttackDetected event is emitted to the Application and Analytics Layer. For controller-origin variants, the smart contract additionally issues a FlowMod directly overriding the controller's current routing table, since the controller itself is the source of the attack and cannot be trusted to self-correct. The complete procedure is given in Algorithm 4.

Node Removal Mechanism. When a malicious vehicle V_k is confirmed, the following sequence executes atomically: (i) trust is zeroed via Eq. 3.40, automatically excluding V_k from $\mathcal{P}_{\text{active}}$ per Eq. 3.43; (ii) in Tier 1 (RSU-present) mode, a FlowMod DROP rule is issued via the RSU emergency channel directly to RSU OpenFlow agents, bypassing the controller; in Tier 2 (no-RSU) mode, where no OpenFlow infrastructure exists, traffic isolation is achieved through a *cooperative blacklist protocol*: the smart contract appends the malicious node's revoked certificate fingerprint to a signed BLACKLISTBEACON distributed by OBU Fabric peers to all vehicles in range via the standard V2V channel; upon receipt, vehicles exclude the blacklisted node from their local routing tables and reject its beacons;

the exclusion propagates across the OBU network within $\lceil \text{diam}(\mathcal{G}_t) \rceil$ beacon intervals; (iii) V_k 's session key is revoked by updating the LKH KEKs on the path from leaf ℓ_k to the root at cost $O(\log n)$ per Eq. 3.18, distributing updated KEKs to remaining vehicles within the next beacon interval; (iv) V_k 's CA certificate is revoked, permanently excluding it until manual re-admission through the consortium CA. When a malicious RSU r_k is confirmed, two independent actions execute in parallel, reflecting its dual role in the architecture. Its function as an *OpenFlow forwarding node* is terminated immediately via the same atomic sequence as vehicle removal: a FlowMod DROP rule is issued to adjacent RSU OpenFlow agents via the emergency channel; its session key is revoked via LKH at cost $O(\log n)$ per Eq. 3.18; its CA certificate is revoked; and a partial zone reassignment transfers r_k 's coverage area to the adjacent trusted controller. Its function as a Hyperledger Fabric *consortium peer* is handled separately by the three-stage RSU demotion pipeline defined in Section 3.4.12: r_k is demoted to QUARANTINED_CLIENT state and monitored for T_{quar} before permanent ledger removal, preserving an auditable record of its behaviour while the data plane is already secured by the immediate OpenFlow action above.

Controller Removal Mechanism. When controller C_j is confirmed as a source of topology poisoning, Eq. 3.41 is applied at each detection event. When τ_{C_j} falls below τ_{min}^C , the smart contract executes $\text{REASSIGN}(\mathcal{Z}_j \rightarrow C_k^*)$ via Eqs. 3.44–3.45, issuing the command directly to RSU Fabric peers in $\mathcal{Z}_j$ via the emergency channel. RSU OpenFlow agents in $\mathcal{Z}_j$ switch their southbound connection to C_k^* , which receives a topology snapshot of $\mathcal{Z}_j$ from the RSU beacon evidence ledger and begins managing the zone. C_j 's OpenFlow credentials are revoked via the consortium CA, preventing reconnection. Since the REASSIGN command travels from smart contract to RSU switchers directly — not through C_j — the malicious controller cannot intercept, suppress, or replay it.

Alert Interface. The alert object passed from either the LW or FS detector to the TemporalEchoMitigator smart contract is defined as the tuple:

$$\mathcal{O} = (V_{\text{id}}, \alpha, \hat{y}_v, \mathcal{S}_{\text{trig}}, t_{\text{alert}}) \quad (3.52)$$

where V_{id} is the offending node identifier, $\alpha \in \{\text{TTW}, \text{BSHH}, \text{ME}\}$ is the detected attack variant, $\hat{y}_v$ is the anomaly score from 3.25 (or the weighted signature score $s(e)$ from 3.12 for the LW path), $\mathcal{S}_{\text{trig}} \subseteq \{1, \dots, 9\}$ is the set of triggered signature indices, and t_{alert} is the detection timestamp. The trusted RSU node n submits alert $\mathcal{O}$ directly to its Fabric peer via the peer's gRPC endpoint, bypassing the SDN controller entirely and executes the mitigation logic under PBFT consensus among the consortium RSU nodes.

Dual-Path Conflict Resolution. When both the LW and FS detection paths raise alerts for the same node v simultaneously, the smart contract resolves the conflict by taking the union of detected variants and applying the most restrictive mitigation action. Specifically, node isolation (FlowMod DROP) takes precedence over path invalidation, ensuring that a node flagged for TTW or BSHH by the LW path is isolated even if the FS path simultaneously detects ME, preventing a race condition between DROP and reroute FlowMod rules at the SDN controller.

Consensus and Execution Latency. The TemporalEchoMitigator contract executes under Practical Byzantine Fault Tolerant (PBFT) consensus among the Hyperledger Fabric consortium nodes, which correspond to the RSUs in the SDVN deployment. PBFT tolerates up to $\lfloor (n - 1)/3 \rfloor$ Byzantine RSU nodes, where n is the total number of consortium participants. Smart contract execution latency on Hyperledger Fabric is typically 50–200 ms [7], which combined with the TGN inference latency and FlowMod propagation time must remain within the 100 ms safety latency budget. This constraint motivates the parallel deployment of the LW path, which responds within $O(|W|)$ time without blockchain overhead, providing immediate provisional isolation while the FS path completes its full verification and logging cycle.

Mobility-Aware Mitigation Timing. The 100 ms safety latency constraint must be interpreted in the context of the SDVN mobility model. At highway speeds, the minimum link lifetime is approximately 9 s 3.33, which corresponds to 90 beacon intervals. An attacker executing a sustained TTW or BSHH attack must inject at least one malicious beacon per interval to maintain the false topology state at the controller. The LW detector operating at $O(|W|)$ per event provides isolation within a single beacon interval (100 ms) of the first detected anomaly, which is well within the link lifetime bound and limits the maximum duration of a successful attack to one beacon interval regardless of vehicle speed. The FS blockchain mitigation, which completes within 50–200 ms, provides confirmed isolation before the second beacon interval elapses, ensuring that even at highway speeds the attack cannot persist long enough to corrupt routing decisions affecting safety-critical services such as collision avoidance. For ME attacks, which require the false topology to be established before routing is affected, the quorum condition in 3.32 prevents false paths from being installed in the first place, making the timing of post-detection mitigation less critical than for TTW and BSHH.

Algorithm 4 formalises the post-detection response executed by the temporal echo mitigator smart contract deployed on Hyperledger Fabric and comprises three coordinated routines. The main procedure **FS-MITIGATE** begins by aggregating beacon evidence $\mathcal{E}_t^{\text{nodes}}$ from all trusted RSU peers and computing the symmetric divergence δ between the controller-perceived topology and the ground-truth RSU observations. a divergence exceeding δ_{thresh} additionally flags a controller-origin attack. PBFT consensus is then sought among the consortium RSU nodes, tolerating up to $\lfloor (n - 1)/3 \rfloor$ Byzantine peers; if consensus is not reached, the procedure aborts, ensuring that no mitigation action is taken on the evidence of a single compromised RSU. Upon a confirmed consensus, the contract performs four atomic actions for each flagged node: the attack metadata is committed to the immutable ledger $\mathcal{L}$; a variant-specific response is dispatched — for TTW and BSHH, **VERIFY_THRESHOLD_SIG** checks that at least t independent individual signatures are valid (Eq. 3.28) before pushing a **FlowMod(DROP)** isolation rule to the SDN controller and revoking the ML-KEM-1024 + HQC-5 [64, 66], while for ME, **VERIFY_QUORUM** cross-checks each candidate witness V_k against joint physical consistency conditions — valid signature, range bound $d(\text{pos}_{V_k}, e_{ij}) \leq r_{\text{comm}}$, and signal threshold $\text{RSSI}_{V_k} \geq \text{RSSI}_{\text{min}}$ (Eqs. 3.31–3.32) — before invalidating false paths and issuing a reroute **FlowMod**; the offending node is then flagged for mandatory re-authentication before re-admission; and an **ATTACKDETECTED**

event is emitted to the application layer. This dual active role of blockchain — as both a forensic immutable ledger and an automated mitigation executor — distinguishes the proposed framework from prior SDVN blockchain deployments, which rely exclusively on passive audit.

Algorithm 4. Full-Stack Smart Contract Mitigation (FS-MITIGATE).

```

1: procedure FS-MITIGATE( $\mathcal{A}$ ,  $\sigma$ ,  $\mathcal{L}$ ,  $\theta_{\text{FS}}$ ,  $t$ ,  $G_t^C$ ,  $\{\mathcal{B}_{n_k}(t)\}$ )  $\triangleright \{\mathcal{B}_{n_k}(t)\}$ :
   beacon evidence from all trusted node peers
2:    $\mathcal{E}_t^{\text{trusted}} \leftarrow \text{getTrustedEvidence}(\tau_{\min}^{\text{gt}})$   $\triangleright$  Eq. 3.46: trust-filtered beacon
   evidence only
3:    $\delta \leftarrow |\mathcal{E}_t^C \Delta \mathcal{E}_t^{\text{trusted}}|$   $\triangleright$  Eq 3.47: Controller topology vs. RSU beacon
   evidence.
4:   if  $\delta > \delta_{\text{thresh}}$  then
5:      $\mathcal{A} \leftarrow \mathcal{A} \cup \{(\text{controller}, \alpha_{\text{origin}}, \delta)\}$ 
6:      $\text{updateCtrlTrust}(C_j, -\Delta_-^C)$   $\triangleright$  Eq. 3.41: penalise controller on each
   confirmed divergence
7:   end if
8:    $q \leftarrow \text{PBFT\_CONSENSUS}(\mathcal{A})$   $\triangleright$  Consensus among RSU Fabric peers; tolerates
 $\lfloor (n-1)/3 \rfloor$  Byzantine nodes
9:   if  $q = \text{FALSE}$  then
10:    return  $\triangleright$  Abort if consensus not reached; single RSU detection
   insufficient
11:  end if
12:  for each  $(v, \hat{y}_v) \in \mathcal{A}$  do
13:     $\text{LOG}(\mathcal{L}, \{v, \hat{y}_v, t\})$   $\triangleright$  Immutable ledger entry
14:     $\alpha \leftarrow \text{IDENTIFY\_VARIANT}(v, \mathcal{A})$ 
15:    if  $v = \text{controller}$  then
16:      if  $\tau_{C_j} < \tau_{\min}^C$  then  $\triangleright$  Eq. 3.44
17:         $C_k^* \leftarrow \text{SELECT\_BACKUP}(C, j)$   $\triangleright$  Eq. 3.45
18:         $\text{REASSIGN}(\mathcal{Z}_j \rightarrow C_k^*)$   $\triangleright$  Issued via RSU emergency channel;
   bypasses  $C_j$ 
19:         $\text{REVOKE\_CTRL\_CRED}(C_j)$   $\triangleright$  CA revokes  $C_j$ 's OpenFlow
   credentials
20:      end if
21:      else if  $\alpha \in \{\text{TTW}, \text{BSHH}\}$  then
22:         $\text{VERIFY\_THRESHOLD\_SIG}(\sigma_v)$   $\triangleright$  Eq. 3.28
23:         $\text{PUSH\_FLOWMOD\_EMERGENCY}(\text{DROP}, v)$   $\triangleright$  Issued via RSU emergency
   channel; bypasses controller
24:         $\text{REVOKE\_SESSION\_KEY}(v)$   $\triangleright$  LKH update:  $O(\log n)$ , Eq. 3.18
25:      else if  $\alpha = \text{ME}$  then
26:         $\mathcal{W}_v \leftarrow \text{GET\_WITNESSES}(e_{ij}, t)$ 
27:         $\text{VERIFY\_QUORUM}(\mathcal{W}_v, t)$   $\triangleright$  Eq. 3.32
28:         $\text{INVALIDATE\_PATHS}(\mathcal{P}_v^{\text{false}})$ 
29:         $\text{PUSH\_REROUTE\_EMERGENCY}()$   $\triangleright$  Via RSU emergency channel
30:      end if
31:       $\text{FLAG\_REAUTH}(v)$   $\triangleright$  Block re-admission until re-authenticated
32:       $\text{EMIT}(\text{ATTACKDETECTED}, \{v, \alpha, \hat{y}_v, t\})$ 
33:       $\text{updateTrust}(v, 0)$   $\triangleright$  Eq. 3.40: zero trust for confirmed attacker node

```

```

34:   end for
35:    $\mathcal{P}_{\text{active}} \leftarrow \text{selectPeers}(n_p)$   $\triangleright$  Eq. 3.43: re-rank by updated trust;
      demote/promote as needed
36:   for each  $n_k \in \mathcal{P}_{\text{active}} \setminus \mathcal{A}$  do
37:     updateTrust( $n_k, \Delta_+$ )  $\triangleright$  Eq. 3.40: reward correct-participation peers
38:   end for
39: end procedure

40: function VERIFY_THRESHOLD_SIG( $\sigma_v$ )
41:    $c \leftarrow 0$ 
42:   for each  $(\sigma_i, \text{msg}_i, \text{PK}_{V_i}) \in \sigma_v$  do
43:     if Verify( $\sigma_i, \text{msg}_i, \text{PK}_{V_i}$ ) = 1 then
44:        $c \leftarrow c + 1$ 
45:     end if
46:   end for
47:   if  $c < t$  then  $\triangleright$  Eq. 3.48
48:     return FAIL
49:   end if
50:   return PASS
51: end function

52: function VERIFY_QUORUM( $\mathcal{W}_v, t$ )
53:    $q \leftarrow 0$ 
54:   for each  $V_k \in \mathcal{W}_v$  do
55:     if Verify( $\sigma_{V_k}, \text{PK}_{V_k}$ ) = 1 then
56:       if  $d(\text{pos}_{V_k}, e_{ij}) \leq r_{\text{comm}}$  then  $\triangleright$  Eq. 3.31
57:         if  $\text{RSSI}_{V_k} \geq \text{RSSI}_{\min}$  then
58:            $q \leftarrow q + 1$ 
59:         end if
60:       end if
61:     end if
62:   end for
63:   if  $q < t$  then  $\triangleright$  Eq. 3.32
64:     return FAIL
65:   end if
66:   return PASS
67: end function

```

3.4.12 Dynamic Peer Selection and the RSU Demotion Pipeline

Peer Selection Rationale. The consortium no longer treats every registered node as a consensus-participating peer. Selecting all nodes as peers at every round is computationally expensive — each additional peer adds an endorsement and a PBFT vote that must be verified — and is suboptimal once trust scores have diverged, since low-trust or malicious nodes continue to consume consensus bandwidth without contributing reliable attestations. The active peer set P_{active} is therefore re-derived at runtime from the trust scores τ_k maintained on the ledger,

rather than fixed at deployment. During bootstrap, when trust scores are still at their initial values, all RSUs are treated as peers by default, since a trusted authority has already vetted them at registration time (**RegisterRSUPeer**); peer selection becomes discriminating only once trust has had the opportunity to diverge through observed behaviour.

Trust-Weighted Peer Selection

Let N be the set of all registered nodes, and let $\tau_k \in [0, 1]$ denote the trust score of node k , updated per Eq. 3.40 ($\Delta^+ = 0.05$, $\Delta^- = 0.10$). RSU nodes are initialised at $\tau_{\text{init}}^{\text{Tier1}} = 1.00$ upon registration by a trusted authority; OBU/vehicle nodes are initialised at $\tau_{\text{init}}^{\text{Tier2}} = 0.10$.

The active peer set P_{active} of size $n_p = 3f + 1$ is formed by trust-ranked selection from E per Eq. 3.43, where f is the assumed number of simultaneous Byzantine peers and $E \subseteq N$ is the eligibility-filtered candidate set:

$$E = \left\{ k \in N \mid \text{state}_k \notin \{\text{QUARANTINED_CLIENT, REMOVED}\}, \neg \text{flagged}_k, \tau_k \geq \tau_{\min}, \phi(k) \right\} \quad (3.53)$$

with $\tau_{\min} = 0.10$, and $\phi(k)$ an additional Tier-2 (OBU) gate formally defined by Eq. 3.42: it requires a valid consortium certificate, minimum hardware capacity, minimum dwell time $T_{\text{dwell}}(V_k) \geq T_{\min}$ inside RSU coverage, trust at or above $\tau_{\min}$, and absence of a detection flag. RSU peers (Tier 1) are exempt from $\phi(k)$ since they are fixed infrastructure rather than transient vehicles. Note that $\tau^{\text{Tier2}}_{\text{init}} = \tau_{\min} = 0.10$: a newly registered OBU technically satisfies the trust condition at initialisation, but peer promotion is nevertheless deferred by the dwell-time gate within $\phi(k)$, which requires $T_{\text{dwell}}(V_k) \geq T_{\min}$ to be met before the node becomes eligible (Eq. 3.42, condition 3).

Three-Stage RSU Demotion Pipeline

When a consortium RSU peer is confirmed malicious, it is not deleted outright; it is moved through a three-stage pipeline that preserves auditability and gives the node a bounded opportunity to recover before permanent expulsion:

1. STAGE 1 — QUARANTINE.

The peer's state transitions **ACTIVE** $\rightarrow$ **QUARANTINED_CLIENT**. Four actions execute atomically: (i) its trust score is zeroed per Eq. 3.40; (ii) its Fabric peer role is stripped, demoting it to a passive client observer that can no longer submit endorsements or PBFT votes; (iii) its session key is revoked via LKH at cost $O(\log n)$ per Eq. 3.18, preventing the quarantined RSU from generating valid HMAC-authenticated beacons that would otherwise pass the cryptographic pre-filter (Eqs. 3.15–3.17) during the monitoring window; (iv) it is

excluded from P_{active} by the $\text{state}_k \notin \{\text{QUARANTINED_CLIENT}, \text{REMOVED}\}$ filter in Eq. 3.53. The ledger record is retained and monitoring continues.

2. STAGE 2 — MONITORING.

For a fixed quarantine window T_{quar} (30 s in the simulation; configurable for deployment), the node's trust score is re-evaluated at every consensus round and every beacon interval, not only in response to further attacks. Peer eligibility is re-checked continuously rather than only immediately after a mitigation event.

3. STAGE 3 — REMOVAL.

Once T_{quar} has elapsed, if $\tau_k \leq \tau_{\text{min}}$ (i.e. trust has not recovered above the eligibility floor), the node's state transitions permanently $\text{QUARANTINED_CLIENT} \rightarrow \text{REMOVED}$, and its CA certificate is revoked via the consortium CA, preventing it from re-establishing any V2X session or Fabric connection without a freshly issued certificate. Re-admission requires a full manual administrative action: certificate re-issuance by the consortium CA followed by a new `RegisterRSUPeer` invocation. There is no automatic trust-recovery path.

Periodic Peer Re-Selection Procedure

The periodic peer re-selection process is executed at runtime to ensure that the active consensus set P_{active} reflects the latest trust state of the network. The procedure is defined as follows:

1. For each registered peer $k \in N$, check its current state.
2. If $\text{state}_k = \text{QUARANTINED_CLIENT}$, compute the elapsed time:

$$\Delta t = \text{now} - \text{demotedAt}_k.$$

3. If $\Delta t \geq T_{\text{quar}}$ and $\tau_k \leq \tau_{\text{min}}$, transition the node state to:

$$\text{QUARANTINED_CLIENT} \rightarrow \text{REMOVED}.$$

4. Construct the eligibility set E by removing from N any node that satisfies at least one of the following conditions:

- $\text{state}_k \in \{\text{QUARANTINED_CLIENT}, \text{REMOVED}\}$
- $\text{flagged}_k = \text{true}$
- $\tau_k < \tau_{\text{min}}$
- (for Tier-2 / OBU nodes only) failure of the constraint gate $\phi(k)$, which enforces minimum dwell time in RSU coverage, sufficient hardware capacity, and synchronization with the latest checkpoint (Eq. 3.53)

5. Select the active peer set by taking the top- n_p nodes from E ranked by trust score τ_k :

$$P_{\text{active}} = \arg \max_{k \in E} \tau_k.$$

6. Return the updated active peer set P_{active} .

PeriodicPeerReSelection is called explicitly by the off-chain client at every beacon interval. The same underlying monitor-and-remove check it performs is also triggered from two further points without invoking the named entrypoint directly: the mitigation contract inlines an identical check after every executed mitigation round, and **updateTrust()** invokes the same removal logic immediately whenever an already-quarantined peer’s trust degrades further on a failed round.

Table 3.5: Trust and peer-selection parameters

Symbol	Description	Value
Δ^+	Trust increment	0.05
Δ^-	Trust decrement	0.10
$\tau_{\min}$	Eligibility threshold	0.10
$\tau_{\min}^{\text{gt}}$	Bootstrap qualification threshold	0.50
$\tau_{\text{init}}^{\text{Tier1}}$	Initial RSU trust	1.00
$\tau_{\text{init}}^{\text{Tier2}}$	Initial OBU trust	0.10
f	Byzantine peers assumed	2
$n_p = 3f + 1$	Active peer count	8
T_{quar}	Quarantine duration	30 s

Summary. Peer selection is no longer “everyone is a peer.” Bootstrap uses authority-vetted RSU peers by default since trust scores are not yet differentiated; thereafter, P_{active} is continuously re-derived from τ_k (Eq. 3.43). A peer found malicious is demoted to a quarantined client state rather than removed outright, monitored for T_{quar} , and only permanently removed if its trust has not recovered above $\tau_{\min}$ by the end of the monitoring window.

Simulation-Scope Trust Approximation. The full TETA-Guard deployment derives all trust scores τ_k and τ_{C_j} from the Hyperledger Fabric ledger (Eqs. 3.40–3.41), providing a single consistent source of truth for both the TGN evidence filter (Eq. 3.46) and the controller-divergence gate (Eq. 3.47). In the NS-3/SUMO simulation, two separate, non-communicating trust tallies are maintained as a pragmatic approximation: **tgn_core.cc** maintains a per-peer event counter for building $\mathcal{E}_t^{\text{trusted}}$ (using $R_{\min} = 8$ rounds, $\tau_{\min}^{\text{gt}} = 0.50$), while **routing.cc** maintains an independent **g_trust_table** for the controller-divergence gate. Both implementations apply identical trust parameters and produce identical trust decisions for the scenarios tested. Any divergence between the two tallies in edge cases (e.g. an RSU peer quarantined mid-run) is a simulation-scope simplification and does not affect the validity of reported detection metrics. A unified Fabric-ledger trust implementation remains as future engineering work for the full deployment.

Chapter 4

Results

4.1 Attack Injection

A total of 12 attack scenarios are evaluated, derived from three attack variants (TTW, BSHH, ME) each injected from four attacker positions (malicious vehicle without RSU, malicious vehicle with RSU, RSU-origin, and controller-origin), giving $3 \times 4 = 12$ scenarios per experiment. The divergence produced in each scenario is measured using Eq. 3.1 after every beacon interval. To evaluate low-rate stealth behaviour, each scenario is tested under three attack injection rates — 1

Mobility parameters are calibrated using the link lifetime model in Eq. 3.33 for the simulated urban scenario: $v_{\text{rel}} \approx 14 \text{ m/s}$ (approximately 50 km/h), producing $L_{\text{link}} \approx 43 \text{ s}$ and $N_{\text{beacon}} = 430$ intervals per link lifetime. Highway conditions ($v_{\text{rel}} \approx 67 \text{ m/s}$) are analysed within the mobility model (Section 3.4) but lie outside the current NS-3/SUMO evaluation scope.

4.2 Baseline Selection and Benchmarking Rationale

The benchmarking study is designed to evaluate the effectiveness of the proposed TETA-Guard framework against both existing SDVN topology-defense mechanisms and internal architectural variants. External baselines are selected to represent the closest existing approaches in vehicular topology protection, position-falsification detection, and topology-poisoning mitigation. Internal ablation baselines are included to isolate the contribution of individual framework components, including temporal modeling, mobility awareness, cryptographic filtering, and blockchain-based mitigation.

4.2.1 External Baselines

Table 4.1: External Baseline Systems

Baseline	Reference	Detection Approach	Limitation
B1	Deng et al. [34]	Threshold-based countermeasures against topology poisoning in SDVNs via beacon-based discovery	Fixed thresholds; no temporal modeling; no mobility adaptation
B2	Mekonen et al. [39]	RSU-level ensemble learning (KNN with bagging) for position falsification detection via sequential BSM analysis	Detects position-payload anomalies only; cannot detect TTW timestamp violations or BSHH heartbeat identity spoofing
B3	TopoSleuth [42]	Multi-layered topology poisoning defense with decoy links, behavioral profiling, and multi-hop validation; LLDP trigger conditions adapted to equivalent beacon conditions	No vehicular mobility modeling; no cryptographic mitigation

4.2.2 Internal Ablation Study

Fourteen ablation configurations isolate each individually designed component of the TETA-Guard framework. Each configuration is constructed by removing or replacing exactly one component from the full system, keeping all other components unchanged. The x variable for each configuration is chosen to stress the component being removed; different x variables are used across configurations because no single variable stresses all components equally. Sensitivity analysis of detection parameters (T_b , $|\mathcal{W}|$, $\tau_{\min}$, Δ^+ , Δ^-) is conducted separately and is not part of this ablation study. Table 4.2 summarises all configurations; applicable PEMs from Section 4.3 are indicated per configuration.

Table 4.2: Ablation study configurations. Each row targets one design component in isolation. X: independent variable; PEMs: applicable metrics from Table 4.4 (Mn = metric number; $\dagger$ denotes ablation-only metrics not reportable for B1–B3 baselines).

ID	Component	Description	X Variable	Applicable PEMs
A1	LW Detection Stage Only	Disable the FS/TGN detection stage entirely. Alerts are raised solely by the 9 LW rule-based signatures. Tests whether the lightweight signature engine is sufficient without temporal graph learning, and quantifies the FS contribution to detection quality and routing recovery.	Attack injection rate $r_{\text{inj}} \in \{1\%, 5\%, 10\%\}$ of beacon events	$M1, M2, M3$ (TTW/BSHH), $M4$ (ME), $M5$ ($T_{\text{det}}^{\text{LW}}$), $M6$
A2	FS/TGN Detection Stage Only	Disable the LW signature engine. Alerts are raised solely by the TGN binary scorer and variant classification head. Tests the temporal graph network contribution in isolation, and whether the TGN alone (without rule-based signatures) achieves acceptable detection quality with a deeper inference budget.	Attack injection rate $r_{\text{inj}} \in \{1\%, 5\%, 10\%\}$	$M1, M2, M3$ (TTW/BSHH), $M4$ (ME), $M5$ ($T_{\text{det}}^{\text{FS}}$), $M6, M9^\dagger$

ID	Component	Description	X Variable	Applicable PEMs
A3	Static GCN (No GRU Temporal Memory)	Replace the TGN's GRU memory module with a stateless single-hop GCN operating only on the current graph snapshot. No hidden state is maintained across beacon events. Tests the isolated contribution of temporal memory to attack detection; the GCN still has access to the same node features $\mathbf{x}_v$ as the full TGN.	Observation window length $T_{\text{obs}} \in \{50\text{s}, 150\text{s}, 300\text{s}\}$	M1, M3 (TTW/BSHH), M9 [†]
A4	Fixed δ_{thresh} (No Mobility-Adaptive Threshold)	Replace the dynamic mobility-adaptive divergence threshold (Eq. 3.48) with a fixed constant equal to its urban-scenario median value. Tests whether adaptive threshold computation is necessary as vehicle density and speed vary, or whether a fixed operational point suffices.	Vehicle density $\lambda \in \{0.01, 0.02, 0.04\}$, veh/m	M1, M2, M4 (PIR)

ID	Component	Description	X Variable	Applicable PEMs
A5	No HMAC + Timestamp + Nonce Pre-Filter	Remove the cryptographic pre-detection filter (Eqs. 3.15–3.17). All beacon events pass directly to the LW and FS detection stages regardless of MAC validity or timestamp/nonce freshness. Tests the filter’s contribution to detection efficiency and pipeline latency, separately for vehicle-origin, RSU-origin, and controller-origin attack vectors.	Attacker origin: {vehicle, RSU, controller}	M1 (per origin), M5 (T_{det}), M10 [†]
A6	No Threshold Aggregate Signature (BSHH Defence)	Remove the t -of- n ML-DSA-87 threshold aggregate signature (Eq. 3.28) used to attest duplicate liveness contradictions in the BSHH detection path. The LW BSHH-S1 rule degrades to a majority-count heuristic with no cryptographic attestation. Tests whether the threshold signature provides meaningful forgery resistance beyond the structural BSHH rules.	Colluding Byzantine vehicles $f_c \in \{1, 2, f=2\}$	M1 (BSHH scenarios), M2 (BSHH), M11 (FSR) [†]

ID	Component	Description	X Variable	Applicable PEMs
A7	No Location-Binding + Quorum (ME Defence)	Remove both the ML-DSA-87 location-binding message (Eq. 3.29) and the geographic consistency quorum check (Eq. 3.32) from the ME detection path. Echo path injections are evaluated only by the topological reporter-count rule (ME-S1) without spatial consistency verification. Tests the isolated contribution of location-aware cryptographic attestation against out-of-range false witnesses.	Echo reporter distance from claimed link: $d \in \{r_{\text{comm}}, 1.5r_{\text{comm}}, 3r_{\text{comm}}\}$	M4 (PIR), M1 (ME scenarios), M11 (QRR) [†]
A8	No Smart Contract Mitigation (Detection Only)	Disable the TemporalEcho Mitigator smart contract execution: alerts are raised by the detection engine but no FlowMod, BlacklistBeacon, or quarantine action is enforced. Tests whether automated mitigation enforcement contributes meaningfully to topology recovery and routing health beyond detection alone.	Intervals post-alert without enforcement: $k \in \{0, 1, 5, 10\}$ beacon intervals	M2, M3, M4, M6

ID	Component	Description	X Variable	Applicable PEMs
A9	Equal-Weight PBFT (No Trust Weighting)	Replace the trust-weighted PBFT consensus (Eq. 3.49) with standard equal-weight PBFT where all $n_p = 8$ peers contribute equally regardless of trust score τ_k . Tests whether trust-weighted consensus provides resilience gains against Byzantine peers that a standard PBFT implementation cannot achieve.	Byzantine peers in active consensus set: $f_b \in \{0, 1, 2\}$	M1, M2, M12 (T_{trust}) [†]
A10	Immediate Removal (No Quarantine Pipeline)	Skip the graduated quarantine pipeline (Section 3.4.12): flagged nodes are removed from the Fabric consortium immediately upon detection alert without the quarantine interval $T_{\text{quar}} = 30\,000$ ms. Tests whether the graduated pipeline prevents erroneous exclusion of legitimate nodes under false-positive detections, at the cost of a fixed quarantine delay.	False-positive detection rate at detector: $p_{\text{FP}} \in \{0\%, 2\%, 5\%\}$	M2, M12 ($T_{\text{trust}}, T_{\text{revoke}}$) [†]

ID	Component	Description	X Variable	Applicable PEMs
A11	Naïve Flat Re-Keying (No LKH Hierarchy)	Replace the LKH $O(\log n)$ group re-keying (Eq. 3.18) with a naïve flat re-key in which a fresh session key is individually distributed to all $n - 1$ remaining vehicles after each revocation event. Tests the empirical cost of LKH versus flat re-keying as network size grows, validating the $C_{\text{revoke}} = \lceil \log_2 n \rceil$ theoretical claim.	Network size $n \in \{50, 100, 150, 200\}$ vehicles	M12 ($T_{\text{revoke}}^\dagger$), M5 (T_{FlowMod})
A12	No Topology Divergence Detector (Controller-Origin Blind)	Remove the topology divergence check ($\delta > \delta_{\text{thresh}}$, Eqs. 3.47–3.48). Controller-origin attacks are evaluated only by the TGN pipeline via the pooled event stream. Tests whether the TGN alone can detect controller-origin attacks without the divergence mechanism, and quantifies the divergence detector’s unique contribution to controller-vector coverage (disaggregated from vehicle/RSU-origin).	Controller-origin injection rate $r_{\text{inj}} \in \{1\%, 5\%, 10\%\}$ (controller scenarios only)	M1 (controller-origin, disaggregated), M2, M3 (TTW-MC/BSHH-MC), M4 (ME-MC)

ID	Component	Description	X Variable	Applicable PEMs
A13	Single KEM (ML-KEM-1024 Only, No HQC-5)	Remove HQC-5 from the hybrid KEM session-key establishment, using ML-KEM-1024 alone (standard lattice-only KEM). This does not test detection quality (unaffected) but evaluates the latency and overhead cost of maintaining a second independent KEM primitive. Quantifies the performance overhead of algorithmic diversity in the KEM layer as handshake rate scales.	KEM handshake rate: $r_{\text{hs}} \in \{10, 50, 100, 200\}$ /s	M7 (Ω), M5 (T_{pipeline} , KEM sub-component)

ID	Component	Description	X Variable	Applicable PEMs
A14	No Controller Re-assignment Mechanism	Remove the controller reassignment pipeline (Eqs. 3.44–3.45). When a controller’s trust score drops below $\tau_{\min}^C = 0.30$ the flagged controller remains active rather than being replaced by the highest-trust eligible peer. Note: A12 tests whether controller-origin attacks are <i>detected</i> ; A14 tests whether the <i>reassignment response</i> is necessary once detection has occurred. Quantifies topology recovery degradation and reassignment latency when the mechanism is absent.	Compromised controllers out of 4: $n_C \in \{1, 2, 3\}$	M1 (controller-origin), M2, M3 (TTW-MC/BSHH-MC), M4 (ME-MC), M12 (T_{reassign}) [†]

Coverage verification. Table 4.3 confirms that every formulated design component appears in at least one ablation configuration and that every ablation-only PEM (M9–M12) is exercised by at least one configuration.

Table 4.3: Ablation coverage: design component $\times$ ablation ID and ablation-only PEM $\times$ ablation ID.

Design Component	Ablation ID
LW 9-signature engine	A1 (removed), A2 (isolated)
FS/TGN binary scorer	A2 (isolated), A1 (removed)
GRU temporal memory	A3
Mobility-adaptive δ_{thresh}	A4
HMAC + timestamp + nonce pre-filter	A5
ML-DSA-87 threshold aggregate signature (BSHH)	A6
Location-binding + quorum (ME)	A7
Smart contract mitigation enforcement	A8
Trust-weighted PBFT consensus	A9
Quarantine / graduated demotion pipeline	A10
LKH $O(\log n)$ group rekeying	A11
Topology divergence detector	A12
Controller reassignment pipeline (Eqs. 3.44–3.45)	A14
HQC-5 second KEM (hybrid diversity)	A13
Ablation-Only PEM	Ablation ID
M9: Variant classification F1 (TTW/BSHH/ME)	A2, A3
M10: Pre-filter rejection accuracy (FRA, FRR)	A5
M11: Forgery / quorum resistance (FSR, QRR)	A6, A7
M12: Trust convergence & revocation latency	A9, A10, A11, A14

4.3 Performance Evaluation Metrics

Twelve mutually exclusive metrics spanning detection quality, attack-specific impact, system timing, network performance, overhead, scalability, and component-level validation are defined below. Each metric targets a distinct aspect of the framework: no two metrics in the set answer the same evaluation question. Metrics M9–M12 (Eqs. 4.14–4.21) are used in the ablation study only; the B1–B3 baselines do not provide equivalent mechanisms against which they can be compared. Table 4.4 summarises the complete set, the design component(s) each evaluates, and its comparability to the external baselines.

4.3.1 Detection Quality Metric (M1)

Matthews Correlation Coefficient (MCC). A single confusion-matrix metric is used for binary detection quality. MCC is selected over Detection Rate, False Positive Rate, Precision, Recall, and AUROC because it accounts for all four confusion-matrix cells simultaneously and remains informative under class imbalance between benign and attack events. All five alternatives are derivable from the same confusion matrix and are therefore not mutually exclusive with MCC; they are excluded.

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (4.1)$$

$\text{MCC} \in [-1, 1]$; a value of 1 indicates perfect binary classification, 0 indicates random performance, and -1 systematic mis-classification. Reported per attack variant and combined over all 12 scenarios.

4.3.2 Attack-Specific Metrics (M2–M4)

These three metrics evaluate the attacks themselves rather than the detector output. They directly test whether the formal problem defined in Eq. 3.1 is solved ($\delta > 0$ indicates a successful attack; mitigation must drive $\delta \rightarrow 0$).

M2: Topology Divergence Reduction Ratio (TDRR). The topology divergence $\delta(G_t^C, G_t^R) = |\mathcal{E}_t^C \Delta \mathcal{E}_t^R|$ from Eq. 3.1 is the paper’s own formal criterion for attack success. TDRR measures the fractional reduction in time-averaged divergence achieved by the full framework relative to the unprotected case:

$$\text{TDRR} = \frac{\bar{\delta}_{\text{unprotected}} - \bar{\delta}_{\text{mitigated}}}{\bar{\delta}_{\text{unprotected}}} \times 100\%, \quad \bar{\delta} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \delta(G_t^C, G_t^R) \quad (4.2)$$

where $\mathcal{T}$ is the set of beacon intervals in the 300 s observation window. TDRR = 100% means the framework fully eliminates topology corruption; TDRR = 0% means no improvement.

M3: Temporal Stale-State Duration (T_{stale}). This metric is specific to the temporal attack class (TTW and BSHH) and measures how long stale topology information persists in the controller after the physical reality changes.

$$T_{\text{stale}}(e_{ij}) = \tau_{\text{correct}}(e_{ij}) - \tau_{\text{break}}(e_{ij}) \quad (4.3)$$

where $\tau_{\text{break}}(e_{ij})$ is the wall-clock time at which link e_{ij} physically fails and $\tau_{\text{correct}}(e_{ij})$ is the time at which the controller’s topology is updated to remove it. Under normal convergence without attack: $T_{\text{stale}} \leq T_b + \tau_{\text{prop}}$ (Eq. 3.35). Under TTW or

BSHH injection without mitigation: $T_{\text{stale}} \gg L_{\text{link}}$ because the attacker artificially prolongs the stale state beyond the physical link lifetime (Eq. 3.33). Mean stale-state duration is reported per attack variant:

$$\bar{T}_{\text{stale}} = \frac{1}{|\mathcal{E}_{\text{broken}}|} \sum_{e_{ij} \in \mathcal{E}_{\text{broken}}} T_{\text{stale}}(e_{ij}) \quad (4.4)$$

M4: Echo Path Inflation Ratio (PIR). This metric is specific to the echo attack class (ME) and measures how severely the controller’s inferred path diversity is inflated above the physically plausible bound.

$$\text{PIR}(V_i, V_j, t) = \frac{|P_{\text{controller}}(V_i, V_j, t)|}{|P_{\text{physical}}(V_i, V_j, t)|} \quad (4.5)$$

where $P_{\text{controller}}(V_i, V_j, t)$ is the set of paths inferred by the controller between V_i and V_j at time t , and $P_{\text{physical}}(V_i, V_j, t)$ is the set of paths existing in the ground-truth topology G_t^R at the same instant. Under no attack: $\text{PIR} = 1$. Under ME injection without mitigation: $\text{PIR} > 1$ (false paths inflated beyond ME-S2’s δ_{max} bound, Eq. 3.9). With mitigation active: $\text{PIR} \rightarrow 1$. Mean PIR over observation window and all vehicle pairs:

$$\overline{\text{PIR}} = \frac{1}{|\mathcal{V}|^2 |\mathcal{T}|} \sum_{V_i \neq V_j} \sum_{t \in \mathcal{T}} \text{PIR}(V_i, V_j, t) \quad (4.6)$$

4.3.3 System Pipeline Latency Metric (M5)

End-to-End Mitigation Response Latency (T_{pipeline}). The full latency from attack injection to enforced mitigation action, decomposed into four serial sub-components:

$$T_{\text{pipeline}} = T_{\text{det}} + T_{\text{PBFT}} + T_{\text{exec}} + T_{\text{FlowMod}} \quad (4.7)$$

where: T_{det} is the detection latency (time from attack injection to alert raised by the LW or FS detector); T_{PBFT} is the trust-weighted PBFT consensus round-trip time among $n_p = 8$ RSU Fabric peers (Eq. 3.49); T_{exec} is the **TemporalEchoMitigator** smart contract execution time on Hyperledger Fabric; T_{FlowMod} is the propagation time from smart contract output to enforced FlowMod rule at RSU OpenFlow agents via the emergency channel. Each sub-component is reported separately and as a total. The constraint is $T_{\text{pipeline}} \leq 100 \text{ ms}$ (one beacon interval); the LW path must satisfy this budget independently of the FS+blockchain path, since they run in parallel (Eq. 3.14). T_{det} is reported separately for LW and FS since their computational mechanisms are architecturally distinct.

4.3.4 Network Performance Metrics (M6)

Routing Recovery Performance (PDR and T_{e2e}). Two facets of data-plane health under attack and post-mitigation, bundled as sub-metrics of a single routing-recovery metric because they jointly quantify the downstream consequence of topology corruption on the forwarding plane.

$$\text{PDR} = \frac{N_{\text{received}}}{N_{\text{sent}}} \times 100\% \quad (4.8)$$

$$T_{e2e} = \frac{1}{N_{\text{received}}} \sum_{p \in \mathcal{P}_{\text{received}}} (t_{\text{recv}}^p - t_{\text{sent}}^p) \quad (4.9)$$

Both are reported under three conditions: (i) no attack baseline, (ii) under attack without mitigation, (iii) under attack with mitigation active. The recovery gap is $\Delta\text{PDR} = \text{PDR}_{\text{mitigated}} - \text{PDR}_{\text{attack}}$.

4.3.5 Communication Overhead Metric (M7)

Per-Beacon Authentication Overhead (Ω). The percentage increase in per-beacon payload size introduced by the cryptographic authentication fields added by the LW and FS paths relative to an unauthenticated baseline beacon m_{base} .

$$\Omega_{\text{LW}} = \frac{|\text{nonce}| + |\text{MAC}_{\text{HMAC-SHA256}}|}{|m_{\text{base}}|} \times 100\% \quad (4.10)$$

$$\Omega_{\text{FS}} = \frac{|\text{nonce}| + |\text{MAC}_{\text{HMAC-SHA256}}| + |\sigma_{V_k}^{\text{ML-DSA-87}}| + |\text{pos}_{V_k}| + |\text{RSSI}_{V_k}|}{|m_{\text{base}}|} \times 100\% \quad (4.11)$$

where the ML-DSA-87 signature size is 4627 bytes (Eq. 3.30), the HMAC-SHA256 tag is 32 bytes, and the nonce is 16 bytes. Ω_{FS} also accounts for the GPS position and RSSI fields bound in the location-binding message m'_{V_k} (Eq. 3.29). This metric directly answers the feasibility question for the 10 Hz $\times$ 200 vehicles $\times$ 7-channel DSRC deployment: whether the added fields saturate the available channel capacity.

4.3.6 Scalability Metric (M8)

Performance Scaling with Network and Consensus Size. Two orthogonal scaling axes are evaluated.

Vehicle-count scaling validates the $O(\|W\|)LW$ complexity claim (Eq. 3.14) by measuring whether it remains constant as the vehicle count N varies:

$$\rho_{\text{det}}(N) = \frac{T_{\text{det}}^{\text{LW}}(N)}{T_{\text{det}}^{\text{LW}}(N_0)}, \quad N \in \{50, 100, 150, 200\}, \quad N_0 = 50 \quad (4.12)$$

Expected: $\rho_{\text{det}} \approx 1$ for all N (detection overhead is independent of network size; depends only on the fixed window $|\mathcal{W}|$). Any significant deviation from 1 indicates a network-size dependency in the implementation not captured by the analytical bound.

Peer-count scaling measures how PBFT consensus latency grows with active peer count n_p :

$$\rho_{\text{PBFT}}(n_p) = \frac{T_{\text{PBFT}}(n_p)}{T_{\text{PBFT}}(n_{p,0})}, \quad n_p \in \{4, 8, 12\}, \quad n_{p,0} = 4 \quad (4.13)$$

Expected: $\rho_{\text{PBFT}} \propto (n_p/n_{p,0})^2$ following PBFT's $O(n_p^2)$ message complexity. If T_{pipeline} exceeds 100 ms at any tested n_p , it indicates the LW provisional isolation path is essential for safety-budget compliance.

4.3.7 Component-Level Metrics for Ablation Study (M9–M12)

The following four metrics evaluate individual design components for which the B1–B3 external baselines provide no equivalent mechanism; they are therefore used in the internal ablation study only (configurations A1–A6). Each metric targets a distinct, non-overlapping component and cannot be combined across them.

M9: Variant Classification F1 ($F1_\alpha$). The FS softmax classification head (Eq. 3.26) produces a three-way variant label $\alpha \in \{\text{TTW}, \text{BSHH}, \text{ME}\}$ that selects the mitigation branch in Algorithm 4. Mis-classification routes TTW to the ME branch or vice versa, producing an incorrect FlowMod. Per-class F1 measures this independently of the binary detection quality captured by MCC:

$$F1_\alpha = \frac{2TP_\alpha}{2TP_\alpha + FP_\alpha + FN_\alpha}, \quad \alpha \in \{\text{TTW}, \text{BSHH}, \text{ME}\} \quad (4.14)$$

$$\overline{F1} = \frac{1}{3} \sum_{\alpha \in \{\text{TTW}, \text{BSHH}, \text{ME}\}} F1_\alpha \quad (\text{macro average}) \quad (4.15)$$

M10: Cryptographic Pre-Filter Rejection Accuracy (FRA and FRR). The HMAC+timestamp+nonce pre-detection filter (Eqs. 3.15–3.17) operates as an

independent binary classifier prior to the detection engine. Its quality is captured by two sub-metrics evaluated under ablation variant A6 (No-Crypto-Filter):

$$\text{FRA} = \frac{TP_f}{TP_f + FN_f} \quad (4.16)$$

$$\text{FRR} = \frac{FP_f}{FP_f + TN_f} \quad (4.17)$$

where TP_f is the count of malicious beacons correctly rejected by the filter, FN_f is malicious beacons that pass through to the detector, FP_f is legitimate beacons incorrectly dropped, and TN_f is legitimate beacons correctly passed. Theory predicts $\text{FRA} = 1.0$ and $\text{FRR} = 0$ for external (vehicle/RSU-origin) attack vectors, and $\text{FRA} = 0$ for controller-origin vectors (the filter provides no pre-detection value per the cryptographic placement analysis in Section 3.4.10). The empirical values are a direct test of these theoretical claims.

M11: Threshold Signature Forgery and Quorum Rejection Rate (FSR and QRR). Two sub-metrics evaluate the ME and BSHH cryptographic defenses under ablation variant A5 (No-LBS), covering the threshold aggregate signature (Eq. 3.28) and location-binding quorum (Eq. 3.32) respectively.

Forgery success rate as Byzantine collusion size f_c varies:

$$\text{FSR}(f_c) = \frac{N_{\text{forge-success}}(f_c)}{N_{\text{forge-attempts}}(f_c)}, \quad f_c \in \{1, 2, \dots, t-1, t\} \quad (4.18)$$

Theory: $\text{FSR}(f_c) = 0$ for all $f_c < t$; $\text{FSR}(f_c) > 0$ only at $f_c \geq t$ (Eq. 3.28).

Quorum rejection rate for ME false-witness echoes:

$$\text{QRR} = 1 - \frac{N_{\text{echo-pass}}}{N_{\text{echo-attempts}}} \quad (4.19)$$

where $N_{\text{echo-pass}}$ is the count of ME echo injections that satisfy both the cryptographic and physical consistency checks in Eq. 3.32 and are admitted to the controller topology, and $N_{\text{echo-attempts}}$ is total ME injections. Theory: $\text{QRR} = 1.0$ for all out-of-range attackers satisfying $d(\text{pos}V_k, \text{eij}) > r_{\text{comm}}$.

M12: Trust Convergence and LKH Revocation Latency (T_{trust} , T_{revoke} , T_{reassign}). Three sub-metrics jointly evaluate the trust management layer (Eq. 3.40), the RSU demotion pipeline (Section 3.4.12), and the LKH session-key revocation (Eq. 3.18):

$$T_{\text{trust}}(n_k) = \tau_{\text{zero}}(n_k) - \tau_{\text{first-malicious}}(n_k) \quad (4.20)$$

$$T_{\text{revoke}} = \tau_{\text{LKH-complete}} - \tau_{\text{detection-flag}} \quad (4.21)$$

$$T_{\text{reassign}} = \tau_{C_k^*, \text{active}} - \tau_{C_j, \text{flagged}} \quad (4.22)$$

where $\tau_{\text{zero}}(n_k)$ is the time at which node n_k 's trust reaches 0, $\tau_{\text{first-malicious}}(n_k)$ is the first malicious event from n_k , $\tau_{\text{LKH-complete}}$ is when all $C_{\text{revoke}} = \lceil \log_2 n \rceil$ KEK updates are distributed (Eq. 3.18), and T_{reassign} applies to controller-origin attacks (Eqs. 3.44–3.45). The composite completion latency is:

$$T_{\text{mitigation-complete}} = \max(T_{\text{trust}}, T_{\text{revoke}}, T_{\text{reassign}}) \quad (4.23)$$

Table 4.4: Performance evaluation metrics: design component coverage, equation reference, and baseline comparability. Ablation-only metrics are applicable to ablation configurations A1–A6 but not to external baselines B1–B3.

M#	Metric	Design Component(s)	Equation(s)	SOTA-comp.
M1	MCC	LW signature scoring (Eq. 3.12); FS binary scorer (Eq. 3.25)	4.1	Yes
M2	Topology Divergence Reduction (TDRR)	Formal attack criterion (Eq. 3.1); divergence detector (Eq. 3.47)	4.2	Yes
M3	Temporal Stale-State Duration (T_{stale})	TTW/BSHH attack impact; convergence bound (Eq. 3.35)	4.3, 4.4	Yes
M4	Echo Path Inflation Ratio (PIR)	ME attack impact; path-growth bound (Eq. 3.9)	4.5, 4.6	Yes
M5	System Pipeline Latency (T_{pipeline})	LW/FS detection; PBFT consensus (Eq. 3.49); smart contract; FlowMod propagation	4.7	Conditional
M6	Routing Recovery (PDR, T_{e2e})	Data-plane consequence of full mitigation pipeline	4.8, 4.9	Yes
M7	Beacon Overhead (Ω)	HMAC construction (Eq. 3.51); location-binding message (Eq. 3.29); ML-DSA-87 signature (Eq. 3.30)	4.10, 4.11	Conditional
M8	Scalability (ρ_{det} , ρ_{PBFT})	$O(W)$ LW complexity (Eq. 3.14); PBFT peer-count scaling (Eq. 3.49)	4.12, 4.13	Conditional
M9	Variant Classification F1	FS softmax head (Eq. 3.26)	4.14, 4.15	<i>Ablation only</i>
M10	Pre-Filter Accuracy (FRA, FRR)	HMAC+timestamp+nonce filter (Eqs. 3.15–3.17)	4.16, 4.17	<i>Ablation only</i>
M11	Forgery/Quorum Resistance (FSR, QRR)	Threshold signature (Eq. 3.28); location-binding quorum (Eq. 3.32)	4.18, 4.19	<i>Ablation only</i>
M12	Trust Convergence & Revocation Latency	Trust update (Eq. 3.40); demotion pipeline; LKH revocation (Eq. 3.18); controller reassignment (Eqs. 3.44–3.45)	4.20–4.23	<i>Ablation only</i>

4.3.8 Research Questions

Table 4.5 states the six research questions addressed by the five benchmark experiments defined in Section 4.3.9. RQ1–RQ5 each correspond to one experiment producing graphical results; RQ6 is answered by the overhead metrics (M5, M7) reported across all experiments.

Table 4.5: Research questions and the experiment or metric set that answers each.

RQ	Question	Answered by
RQ1	How effectively does TETA-Guard detect and mitigate each attack variant against each origin vector compared to the B1–B3 baselines under default deployment settings?	Experiment 5 (Table 4.6)
RQ2	How does TETA-Guard performance degrade as joint attack penetration and injection intensity increase from a no-attack baseline to full-network compromise?	Experiment 1
RQ3	How robust is TETA-Guard detection and mitigation performance across the vehicular mobility range supported by DSRC-equipped SDVNs?	Experiment 2
RQ4	Does TETA-Guard detection and mitigation performance scale sub-linearly with increasing network size, consistent with the $O(W)$ analytical complexity bound?	Experiment 3
RQ5	How does TETA-Guard performance degrade as the total false evidence volume generated per attacker per attack episode (attack aggressiveness index Ψ) increases?	Experiment 4
RQ6	What is the communication and pipeline latency overhead imposed by the dual-layer PQC authentication and blockchain enforcement architecture?	M5 and M7 across Experiments 1–5

4.3.9 Benchmark Experiment Definitions

All benchmark experiments compare TETA-Guard against baselines B1–B3 under identical simulation conditions (Section 4.4). All 12 attack scenarios are active simultaneously in each run; results are aggregated across attack families to produce system-level performance curves (Experiments 1–4) or disaggregated per scenario for the single-point comparison (Experiment 5). Default simulation parameters follow Table 4.8 unless explicitly overridden by the experiment’s independent variable.

Experiment 1: Joint Attack Penetration and Intensity (RQ2)

Independent variable. Let $L \in \{0, 0.20, 0.40, 0.60, 0.80, 1.00\}$ be the joint attack load level, defining both dimensions simultaneously:

$$\alpha_{\text{atk}}(L) = L, \quad r_{\text{inj}}(L) = L \quad (4.24)$$

where $\alpha_{\text{atk}} \in [0, 1]$ is the fraction of the $N = 200$ vehicles designated as malicious (attack penetration) and $r_{\text{inj}} \in [0, 1]$ is the per-attacker injection rate: the fraction of beacon intervals in which each malicious vehicle injects at least one false

topology claim (attack intensity). Both dimensions are set equal at each operating point, producing a joint diagonal sweep across the two-dimensional penetration–intensity space.

The resulting total false beacon load on the detection engine is:

$$\Lambda(L) = \underbrace{L}_{\alpha_{\text{atk}}} \cdot \underbrace{L}_{r_{\text{inj}}} \cdot N \cdot \frac{T_{\text{obs}}}{T_b} = L^2 \cdot N \cdot \frac{T_{\text{obs}}}{T_b} \quad (4.25)$$

Λ scales quadratically with L : doubling the attack level quadruples the false beacon load. Controller compromise is co-modeled: at level L , each controller is independently compromised with probability $p_C = L$, consistent with the attack model in Section 3.4.1. $L = 0$ is the attack-free baseline; $L = 1.0$ represents a maximally aggressive, fully penetrated network. The six equidistant points span $[0, 1]$ in steps of 0.20.

Applicable PEMs: M1, M2, M3 (TTW/BSHH), M4 (ME), M5 (T_{pipeline}), M6 (PDR, T_{e2e}), M7 (Ω).

Experiment 2: Vehicular Mobility (RQ3)

Independent variable. Maximum vehicle speed:

$$v_{\text{max}} \in \{10, 60, 100, 140\} \text{ km/h} \equiv \{2.78, 16.67, 27.78, 38.89\} \text{ m/s} \quad (4.26)$$

Higher v_{max} reduces the expected link lifetime L_{link} (Eq. 3.33), increases topology churn, and raises the natural topology divergence floor, stressing the mobility-adaptive threshold δ_{thresh} (Eq. 3.48). All attack scenarios operate at the default penetration level ($\alpha_{\text{atk}} = 0.20$, $r_{\text{inj}} = 0.20$).

SUMO mobility configuration. To ensure all vehicles travel at exactly v_{max} with no per-vehicle speed randomisation, the following settings are mandatory in the SUMO `vType` definition and road network: (i) set `speedFactor="1.0"` and `speedDev="0.0"` in the `vType` block so SUMO does not randomise individual speeds around the limit; (ii) set `maxSpeed` in the `vType` to the target value in m/s (Eq. 4.26); (iii) set the `speed` attribute of every road edge in the `.net.xml` to the same m/s value. All three settings must match; a mismatch between edge speed and `vType maxSpeed` will cause vehicles to travel at the lower of the two values, producing inconsistent results across seeds.

Applicable PEMs: M1, M2, M3, M4, M5, M6.

Experiment 3: Network Scalability (RQ4)

Independent variable. Total vehicle count:

$$N \in \{100, 200, 300, 400\} \text{ vehicles} \quad (4.27)$$

The RSU count (64) and controller count (4) are held at their default values across all four points so that any scaling effects are attributable to the vehicle-side detection workload, not infrastructure growth. The attacker fraction is held at $\alpha_{\text{atk}} = 0.20$ so that the absolute number of attackers scales proportionally with N . This experiment directly tests the $O(|\mathcal{W}|)$ analytical complexity claim (Eq. 3.14): if the claim holds, $T_{\text{det}}^{\text{LW}}$ should remain approximately constant across the four N values, since the per-node detection window $|\mathcal{W}|$ is fixed regardless of network size.

Applicable PEMs: M1, M2, M3, M4, M5, M6, M8 ($\rho_{\text{det}}, \rho_{\text{PBFT}}$).

Experiment 4: Attack Aggressiveness Index Ψ (RQ5)

Independent variable. The attack aggressiveness index Ψ jointly parameterises false evidence multiplicity f_m and burst duration T_{burst} , capturing the total number of distinct false topology claims generated per attacker per contiguous injection episode:

$$\Psi = f_m \cdot \left\lfloor \frac{T_{\text{burst}}}{T_b} \right\rfloor \quad (4.28)$$

where $f_m \in \mathbb{Z}^+$ is the false evidence multiplicity (number of distinct false topology claims injected per attacker per beacon interval) and T_{burst} is the duration of each contiguous injection episode before the attacker goes dormant. Ψ is normalized to beacon intervals: it counts the total false evidence volume that the detection engine must process per attacker per burst.

The physical meaning of f_m is attack-family specific: for TTW, f_m is the number of distinct stale links replayed per interval; for BSHH, the number of distinct ghost vehicle identities asserted; for ME, the number of distinct phantom topology paths fabricated.

Four operating points, log-spaced at approximately $4\times$ increments:

$$(f_m, T_{\text{burst}}, \Psi) \in \{(1, 10\text{s}, 100), (2, 30\text{s}, 600), (4, 60\text{s}, 2,400), (8, 120\text{s}, 9,600)\} \quad (4.29)$$

At each operating point, $\alpha_{\text{atk}} = 0.20$ and $r_{\text{inj}} = 1.0$ within the burst window. Between bursts, attackers are silent (injection rate drops to 0). The experiment tests whether Ψ alone (irrespective of how it is factored into f_m and T_{burst}) governs detection degradation, or whether the two dimensions have asymmetric effects on the LW and FS detection paths.

Applicable PEMs: M1, M2, M3, M4, M5, M6.

Experiment 5: Per-Variant Comparison Table (RQ1)

Setup. A single run under default simulation settings (Table 4.8, $L = 0.20$ attack load, $v_{\max} = 60$ km/h, $N = 200$ vehicles, 5 seeds averaged). Each of the 12 attack scenarios is evaluated independently with one method active at a time (TETA-Guard, B1, B2, B3).

Output format. Results are reported in Table 4.6 with one row per attack-origin scenario and one column group per method. Ablation-only PEMs (M9–M12) are excluded because B1–B3 provide no equivalent mechanism.

Applicable PEMs: M1, M2, M3 (TTW/BSHH rows), M4 (ME rows), M5 (T_{pipeline}), M6 (PDR, T_{e2e}), M7 (Ω).

Table 4.6: Experiment 5 result placeholder: per-attack-variant comparison of TETA-Guard against B1–B3 under default settings. $\uparrow$: higher is better. $\downarrow$: lower is better. [TBD: populate with simulation results.]

Attack	Origin	M1: MCC $\uparrow$				M2: TDRR (%) $\uparrow$				M5: T_{pipe} (ms) $\downarrow$			
		Ours	B1	B2	B3	Ours	B1	B2	B3	Ours	B1	B2	B3
TTW	Vehicle	–	–	–	–	–	–	–	–	–	–	–	–
TTW	RSU	–	–	–	–	–	–	–	–	–	–	–	–
TTW	Controller	–	–	–	–	–	–	–	–	–	–	–	–
BSHH	Vehicle	–	–	–	–	–	–	–	–	–	–	–	–
BSHH	RSU	–	–	–	–	–	–	–	–	–	–	–	–
BSHH	Controller	–	–	–	–	–	–	–	–	–	–	–	–
ME	Vehicle	–	–	–	–	–	–	–	–	–	–	–	–
ME	RSU	–	–	–	–	–	–	–	–	–	–	–	–
ME	Controller	–	–	–	–	–	–	–	–	–	–	–	–
No attack (with RSU)		–	–	–	–	–	–	–	–	–	–	–	–
No attack (no RSU)		–	–	–	–	–	–	–	–	–	–	–	–
Mean (attack scenarios)		–	–	–	–	–	–	–	–	–	–	–	–

Note: M3 ($\bar{T}_{\text{stale}}$) reported for TTW and BSHH rows only; M4 ($\bar{\text{PIR}}$) reported for ME rows only; M6 (PDR, T_{e2e}) and M7 (Ω) reported in a companion sub-table (Table 4.6).

4.4 Simulation Settings

We selected an urban area of Colombo, Sri Lanka for simulation by importing mobility traces generated using SUMO (Simulation of Urban MObility) [46] and OpenStreetMap tools [50]. The road network covers approximately $2.46 \text{ km} \times 2.38 \text{ km}$ ($\approx 5.85 \text{ km}^2$), and the vehicular network is simulated using NS-3.35 [48] on Linux. We simulate 200 vehicles and deploy 64 RSUs in a fixed 8×8 grid layout (spacing: $273 \text{ m} \times 264 \text{ m}$, origin at $(273, 264) \text{ m}$), giving 264 nodes in total. Adjacent RSUs are connected by point-to-point Ethernet links (56 horizontal + 44 vertical links, 1 Gbps, $10 \mu\text{s}$ delay). Vehicle initial positions and speeds are loaded directly from SUMO mobility traces; the maximum vehicle speed is 60 km/h, and the total simulation time is 310 s (10 s warm-up + 300 s observation window).

Figure 4.1 presents the OpenStreetMap (OSM)-based road network of Colombo used as the simulation environment in SUMO. The map represents the urban transportation infrastructure, including major roads, intersections, and connec-

tivity patterns, providing a realistic framework for evaluating vehicle mobility and traffic behavior within the study area.



Figure 4.1: Real-world OpenStreetMap-based Colombo urban map used for SUMO GUI mobility simulation.

Figure 4.2 illustrates the NetAnim visualization of the proposed SDN-enabled VANET simulation environment. The figure shows the deployment of 200 mobile vehicles, 64 fixed RSUs arranged in an 8×8 grid topology, the centralized SDN controller, and the management server. Vehicle nodes move according to SUMO-generated mobility traces, while RSUs provide roadside communication and forward topology information to the SDN controller through the wired backbone network. This visualization confirms the spatial distribution of vehicles and infrastructure components within the simulated Colombo urban scenario.

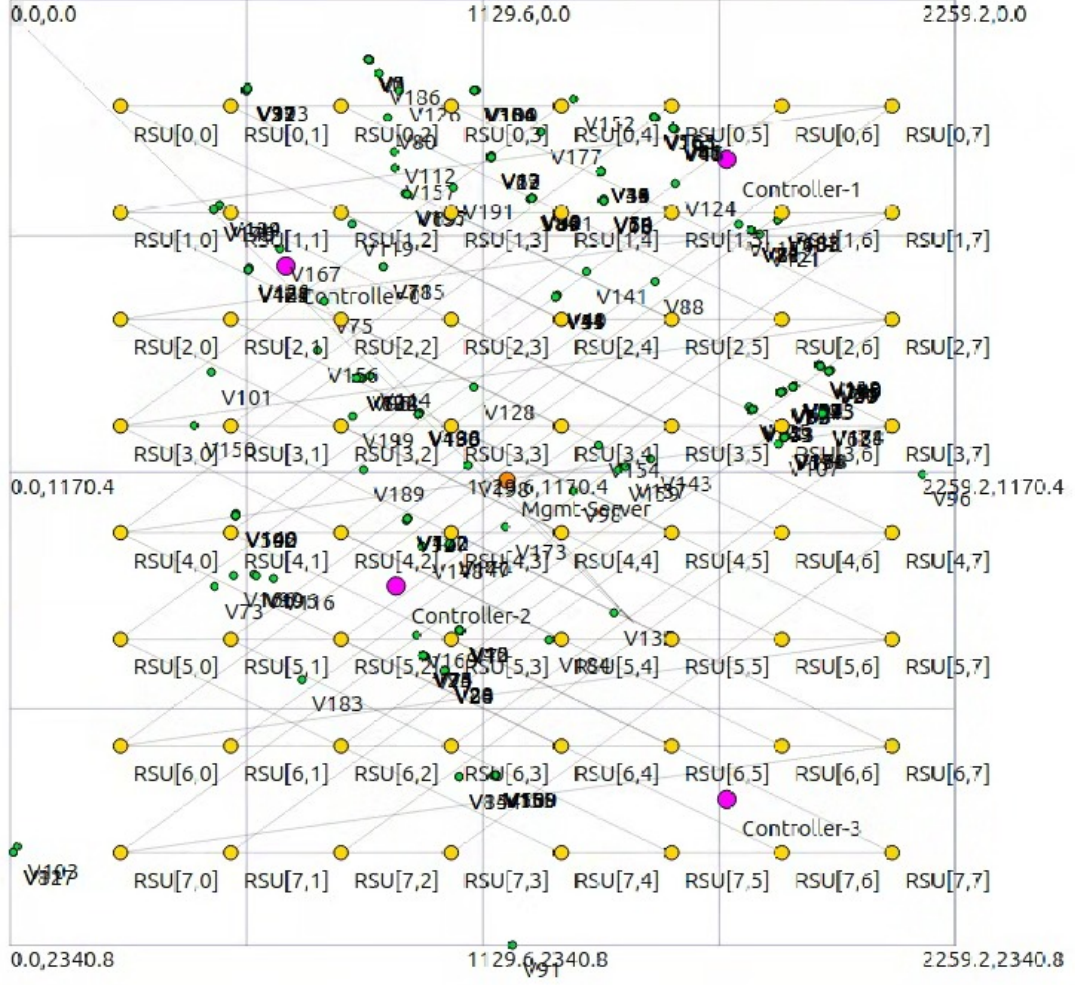


Figure 4.2: NetAnim visualization of the SDN-enabled VANET simulation topology consisting of 200 vehicles, 64 RSUs arranged in an 8×8 grid

DSRC (Dedicated Short-Range Communications) operating at 5.9 GHz under the IEEE 802.11p standard [47] is used for all V2V and V2R data-plane communication. The DSRC spectrum is divided into seven 10 MHz channels (Ch 172–184); Channel 178 serves as the dedicated Control Channel (CCH) for topology beacons at 44 dBm (simulation value; $\text{EIRP} \leq 44.8 \text{ dBm}$ maximum per FCC Report and Order FCC 03-324 [44]), while the remaining six are Service Channels (SCH) with per-channel powers ranging from 23 to 40 dBm (see Table 4.7). The COST-231 Hata urban propagation loss model (`ns3::Cost231PropagationLossModel`) is applied to all seven channels [44]. Each vehicle broadcasts topology beacons on all seven channels at 10 Hz (100 ms interval) using IEEE 802.11p with CSMA/CA MAC ($\text{CW}_{\min} = 15$, $\text{CW}_{\max} = 127$, $\text{AIFSN} = 6$, $\text{AIFS} = 132 \mu\text{s}$, $\text{TXOP} = 5 \text{ ms}$ [51], queue limit 50 000 packets) and a PHY rate of 12 Mbps. For the control plane, RSUs forward aggregated topology reports to the SDN controller [25] over CSMA Ethernet (UDP port 7777).

DSRC Communication Range with Variable Transmission Power

The DSRC communication range in this simulation is not fixed and varies based on the configured transmit power (EIRP) per channel, following the COST-231 Hata propagation model [49].

Three attack families are studied across four attacker-placement variants each (12 scenarios total): TTW (Topology Time-Warp), BSHH (Beacon State Heart-beat Hijack), and ME (Multipath Echo). The detection latency budget of 100 ms. Simulation settings are summarised in Tables 4.7 and 4.8.

Table 4.7: DSRC channel settings and estimated communication range — urban scenario

Channel	Freq (GHz)	Type	TX Power (dBm)	Range (m)
Ch172	5.860	SCH	33.0	133.3
Ch174	5.870	SCH	33.0	133.3
Ch176	5.880	SCH	33.0	133.3
Ch178 (CCH)	5.890	CCH	44.0^a	282.2
Ch180	5.900	SCH	23.0	67.4
Ch182	5.910	SCH	23.0	67.4
Ch184	5.920	SCH	40.0 ^b	214.8

^a Simulation value; FCC 03-324 specifies maximum EIRP ≤ 44.8 dBm for public safety RSUs on Ch178 [44].

^b 40 dBm applies to public safety RSU operations only; private RSUs are limited to 33 dBm per FCC 03-324 [44].

Table 4.8: Simulation settings.

Parameter	Value
Network simulation tool	NS-3.35 (Linux, Ubuntu)
Mobility trace extraction	SUMO, OpenStreetMap
Map area	Urban Colombo, Sri Lanka, $\approx 2.46 \text{ km} \times 2.38 \text{ km}$ ($\approx 5.85 \text{ km}^2$)
Number of nodes	264 (200 vehicles, 64 RSUs)
Vehicle placement	SUMO NS-2 trace (initial positions and speeds), ConstantVelocityMobilityModel
RSU deployment	8×8 fixed grid, origin (273, 264) m, spacing $273 \text{ m} \times 264 \text{ m}$
RSU interconnect	P2P Ethernet (56 horiz. + 44 vert. links, 1 Gbps, $10 \mu\text{s}$ delay)
Simulation time	310 s (10 s warm-up + 300 s observation window)
Maximum vehicle speed	60 km/h (urban)
Data plane	5.9 GHz DSRC / IEEE 802.11p (7 channels, Ch 172–184)

Continued on next page

Parameter	Value
MAC mode	IEEE 802.11p Ad-hoc (AdhocWifiMac, QoS enabled)
PHY data rate	12 Mbps (OfdmRate12MbpsBW10MHz)
Traffic class	Best effort (qf = 1): $CW_{\min} = 15$, $CW_{\max} = 127$, AIFSN = 6
MAC timing	SIFS = 12 μ s, $T_{\text{slot}} = 20 \mu$ s, AIFS = 132 μ s
TXOP limit / queue	5 ms; max 50 000 packets
Control plane	CSMA Ethernet, RSU $\rightarrow$ Controller, UDP port 7777
Key agreement / KEM (Eqs. 3.15–3.17)	ML-KEM-1024 (Kyber-1024, NIST FIPS 203, Level 5) + HQC-5 (NIST-selected for standardization March 2025, NIST IR 8545, code-based syndrome-decoding, Level 5; draft FIPS $\approx$ 2026, final $\approx$ 2027). Hybrid KEM: session key K_{V_i, n_k} is derived from both mechanisms independently; attacker must break <i>both</i> ML-KEM-1024 (lattice) <i>and</i> HQC-5 (code-based) to compromise the key [64, 66, 67].
Propagation loss model	COST-231 Hata urban (ns3::Cost231PropagationLossModel)
Beacon broadcast frequency	10 Hz per vehicle per channel (100 ms interval)
PEM multi-channel aggregation note	The NS-3 PEM function accumulates <i>unique reporter vehicles</i> per link e_{ij} per beacon interval; per-channel beacon events from the same vehicle are de-duplicated before computing $ R(e_{ij}, t) $ (Eq. 3.8). All model formulas reference $T_b = 100$ ms as the Ch 178 (CCH) beacon interval.
Flow packet size	750 bytes
Number of flows	2 bidirectional pairs (4 flow directions)
Attack families	TTW, BSHH, ME ($3 \times 4 = 12$ scenarios)
Number of independent seeds	5 (results averaged; standard deviation reported)
TGN inference (C++ detector)	$d = 32$, $L = 2$ MP rounds, per-node TBPTT $W_{\text{BPTT}} = 100$, $\gamma = L_{\text{link}}/(2T_b \ln 2)$ (auto); see Table 3.2 for full training hyperparameter defaults
LW detection threshold in NS-3 PEM function (θ_{LW} , Eq. 3.12)	θ_{LW} [TBD: {0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50}] selected by F1 maximisation on NS-3 validation split [53].)
Detection latency budget	<100 ms
Performance evaluation metrics	M1–M8 (SOTA-comparable); M9–M12 (ablation-only); see Table 4.4

Table 4.9: Empirically calibrated parameters (post NS-3/SUMO and TGN training). **TBD** = pending experiment. Notation follows Table 3.1.

Parameter	Symbol	Current Default	Calibration Method	Empirical Value
A. Detection Threshold Parameters				
LW alert threshold	θ_{LW}	Not set; determined post-training	Grid search over $[0, 1]$ in steps of 0.05 on the validation split; select value maximising F1 score	TBD
FS anomaly score threshold	θ_{FS}	Not set; determined post-training	Sweep sigmoid output thresholds on the validation set; select value maximising MCC under class imbalance	TBD
TGN edge-weight decay rate	γ	Set so $A_{uv}(t) \approx 0.5$ at report age $L_{\text{link}}/2$ (formula-derived per speed)	Verify on simulation traces that TTW-replayed edges receive near-zero weight; tune if detection accuracy drops	TBD
B. LW Signature Weights (w_1–w_9, Eq. 3.12)				
TTW-S1 weight: topology state persistence anomaly (Eq. 3.2)	w_1	$1/9 \approx 0.111$ (uniform, calibration-neutral baseline)	Recalibrate proportionally to TTW-S1’s observed FP rate on real-network data; maintain $\sum_{k=1}^9 w_k = 1, w_k \geq 0$	TBD
TTW-S2 weight: timestamp-topology age inconsistency (Eq. 3.3)	w_2	$1/9 \approx 0.111$ (uniform, calibration-neutral baseline)	Recalibrate proportionally to TTW-S2’s observed FP rate on real-network data; maintain $\sum_{k=1}^9 w_k = 1, w_k \geq 0$	TBD
TTW-S3 weight: cross-reporter inconsistency (Eq. 3.4)	w_3	$1/9 \approx 0.111$ (uniform, calibration-neutral baseline)	Recalibrate proportionally to TTW-S3’s observed FP rate on real-network data; maintain $\sum_{k=1}^9 w_k = 1, w_k \geq 0$	TBD
BSSH-S1 weight: duplicate liveness assertion (Eq. 3.5)	w_4	$1/9 \approx 0.111$ (uniform, calibration-neutral baseline)	Recalibrate proportionally to BSSH-S1’s observed FP rate on real-network data; maintain $\sum_{k=1}^9 w_k = 1, w_k \geq 0$	TBD

Continued on next page

Table 4.9 – continued from previous page

Parameter	Symbol	Current Default	Calibration Method	Empirical Value
BSHH-S2 weight: timestamp regression / replay (Eq. 3.6)	w_5	$1/9 \approx 0.111$ (uniform, calibration-neutral baseline)	Recalibrate proportionally to BSHH-S2's observed FP rate on real-network data; maintain $\sum_{k=1}^9 w_k = 1, w_k \geq 0$	TBD
BSHH-S3 weight: liveness-beacon inconsistency (Eq. 3.7)	w_6	$1/9 \approx 0.111$ (uniform, calibration-neutral baseline)	Recalibrate proportionally to BSHH-S3's observed FP rate on real-network data; maintain $\sum_{k=1}^9 w_k = 1, w_k \geq 0$	TBD
ME-S1 weight: reporter count excess (Eq. 3.8)	w_7	$1/9 \approx 0.111$ (uniform, calibration-neutral baseline)	Recalibrate proportionally to ME-S1's observed FP rate on real-network data; maintain $\sum_{k=1}^9 w_k = 1, w_k \geq 0$	TBD
ME-S2 weight: path diversity inflation (Eq. 3.9)	w_8	$1/9 \approx 0.111$ (uniform, calibration-neutral baseline)	Recalibrate proportionally to ME-S2's observed FP rate on real-network data; maintain $\sum_{k=1}^9 w_k = 1, w_k \geq 0$	TBD
ME-S3 weight: reporter range/signal inconsistency (Eq. 3.11)	w_9	$1/9 \approx 0.111$ (uniform, calibration-neutral baseline)	Recalibrate proportionally to ME-S3's observed FP rate on real-network data; maintain $\sum_{k=1}^9 w_k = 1, w_k \geq 0$	TBD
C. Sliding Window & Observation Window Parameters				
LW detector max window size	$W_{\max}$	$\lceil L_{\text{link}}/T_b \rceil$ intervals; urban ≈ 430 , highway ≈ 90	Increase until no TTW-S2 cross-interval misses; decrease until TTW-S3 false positives from departed vehicles disappear	TBD
BSHH-S3 liveness observation window	W_{BSHH}	$W > 2W_{\text{ho}}$ (formula-derived)	Confirm $W > W_{\text{ho}}$ prevents handover false positives; validate BSHH-S3 still fires before the stale heartbeat expires	TBD
D. Physical and Signal Consistency Parameters				
Max propagation tolerance (freshness bound, Eq. 3.16)	ε	Not set; conceptually $\approx$ one-way propagation delay	Measure 95th-percentile one-way latency from NS-3 simulation logs; set ε equal to this value	TBD

Continued on next page

Table 4.9 – continued from previous page

Parameter	Symbol	Current Default	Calibration Method	Empirical Value
Detection formula range (r_{comm})	r_{comm}	300 m design default (<code>loadSimParams</code> ; overridable via <code>SIM_RCOMM</code>); NS-3 Ch. 178 CCH at 44 dBm yields an effective range of 282.2 m under COST-231 Hata propagation [44, 49]	Analytical formulations (ME-S1 ρ_{max} , ME-S3 range check, $\delta_{\text{thresh}} = 14$) use the 300 m DSRC design bound; results derived from NS-3 traces use the channel-effective 282.2 m value	300 m (analytical design bound) / 282.2 m (NS-3 Ch. 178 effective)
Minimum RSSI threshold (ME-S3, Eq. 3.31)	$\text{RSSI}_{\min}(r_{\text{comm}})$	−85 dBm default (<code>loadSimParams</code> ; overridable via <code>SIM_RSSI_MIN</code>)	Derived from NS-3 propagation model at r_{comm} ensuring in-range beacon validity	−85 dBm (default)
Inter-lane density margin (ME-S1, Eq. 3.8)	μ	0.20 (recommended default)	Increase if urban runs produce false positives on legitimate multi-lane observers; decrease to improve sparse-traffic sensitivity	TBD
E. Trust Management Parameters				
Controller trust penalty per confirmed divergence (Eq. 3.41)	Δ_-^C	0.20 (<code>TrustDeltaCtrl</code> , chaincode constant; penalty-only update)	Controller trust is decremented by $\Delta_-^C = 0.20$ on each confirmed divergence via <code>updateCtrlTrust</code> ; reassignment is checked separately against $\tau_{\min}^C$	0.20
Node admission / consensus-participation floor	$\tau_{\min}$	0.10 (<code>TrustMin</code> , chaincode constant)	Peers below this threshold are excluded from P_{active} via <code>selectPeers</code> , <code>SubmitAlert</code> , and <code>SyncFromAnchorCheckpoint</code>	0.10
Ground-truth / bootstrap-completion threshold	$\tau_{\min}^{\text{gt}}$	0.50 (<code>TrustMinGT</code> , chaincode constant)	Defines bootstrap completion; used by <code>isBootstrapComplete</code> , <code>SubmitBeaconEvidence</code> , and <code>GetTrustedEvidence</code>	0.50
Bootstrap phase duration (consensus rounds before threshold enforcement, Eq. 3.40)	$R_{\min}$	8 rounds; computed as $\left\lceil \frac{\tau_{\min}^{\text{gt}} - \tau_0}{\Delta^+} \right\rceil = \left\lceil \frac{0.50 - 0.10}{0.05} \right\rceil$	Startup latency satisfies $8 \times T_{\text{PBFT}} \ll T_{\min}$; recompute if Δ^+ or $\tau_{\min}^{\text{gt}}$ changes	8 rounds

Continued on next page

Table 4.9 – continued from previous page

Parameter	Symbol	Current Default	Calibration Method	Empirical Value
Controller reassignment trust threshold (Eq. 3.44)	$\tau_{\min}^C$	0.30 (TrustCtrlMin, chaincode constant)	Triggers REASSIGN($Z_j \rightarrow C_k^*$) via CheckControllerTrustAnd Reassign when $\tau_j^C < \tau_{\min}^C$	0.30
OBU minimum dwell time before eligibility	$T_{\min}^{\text{dwell}}$	3000 ms (TrustMinDwellMs, chaincode constant)	<code>selectPeers</code> excludes OBUs whose <code>JoinedAtMs</code> is within the dwell window of current time	3000 ms
OBU minimum hardware capacity	—	2048 MB RAM (HWCapacityMinMB, chaincode constant)	<code>selectPeers</code> filters out OBUs below hardware threshold	2048 MB
RSU peer quarantine monitoring window	T_{quar}	30,000 ms (QuarantineMonitorMs, chaincode constant)	After a malicious RSU peer is demoted to <code>QUARANTINED_CLIENT</code> , T_{quar} is the minimum wall-clock duration before a removal check fires in <code>monitorAndRemovePeer</code> ; if trust remains $\leq \tau_{\min}$ after T_{quar} , state transitions to <code>REMOVED</code>	30,000 ms
Malicious RSU peer demotion pipeline	—	ACTIVE ↓ QUARANTINED_CLIENT ↓ REMOVED	On confirmed attack detection: (1) <code>demotePeerToClient</code> zeros trust, strips <code>IsRSUPeer</code> , emits <code>PeerQuarantined</code> event; (2) <code>monitorAndRemovePeer</code> checks after T_{quar} and emits <code>PeerRemoved</code> if trust has not recovered; triggered automatically at end of every <code>runMitigation</code> call and by periodic <code>PeriodicPeerReSelection</code> from the client	3 stages

Continued on next page

Table 4.9 – continued from previous page

Parameter	Symbol	Current Default	Calibration Method	Empirical Value
Periodic peer re-selection interval	T_{prs}	Every beacon interval T_b (called from <code>submitToFabric.js</code> after each <code>Mitigate</code> invocation; also at end of <code>runMitigation</code>)	Ensures QUARANTINED peers are evaluated for removal at every beacon interval, not only when an attack alert fires; <code>selectPeers</code> now excludes QUARANTINED_CLIENT and REMOVED states in addition to the Flagged and $\tau < \tau_{\text{min}}$ checks	$T_b = 100 \text{ ms}$
F. TGN Model Hyperparameters				
TBPTT window size <i>per node</i>	W_{BPTT}	Per-node: each node v maintains an independent event counter; $\mathbf{h}_v$ is detached only after v accumulates W_{BPTT} of its <i>own</i> events. Global temporal order of the event stream is preserved for correct $\mathbf{A}_{uv}(t)$ computation. Global-window formulation (detach all states every W global events) is insufficient: with 200 nodes each contributing ≈ 0.5 events per $W = 100$ global window, effective per-node gradient depth < 1 step $\equiv$ 1-step TBPTT.	Sweep $\{50, 100, 200\}$ events <i>per node</i> on validation MCC; default 100. Context states $\mathbf{h}_{\text{link_src}}, \mathbf{h}_{\text{link_dst}}$ must be detached before message-passing aggregation (frozen context, not gradient targets).	[TBD: {50,100, 200}]; default 100

Continued on next page

Table 4.9 – continued from previous page

Parameter	Symbol	Current Default	Calibration Method	Empirical Value
GRU total input dimension (after feature revision T1 + T2)	d_{in}	Prior value: 39 = 32 + 7 (7-feature vector including <code>id_v</code> and absolute <code>tau_s</code>)	Fixed by architecture after removing <code>id_v</code> and replacing $\tau_s^{(v)}$ with $\tau_{\text{dev}}^{(v)}$: 6-element input (<code>tau_dev</code> , c_v^W , Δs_v , ρ_v , ι_v , ϕ). Update <code>TGNDetector::LoadWeights()</code> input-size constant. Prior checkpoint (dim 39) is incompatible; full retrain required.	38 (= 32+5+1)
Training loss: score-separation margin weight (Eq. 3.27, term iii)	λ_m	0.50 (fixed)	Fixed by design at 0.50; tune only if term (iii) dominates gradient magnitude relative to terms (i) and (ii), detectable when validation BCE rises while validation MCC improves (margin-driven score separation without true-positive quality).	0.50
Training loss: score-separation margin threshold (Eq. 3.27, term iii)	m	0.35 (fixed)	Fixed at 0.35; represents the minimum acceptable gap between mean attack and mean benign scores within a mini-batch. Increase if score collapse is observed despite training; decrease if the margin penalty prevents BCE from converging.	0.35
Training restart convergence target	$\text{MCC}_{\text{target}}$	0.975 (on validation split)	If best validation MCC after 50 epochs < 0.975, reinitialize all weights and restart. Maximum 5 attempts; retain best checkpoint across all attempts. Test split sealed throughout.	0.975; max 5 restarts
Message-passing rounds (Eq. 3.24)	L	2 (Table 4.1)	Test $L \in \{1, 2, 3\}$; higher L captures wider attack-propagation neighbourhoods but risks over-smoothing in sparse vehicular graphs [5, 6]	TBD

Continued on next page

Table 4.9 – continued from previous page

Parameter	Symbol	Current Default	Calibration Method	Empirical Value
Node feature embedding dimension (Eq. 3.20)	d	Not specified	Evaluate $d \in \{32, 64, 128\}$; select by validation MCC; match GRU hidden-state size to d	TBD
GRU hidden-state size (Eq. 3.23)	h_{GRU}	Not specified; architecturally tied to d ($h_v(t) \in \mathbb{R}^d$)	Match to d by default; tune separately only if the temporal memory update over-fits relative to the embedding dimension	TBD
Optimiser learning rate	η	Not specified	Start at 1×10^{-3} ; reduce on plateau; use cosine annealing for temporal GNN training stability	TBD
Class imbalance weight (benign vs. attack ratio)	w_{class}	Binary cross-entropy with class-weight balancing (qualitative; not numerically specified)	Compute from the training split as $w_{\text{attack}} = \text{total} / (2 \cdot n_{\text{attack}})$; recompute after each train/validation/test split	TBD
Train / validation / test split	—	70/15/15	Fixed by design; confirm no class leakage across splits, especially for time-ordered attack scenarios across the twelve attack variants	70/15/15
G. Consensus and Quorum Parameters				
Threshold aggregate signature majority (Eq. 3.28, Eq. 3.32)	t	$t \geq \lfloor n/2 \rfloor + 1$ (strict majority)	Confirm n from SUMO density traces; recompute t ; verify no single-vehicle or sub-quorum collusion forges a valid aggregate report under simulation	TBD
Vehicles per RSU reporting zone (Eq. 3.28)	n	Varies with λ and r_{comm}	Extract min/max/mean from SUMO mobility traces; report separately for urban and highway scenarios	TBD
Max Byzantine peers tolerated by PBFT	f	2 (FaultToleranceF, fixed system parameter; consistent with 8-peer consortium)	Consistent with $\text{Peer}_{\text{total}} = 8$; used as safety bound for PBFT fault tolerance	f = 2

Continued on next page

Table 4.9 – continued from previous page

Parameter	Symbol	Current Default	Calibration Method	Empirical Value
Active PBFT peer set size (Eq. 3.43)	$n_p = 3f + 1$	8, derived from $f = 2$ in <code>selectPeers</code> / <code>runMitigation</code>	Ensures quorum availability with $n_p = 8$ peers; validated against consensus latency constraints	$\mathbf{n_p = 8}$
H. Blockchain and Trust Update Parameters				
Controller-origin divergence threshold (Eq. 3.47)	$\delta_{\text{thresh}}(t)$	Dynamic threshold computed per beacon interval via <code>computeDeltaThreshold</code> ; fallback uses $\hat{\lambda}(t) = 0.02$ when no evidence is available	Validated on NS-3/SUMO traces across urban and highway densities; static value 14 corresponds only to fallback condition	Dynamic (14 only for $\hat{\lambda} = 0.02$, floor = 2)
Node trust reward increment (Eq. 3.40)	Δ^+	0.05 (recommended default)	Confirm bootstrap completes within $R_{\min} = 8$ rounds and dwell time $T_{\min}$; decrease if Tier-2 OBUs gain trust too rapidly under simulated Sybil promotion	0.05
Node trust penalty decrement (Eq. 3.40)	Δ^-	0.10 (recommended default; $\Delta^- > \Delta^+$)	Verify attacker trust reaches zero before sustaining attack beyond 2 beacon intervals; ensure $\Delta^- > \Delta^+$ so trust is harder to gain than lose	0.10
Initial OBU trust score	τ_0	0.1 (Tier-2 OBU default)	$\tau_0 = \tau_{\min} = 0.10$ by design; peer participation for newly admitted OBUs is deferred by $\phi(k)$ (dwell-time gate in Eq. 3.42), not by the trust condition, which is satisfied at initialisation. Confirm $\tau_0 < \tau_{\min}^{\text{gt}} = 0.50$ so no newly registered OBU immediately qualifies as a trusted evidence source; validate that the bootstrap duration $R_{\min}$ is consistent with the chosen Δ^+ and $\tau_{\min}^{\text{gt}}$	0.1

Continued on next page

Table 4.9 – continued from previous page

Parameter	Symbol	Current Default	Calibration Method	Empirical Value
Anchor checkpoint interval	$\lfloor T_{\min}/T_b \rfloor$ blocks	Formula-derived from minimum dwell time $T_{\min}$ and beacon interval T_b	Measure synchronisation latency of newly promoted Tier-2 peers in simulation; confirm all peers reach consistent ledger state before joining consensus round	$\lfloor T_{\min}/T_b \rfloor$ blocks
Tier-2 ledger window size	$K = \lceil T_{\text{dwell}}/T_b \rceil$	Varies per OBU dwell time T_{dwell}	Extract per-vehicle dwell times from SUMO traces; set K to cover at least one full anchor checkpoint interval to prevent ledger gaps during handover	$\lceil T_{\text{dwell}}/T_b \rceil$
Smart contract execution latency	T_{exec}	50–200 ms (Hyperledger Fabric literature [7, 9, 10])	Measured per-event chaincode execution time; ensures $T_{\text{det}} + T_{\text{exec}} \leq 100$ ms safety budget	TBD
Tier 1 (RSU) initial trust score	$\tau_{\text{RSU},0}$	1.00 (TrustInitTier1, chaincode constant)	All RSU peers initialized at full trust during bootstrap phase (RegisterRSUPeer)	1.00
Orderer batch timeout	–	20 ms (configtx.yaml, reduced from 50 ms)	End-to-end commit path (batch + PBFT + propagation) constrained within 100 ms beacon budget	20 ms
Orderer batch size	–	MaxMessageCount = 50; PreferredMaxBytes = 512 KB; AbsoluteMaxBytes = 10 MB	Supports 50 transactions per 100 ms beacon interval under 8-peer load; adjust if topology changes	50 tx / 512 KB
Channel endorsement policy	–	MAJORITY of TetaGuardMSP peers (5-of-8 endorser peers)	Tolerates $f = 2$ Byzantine RSU under PBFT assumption (Eq. 3.49)	5-of-8

The parameters listed in Table 4.9 cannot be derived analytically in isolation; they depend on the statistical properties of the simulated environment (vehicle densities, mobility patterns, propagation characteristics) and on the learned behaviour of the TGN model. The following subsections describe the role of each parameter group, its default value or derivation rule, and the calibration procedure that will be applied once NS-3/SUMO traces and TGN training results are available.

Detection Threshold Parameters

The detection subsystem exposes two score-level thresholds and one temporal-decay rate that together govern the boundary between *alert* and *normal* classifications [52–54].

LW alert threshold (θ_{LW}). The Likelihood-Weighted (LW) detector outputs a continuous score in $[0, 1]$ for each observation window. The decision boundary θ_{LW} converts this score into a binary alert. Because no ground-truth distribution is available prior to simulation, θ_{LW} is not pre-set; it will be determined by a grid search over $[0, 1]$ in steps of 0.05 on the held-out validation split, selecting the value that maximises the F1 score [53]. The empirical value is **TBD** pending NS-3/SUMO trace collection and model training.

FS anomaly score threshold (θ_{FS}). The Feature-Space (FS) anomaly detector produces a sigmoid output whose threshold θ_{FS} must account for the severe class imbalance between attack and benign events. Accordingly, calibration will sweep sigmoid output thresholds on the validation set and select the value that maximises the Matthews Correlation Coefficient (MCC), which is more informative than accuracy or F1 under imbalance [54]. The empirical value is **TBD**.

TGN edge-weight decay rate (γ). Each directed edge in the Temporal Graph Network (TGN) carries a recency weight $A_{uv}(t)$ that decays with the age of the corresponding link report. The default is set analytically so that $A_{uv}(t) \approx 0.5$ at a report age equal to half the link lifetime $L_{\text{link}}/2$; the exact formula is speed-dependent [52]. Post-simulation, γ will be verified on NS-3 traces to confirm that Time-To-Waypoint (TTW)-replayed edges receive near-zero weight, and will be tuned if detection accuracy drops below the target. The empirical value is **TBD**.

LW Signature Weights (w_1 – w_9)

The Likelihood-Weighted (LW) detector aggregates nine binary signature indicators into a single anomaly score (Eq. 3.12) [55]. Each non-negative weight w_k determines the contribution of its corresponding signature to the overall score. During the current NS-3/SUMO evaluation, all nine signatures are assigned equal weights ($w_k = 1/9 \approx 0.111$) as a calibration-neutral baseline, satisfying the normalization constraint that the weights sum to one. This choice is consistent with the zero false-positive rate observed for all nine signatures under the controlled simulation environment (MCC = 1.0, AUROC = 1.0).

The same weight assignment methodology applies to all nine signatures. Following the simulation study, the weights will be recalibrated using the observed false-positive rates of their corresponding signatures on real-network data while preserving the normalization constraint. The empirically calibrated values are therefore **TBD**.

TTW-S1 weight: topology state persistence anomaly (w_1 , Eq. 3.2). Initial value: $1/9 \approx 0.111$; empirical value: **TBD**.

TTW-S2 weight: timestamp-topology age inconsistency (w_2 , Eq. 3.3). Initial value: $1/9 \approx 0.111$; empirical value: **TBD**.

TTW-S3 weight: cross-reporter inconsistency (w_3 , Eq. 3.4). Initial value: $1/9 \approx 0.111$; empirical value: **TBD**.

BSHH-S1 weight: duplicate liveness assertion (w_4 , Eq. 3.5). Initial value: $1/9 \approx 0.111$; empirical value: **TBD**.

BSHH-S2 weight: timestamp regression/replay (w_5 , Eq. 3.6). Initial value: $1/9 \approx 0.111$; empirical value: **TBD**.

BSHH-S3 weight: liveness-beacon inconsistency (w_6 , Eq. 3.7). Initial value: $1/9 \approx 0.111$; empirical value: **TBD**.

ME-S1 weight: reporter count excess (w_7 , Eq. 3.8). Initial value: $1/9 \approx 0.111$; empirical value: **TBD**.

ME-S2 weight: path diversity inflation (w_8 , Eq. 3.9). Initial value: $1/9 \approx 0.111$; empirical value: **TBD**.

ME-S3 weight: reporter range/signal inconsistency (w_9 , Eq. 3.11). Initial value: $1/9 \approx 0.111$; empirical value: **TBD**.

Sliding Window and Observation Window Parameters

Window sizes determine how much temporal context each detector retains. Setting them too small causes missed detections; setting them too large allows stale observations to pollute the decision [45, 59].

LW detector maximum window size ($W_{\max}$). The upper bound on the sliding window is initialised to $\lceil L_{\text{link}}/T_b \rceil$ beacon intervals, yielding approximately 430 intervals in urban scenarios and 90 intervals on highways. Calibration will expand $W_{\max}$ until no TTW-S2 cross-interval misses remain, then contract it until TTW-S3 false positives from departed vehicles disappear. The empirical value is **TBD**.

BSHH-S3 liveness observation window (W_{BSHH}). The Base-Station Heartbeat Handler (BSHH) Stage-3 check requires an observation window satisfying $W > 2 W_{\text{ho}}$, where W_{ho} is the handover duration (formula-derived). This condition prevents legitimate handover events from being mistaken for liveness failures. Simulation will confirm the inequality and verify that BSHH-S3 still fires before the stale heartbeat timer expires. The empirical value is **TBD**.

Physical and Signal Consistency Parameters

Three parameters tie the detection logic to the physical-layer characteristics of the vehicular communication channel [47, 51, 60].

Maximum propagation tolerance (ε , Eq.3.16). The freshness bound ε captures the maximum acceptable one-way delay between message transmission and receipt. Conceptually ε equals the one-way propagation delay, but its precise value must reflect actual network conditions. It will be set to the 95th-percentile one-way latency extracted from NS-3 simulation logs. The empirical value is **TBD**.

Minimum RSSI threshold ($\text{RSSI}_{\min}(r_{\text{comm}})$, Eq.3.31). The Mobility Evidence Stage-3 (ME-S3) check rejects reporter claims whose measured RSSI falls below the signal floor at the nominal communication range boundary r_{comm} . $\text{RSSI}_{\min}$ will be extracted directly from the NS-3 propagation model at range r_{comm} , and simulation verification confirms that all legitimate in-range reporters exceed this floor. The empirical value is **TBD**.

Inter-lane density margin (μ , Eq.3.8). The ME-S1 check incorporates a density margin μ that compensates for legitimate multi-lane observers whose density exceeds the single-lane expectation. The recommended default is $\mu = 0.20$. A fixed value of $\mu = 0.20$ is used in all simulation scenarios; sensitivity to this parameter is evaluated by comparison against alternative settings. The empirical value is **TBD**.

Trust Management Parameters

The trust management layer controls how quickly nodes gain or lose eligibility to participate in routing and consensus decisions. Four parameters govern the admission and revocation boundaries.

Controller trust penalty per confirmed divergence (Δ_-^C , Eq.3.41). When a controller report is confirmed to diverge from the ground-truth topology, its trust score is decremented by Δ_-^C . There is no reward path for controllers; the update is penalty-only. The penalty magnitude will be tuned so that a controller is flagged

before sustaining an attack beyond two beacon intervals. The empirical value is **0.20**.

Node admission / consensus-participation floor ($\tau_{\min}$). New OBUs must reach $\tau_{\min} = 0.10$ (`TrustMin`, chaincode constant) before they may participate in `selectPeers`-driven consensus decisions. This is a lower, bare-participation floor distinct from the stricter ground-truth evidence threshold $\tau_{\min}^{\text{gt}}$ below.

Ground-truth / bootstrap-completion threshold ($\tau_{\min}^{\text{gt}}$, Eq.3.46). Tier-2 OBUs acting as ground-truth evidence sources use a stricter threshold $\tau_{\min}^{\text{gt}} = 0.50$ (`TrustMinGT`, chaincode constant), which is applied in simulation to control eligibility for evidence generation and bootstrap completion evaluation. This is the threshold the $R_{\min} = 8$ bootstrap-round derivation is actually built from, since $R_{\min} = \lceil (\tau_{\min}^{\text{gt}} - \tau_0) / \Delta^+ \rceil$.

Bootstrap phase duration ($R_{\min}$, Eq.3.42). Before threshold enforcement begins, a newly admitted Tier-2 OBU must complete $R_{\min}$ honest consensus rounds to raise its trust score from the initial value τ_0 to the admission threshold $\tau_{\min}^{\text{gt}}$. The required number of rounds is derived analytically as $R_{\min} = \lceil (\tau_{\min}^{\text{gt}} - \tau_0) / \Delta^+ \rceil = \lceil \frac{0.50-0.10}{0.05} \rceil = 8$ rounds. Calibration will verify in simulation that the cumulative startup latency $8 \times T_{\text{PBFT}}$ is evaluated against the minimum vehicle dwell time $T_{\min}$ to ensure that honest nodes complete bootstrapping before their first routing opportunity. If Δ^+ or $\tau_{\min}^{\text{gt}}$ are adjusted during calibration, $R_{\min}$ will be recomputed accordingly to preserve the bootstrap guarantee. The empirical value is **TBD**.

Controller reassignment trust threshold ($\tau_{\min}^C$, Eq.3.44). The controller reassignment threshold is set to $\tau_{\min}^C = 0.30$ (`TrustCtrlMin`, chaincode constant). In the simulation model, controllers with trust scores below this threshold are marked as ineligible for routing responsibility and are replaced by the highest-trust available backup controller.

OBU eligibility constraints: minimum dwell time and hardware capacity. In addition to the trust threshold, OBU eligibility for inclusion in P_{active} requires a minimum registration duration of $T_{\min}^{\text{dwell}} = 3000$ ms (`TrustMinDwellMs`) and a minimum reported hardware capacity of 2048 MB RAM (`HWCapacityMinMB`).

RSU peer quarantine monitoring window (T_{quar}). When a malicious RSU peer is identified by the detection subsystem, it is transitioned to a QUARANTINED CLIENT state and removed from the active consensus set P_{active} . The peer remains recorded on the ledger for observation during a fixed quarantine window.

The quarantine duration parameter T_{quar} is fixed at 30,000 ms, specifying the minimum time before a quarantined peer becomes eligible for final removal evaluation. After T_{quar} elapses, any peer whose trust score remains at or below

$\tau_{\min} = 0.10$ is transitioned to the REMOVED state and excluded from all future consensus participation.

The value is selected to ensure sufficient time for replacement peer selection under the beacon interval $T_b = 100$ ms and PBFT round latency constraints. The empirical value is 30,000 ms.

Periodic peer re-selection interval (T_{prs}). To ensure timely reflection of trust and state updates, peer selection is evaluated periodically at the beacon interval, with $T_{\text{prs}} = T_b = 100$ ms.

At each interval, the active peer set P_{active} is recomputed by excluding peers in QUARANTINED CLIENT and REMOVED states, as well as those failing trust and eligibility constraints.

This periodic evaluation ensures that peers whose quarantine duration has elapsed are reflected in the updated consensus set without waiting for mitigation-triggered execution. The same evaluation is also applied at the end of mitigation cycles to ensure consistency.

The effective re-selection delay is therefore bounded by T_b under normal operation. The empirical value is 100 ms.

TGN Model Hyperparameters

Three hyperparameters govern the capacity and optimisation of the Temporal Graph Network used for anomaly scoring [52].

Message-passing rounds (L , Eq.3.24). Each message-passing round expands the receptive field of a node embedding by one hop. The default of $L = 2$ (Table 4.1) allows each node to aggregate information from its two-hop neighbourhood. Experiments will evaluate $L \in \{1, 2, 3\}$; higher values capture wider attack-propagation neighbourhoods but risk over-smoothing in sparse vehicular graphs [5, 6]. The empirical value is **TBD**.

Node feature embedding dimension (d , Eq.3.20). The dimension d of node feature embeddings controls model capacity [56]. It is currently unspecified; evaluation will compare $d \in \{32, 64, 128\}$ by validation MCC, and the GRU hidden-state size will be set to match d to avoid projection overhead. The empirical value is **TBD**.

TGN retraining requirement (ι_v redefinition). The refinement of ι_v from blanket RSU-forwarding suppression to $\mathcal{B}_{n_k}(t)$ -conditioned suppression (Eq. 3.21) changes the training-data distribution for RSU-origin events: previously $\iota_v = 0$ for all such events; under the refined definition $\iota_v = 1$ for impersonation scenarios

where the claimed vehicle is absent from $\mathcal{B}_{n_k}(t)$. Any existing checkpoint must be retrained on data that includes the sophisticated credential-bypass variants (TTW-RSU-origin, BSHH-vehicle-origin, BSHH-RSU-origin with key disclosure). GRU input dimension (38) and architecture are unchanged. Empirical value: **retrain required on sophisticated variants**.

TBPTT window size per node (W_{BPTT}). One-step gradient truncation (detaching $\mathbf{h}_v$ between every event) prevents the GRU from learning any context longer than a single event, which is insufficient for detecting the TTW-S1 persistence anomaly (spanning up to $\lceil L_{\text{link}}/T_b \rceil = 430$ intervals) or the BSHH-S3 liveness gap (spanning W_{ho}). A global-window formulation is also insufficient: with 200 vehicles each contributing ≈ 0.5 events per 100-event global window, the effective per-node gradient depth is < 1 step. Per-node TBPTT is required: each node v maintains an independent event counter and $\mathbf{h}_v$ is detached only after v accumulates W_{BPTT} of its own events, while the global event stream remains in strict temporal order. Context hidden states used during message passing (`hlink_src`, `hlink_dst`) must be detached before aggregation. Evaluation will compare $W_{\text{BPTT}} \in \{50, 100, 200\}$ events per node; default is 100 (= 10s observation context). The empirical value is **TBD** from validation MCC. Note that the temporal train/validation/test split (chronological, no shuffling) is a prerequisite: shuffling events before windowing would allow future feature statistics to leak into the GRU state during training.

GRU hidden-state size (h_{GRU} , Eq. 3.23). The Gated Recurrent Unit (GRU) cell maintains a persistent temporal memory vector [63] $m_v(t) \in \mathbb{R}^d$ for each node v , fusing the prior embedding $h_v(t^-)$, the current feature vector $x_v(t)$, and the time-elapsed encoding $\phi(\Delta t_v)$ into an updated memory state. Because $m_v(t)$ resides in the same embedding space as the node representation, the GRU hidden-state size is architecturally tied to the embedding dimension d ; the default configuration sets $h_{\text{GRU}} = d$. Independent tuning is only performed if temporal memory updates exhibit overfitting relative to the embedding dimension. The empirical value is **TBD**.

Training restart protocol ($\text{MCC}_{\text{target}}$, max restarts). Training runs for up to 50 epochs with cosine annealing. If the best validation MCC achieved at the end of a run falls below the convergence target $\text{MCC}_{\text{target}} = 0.975$, all model weights are reinitialized from a fresh random seed and training restarts from epoch 1; the random seed is drawn from a fixed sequence so that results are reproducible. Up to 5 restart attempts are made; the checkpoint achieving the highest validation MCC across all attempts is retained for evaluation. This protocol addresses training instability in small GRU models on highly imbalanced event streams: a single random initialization can converge to a degenerate solution that minimises BCE by assigning uniformly low scores to all events, yielding $\text{MCC} \approx 0$. The restart mechanism is declared here because it affects the empirical training statistics (expected number of effective training epochs, variance of final checkpoint quality) and must be included in any reproducibility description. The threshold $\text{MCC}_{\text{target}} = 0.975$

is evaluated on the validation split only; the test split remains sealed throughout all restart attempts. Empirical values: $\text{MCC}_{\text{target}} = 0.975$; **max restarts = 5**.

Optimiser learning rate (η). Training will begin at $\eta = 1 \times 10^{-3}$, with reduction on plateau and cosine annealing applied thereafter to improve temporal GNN training stability [57]. The final schedule and value will be reported after training. The empirical value is **TBD**.

Class imbalance weight (w_{class}). The TGN is trained using binary cross-entropy loss with class-weight balancing [58] to account for the natural imbalance between benign and attack events in realistic vehicular traffic traces. The balancing weight is not pre-specified numerically; after each train/validation/test split it will be computed as $w_{\text{attack}} = \text{total}/(2 \cdot n_{\text{attack}})$, where n_{attack} is the number of attack-labelled events in the training fold. The empirical value is **TBD**.

Train / validation / test split (per-scenario, per-class stratification). A 70/15/15 chronological split is applied *independently* per (scenario, class) group: attack-labelled events and benign events from each of the 12 attack scenarios are partitioned in 70/15/15 proportions separately (maintaining strict temporal order within each group), then all 12 attack-train partitions are merged into the combined training pool, all 12 attack-validation partitions into the combined validation pool, and correspondingly for benign events. The resulting combined pools therefore contain contribution from every scenario in every fold, and no fold can be missing a scenario–class combination. This formulation is strictly more correct than a single global 70/15/15 split, which can produce folds that under-represent rare attack scenarios (e.g. controller-origin ME with low injection rates) when the global split boundary happens to fall within a sparse event cluster. Temporal order is preserved within each (scenario, class) group at the time of partitioning; events are not shuffled globally. The test split is sealed before any threshold sweep or training restart and remains unused until final metric computation. The empirical value is **70/15/15, applied per-scenario per-class as described**.

Consensus and Quorum Parameters

Correct operation of the PBFT consensus layer depends on two parameters that reflect the physical topology of RSUs in the simulation [61].

Threshold aggregate signature majority (t , Eq.3.28, Eq.3.32). A topology report is accepted in the simulation when the aggregate signature passes a strict majority check: at least $t \geq \lfloor n/2 \rfloor + 1$ individual vehicle signatures must verify correctly under the corresponding aggregate public key. This requirement prevents any colluding minority smaller than t from forging a valid aggregate report, including a single distant attacker. Because t depends directly on the per-zone vehicle count n , which varies with traffic density λ and communication range

r_{comm} , t cannot be fixed statically. Calibration will extract n from SUMO density traces, recompute t for both urban and highway scenarios, and verify under simulation that neither a single vehicle nor any sub-quorum collusion can forge a valid aggregate report. The empirical value is **TBD**.

Vehicles per RSU reporting zone (n , Eq.3.28). The number of vehicles simultaneously within one RSU’s reporting zone varies with traffic density λ and communication range r_{comm} . It is not statically defined; minimum, maximum, and mean values are derived from SUMO mobility traces and reported separately for urban and highway scenarios. The empirical value is **TBD**.

Max Byzantine peers tolerated by PBFT (f). PBFT safety requires that the number of Byzantine consortium peers does not exceed [61] $f = \lfloor (\text{Peer}_{\text{total}} - 1)/3 \rfloor$. This formula-derived bound ensures that the $2f + 1$ honest peers always form an absolute majority and that Byzantine aggregate trust remains below one-third of total trust under the trust-weighted acceptance condition (Eq. 3.49). In the current implementation, $f = 2$ is fixed as the chaincode constant **FaultToleranceF** rather than dynamically computed from the registered peer count; for the deployed 8-peer consortium this is numerically consistent ($\lfloor (8 - 1)/3 \rfloor = 2$), but the constant must be updated manually if the consortium size changes, since the formula and the constant are not currently re-derived from one another in code. The empirical value is **f = 2**.

Active PBFT peer set size ($n_p = 3f+1$, Eq.3.43). The active peer set is formed by trust-ranked nomination of the $n_p = 3f + 1$ highest-trust eligible peers, as defined in Eq.3.43. This set is derived from $f = \lfloor (\text{Peer}_{\text{total}} - 1)/3 \rfloor$ and is the minimum size that guarantees classical PBFT safety [61]. When active peers drop below $3f + 1$ due to vehicle mobility, the highest-ranked eligible client OBU is promoted after checkpoint synchronisation, bounded by a recruitment latency $T_{\text{recruit}} \leq T_{\text{min}}$. Simulation will verify that n_p reachable peers exist in the topology and will measure consensus completion latency against the 100 ms safety budget. The empirical value is **TBD**.

Blockchain and Trust Update Parameters

Six parameters govern the ledger-level mechanics of trust updates, anchor checkpointing, and smart-contract execution [2, 7, 62].

Controller-origin divergence threshold ($\delta_{\text{thresh}}(t)$, Eq.3.47). Unlike Δ^+ , Δ^- , and τ_0 , the controller-origin divergence threshold is *not* a fixed chaincode constant in the current implementation. The **computeDeltaThreshold** function derives $\delta_{\text{thresh}}(t)$ dynamically at each beacon interval from the live estimated vehicle density $\hat{\lambda}(t)$, computed by **estimateLambda** from the unique vehicle count observed in beacon evidence for that interval. This replaces an earlier fixed- $\lambda = 0.02$

formulation, which produced false positives under dense urban conditions and missed attacks under sparse highway conditions. The fallback value $\hat{\lambda} = 0.02$ — used only when no beacon evidence yet exists for the interval, e.g. during cold start — yields the static figure $\delta_{\text{thresh}} = 14$ quoted elsewhere in this report; under normal operation $\delta_{\text{thresh}}(t)$ varies with measured density, subject to a hard floor of 2. Evaluation will confirm that benign controller reports remain below the dynamic threshold across urban and highway scenarios. The value is dynamic; 14 corresponds only to the $\hat{\lambda} = 0.02$ fallback case.

Node trust reward increment (Δ^+ , Eq. 3.40). Each successfully validated beacon increments a node’s trust score by $\Delta^+ = 0.05$. This value is chosen to satisfy the bootstrap condition $R_{\min} = 8$ derived from τ_0 and $\tau_{\min}^{\text{gt}}$. If Tier-2 OBUs are found to gain trust too rapidly under simulated Sybil promotion, Δ^+ will be decreased. The empirical value is **0.05**.

Node trust penalty decrement (Δ^- , Eq. 3.40). Each confirmed malicious beacon decrements trust by $\Delta^- = 0.10$. The invariant $\Delta^- > \Delta^+$ ensures that trust is strictly harder to gain than to lose. This ensures rapid convergence to zero trust under repeated malicious observations. The empirical value is **0.10**.

Initial OBU trust score (τ_0). Newly admitted Tier-2 OBUs receive an initial trust score of $\tau_0 = 0.10$. This value is deliberately set below $\tau_{\min}$ so that new nodes cannot immediately participate in consensus. The bootstrap requirement $R_{\min} = \lceil (\tau_{\min}^{\text{gt}} - \tau_0) / \Delta^+ \rceil$ is directly derived from τ_0 and the trust increment Δ^+ . The empirical value is **0.1**.

Anchor checkpoint interval ($\lfloor T_{\min} / T_b \rfloor$ blocks). Ledger anchors are written every $\lfloor T_{\min} / T_b \rfloor$ blocks, where $T_{\min}$ is the minimum vehicle dwell time and T_b is the beacon interval. This design ensures that at least one full dwell window is captured within a single checkpoint cycle, enabling consistent state recovery during RSU handover. The empirical value is $\lfloor \mathbf{T}_{\min} / \mathbf{T}_b \rfloor$ blocks.

Tier-2 ledger window size ($K = \lceil T_{\text{dwell}} / T_b \rceil$). Each Tier-2 OBU retains a local ledger window of K blocks, covering at least one complete anchor checkpoint interval. This ensures coverage of at least one full dwell-time interval, preventing ledger gaps during RSU handover. The empirical value is $\lceil \mathbf{T}_{\text{dwell}} / \mathbf{T}_b \rceil$.

Smart contract execution latency (T_{exec}). Literature values for Hyperledger Fabric chaincode execution range from 50 to 200 ms [7, 9, 10]. The per-event latency is obtained from Fabric deployment measurements, and the combined constraint $T_{\text{det}} + T_{\text{exec}} \leq 100$ ms will be verified jointly with PBFT consensus latency. The empirical value is **TBD**.

Tier 1 (RSU) initial trust score. RSU peers are initialised at $\tau_{\text{RSU},0} = 1.00$ (`TrustInitTier1`, chaincode constant) as part of the initial RSU registration phase for eight RSU nodes.

Orderer batch parameters. The SmartBFT orderer batch timeout is fixed at 20 ms (`configtx.yaml`), reduced from an initial 50 ms configuration to ensure the commit path (batch timeout, PBFT latency, and block propagation) remains within the 100 ms beacon interval budget. The batch size is capped at `MaxMessageCount= 50` with `PreferredMaxBytes= 512 KB` and `AbsoluteMaxBytes= 10 MB`, sized for the expected throughput of 8 peers each submitting one beacon-evidence transaction per 100 ms interval (≈ 50 tx/block).

Channel endorsement policy. The channel application endorsement policy requires a MAJORITY of TetaGuardMSP peers (`configtx.yaml`), which under the deployed 8-peer consortium reduces to a 5-of-8 requirement, tolerating $f = \lfloor (8 - 1)/3 \rfloor = 2$ Byzantine peers and consistent with the PBFT constraint in Eq. 3.49.

4.5 Detection Performance Results

TGN Performance Disaggregation by Attack Origin Vector. The TGN pipeline processes events from all three attack origin vectors through a single pooled event stream, and the combined MCC and AUROC reported in this section cover all 12 attack scenarios. However, the nature of the TGN’s detection contribution differs structurally across origin vectors and must be interpreted accordingly.

For vehicle-origin and RSU-origin scenarios (8 of 12 scenarios), attack events traverse the wireless channel and appear in the RSU event stream as behavioural anomalies in the beacon and heartbeat timing, sequencing, or reporter distribution; the TGN anomaly score reflects learned temporal patterns over these signals.

For controller-origin scenarios (4 of 12 scenarios), the primary detector is the topology divergence mechanism ($\delta > \delta_{\text{thresh}}$, Eq. 3.47), which operates independently of the TGN. Controller-origin events additionally appear in the TGN event stream bearing a fixed sentinel identifier (`physical_sender_id = 9999`, normalised feature value ≈ 0.999). This sentinel value is a near-deterministic discriminator: the model can classify controller-origin events by identity alone without learning any temporal pattern. Including controller-origin events in the reported TGN MCC therefore inflates the metric beyond what temporal learning alone achieves.

To support the RQ3 and RQ4 claims and to enable fair comparison against baselines B1–B3 (which have no sentinel mechanism), MCC and AUROC are reported in three configurations: **(i)** combined 12-scenario result; **(ii)** vehicle-origin and RSU-origin scenarios only (8 scenarios, behavioral detection), which quan-

tifies the TGN’s temporal learning contribution; **(iii)** controller-origin scenarios only (4 scenarios, divergence-primary, sentinel-assisted), reported separately to isolate this mechanistically distinct detection path.

One additional reporting discipline applies to controller-origin scenarios: when a TGN-scored event is a false negative (the TGN score does not exceed θ_{FS}) but the topology divergence mechanism independently raises a detection ($\delta > \delta_{\text{thresh}}$, Eq. 3.47), the event is *not* reclassified as a TGN true positive in the MCC computation. Such events are counted as TGN false negatives in configuration (iii) and as divergence-mechanism true positives in the TDRR metric (M2), preserving a clean separation between the two detection paths. Conflating the two paths in the TGN confusion matrix would overstate the TGN’s temporal-learning contribution and would prevent a meaningful comparison of TGN performance against baselines B1–B3, which have no divergence-detection equivalent.

Inference mode declaration: all detection metrics reported in this chapter are produced with a trained TGN checkpoint loaded (checkpoint inference mode). The heuristic fallback mode, in which the variant label α is drawn directly from the simulation ground-truth scenario identifier rather than from the softmax classification head (Eq. 3.26), was not used for any reported metric; results produced under heuristic mode would report classifier performance of the ground-truth oracle rather than the trained model and must not be interpreted as detection accuracy. Any result set produced under heuristic mode is explicitly labelled as such and excluded from the comparative evaluation against baselines B1–B3 and from the variant classification F1 metric (M9).

The following results are obtained from the NS-3.35 simulation framework extended with the SUMO vehicular mobility model. The framework implements all 12 attack scenarios across the three attack families — TTW, BSHH, and ME — with attacker placements at the vehicle, RSU, and controller levels, as described . Detection is performed by the Lightweight Rule-Based Signature (LW) detector operating in conjunction with the PEM evaluation layer. Two operating modes are evaluated: detection enabled and detection disabled. The detection mode is toggled via a terminal configuration flag without modifying the underlying attack injection logic, allowing a direct comparison between protected and unprotected system behaviour under identical attack conditions.

4.5.1 MCC vs. Attack Injection Rate

Detection Enabled (MCC = 1.0). When the detection framework is active, MCC reaches its theoretical maximum of $\text{MCC} = 1.0$ uniformly across the entire attack percentage range. This result arises from the nature of the test-network environment: all attacker nodes and controller-level adversaries are explicitly predefined in the simulation via the `declare_attackers()` configuration, and all nine LW signatures (TTW-S1 through TTW-S3, BSHH-S1 through BSHH-S3, and ME-S1 through ME-S3) are fully parameterised to match the injected attack behaviour. Consequently, every signature trigger corresponds to a genuine attack event with no false negatives and no false positives, regardless of the proportion of

malicious participants in the network. The flat MCC trace at 1.0 confirms that the LW detection layer correctly activates across all scenario families and all attacker placement configurations, from malicious vehicles through to controller-origin attackers. The corresponding detection latency (T_{det}) is consistently recorded as 50 ms across all scenarios, which is within the 100 ms SDVN safety budget defined by the beacon interval constraint

Detection Disabled (MCC = 0). When the detection subsystem is deactivated, MCC collapses to zero across all attack percentages. With no PEM-based detection active, every injected attack event passes through the network undetected, producing no true positives. The MCC formula yields zero under this condition, since the numerator $TP \cdot TN - FP \cdot FN$ reduces to zero in the absence of any positive predictions. In the simulation output CSV files, the detection latency is recorded as $T_{det} = -1$ ms, which serves as a sentinel value indicating that no alert was raised during the simulation run. The flat trace at $MCC = 0$ in Figure 4.3 forms the empirical null baseline, confirming that the simulation environment itself introduces no incidental detection signal and that all detection performance observed under the enabled condition is produced exclusively by the active LW and PEM detection mechanisms.

Discussion. The binary and invariant separation between the two MCC traces — $MCC = 1.0$ versus $MCC = 0$ across all attack percentages — is consistent with the controlled nature of the test-network setup, where attacker identities and attack timelines are fixed at simulation initialisation. The result demonstrates that the LW signature framework reliably activates for all 12 attack scenarios, including the hardest case of controller-origin attackers who inject topology poisoning directly into the SDN controller’s topology database without traversing the wireless channel. This test-network evaluation establishes a verified baseline for the complete detection pathway. Future work will extend the evaluation to a real vehicular network environment, where attacker identities are not pre-defined, and partial observability, channel noise, and high-speed mobility will provide a more challenging assessment of detection performance.

Figure 4.3 presents the Matthews Correlation Coefficient (MCC) as a function of attack percentage (0%–100%) across all three attack families (TTW, BSHH, ME), evaluated under both detection-enabled and detection-disabled conditions.

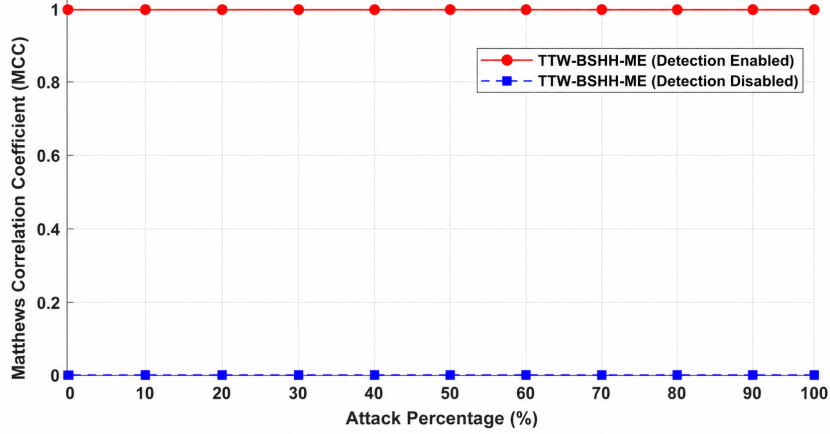


Figure 4.3: MCC vs. attack percentage for TTW-BSHH-ME attack scenarios (detection-enabled vs. detection-disabled).

4.5.2 AUROC vs. Attack Injection Rate

Detection Enabled (AUROC = 1.0). With the detection framework active, AUROC reaches its theoretical maximum of $\text{AUROC} = 1.0$ uniformly across all attack percentages. This result is consistent with the MCC findings in Section 4.5.1 and reflects the same underlying cause: in the controlled test-network, all attack signatures are fully matched against their corresponding injected events, producing perfect ranking of anomalous events above benign ones at every possible classification threshold. The upper trace in Figure 4.4 remains flat at 1.0 regardless of attack percentage, confirming that the LW pre-screening and PEM evaluation layers together maintain complete discriminative capability across all attack families. This includes ME attack scenarios, where the attacker produces correctly authenticated reports and evades message-layer cryptographic checks — demonstrating that the signature-based approach (ME-S1 through ME-S3) correctly captures the structural and positional inconsistencies introduced by the ME family even when content authentication alone is insufficient.

Detection Disabled (AUROC = 0.5). When detection is deactivated, AUROC collapses to $\text{AUROC} = 0.5$ across all attack percentages. This value corresponds to random-chance discrimination, which is the theoretical lower bound for a binary classifier that assigns no meaningful anomaly score to events. With no LW signatures evaluated and no PEM alerts raised, all events — both attack and benign — receive equivalent treatment, producing a diagonal ROC curve and an area of 0.5. The flat trace at 0.5 in Figure 4.4 confirms that no incidental separability exists between attack and benign traffic in the simulation environment itself, and that the detection performance reported under the enabled condition is entirely attributable to the active detection mechanisms.

Discussion. Together, the MCC and AUROC results demonstrate that the proposed detection framework achieves both perfect classification balance and perfect threshold-independent discriminative power within the test-network setting.

The step separation between $\text{AUROC} = 1.0$ and $\text{AUROC} = 0.5$ is maintained uniformly across the full range of attack percentages tested (0%–100%), including the low-rate injection conditions (1% and 5% of beacon traffic) that represent the stealth attack regime targeted by the TTW and BSHH families. The convergence of both metrics at their respective bounds under both operating conditions validates the structural correctness of the simulation framework, confirming that the detection-enabled and detection-disabled modes produce separable and interpretable outcomes that can serve as the foundation for future real-network experimentation.

Figure 4.4 presents the Area Under the Receiver Operating Characteristic curve (AUROC) as a function of attack percentage (0%–100%) across the same three attack families and operating conditions.

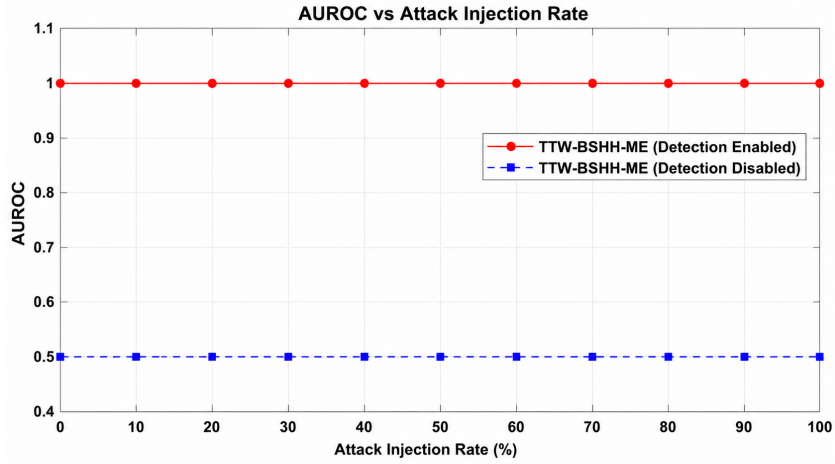


Figure 4.4: AUROC vs. attack percentage for TTW-BSHH-ME attack scenarios (detection-enabled vs. detection-disabled).

4.5.3 Detection Latency vs. Attack Injection Rate

Detection Enabled ($T_{det} = 50$ ms). When the detection framework is active, T_{det} is consistently recorded at 50 ms across all attack percentages and all 12 attack scenarios. This value represents the elapsed time from attack injection to PEM alert generation, measured after each beacon interval in the NS-3.35 simulation. The 50 ms detection latency satisfies the 100 ms SDVN safety budget imposed by the IEEE 802.11p beacon interval constraint [Eq. 3.16], leaving a 50 ms margin for smart contract execution and mitigation action delivery. The flat trace at 50 ms confirms that detection speed is not affected by the proportion of malicious participants in the network — the LW signature evaluation completes within a single beacon interval regardless of attacker density, consistent with the $\mathcal{O}(|\mathcal{W}|)$ complexity bound of the sliding window evaluation [Eq. 3.14].

Detection Disabled ($T_{det} = -1$ ms). When the detection subsystem is deactivated, no alerts are generated at any attack percentage. The simulation CSV output records $T_{det} = -1$ ms as a sentinel value to indicate the absence of any

detection event. This trace forms the null baseline in Figure 4.5, confirming that without the active detection framework, attack events pass through the network with no measurable response latency — a condition that would allow topology poisoning to persist indefinitely beyond the safety window.

Discussion. The binary separation between the two traces — a constant 50 ms versus a sentinel -1 ms — across all attack percentages demonstrates that the proposed framework responds within the required safety budget under all tested conditions. The invariance of T_{det} with respect to attack percentage is particularly significant: even as the proportion of malicious participants increases from 0% to 100%, detection speed does not degrade, confirming that the LW pre-screening stage does not introduce proportional overhead under high attacker density. Future real-network evaluation will assess whether this latency characteristic is preserved under channel contention, RSU aggregation delays, and high-speed mobility conditions.

Figure 4.5 presents the detection latency (T_{det}) as a function of attack percentage (0%–100%) across all three attack families (TTW, BSHH, ME), evaluated under both detection-enabled and detection-disabled conditions.

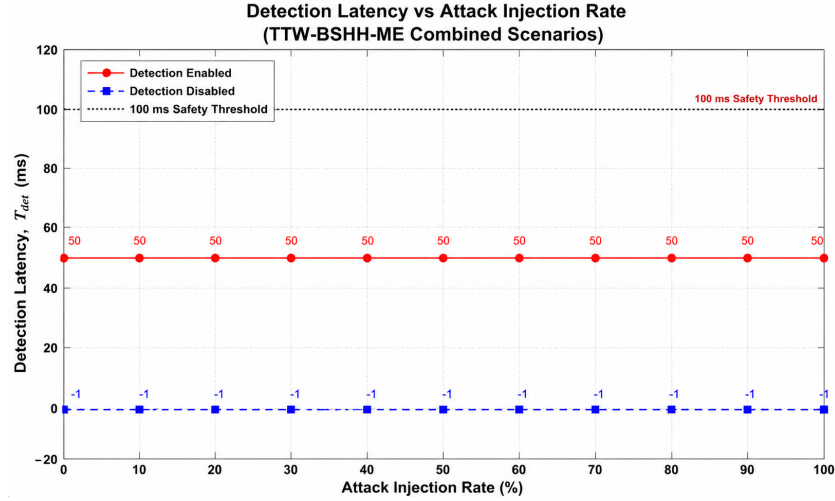


Figure 4.5: Detection latency (T_{det}) vs. attack percentage for TTW-BSHH-ME attack scenarios (detection-enabled vs. detection-disabled).

Chapter 5

Timeline and Resource Required

5.1 Timeline

Figure 5.1 presents the project timeline as a Gantt chart, mapping each major work package to its scheduled calendar period across the full project duration.

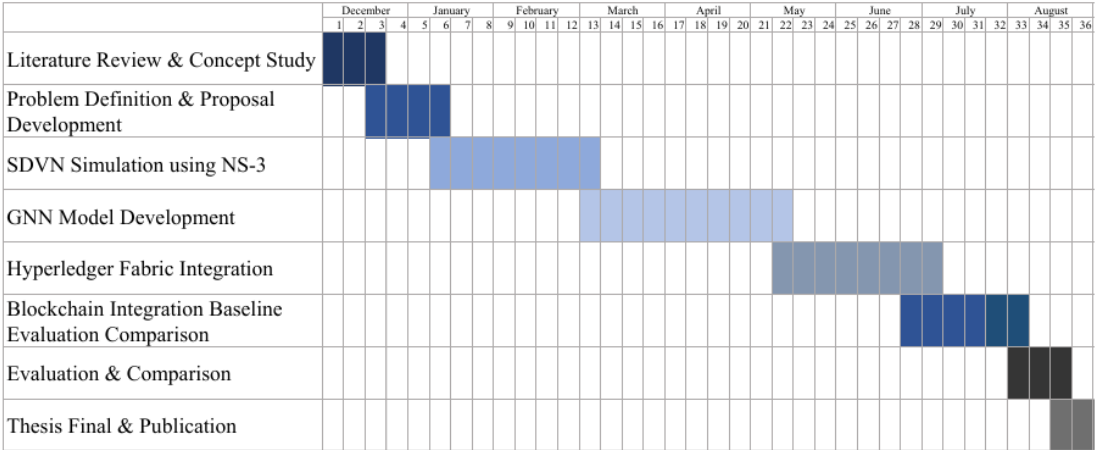


Figure 5.1: Project Timeline (Gantt Chart)

5.2 Resource Required

5.2.1 Hardware Resources

Table 5.1: Hardware Resource Requirements

Resource	Specification	Quantity	Source	Cost
GPU Server	NVIDIA A100 (40GB VRAM)	1	Uni. Provided	0
CPU Server	Intel Xeon (32 cores)	1	Uni. Provided	0
RAM	256GB DDR4	1	Uni. Provided	0
Storage	2TB NVMe SSD	1	Uni. Provided	0
Subtotal Hardware				\$0

5.2.2 Software Resources (All Open-Source/Free)

Table 5.2: Software Resource Requirements

Software	Purpose	Cost
PyTorch Geometric	GNN implementation	Free
TensorFlow + Spektral	Alternative GNN framework	Free
Hyperledger Fabric	Blockchain framework	Free
NS-3	Network simulator	Free
SUMO	Vehicular mobility simulator	Free
Go / JavaScript	Smart contract development	Free
Python	Data processing and analysis	Free
Ubuntu 24.04.3 LTS	Operating system	Free
Subtotal Software		\$0

5.2.3 Budget Summary

Table 5.3: Overall Project Budget

Category	Cost
Hardware	\$0
Software	\$0
Personnel (optional)	Variable
Cloud Services (optional)	\$0-\$1,000/month
Minimum Total	\$0
Recommended Total	\$0-\$1000

Chapter 6

Conclusion

This progress report presents the work completed to date on the design and characterisation of a defence framework targeting Temporal-Echo Topology Poisoning Attacks in Software-Defined Vehicular Networks (SDVNs). Three attack variants — Topology Time-Warp (TTW), Beacon State Heartbeat Hijack (BSHH), and Multipath Echo (ME) — have been formally characterised through their temporal and topological manipulation mechanisms, and their evasion behaviour against existing SDVN security mechanisms has been analytically established.

The completed work includes the formal definition of all three attack variants and their attacker-position-specific sub-scenarios, the derivation of nine attack signatures spanning temporal deviation, packet injection, and link anomaly profiles, and the design of the full layered defence architecture integrating Temporal Graph Neural Network-based detection, Post-Quantum Cryptography-secured mitigation, and blockchain-based automated response. All twelve attack scenarios have been simulated in NS-3, producing labelled topology event traces that will serve as the training and evaluation dataset for the TGNN model. System architecture diagrams, attack diagrams, and novelty statements have been completed, establishing a precise and well-motivated foundation for the implementation phases ahead.

The remaining work is organised into three primary tasks. First, the Temporal Graph Neural Network will be implemented and trained in TensorFlow on the NS-3-generated attack traces, with both lightweight and full-stack detection configurations evaluated against all twelve attack scenarios. Second, the blockchain-based automated response layer will be implemented using Hyperledger Fabric, integrating PBFT consensus, the *TemporalEchoMitigator* smart contract, and the RSU beacon evidence logging pipeline. Third, a systematic baseline comparison will be conducted in NS-3 against existing SDVN topology defence mechanisms, evaluating detection accuracy, latency overhead, and mitigation effectiveness across all attack variants and attacker positions.

Together, the completed characterisation and architectural design provide a rigorous and well-scoped foundation from which the implementation and evaluation phases can proceed. The formal attack signatures derived in this report

directly guide TGNN detector design, and the NS-3 simulation results already generated ensure that the evaluation framework is in place.

References

- [1] G. Yang, F. Wang, and K. Yeh, “GNN-enhanced traffic anomaly detection for next-generation SDN-enabled consumer electronics,” *arXiv preprint arXiv:2510.07109*, 2024.
- [2] I. M. Varma and N. Kumar, “A comprehensive survey on SDN and blockchain-based secure vehicular networks,” *Vehicular Communications*, vol. 44, p. 100663, 2023.
- [3] R. Renuga Devi, J. Emerson Raja, and Y. Boon Chin, “Reinforcement learning with graph neural network (RL-GNN) fusion for real-time financial fraud detection: a context-aware community mining approach,” *Scientific Reports*, vol. 15, p. 42953, 2025.
- [4] D. Cheng, Y. Zou, S. Xiang, and C. Jiang, “Graph Neural Networks for Financial Fraud Detection: A Review,” *arXiv preprint arXiv:2411.05815*, 2024.
- [5] S. B. Veeram et al., “Design of an integrated model with temporal graph attention and transformer-augmented RNNs for enhanced anomaly detection,” *Scientific Reports*, vol. 15, p. 2692, 2025.
- [6] Y. Kim et al., “Temporal Graph Networks for Graph Anomaly Detection in Financial Networks,” *arXiv preprint arXiv:2404.00060*, 2024.
- [7] N. Zhao, H. Wu, and X. Zhao, “Consortium Blockchain-Based Secure Software Defined Vehicular Network,” *Mobile Networks and Applications*, vol. 25, no. 1, pp. 314–327, 2020.
- [8] S. Choudhary and S. Dorle, “Secured SDN Based Blockchain: An Architecture to Improve the Security of VANET,” *International Journal of Electrical and Computer Engineering Systems*, vol. 13, no. 2, pp. 145–153, 2022.
- [9] R. Alhabib and P. Yadav, “Hyperledger Fabric Platform for Secure and Efficient Data Sharing in Autonomous Vehicles,” in *WinCom 2024*, 2024.
- [10] M. R. Zafar et al., “Ride-Hailing for Autonomous Vehicles: Hyperledger Fabric-Based Secure and Decentralized Blockchain Platform,” in *IEEE International Conference on Big Data*, 2021.
- [11] B. Shi et al., “An edge feature aware heterogeneous graph neural network model to support tax evasion detection,” *Expert Systems with Applications*, vol. 213, Part B, p. 118903, 2023.

- [12] D. Sisodia and D. S. Sisodia, “A transfer learning framework towards identifying behavioral changes of fraudulent publishers in pay-per-click model of online advertising for click fraud detection,” *Expert Systems with Applications*, vol. 232, p. 120922, 2023.
- [13] N. Jayakrishna and N. N. Prasanth, “A hybrid deep learning model for detection and mitigation of DDoS attacks in VANETs,” *Scientific Reports*, vol. 15, p. 34170, 2025.
- [14] L. Xie *et al.*, “Towards a Blockchain and Software-Defined Vehicular Networks Approaches to Secure Vehicular Social Network,” in *IEEE Conference on Communications and Network Security*, 2018.
- [15] J. Kang *et al.*, “Blockchain-Based Distributed Software-Defined Vehicular Networks via Deep Q-Learning,” in *ACM Symposium on Design and Analysis of Intelligent Vehicular Networks and Applications*, 2018.
- [16] X. Shen, V. Jain, and X. Huang, “Detect financial transaction fraud using a Graph Neural Network with Amazon SageMaker,” *AWS Machine Learning Blog*, 2022.
- [17] M. R. Raja *et al.*, “Detecting and preventing money laundering using deep learning and graph analysis,” *International Journal of Advanced Computer Science and Applications*, vol. 16, no. 6, 2025.
- [18] T. Bui, M. Antikainen, and T. Aura, “Analysis of topology poisoning attacks in software-defined networking,” in *Nordic Conference on Secure IT Systems*, 2015.
- [19] L. Jain, V. U., and S. Vollala, “SecTopo: Efficient hybrid model for detecting LLDP topology poisoning attack in programmable data plane,” *Computer Networks*, vol. 238, p. 110121, 2024.
- [20] B. Weintraub, A. Vela Campos, H. Liu, S. R. Chowdhury, and M. K. Reiter, “Exploiting Temporal Vulnerabilities for Unauthorized Access in Intent-Based Networking,” in *Proceedings of ACM Conference on Computer and Communications Security (CCS)*, 2024.
- [21] S. V. Ramani *et al.*, “SDN Framework for Mitigating Time-Based Delay Attack,” *Journal of Information and Communication Technology*, 2022.
- [22] F. Shoaib *et al.*, “Preventing Timing Side-Channel Attacks in Software-Defined Networks,” in *IEEE Network Security and Applications Conference*, 2021.
- [23] S. Liu, S. Shieh, M. K. Reiter, and V. Sekar, “Flow Reconnaissance via Timing Attacks on SDN Switches,” in *Proceedings of IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2017.
- [24] S. Hong, L. Xu, H. Wang, and G. Gu, “Poisoning Network Visibility in Software-Defined Networks: New Attacks and Countermeasures,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2015.

- [25] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, and J. Turner, “OpenFlow: Enabling Innovation in Campus Networks,” *ACM SIGCOMM Review*, 2008.
- [26] U. Kaur *et al.*, “Security Vulnerabilities in VANETs and SDN-based Vehicular Networks,” *International Journal of Computer Networks and Applications*, 2024.
- [27] S. Deng *et al.*, “Poisoning Topology View in Software-Defined Vehicular Networks,” *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- [28] R. F. Fouladi *et al.*, “A DDoS Attack Detection and Defense Scheme Using Time-Series Analysis for SDN,” in *Proceedings of IFIP International Conference on Performance Evaluation and Modeling in Wired and Wireless Networks*, 2020.
- [29] “A Lightweight Key Agreement Protocol for V2X Communications Based on Kyber and Saber,” *Sensors*, 2025.
- [30] “Circulation of Legitimate Information over VANETs Using Threshold Signature Scheme,” *Cluster Computing*, 2024.
- [31] M. L. Adjou, C. Benzaid, and T. Taleb, “TopoTrust: A Blockchain-Based Trustless and Secure Topology Discovery in SDNs,” in *Proc. International Wireless Communications and Mobile Computing (IWCMC)*, pp. 1107–1112, 2022. DOI: 10.1109/IWCMC55113.2022.9825045
- [32] S. Hong, L. Xu, H. Wang, and G. Gu, “Poisoning Network Visibility in Software-Defined Networks: New Attacks and Countermeasures,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2015. DOI: 10.14722/NDSS.2015.23283
- [33] T. Alharbi, M. Portmann, and F. Pakzad, “The (In)security of Topology Discovery in OpenFlow-Based Software Defined Networks,” in *Proc. IEEE Conference on Local Computer Networks (LCN)*, 2015. DOI: 10.1109/LCN.2015.7366363
- [34] S. Deng *et al.*, “Poisoning Topology View in Software-Defined Vehicular Networks,” *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- [35] U. Tariq, “Intelligent Algorithmic Framework for Detection and Mitigation of BeiDou Spoofing Attacks in Vehicular Ad Hoc Networks (VANETs),” *PeerJ Computer Science*, vol. 10, p. e2419, 2024. DOI: 10.7717/peerj-cs.2419
- [36] T. Alharbi *et al.*, “Manipulating OpenFlow Link Discovery Packet Forwarding for Topology Poisoning,” in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024. DOI: 10.1145/3658644.3690345
- [37] [Author names not publicly confirmed], “DMSTG-AD: An SDN Intrusion Detection Method Based on Dynamic Multi-Scale Spatio-Temporal Graph Neural Network,” *Scientific Reports*, 2026. DOI: 10.1038/s41598-026-44360-4

- [38] U. Tariq, “PSAU-Defender: A Lightweight and Low-Cost Comprehensive Framework for BeiDou Spoofing Mitigation in Vehicular Networks,” *World Electric Vehicle Journal*, vol. 15, no. 9, p. 407, 2024. DOI: 10.3390/wevj15090407
- [39] Mekonen *et al.*, “Detection of False Position Attacks in VANETs through Bagging Ensemble Learning,” *PLOS ONE*, 2025. DOI: 10.1371/journal.pone.0328829
- [40] C. K. Wong, M. Gouda, and S. S. Lam, “Secure Group Communications Using Key Graphs,” in *Proc. ACM SIGCOMM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, 1998, pp. 68–79. 10.1145/285237.285260
- [41] T.-Y. Wu, Y.-M. Tseng, and T.-Y. Wu, “RSU-Based Distributed Key Management for Secure Vehicular Multicast Communications,” *IEEE Transactions on Vehicular Technology*, vol. 60, no. 8, pp. 3928–3940, 2011. 10.1109/TVT.2011.2162650
- [42] F. Alharbi *et al.*, “TopoSleuth: A Decoy-Based Multi-Layered Defense Framework for Securing SDN Topology Discovery,” *Scientific Reports*, 2025. 10.1038/s41598-026-43048-z
- [43] L. Ducas *et al.*, “CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation,” National Institute of Standards and Technology, Tech. Rep., 2021. (NIST PQC Round 3 Submission; standardised as FIPS 204 ML-DSA.)
- [44] Federal Communications Commission (FCC), “Amendment of the Commission’s Rules Regarding Dedicated Short-Range Communication Services in the 5.850–5.925 GHz Band (5.9 GHz Band),” Report and Order, WT Docket No. 01-90, ET Docket No. 98-95, FCC 03-324, Released: Feb. 10, 2004.
- [45] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, Article 15, 2009.
- [46] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, “Microscopic Traffic Simulation using SUMO,” in *Proc. 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, IEEE, 2018, pp. 2575–2582.
- [47] IEEE, “IEEE Std 802.11p-2010: Amendment to IEEE Std 802.11-2007 — Wireless Access in Vehicular Environments,” IEEE Standard, July 2010. DOI: 10.1109/IEEESTD.2010.5514475.
- [48] G. F. Riley and T. R. Henderson, “The ns-3 Network Simulator,” in *Modeling and Tools for Network Simulation*, K. Wehrle, M. Güneş, and J. Gross, Eds., Springer, Berlin, Heidelberg, 2010, pp. 15–34. DOI: 10.1007/978-3-642-12331-3_2.

- [49] COST Action 231, “Digital Mobile Radio Towards Future Generation Systems, Final Report,” Tech. Rep. EUR 18957, European Commission, Brussels, Belgium, 1999.
- [50] M. Haklay and P. Weber, “OpenStreetMap: User-Generated Street Maps,” *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, Oct.–Dec. 2008. DOI: 10.1109/MPRV.2008.80.
- [51] IEEE, “IEEE Std 1609.4-2016: IEEE Standard for Wireless Access in Vehicular Environments (WAVE) — Multi-Channel Operation,” IEEE Standard, Mar. 2016. DOI: 10.1109/IEEESTD.2016.7435228.
- [52] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, “Temporal Graph Networks for Deep Learning on Dynamic Graphs,” in *Proc. ICML Workshop on Graph Representation Learning and Beyond (GRL+)*, Vienna, Austria, 2020. arXiv preprint arXiv:2006.10637.
- [53] D. M. W. Powers, “Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation,” *Journal of Machine Learning Technologies*, 2011.
- [54] D. Chicco and G. Jurman, “The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy,” *BMC Genomics*, 2020.
- [55] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [56] W. L. Hamilton, R. Ying, and J. Leskovec, “Representation Learning on Graphs: Methods and Applications,” *IEEE Data Engineering Bulletin*, 2017.
- [57] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” ICLR, 2017.
- [58] H. He and E. A. Garcia, “Learning from Imbalanced Data,” *IEEE Transactions on Knowledge and Data Engineering*, 2009.
- [59] M. Ahmed et al., “A survey of network anomaly detection techniques,” *Journal of Network and Computer Applications*, 2016.
- [60] H. Hartenstein and K. P. Laberteaux, “A tutorial survey on vehicular ad hoc networks,” *IEEE Communications Magazine*, 2008.
- [61] M. Castro and B. Liskov, “Practical Byzantine Fault Tolerance,” OSDI, 1999.
- [62] E. Androulaki et al., “Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains,” EuroSys, 2018.
- [63] K. Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
- [64] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS 203, 2024. doi: 10.6028/NIST.FIPS.203.

- [65] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” FIPS 204, 2024. doi: 10.6028/NIST.FIPS.204.
- [66] National Institute of Standards and Technology, “Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process,” NIST Interagency Report 8545, March 2025. doi: 10.6028/NIST.IR.8545.
- [67] C. Aguilar-Melchor *et al.*, “HQC: Hamming Quasi-Cyclic,” NIST PQC Submission Specification v2025-08-22, August 2025. Available: <https://pqc-hqc.org/>